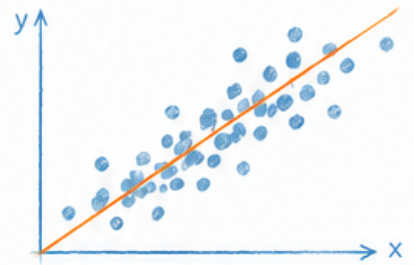
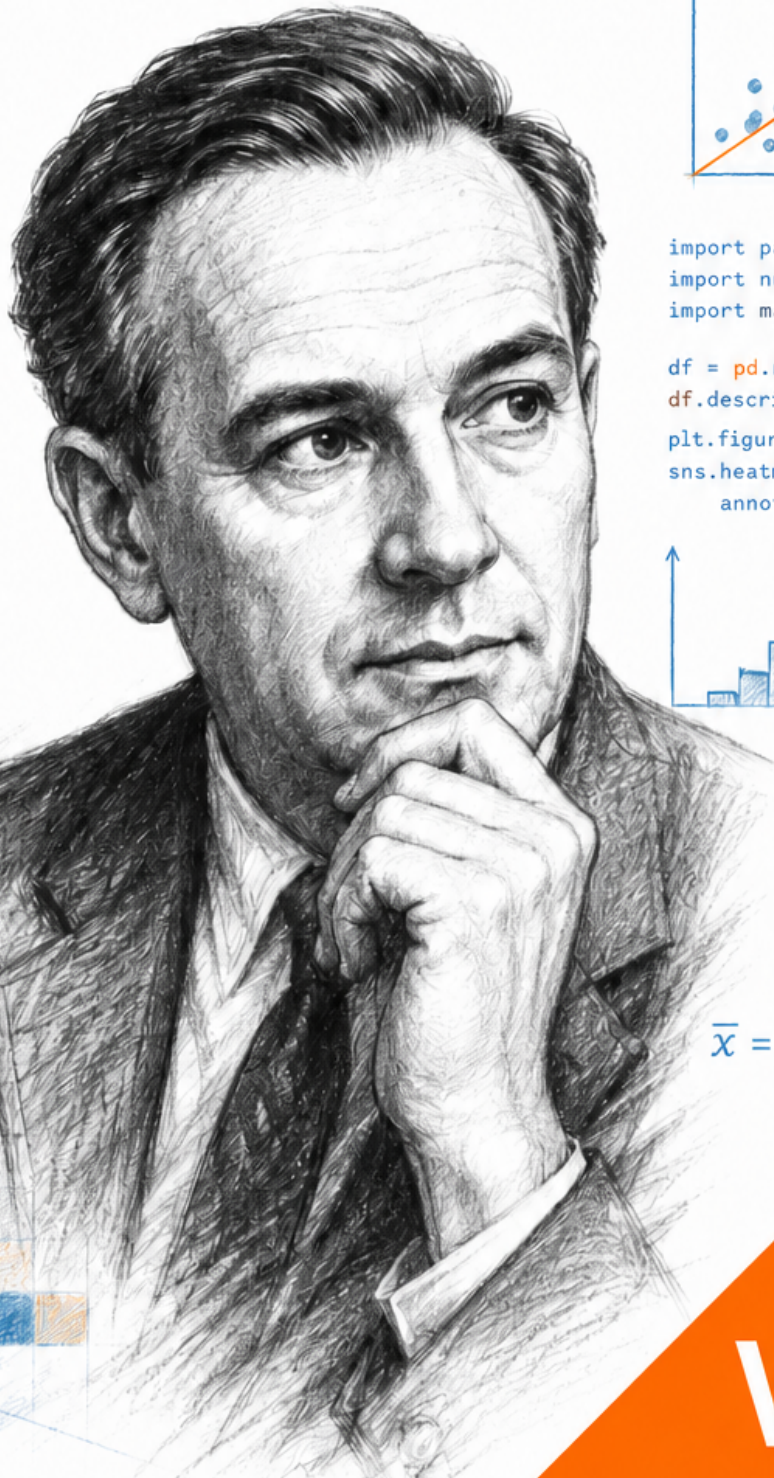


CIENCIA DE DATOS CON PYTHON

— DE LOS DATOS BRUTOS AL CONOCIMIENTO —



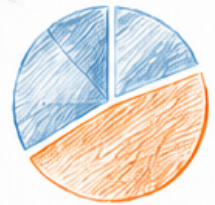
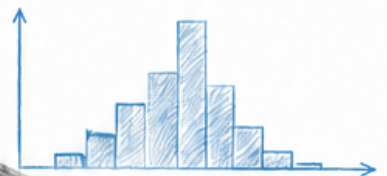
John Tukey
(1915–2000)



```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv('datos.csv')
df.describe()

plt.figure(figsize=(8,5))
sns.heatmap(df.corr(),
            annot=True, cmap='coolwarm')
```



$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

Vol. I

MATERIAL GRATUITO

Última versión y apoyo al proyecto

Este documento forma parte de VintaBytes y se comparte de manera gratuita.

Escaneá el código QR para acceder a la última versión disponible, encontrar otros recursos del proyecto o colaborar de forma voluntaria.

Si este material te resultó útil, cualquier aporte ayuda a seguir preparando libros, cuadernos y ejemplos en español. También ayuda compartirlo, recomendarlo o avisarme si encontrás algo para mejorar.

Y si querés colaborar con el café que acompaña muchas de estas páginas, prometo usarlo con responsabilidad pedagógica.

Muchas gracias por formar parte de VintaBytes.



Escaneá para ir a la página oficial

También podés entrar desde:

<https://vintabytes.github.io/>

John Tukey

John W. Tukey fue un matemático y estadístico estadounidense, considerado una figura clave en el desarrollo de la estadística moderna y del análisis exploratorio de datos. Nacido en 1915, trabajó durante gran parte de su carrera entre la Universidad de Princeton y Bell Labs, dos espacios desde los que influyó profundamente en la forma en que se estudian, resumen y visualizan los datos.

Su aporte más importante para la ciencia de datos fue haber impulsado el análisis exploratorio de datos, una manera de trabajar que propone observar los datos antes de aplicar modelos o conclusiones formales. Para Tukey, analizar datos no era solo calcular resultados, sino también mirar distribuciones, detectar patrones, encontrar valores atípicos y formular mejores preguntas a partir de la evidencia disponible.

También desarrolló herramientas que hoy siguen siendo habituales en análisis de datos, como el diagrama de caja o boxplot, y promovió resúmenes simples pero potentes, como el resumen de cinco números. Estas herramientas permiten comprender rápidamente la distribución de una variable, su mediana, su dispersión y la posible presencia de valores extremos.

Además, Tukey realizó aportes importantes en áreas como el procesamiento de señales, la transformada rápida de Fourier y la estadística computacional. Incluso se le atribuye la creación o popularización de términos fundamentales para la informática, como “bit” y “software”.

Por todo esto, John Tukey ocupa un lugar central en la historia de la ciencia de datos. Su trabajo ayudó a instalar una idea que hoy parece natural: antes de modelar, conviene explorar; antes de concluir, conviene mirar; y antes de confiar en una respuesta automática, conviene entender qué muestran realmente los datos.

Créditos y uso del material

Ciencia de Datos con Python - Tomo 2

Fundamentos del trabajo con datos

Autor: **Ariel Palazzesi**

Año: **2026**

Versión actualizada de este documento:

<https://vintabytes.github.io/>

Este libro forma parte de una serie de materiales educativos sobre ciencia de datos con Python. La presente versión reúne en formato PDF el contenido desarrollado originalmente como cuadernos de trabajo para Google Colab.

Los cuadernos originales están disponibles de forma gratuita en el repositorio del proyecto:

Repositorio:

<https://github.com/VintaBytes/Ciencia-de-datos>

Desde allí, el lector puede acceder a los notebooks, abrirlos en Google Colab, ejecutar el código, modificar ejemplos y experimentar con los datos como parte del proceso de aprendizaje.

Derechos de autor

© 2026 Ariel Palazzesi. Todos los derechos reservados.

Este material se publica con fines educativos. Se permite su lectura, descarga y uso personal o educativo, así como la ejecución y modificación de los cuadernos en Google Colab con fines de aprendizaje.

No se permite la venta, redistribución, publicación de versiones modificadas, incorporación en cursos comerciales, reproducción total o parcial con fines comerciales, ni la creación de obras derivadas para su distribución pública sin autorización previa y por escrito del autor.

La disponibilidad pública de este libro o de sus cuadernos complementarios no implica cesión de derechos ni autorización para usos distintos de los indicados en esta página.

Para solicitar permisos de uso, adaptación o distribución, contactar al autor.

Uso educativo

Este material puede utilizarse como apoyo para el estudio personal, la consulta docente o el acompañamiento de cursos introductorios de programación, análisis de datos o ciencia de datos con Python, siempre respetando las condiciones de uso indicadas anteriormente.

Se recomienda acompañar la lectura del PDF con la ejecución de los cuadernos online, ya que buena parte del aprendizaje ocurre al probar instrucciones, observar resultados, modificar el código y revisar cómo cambian los datos después de cada operación.

Herramientas utilizadas

Los ejemplos y actividades de este tomo utilizan principalmente:

- Python;
- Pandas;
- Google Colab;
- Jupyter Notebooks.

Las menciones a tecnologías, bibliotecas, plataformas o servicios de terceros se realizan con fines educativos y descriptivos. Cada herramienta conserva sus propias marcas, licencias y condiciones de uso.

Nota sobre el alcance del material

Este tomo está pensado como una introducción progresiva al trabajo con datos tabulares en Python. No pretende cubrir todos los temas de ciencia de datos, sino construir una base sólida para continuar luego con análisis exploratorio, visualización, comunicación de resultados y machine learning.

Introducción

Trabajar con datos se volvió una habilidad cada vez más necesaria en contextos muy distintos: educación, investigación, administración, comunicación, comercio, tecnología, salud, gestión pública y muchas otras áreas. En todos esos casos aparecen tablas, registros, archivos, planillas o bases de datos que necesitan ser explorados, ordenados, limpiados e interpretados para poder transformarse en información útil.

Este libro propone una introducción práctica al trabajo con datos usando Python. No parte de la idea de que analizar datos sea simplemente aplicar comandos, sino de una mirada más amplia: antes de calcular, graficar o construir modelos, necesitamos entender qué tenemos delante, cómo está organizado un dataset, qué preguntas podemos hacerle y qué problemas pueden aparecer en los datos reales.

El contenido de este libro proviene de una serie de cuadernos de trabajo preparados para Google Colab. En esta versión en PDF, esos cuadernos se presentan en formato de texto, para facilitar la lectura continua, la consulta y el estudio. Sin embargo, la experiencia más completa se obtiene al trabajar también con los cuadernos originales en línea: allí es posible ejecutar el código, modificarlo, observar los resultados y experimentar con los ejemplos.

Los cuadernos están disponibles de forma gratuita en un repositorio de GitHub. Si el lector lo desea, puede acceder a ellos desde allí, abrirlos en Google Colab y usarlos como material práctico complementario al libro. Esta forma de trabajo es especialmente recomendable, porque en programación y análisis de datos se aprende mucho al probar, equivocarse, corregir y volver a ejecutar.

Qué vas a encontrar en este tomo

Este primer tomo se concentra en los fundamentos del trabajo con datos tabulares en Python. A lo largo de los capítulos se introducen conceptos esenciales como dataset, DataFrame, filas, columnas, variables, observaciones, tipos de datos y valores. Luego se avanza hacia las primeras operaciones de inspección, selección, filtrado, ordenamiento y creación de columnas.

Una parte importante del recorrido está dedicada a la limpieza y preparación de datos. En los datasets reales es frecuente encontrar valores faltantes, duplicados, textos escritos de distintas maneras, tipos de datos incorrectos, fechas mal interpretadas o columnas que necesitan ser reorganizadas. Por eso, el libro no se

limita a mostrar instrucciones aisladas: también propone criterios para diagnosticar problemas, decidir estrategias de limpieza y verificar los resultados obtenidos.

Hacia el final del tomo, el objetivo es que el lector pueda pasar de una primera tabla cargada en Python a un dataset más ordenado, comprensible y preparado para etapas posteriores de análisis.

Requisitos previos

Para aprovechar este libro no es necesario tener experiencia previa en ciencia de datos, estadística avanzada o machine learning. Sí conviene contar con una base inicial de programación en Python, especialmente:

- uso de variables;
- llamadas a funciones;
- lectura de resultados en pantalla;
- listas y diccionarios a nivel introductorio;
- nociones básicas de condicionales y ciclos;
- familiaridad general con notebooks o disposición para aprender a usarlos.

También resulta útil tener una idea general de qué es una tabla de datos, una fila, una columna o una planilla, aunque estos conceptos se retoman desde el comienzo.

El libro utiliza principalmente Python, Pandas y Google Colab. No hace falta instalar un entorno de desarrollo local para seguir los capítulos, ya que los cuadernos pueden ejecutarse desde el navegador a partir de los archivos disponibles en el repositorio.

A quién está dirigido

Este material está pensado para cualquier persona que necesite empezar a trabajar con datos usando Python. Puede ser útil para estudiantes, docentes, profesionales, analistas, personas que trabajan con planillas, programadores que quieren acercarse al análisis de datos o lectores que se preparan para estudiar ciencia de datos, visualización, inteligencia artificial o machine learning.

El enfoque es introductorio, pero no superficial. Se busca construir una base sólida, con explicaciones claras y ejemplos progresivos, evitando saltar demasiado rápido a herramientas más complejas sin haber desarrollado antes los criterios necesarios para entender y preparar los datos.

Cómo trabajar con este libro y los cuadernos

Cada capítulo puede leerse directamente en este PDF, pero fue pensado originalmente como un cuaderno ejecutable. Por eso, la recomendación es acompañar la lectura con la versión online del cuaderno correspondiente siempre que sea posible.

En los cuadernos de Google Colab, conviene no limitarse a mirar el código: es preferible ejecutar cada celda, observar la salida, comparar resultados y probar pequeñas modificaciones. En análisis de datos, muchas veces la comprensión aparece al mirar con atención qué devuelve cada instrucción y cómo cambia el DataFrame después de cada operación.

También es importante leer las explicaciones que acompañan al código. En varios momentos, el foco no está solamente en aprender una instrucción de Pandas, sino en entender por qué se usa, qué pregunta ayuda a responder y qué precauciones conviene tener al interpretar el resultado.

Los capítulos fueron diseñados como una secuencia progresiva. Algunos conceptos aparecen primero de manera simple y luego se retoman en situaciones más complejas. Esa repetición controlada forma parte del aprendizaje: permite afianzar herramientas, reconocer patrones y ganar autonomía. El PDF puede servir como guía de lectura y consulta; los cuadernos online, como espacio de práctica, exploración y experimentación.

Enfoque del libro

El objetivo principal de este tomo es desarrollar una primera alfabetización práctica en datos con Python. Esto implica aprender instrucciones, pero también aprender a mirar un dataset, hacer preguntas, detectar problemas, justificar decisiones y verificar cambios.

A lo largo del recorrido se priorizan tres ideas:

- comprender antes de transformar;
- trabajar paso a paso, verificando cada resultado;
- preparar datos de manera cuidadosa antes de avanzar hacia análisis más complejos.

Con esa base, el lector queda mejor preparado para continuar con temas como análisis exploratorio, visualización, agrupaciones, conclusiones a partir de datos y, más adelante, machine learning.

Índice general - Tomo 1

Capítulo 1 - Primer contacto con el trabajo con datos

- Presentación general de la serie y del trabajo con datos.
- Diferencia entre datos, información y conocimiento.
- Qué es un dataset y cómo se organiza en filas, columnas, variables, observaciones y valores.
- Tipos de variables: numéricas, categóricas, ordinales, temporales y de texto.
- Primer contacto con un dataset en Python usando Pandas.
- Lectura inicial de una tabla, formulación de preguntas y primeras respuestas.
- Relación entre variables, límites de un dataset y problemas habituales en datos reales.

Capítulo 2 - Primeras inspecciones de un DataFrame

- Importancia de inspeccionar un DataFrame antes de analizarlo.
- Visualización de primeras y últimas filas con `head()` y `tail()`.
- Revisión de dimensiones con `shape`.
- Exploración de nombres de columnas, índice y tipos de datos.
- Uso de `info()` para obtener una vista general del DataFrame.
- Uso de `describe()` para estadísticas iniciales.
- Primeras lecturas interpretativas a partir de la estructura del dataset.

Capítulo 3 - Selección de columnas en un DataFrame

- Qué significa seleccionar columnas dentro de un DataFrame.
- Selección de una columna y de varias columnas.
- Diferencia entre obtener una Serie y obtener un DataFrame.
- Guardado de selecciones en nuevas variables.
- Uso de columnas seleccionadas para responder preguntas simples.
- Selección con corchetes y comparación con la notación de punto.
- Errores frecuentes al seleccionar columnas.

Capítulo 4 - Selección de filas en un DataFrame

- Qué significa seleccionar filas por posición.

- Uso de `iloc` para seleccionar una fila, varias filas y rangos.
- Diferencia entre posición e índice.
- Selección de la última fila y de filas no consecutivas.
- Selección combinada de filas y columnas con `iloc`.
- Errores frecuentes al trabajar con posiciones.

Capítulo 5 - Selección con etiquetas usando `loc`

- Diferencia entre `iloc` y `loc`.
- Selección de filas usando etiquetas del índice.
- Selección de columnas con `loc`.
- Selección simultánea de filas y columnas.
- Casos donde índice y posición no coinciden.
- Efecto de las etiquetas repetidas en el índice.
- Errores frecuentes al seleccionar datos con etiquetas.

Capítulo 6 - Filtros con condiciones

- Qué significa filtrar filas de un DataFrame.
- Construcción de condiciones sobre columnas.
- Máscaras booleanas y aplicación de filtros.
- Filtros con valores numéricos y categóricos.
- Búsqueda de valores distintos.
- Filtrado de filas combinado con selección de columnas.
- Uso de `isin()` para valores incluidos en una lista.
- Filtros sobre textos que contienen una palabra.

Capítulo 7 - Combinar condiciones en filtros

- Revisión de filtros simples.
- Combinación de condiciones con `&`.
- Necesidad de usar paréntesis en condiciones compuestas.
- Combinación de condiciones con `|`.
- Diferencia entre exigir varias condiciones o aceptar alternativas.
- Negación de condiciones con `~`.
- Guardado de condiciones en variables.
- Combinación de filtros y selección de columnas.

Capítulo 8 - Ordenar datos en un DataFrame

- Sentido analítico de ordenar datos.
- Ordenamiento por una columna con `sort_values()`.
- Orden ascendente y descendente.
- Ordenamiento de valores numéricos y columnas de texto.
- Ordenamiento por más de una columna.
- Uso de distintos criterios de orden para varias columnas.
- Combinación de filtros, ordenamiento y selección de columnas.
- Guardado de resultados ordenados y ordenamiento por índice.

Capítulo 9 - Valores extremos y primeros rankings

- Qué significa buscar valores extremos en un dataset.
- Uso de `head()` y `tail()` para revisar registros iniciales o finales.
- Ordenamiento combinado con `head()` y `tail()`.
- Obtención de mayores valores con `nlargest()`.
- Obtención de menores valores con `nsmallest()`.
- Comparación entre distintas formas de buscar extremos.
- Construcción de rankings según diferentes columnas.
- Combinación de filtros y rankings.

Capítulo 10 - Crear nuevas columnas en un DataFrame

- Qué significa crear una nueva columna.
- Creación de columnas calculadas a partir de otras columnas.
- Verificación de columnas nuevas.
- Comparación entre precio, cantidad e importe total.
- Creación de columnas con valores constantes.
- Creación de columnas a partir de condiciones.
- Uso de etiquetas de texto para clasificar registros.
- Asignación de valores con `loc`.

Capítulo 11 - Modificar columnas existentes en un DataFrame

- Qué significa modificar una columna existente.
- Revisión de valores únicos antes de transformar.

- Limpieza de espacios innecesarios.
- Unificación de mayúsculas y minúsculas.
- Reemplazo de valores con `replace()`.
- Decisión entre modificar una columna o crear una nueva.
- Encadenamiento de transformaciones.
- Modificación de columnas numéricas y verificación de cambios.

Capítulo 12 - Organizar columnas en un DataFrame

- Qué significa organizar la estructura de columnas.
- Revisión de nombres actuales.
- Renombrado de columnas con `rename()`.
- Guardado del resultado de un renombrado.
- Eliminación de columnas con `drop()`.
- Eliminación de varias columnas.
- Reordenamiento de columnas.
- Construcción y verificación de una versión organizada del DataFrame.

Capítulo 13 - Qué significa limpiar datos

- La limpieza de datos como proceso de análisis, no solo como aplicación de comandos.
- Problemas frecuentes en datasets reales.
- Diagnóstico antes de transformar datos.
- Decisión de estrategias de limpieza.
- Verificación posterior a cada transformación.
- Primeras inspecciones de un dataset real.
- Detección inicial de valores faltantes y valores problemáticos.
- Registro de problemas detectados.

Capítulo 14 - Valores faltantes

- Qué significa que falte un dato.
- Representación de valores faltantes en Pandas.
- Detección de faltantes con `isna()`.
- Conteo de valores faltantes por columna.
- Cálculo de porcentajes de faltantes.
- Ordenamiento del diagnóstico de valores faltantes.

- Revisión de filas con faltantes.
- Diferencia entre `NaN` y valores textuales como `UNKNOWN` o `ERROR`.

Capítulo 15 - Eliminar, imputar o reconstruir valores faltantes

- Estrategias posibles frente a valores faltantes.
- Eliminación de filas con faltantes.
- Eliminación según columnas específicas.
- Eliminación de columnas con demasiados faltantes.
- Imputación con valores fijos.
- Imputación numérica usando medidas estadísticas.
- Reconstrucción de valores a partir de otras columnas.
- Comparación y verificación de estrategias de tratamiento.

Capítulo 16 - Duplicados en un DataFrame

- Qué significa que una fila esté duplicada.
- Detección de duplicados completos.
- Visualización de filas duplicadas.
- Eliminación de duplicados completos.
- Duplicados según una columna específica.
- Diferencia entre filas parecidas y filas duplicadas.
- Duplicados definidos por varias columnas.
- Uso del parámetro `keep` y verificación posterior.

Capítulo 17 - Limpieza de textos y categorías

- Revisión de valores únicos y frecuencias.
- Limpieza de espacios al comienzo o al final de los textos.
- Unificación de mayúsculas y minúsculas.
- Aplicación de limpiezas similares en varias columnas.
- Reemplazo de valores inconsistentes.
- Tratamiento de valores desconocidos, errores y faltantes.
- Comparación antes y después de limpiar.
- Construcción de una versión limpia del dataset.

Capítulo 18 - Conversión de tipos de datos

- Revisión e interpretación de tipos de datos.
- Identificación de columnas que parecen numéricas pero no lo son.
- Conversión con `pd.to_numeric()`.
- Control de errores durante la conversión.
- Conversión de varias columnas numéricas.
- Verificación de faltantes después de convertir.
- Cálculo de estadísticas luego de corregir tipos.
- Conversión y validación de columnas de fecha.

Capítulo 19 - Fechas y datos temporales

- Revisión inicial de columnas de fecha.
- Conversión de textos a fechas.
- Detección de valores no interpretables.
- Revisión del rango temporal del dataset.
- Ordenamiento de datos por fecha.
- Extracción de componentes de una fecha.
- Obtención del día de la semana.
- Primeras exploraciones y agrupaciones temporales.

Capítulo 20 - Estandarizar nombres y crear nuevas columnas

- Importancia de estandarizar nombres de columnas.
- Renombrado con criterios uniformes.
- Preparación de columnas antes de crear nuevas variables.
- Creación de columnas calculadas.
- Creación de columnas de validación.
- Identificación de coincidencias, diferencias y datos insuficientes.
- Creación de variables a partir de fechas.
- Verificación de las columnas nuevas.

Capítulo 21 - Limpieza encadenada paso a paso

- Organización de una limpieza completa sobre una copia de trabajo.
- Estandarización de nombres de columnas.
- Conversión de columnas numéricas.

- Conversión de columnas de fecha.
- Revisión de valores categóricos problemáticos.
- Creación de columnas calculadas y de validación.
- Creación de columnas temporales.
- Construcción de una versión final de trabajo.

Capítulo 22 - Validación posterior a la limpieza

- Comparación de dimensiones antes y después de limpiar.
- Revisión de tipos de datos finales.
- Revisión de valores faltantes finales.
- Revisión de duplicados.
- Revisión de categorías problemáticas.
- Revisión de rangos numéricos y fechas válidas.
- Validación de consistencia interna.
- Construcción de una tabla resumen de validación y registro de problemas pendientes.

Capítulo 23 - Del primer DataFrame al dataset preparado

- Balance del recorrido desde la primera lectura de una tabla.
- Comprensión progresiva de un DataFrame.
- Selección, filtrado y ordenamiento de información.
- Creación, modificación y organización de columnas.
- Paso de operaciones aisladas a un proceso de limpieza.
- Problemas de datos que se aprendieron a detectar y tratar.
- Qué significa tener un dataset preparado.
- Buenas prácticas

Capítulo 1 — Primer contacto con el trabajo con datos

Presentación de la serie

Vamos a aprender a trabajar con datos usando Python, Pandas y otras herramientas habituales del ecosistema de análisis de datos.

La idea no es solamente aprender comandos o funciones sueltas. El objetivo principal es construir una forma de trabajo: aprender a observar un conjunto de datos, comprender qué representa, detectar problemas, seleccionar información relevante, realizar cálculos, resumir resultados, crear visualizaciones y comunicar conclusiones de manera clara.

A lo largo de los capítulos vamos a trabajar con datasets de distinta complejidad. Algunos serán pequeños y estarán contruidos especialmente para aprender un concepto puntual. Otros serán más parecidos a los que aparecen en situaciones reales, con datos faltantes, columnas desordenadas, valores inconsistentes o información que requiere ser limpiada antes de poder analizarse.

En este primer capítulo vamos a comenzar por una pregunta fundamental: **¿qué significa trabajar con datos?**

Antes de usar Pandas, cargar archivos o hacer gráficos, necesitamos entender qué es un dato, qué es un dataset, qué representa una fila, qué representa una columna y por qué el análisis de datos no consiste solamente en aplicar código, sino también en interpretar la información dentro de un contexto.

Al finalizar este capítulo deberías poder:

- Explicar qué diferencia hay entre datos, información y conocimiento.
- Reconocer qué es un dataset.
- Identificar filas, columnas, variables, observaciones y valores.
- Distinguir algunos tipos básicos de variables.
- Comprender por qué antes de analizar datos necesitamos observarlos y entender su contexto.

Este capítulo funcionará como punto de partida para todo el recorrido. Más adelante iremos incorporando progresivamente herramientas de Pandas para cargar, explorar, limpiar, transformar, resumir y visualizar datos.

¿Por qué aprender a trabajar con datos?

Vivimos rodeados de datos. Cada vez que una persona realiza una compra, completa un formulario, responde una encuesta, usa una aplicación, mira un video, registra una actividad física o consulta una página web, se generan datos.

Sin embargo, tener datos no significa automáticamente comprender una situación.

Una tabla puede tener cientos, miles o millones de filas, pero por sí sola no explica qué está ocurriendo. Para que esos datos sean útiles, necesitamos hacerles preguntas, revisarlos, ordenarlos, limpiarlos, analizarlos e interpretarlos.

Por ejemplo, imaginemos que una escuela registra las notas de sus estudiantes. La tabla puede contener el nombre de cada estudiante, el curso, la materia, la nota obtenida y la fecha de la evaluación. Esos datos podrían ayudarnos a responder preguntas como:

- ¿Cuál fue el promedio general de una evaluación?
- ¿Qué cursos tuvieron mejores resultados?
- ¿En qué temas aparecieron más dificultades?
- ¿Hay estudiantes que necesitan acompañamiento adicional?
- ¿Los resultados mejoraron o empeoraron con el paso del tiempo?

Pero también debemos tener cuidado. Los datos pueden mostrar tendencias, diferencias o relaciones, pero no siempre explican por sí mismos las causas. Si un grupo obtuvo notas más bajas que otro, el dato nos muestra una diferencia, pero no nos dice automáticamente por qué ocurrió.

Trabajar con datos implica combinar herramientas técnicas con interpretación. Python y Pandas nos van a permitir calcular, filtrar, ordenar, agrupar y visualizar información. Pero la parte más importante seguirá siendo entender qué significan esos resultados en el contexto del problema que estamos analizando.

Datos, información y conocimiento

Cuando hablamos de análisis de datos, a veces usamos palabras como *dato*, *información* y *conocimiento* como si fueran lo mismo. Sin embargo, conviene distinguirlas.

Un **dato** es un valor registrado. Puede ser un número, una palabra, una fecha, una categoría o cualquier otro valor que haya sido observado o almacenado.

Por ejemplo:

Ese número, aislado, es un dato. Pero todavía no sabemos qué significa. Podría ser una nota, una edad, una cantidad de ventas, una temperatura, una calificación de una película o muchas otras cosas.

Cuando ese dato aparece dentro de un contexto, empieza a convertirse en **información**.

Por ejemplo:

La nota de Ana en Matemática fue 8.

Ahora el número tiene más sentido. Sabemos que representa una nota, que corresponde a una estudiante llamada Ana y que pertenece a la materia Matemática.

El **conocimiento** aparece cuando podemos interpretar esa información, relacionarla con otros datos y utilizarla para tomar decisiones o comprender mejor una situación.

Por ejemplo:

Ana obtuvo un 8 en Matemática. Su promedio anterior era 6,5 y en las últimas semanas aumentó la cantidad de horas de estudio. Su desempeño parece haber mejorado.

En este caso ya no estamos mirando un dato aislado. Estamos relacionando varios datos y construyendo una interpretación.

A lo largo de esta serie vamos a trabajar constantemente con este recorrido:

datos → información → conocimiento

Python y Pandas nos van a ayudar principalmente a organizar, transformar y analizar datos. Pero la interpretación final siempre requiere pensar qué representan esos datos, de dónde vienen, qué limitaciones tienen y qué conclusiones podemos sacar de ellos.

¿Qué significa analizar datos?

Analizar datos significa observar un conjunto de datos con el objetivo de comprender mejor una situación, responder preguntas o tomar decisiones fundamentadas.

No se trata solamente de aplicar fórmulas o ejecutar código. El análisis de datos suele ser un proceso que incluye varias etapas.

Primero necesitamos **comprender el contexto**. No es lo mismo analizar ventas de una tienda, resultados de una evaluación, datos de salud, registros climáticos o estadísticas deportivas. Cada dataset representa una parte de la realidad, y para interpretarlo correctamente necesitamos saber qué mide y cómo fue construido.

Después necesitamos **observar los datos**. Antes de calcular promedios o hacer gráficos, conviene revisar cuántas filas y columnas tiene la tabla, qué representa cada columna, qué tipos de valores aparecen, si hay datos faltantes, si existen errores evidentes o si algunas columnas necesitan ser transformadas.

Luego podemos comenzar a **hacer preguntas**. Algunas preguntas son simples, por ejemplo:

```
¿Cuántos registros hay?  
¿Cuál es el valor más alto?  
¿Cuál es el promedio?  
¿Qué categoría aparece más veces?
```

Otras preguntas requieren combinar varias operaciones:

```
¿Qué grupo tuvo mejores resultados?  
¿Cómo cambió una variable con el paso del tiempo?  
¿Existe alguna relación entre dos variables?  
¿Qué diferencias aparecen entre distintas categorías?
```

A medida que analizamos, también necesitamos **interpretar los resultados**. Un número o un gráfico no hablan por sí solos. Tenemos que leerlos, relacionarlos con el contexto y evaluar qué conclusiones son razonables.

Por último, muchas veces debemos **comunicar lo encontrado**. Esto puede hacerse mediante tablas, gráficos, dashboards, informes o explicaciones escritas. Un buen análisis no termina cuando obtenemos un resultado, sino cuando podemos explicarlo de manera clara.

Podríamos pensar el trabajo con datos como un recorrido:

```
comprender el contexto  
    ↓  
observar los datos  
    ↓  
limpiar y preparar  
    ↓  
analizar  
    ↓  
visualizar  
    ↓
```

interpretar
↓
comunicar

En esta serie vamos a recorrer cada una de esas etapas usando Python, Pandas y herramientas complementarias.

¿Qué es un dataset?

Un **dataset** es un conjunto organizado de datos. En muchos casos, especialmente cuando trabajamos con Pandas, un dataset se presenta como una tabla formada por filas y columnas.

Cada fila suele representar una observación, un registro o un caso. Cada columna representa una característica, atributo o variable que se registró para cada observación.

Por ejemplo, imaginemos una pequeña tabla con información sobre estudiantes:

nombre	edad	curso	horas_estudio	nota
Ana	15	3A	4	8
Bruno	16	3B	2	6
Carla	15	3A	6	9
Diego	17	4A	1	5
Elena	16	3B	5	8

En esta tabla, cada fila representa a un estudiante. Cada columna describe una característica de esos estudiantes.

Por ejemplo, la fila de Ana nos dice que Ana tiene 15 años, pertenece al curso 3A, estudió 4 horas y obtuvo una nota de 8.

La columna **nota**, en cambio, reúne las notas de todos los estudiantes. La columna **curso** indica a qué curso pertenece cada uno.

Esta forma de organizar los datos es muy frecuente porque permite comparar casos, resumir información, buscar patrones y hacer preguntas como:

¿Cuál fue la nota más alta?
¿Cuántos estudiantes hay en cada curso?
¿Cuál es el promedio de horas de estudio?
¿Los estudiantes que estudiaron más horas obtuvieron mejores notas?

Más adelante vamos a trabajar mucho con datasets en forma de tabla. Por eso es importante acostumbrarnos desde el principio a identificar qué representa cada fila, qué representa cada columna y qué significado tiene cada valor.

Filas, columnas, variables, observaciones y valores

Cuando observamos un dataset tabular, es importante reconocer sus partes principales.

En una tabla de datos, las **filas** suelen representar observaciones, registros o casos. En el ejemplo anterior, cada fila corresponde a un estudiante.

Las **columnas** representan variables o características. Cada columna describe algún aspecto de las observaciones. Por ejemplo, `edad`, `curso`, `horas_estudio` y `nota` son variables del dataset.

Los **valores** son los datos concretos que aparecen en cada celda de la tabla. Por ejemplo, en la fila de Ana y la columna `nota`, el valor es `8`.

Podemos pensarlo de esta manera:

Elemento	Qué representa	Ejemplo
Fila	Una observación, registro o caso	Un estudiante
Columna	Una variable o característica	La nota
Valor	Un dato concreto dentro de la tabla	8
Dataset	El conjunto completo de datos	La tabla de estudiantes

Esta distinción parece simple, pero es fundamental. Muchas operaciones que haremos con Pandas dependen de saber si estamos trabajando con filas, columnas o valores individuales.

Por ejemplo, una cosa es preguntar:

¿Cuál es la nota de Ana?

y otra diferente es preguntar:

¿Cuáles son todas las notas del dataset?

En el primer caso estamos buscando un valor particular. En el segundo, estamos mirando una columna completa.

También podríamos preguntar:

¿Qué información tenemos sobre Diego?

En ese caso estaríamos observando una fila completa.

A medida que avancemos, vamos a usar estas ideas constantemente. Seleccionar una columna, filtrar filas, modificar valores o crear nuevas variables son tareas básicas en el trabajo con datos.

Tipos de variables

No todas las columnas de un dataset representan el mismo tipo de información. Algunas contienen números, otras contienen categorías, otras fechas y otras textos.

Reconocer el tipo de variable es importante porque no todas las operaciones tienen sentido para todos los datos.

Por ejemplo, podemos calcular el promedio de una columna como `nota`, pero no tendría sentido calcular el promedio de una columna como `nombre`.

En una primera aproximación, podemos distinguir algunos tipos frecuentes de variables.

Variables numéricas

Las **variables numéricas** representan cantidades o medidas. Con ellas podemos realizar operaciones matemáticas como sumar, promediar, buscar máximos o mínimos.

Ejemplos:

```
edad
nota
precio
cantidad_vendida
temperatura
horas_estudio
```

Dentro de las variables numéricas, a veces distinguimos entre variables discretas y continuas.

- Una variable numérica **discreta** suele tomar valores contables, como la cantidad de estudiantes, la cantidad de productos vendidos o el número de hermanos.

- Una variable numérica **continua** puede tomar valores dentro de un rango, incluyendo decimales, como la altura, el peso, la temperatura o el tiempo.

Variables categóricas

Las **variables categóricas** representan grupos, clases o etiquetas. No se usan principalmente para hacer cuentas, sino para clasificar observaciones.

Ejemplos:

```
curso
provincia
tipo_producto
medio_de_pago
color
categoría
```

Con una variable categórica podemos contar cuántos registros hay en cada grupo, comparar promedios entre categorías o analizar distribuciones.

Variables ordinales

Las **variables ordinales** son categorías que tienen un orden natural.

Ejemplos:

```
nivel_bajo / nivel_medio / nivel_alto
primario / secundario / terciario / universitario
malo / regular / bueno / muy bueno / excelente
```

En estos casos, el orden importa. No es lo mismo decir “bajo, medio, alto” que tener categorías sin jerarquía, como “rojo, azul, verde”.

Variables temporales

Las **variables temporales** representan fechas, horas o momentos en el tiempo.

Ejemplos:

```
fecha_de_compra
fecha_de_nacimiento
hora_de_ingreso
mes
año
```

Estas variables permiten ordenar eventos cronológicamente, analizar cambios a lo largo del tiempo o agrupar información por días, meses o años.

Variables de texto

Las **variables de texto** contienen frases, comentarios, descripciones u otros fragmentos escritos.

Ejemplos:

```
comentario
descripcion_producto
respuesta_abierta
observaciones
```

Los textos pueden analizarse, pero suelen requerir técnicas diferentes a las que usamos con números o categorías simples.

En general, antes de analizar una columna conviene preguntarnos: ¿qué tipo de información contiene? Esa respuesta nos ayuda a decidir qué operaciones tienen sentido y cuáles no.

Reconocer variables en un ejemplo

Volvamos al pequeño dataset de estudiantes que vimos antes:

nombre	edad	curso	horas_estudio	nota
Ana	15	3A	4	8
Bruno	16	3B	2	6
Carla	15	3A	6	9
Diego	17	4A	1	5
Elena	16	3B	5	8

Podemos analizar qué tipo de variable representa cada columna:

Columna	Tipo de variable	Comentario
nombre	Texto / identificador	Permite reconocer a cada estudiante. No tiene sentido calcular un promedio con esta columna.
edad	Numérica	Representa una cantidad de años. Permite cálculos como promedio, mínimo o máximo.
curso	Categoría	Indica el grupo al que pertenece cada estudiante. Permite contar o comparar cursos.
horas_estudio	Numérica	Representa una cantidad. Permite hacer cálculos y comparaciones.

Columna	Tipo de variable	Comentario
nota	Numérica	Representa el resultado de una evaluación. Permite calcular promedio, máximo, mínimo, etc.

Esta clasificación no es un detalle menor. Si sabemos qué tipo de variable tenemos, podemos elegir mejor qué hacer con ella.

Por ejemplo, con la columna `curso` podríamos preguntar:

¿Cuántos estudiantes hay en cada curso?

Con la columna `nota`, en cambio, podríamos preguntar:

¿Cuál fue la nota promedio?

Y combinando ambas columnas, podríamos preguntar:

¿Cuál fue la nota promedio en cada curso?

Estas preguntas todavía son simples, pero muestran una idea central: muchas veces el análisis de datos consiste en relacionar variables para obtener información más útil.

Nuestro primer dataset en Python

Hasta ahora estuvimos observando una tabla escrita directamente en el capítulo. Ahora vamos a construir ese mismo dataset usando Python.

Para eso vamos a usar una biblioteca llamada **Pandas**, que será una de las herramientas principales de esta serie.

Pandas permite trabajar con datos organizados en forma de tabla. Más adelante vamos a estudiar sus herramientas con mucho más detalle. Por ahora, nos alcanza con saber que Pandas nos permite crear una estructura llamada `DataFrame`.

Un `DataFrame` es una tabla de datos: tiene filas, columnas y valores.

En el siguiente bloque vamos a crear un pequeño dataset de estudiantes. Para hacerlo, primero vamos a usar una estructura que ya conocemos de Python: un diccionario.

En ese diccionario, cada clave será el nombre de una columna, y cada lista asociada a esa clave contendrá los valores de esa columna.

Luego, mediante la instrucción:

```
df = pd.DataFrame(datos)
```

Pandas convertirá ese diccionario en un `DataFrame`, es decir, en una tabla preparada para ser inspeccionada y analizada.

Dicho de manera simple:

diccionario de Python → DataFrame de Pandas

```
# Importamos la biblioteca Pandas.
# Por convención, se suele importar con el alias pd.

import pandas as pd

# Creamos un diccionario de Python.
# Cada clave del diccionario será el nombre de una columna.
# Cada lista contiene los valores de esa columna.

datos = {
    "nombre": ["Ana", "Bruno", "Carla", "Diego", "Elena"],
    "edad": [15, 16, 15, 17, 16],
    "curso": ["3A", "3B", "3A", "4A", "3B"],
    "horas_estudio": [4, 2, 6, 1, 5],
    "nota": [8, 6, 9, 5, 8]
}

# Convertimos el diccionario en un DataFrame de Pandas.
# A partir de este momento, los datos quedan organizados como una tabla.

df = pd.DataFrame(datos)

# Mostramos el DataFrame.

df
```

índice	nombre	edad	curso	horas_estudio	nota
0	Ana	15	3A	4	8
1	Bruno	16	3B	2	6
2	Carla	15	3A	6	9
3	Diego	17	4A	1	5
4	Elena	16	3B	5	8

Leer la salida del DataFrame

Al ejecutar la celda anterior, Pandas muestra una tabla con los datos que cargamos.

La tabla tiene cinco filas y cinco columnas.

Las columnas son:

```
nombre  
edad  
curso  
horas_estudio  
nota
```

Cada fila representa a un estudiante. Por ejemplo, la primera fila corresponde a Ana:

nombre	edad	curso	horas_estudio	nota
Ana	15	3A	4	8

Esto significa que Ana tiene 15 años, pertenece al curso 3A, estudió 4 horas y obtuvo una nota de 8.

A la izquierda de la tabla aparece una columna adicional con números: 0, 1, 2, 3, 4.

Esa columna no forma parte de los datos originales. Es el **índice** del DataFrame. Pandas lo usa para identificar las filas.

En este caso, como no indicamos un índice especial, Pandas creó automáticamente un índice numérico que empieza en 0.

Esto es habitual en Python: muchas estructuras comienzan a contar desde cero.

Más adelante vamos a trabajar con el índice con más detalle. Por ahora, alcanza con reconocer que:

```
las columnas describen variables  
las filas representan observaciones  
el índice identifica las filas dentro del DataFrame
```

Hacer preguntas sobre un dataset

Un dataset no se analiza solamente mirándolo. Para empezar a trabajar con datos, necesitamos hacer preguntas.

Algunas preguntas pueden ser simples y directas:

¿Cuántos estudiantes hay en el dataset?
¿Cuál fue la nota más alta?
¿Cuál fue la nota más baja?
¿Cuántas horas estudió cada estudiante?

Otras preguntas relacionan dos o más variables:

¿Los estudiantes que estudiaron más horas obtuvieron mejores notas?
¿Hay diferencias entre los cursos?
¿Qué curso tiene el promedio de notas más alto?

También pueden aparecer preguntas que requieren más cuidado:

¿Estudiar más horas causa mejores notas?
¿Por qué un estudiante obtuvo una nota más baja?
¿Qué factores explican el rendimiento?

Estas últimas preguntas son más complejas. El dataset puede ayudarnos a explorar posibles relaciones, pero no siempre alcanza para demostrar causas.

Por ejemplo, si observamos que quienes estudiaron más horas obtuvieron mejores notas, podríamos decir que en este pequeño dataset parece existir una relación entre horas de estudio y nota. Pero eso no significa que hayamos demostrado una causa de manera definitiva.

Tal vez también influyeron otros factores que no están en la tabla, como el tema evaluado, la asistencia a clase, los conocimientos previos, el descanso, la dificultad de la prueba o el acompañamiento recibido.

Por eso, una parte importante del trabajo con datos consiste en distinguir entre:

lo que los datos muestran

y

lo que nosotros interpretamos a partir de esos datos

Esta diferencia nos acompañará durante todo el recorrido.

Algunas primeras respuestas usando Pandas

Ya tenemos nuestro primer DataFrame guardado en la variable `df`.

Ahora vamos a usar Pandas para responder algunas preguntas simples sobre el dataset.

No es necesario comprender todavía todos los detalles de cada instrucción. Lo importante, por ahora, es observar que podemos hacer preguntas sobre la tabla y obtener respuestas a partir de sus columnas.

Estas instrucciones nos permiten obtener respuestas rápidas:

- `len(df)` cuenta cuántas filas tiene el DataFrame.
- `df["nota"].max()` busca el valor más alto de la columna `nota`.
- `df["nota"].min()` busca el valor más bajo de la columna `nota`.
- `df["nota"].mean()` calcula el promedio de la columna `nota`.
- `df["curso"].value_counts()` cuenta cuántas veces aparece cada curso.

```
# ¿Cuántos estudiantes hay en el dataset?  
  
cantidad_estudiantes = len(df)  
  
cantidad_estudiantes
```

5

```
# ¿Cuál fue la nota más alta?  
  
nota_mas_alta = df["nota"].max()  
  
nota_mas_alta
```

9

```
# ¿Cuál fue la nota más baja?  
  
nota_mas_baja = df["nota"].min()  
  
nota_mas_baja
```

5

```
# ¿Cuál fue el promedio de notas?  
  
promedio_notas = df["nota"].mean()  
  
promedio_notas
```



```
np.float64(7.2)
```

Nota: expresiones como `np.float64(7.2)` o `dtype: int64` son información técnica de Pandas sobre el tipo de dato usado internamente. En términos prácticos, `np.float64(7.2)` se lee como `7.2`, y `dtype: int64` indica que los valores son enteros.

```
# ¿Cuántos estudiantes hay en cada curso?  
  
estudiantes_por_curso = df["curso"].value_counts()  
  
estudiantes_por_curso
```

curso	count
3A	2
3B	2
4A	1

`dtype: int64`

Más adelante vamos a estudiar estas operaciones con más detalle. Por ahora, lo importante es notar que Pandas nos permite trabajar con columnas completas y obtener información resumida a partir de ellas.

Interpretar las primeras respuestas

Los resultados obtenidos con Pandas nos permiten empezar a describir el dataset.

A partir de las operaciones anteriores, podríamos escribir algunas frases simples:

```
El dataset contiene 5 estudiantes.  
La nota más alta fue 9.  
La nota más baja fue 5.  
El promedio de notas fue 7,2.
```

También podemos interpretar el conteo por curso. Si la salida de `value_counts()` fue:

```
3A    2  
3B    2  
4A    1
```

entonces podemos decir:

Hay 2 estudiantes del curso 3A, 2 estudiantes del curso 3B y 1 estudiante del curso 4A.

Este paso es muy importante. Pandas nos ayuda a calcular, pero nosotros tenemos que interpretar.

Una salida como:

7.2

no dice demasiado por sí sola. Pero si sabemos que representa el promedio de notas de un grupo de estudiantes, entonces se convierte en información útil.

En análisis de datos, muchas veces vamos a realizar este recorrido:

código → resultado → interpretación

El código produce un resultado. Luego debemos leer ese resultado dentro del contexto del problema.

Por eso, cada vez que obtengamos una tabla, un número o un gráfico, conviene hacernos algunas preguntas:

¿Qué representa este resultado?
¿A qué columna o variable se refiere?
¿En qué contexto debe interpretarse?
¿Qué conclusión puedo sacar?
¿Qué conclusión no debería sacar todavía?

Esta costumbre ayuda a evitar interpretaciones apresuradas o conclusiones que los datos no sostienen.

Relacionar variables

Hasta ahora hicimos preguntas sobre columnas individuales. Por ejemplo, miramos la columna `nota` para conocer la nota más alta, la más baja y el promedio.

Pero muchas veces el análisis de datos se vuelve más interesante cuando relacionamos variables.

En nuestro dataset tenemos, entre otras, estas dos columnas:

`horas_estudio`
`nota`

Una pregunta posible sería:

```
¿Los estudiantes que estudiaron más horas obtuvieron mejores notas?
```

Para empezar a explorar esa pregunta, podemos mirar solamente esas dos columnas.

```
# Seleccionamos dos columnas del DataFrame:  
# las horas de estudio y la nota obtenida.  
  
df[["horas_estudio", "nota"]]
```

índice	horas_estudio	nota
0	4	8
1	2	6
2	6	9
3	1	5
4	5	8

Al observar esas dos columnas juntas, podemos comparar cada caso:

horas_estudio	nota
4	8
2	6
6	9
1	5
5	8

En este pequeño ejemplo parece haber cierta relación: quienes estudiaron más horas tienden a tener notas más altas.

Sin embargo, debemos ser prudentes. Este dataset tiene solamente cinco estudiantes. Además, no incluye muchas otras variables que podrían influir en el resultado.

Por eso, por ahora no diremos:

```
Estudiar más horas causa mejores notas.
```

Sería más cuidadoso decir:

En este pequeño dataset, los estudiantes con más horas de estudio parecen tener notas más altas.

Los límites de un dataset

Un dataset puede ayudarnos a comprender una situación, pero no contiene toda la realidad.

Nuestro ejemplo de estudiantes incluye algunas columnas:

```
nombre  
edad  
curso  
horas_estudio  
nota
```

Con esos datos podemos responder ciertas preguntas, como:

```
¿Cuántos estudiantes hay?  
¿Cuál fue la nota más alta?  
¿Cuál fue el promedio de notas?  
¿Qué estudiantes estudiaron más horas?
```

Pero hay muchas otras preguntas que no podemos responder con este dataset.

Por ejemplo:

```
¿Qué tema se evaluó?  
¿La evaluación fue fácil o difícil?  
¿Los estudiantes habían tenido clases de repaso?  
¿Todos estudiaron en las mismas condiciones?  
¿Hubo estudiantes ausentes?  
¿La nota corresponde a una prueba escrita, un trabajo práctico o una exposición oral?  
¿Las horas de estudio fueron registradas con precisión?
```

Estas preguntas podrían ser importantes para interpretar mejor los resultados, pero no aparecen en la tabla.

Esto nos muestra una idea fundamental:

Un dataset siempre es una representación parcial de una situación.

Cuando analizamos datos, debemos prestar atención no solo a lo que el dataset muestra, sino también a lo que no muestra.

Por eso, antes de sacar conclusiones, conviene preguntarnos:

¿Qué información tenemos?
¿Qué información falta?
¿Cómo fueron obtenidos estos datos?
¿Qué tan confiables son?
¿Qué conclusiones permite sostener este dataset?
¿Qué conclusiones serían exageradas?

Esta mirada crítica es parte esencial del trabajo con datos.

Los datasets reales suelen tener problemas

El dataset de estudiantes que usamos hasta ahora es muy pequeño y está bastante ordenado. Todas las columnas tienen nombres claros, no hay datos faltantes, no hay valores duplicados y los tipos de datos parecen adecuados.

Pero en situaciones reales, los datasets suelen ser más desordenados.

Podemos encontrar problemas como estos:

Problema	Ejemplo
Datos faltantes	Una fila no tiene cargada la edad o la nota.
Valores duplicados	El mismo estudiante aparece dos veces por error.
Nombres de columnas confusos	Una columna se llama <code>coll</code> o <code>dato_final</code> .
Espacios innecesarios	El valor " 3A " aparece con espacios antes o después.
Diferencias de escritura	Un curso aparece como 3A, 3a, 3 A o Tercero A.
Tipos de datos incorrectos	Una nota aparece como texto en lugar de número.
Valores imposibles	Una edad de -5 o una nota de 25 si la escala va de 1 a 10.
Fechas mal cargadas	Una misma fecha aparece como 2024-05-10, 10/05/2024 o mayo 10.

Estos problemas no son detalles menores. Si no los detectamos, pueden afectar los resultados del análisis.

Por ejemplo, si un estudiante aparece duplicado, su nota podría contarse dos veces. Si una columna numérica fue cargada como texto, tal vez no podamos calcular correctamente un promedio. Si una categoría está escrita de varias formas, Pandas podría interpretarla como si fueran categorías diferentes.

Por eso, una parte muy importante del trabajo con datos consiste en revisar la calidad del dataset antes de analizarlo.

Más adelante vamos a dedicar bastante tiempo a la limpieza de datos. Por ahora, nos alcanza con recordar esta idea:

Antes de confiar en un resultado, debemos revisar los datos que usamos para obtenerlo.

Del dataset a las herramientas de análisis

Hasta ahora trabajamos con un dataset muy pequeño. Esto nos permitió concentrarnos en las ideas principales sin distraernos con detalles técnicos.

Aprendimos que un dataset tabular está formado por filas y columnas, que cada fila suele representar una observación y que cada columna representa una variable. También vimos que no todas las variables son iguales: algunas son numéricas, otras categóricas, otras temporales y otras textuales.

Este punto es importante porque las herramientas que usamos dependen del tipo de dato que tenemos.

Por ejemplo:

Si queremos...	Necesitamos mirar...
Calcular un promedio	Una variable numérica
Contar cuántos casos hay en cada grupo	Una variable categórica
Analizar cambios a lo largo del tiempo	Una variable temporal
Buscar palabras o patrones escritos	Una variable de texto
Comparar grupos	Una variable categórica y una variable numérica
Explorar una posible relación	Dos o más variables

Pandas nos va a permitir realizar todas esas tareas de manera ordenada.

Pero antes de aplicar cualquier herramienta, siempre conviene hacernos algunas preguntas iniciales:

¿Qué representa este dataset?
¿Qué representa cada fila?
¿Qué representa cada columna?
¿Qué tipo de variable contiene cada columna?
¿Qué preguntas puedo responder con estos datos?
¿Qué preguntas no puedo responder todavía?

Algunas preguntas, algunas instrucciones

Cuando trabajamos con datos, muchas preguntas pueden traducirse a operaciones concretas.

Por ejemplo, si queremos saber cuántas filas y columnas tiene un dataset, Pandas nos permite usar `shape`.

Si queremos conocer los nombres de las columnas, podemos usar `columns`.

Si queremos ver una columna completa, podemos seleccionarla por su nombre.

No hace falta memorizar todavía todos estos comandos. Lo importante es empezar a relacionar una pregunta con una posible herramienta.

```
# ¿Cuántas filas y columnas tiene el DataFrame?
```

```
df.shape
```

```
(5, 5)
```

```
# ¿Cuáles son los nombres de las columnas?
```

```
df.columns
```

```
Index(['nombre', 'edad', 'curso', 'horas_estudio', 'nota'], dtype='object')
```

Nota: `dtype='object'` es una indicación de Pandas. En este caso, significa que los nombres de las columnas están almacenados como texto. No forma parte de los datos, sino de la descripción interna del índice.

```
# ¿Cuáles son todas las notas registradas?
```

```
df["nota"]
```

índice	nota
0	8
1	6
2	9
3	5
4	8

`dtype: int64`

```
# ¿Cuáles son todos los cursos registrados?  
  
df["curso"]
```

índice	curso
0	3A
1	3B
2	3A
3	4A
4	3B

`dtype: object`

Estas instrucciones nos permiten empezar a inspeccionar el DataFrame.

- `df.shape` devuelve la cantidad de filas y columnas.
- `df.columns` muestra los nombres de las columnas.
- `df["nota"]` selecciona la columna `nota`.
- `df["curso"]` selecciona la columna `curso`.

Aunque son operaciones simples, ya muestran una idea muy importante: Pandas nos permite trabajar con la tabla completa, con columnas individuales o con partes específicas del dataset.

A medida que avancemos, vamos a aprender a seleccionar filas, combinar condiciones, limpiar datos, crear nuevas columnas, agrupar información y construir visualizaciones.

Resumen del capítulo

En este primer capítulo comenzamos a construir una base conceptual para el trabajo con datos.

Vimos que un dato aislado puede tener poco significado si no conocemos su contexto. También diferenciamos entre datos, información y conocimiento, y observamos que analizar datos no consiste solamente en aplicar código, sino en interpretar resultados dentro de una situación concreta.

Luego presentamos la idea de dataset. Trabajamos con un pequeño ejemplo de estudiantes y usamos esa tabla para reconocer sus partes principales: filas, columnas, observaciones, variables y valores.

También distinguimos algunos tipos frecuentes de variables: numéricas, categóricas, ordinales, temporales y textuales. Esta clasificación será importante porque las operaciones que podemos realizar dependen del tipo de dato que contiene cada columna.

Finalmente, construimos nuestro primer `DataFrame` con Pandas. No estudiamos Pandas en profundidad todavía, pero sí vimos que nos permite representar datos en forma de tabla y responder algunas preguntas simples, como contar registros, seleccionar columnas, calcular valores máximos, mínimos y promedios.

Las ideas principales de este capítulo fueron:

Un dataset organiza datos en forma de filas y columnas.
Cada fila suele representar una observación.
Cada columna representa una variable.
Cada valor aparece en la intersección entre una fila y una columna.
Antes de analizar datos, necesitamos comprender qué representan.
Pandas nos permite trabajar con datos tabulares dentro de Python.

Más adelante vamos a profundizar en Pandas y aprenderemos a inspeccionar datasets de manera más sistemática.

Próximo paso

Ahora que ya tenemos una primera idea de qué es un dataset y cómo se representa en Pandas, el siguiente paso será conocer mejor la estructura de un `DataFrame`.

Vamos a aprender a revisar sus filas, columnas, índice, tipos de datos, dimensiones y primeras estadísticas generales. Esa inspección inicial es fundamental antes de limpiar, transformar o analizar cualquier conjunto de datos.

Capítulo 2 — Primeras inspecciones de un DataFrame

Breve repaso

En el capítulo anterior comenzamos a construir una base conceptual para el trabajo con datos.

Vimos que un dataset es un conjunto organizado de datos y que, muchas veces, se presenta en forma de tabla. También reconocimos sus partes principales: filas, columnas, observaciones, variables y valores.

Además, hicimos un primer contacto con Pandas y construimos un `DataFrame` sencillo. Ese primer ejemplo nos permitió ver que Pandas puede representar datos tabulares dentro de Python y ayudarnos a responder preguntas simples.

En este capítulo vamos a dar un paso más.

Ya no nos vamos a conformar con “mirar” una tabla. Vamos a aprender a inspeccionar un `DataFrame` de manera más ordenada, revisando sus primeras filas, sus últimas filas, sus dimensiones, sus columnas, su índice, sus tipos de datos y algunas estadísticas iniciales.

Antes de limpiar, transformar, resumir o visualizar datos, necesitamos saber con qué estamos trabajando. Por eso, la inspección inicial es una etapa fundamental en cualquier análisis de datos.

Al finalizar este capítulo deberías poder:

- Mostrar las primeras y últimas filas de un `DataFrame`.
- Conocer cuántas filas y columnas tiene un dataset.
- Revisar los nombres de las columnas.
- Reconocer el índice de un `DataFrame`.
- Consultar los tipos de datos de cada columna.
- Obtener información general con `info()`.
- Calcular estadísticas iniciales con `describe()`.
- Formular primeras observaciones sobre la estructura de un dataset.

Por qué inspeccionar un DataFrame

Cuando recibimos un dataset, puede ser tentador empezar directamente a calcular promedios, crear gráficos o buscar conclusiones.

Sin embargo, antes de analizar los datos necesitamos conocer su estructura.

Inspeccionar un `DataFrame` significa hacer una primera revisión para responder preguntas como estas:

```
¿Cuántas filas tiene?  
¿Cuántas columnas tiene?  
¿Qué representa cada columna?  
¿Cómo se llaman las columnas?  
¿Qué tipos de datos contiene?  
¿Hay columnas numéricas?  
¿Hay columnas categóricas?  
¿Aparecen valores faltantes?  
¿Los datos parecen estar cargados correctamente?
```

Esta revisión inicial nos ayuda a evitar errores.

Por ejemplo, si una columna de precios fue cargada como texto, Pandas podría no permitirnos calcular correctamente un promedio. Si una columna tiene muchos valores faltantes, tal vez necesitemos tratar ese problema antes de analizarla. Si los nombres de las columnas son confusos, puede ser difícil escribir código claro.

La inspección inicial no resuelve todos los problemas del dataset, pero nos permite construir un primer diagnóstico.

Podemos pensarla como una lectura exploratoria:

```
antes de transformar los datos,  
primero necesitamos conocerlos.
```

En este capítulo vamos a aprender algunas herramientas básicas de Pandas para realizar esa primera revisión.

Dataset de trabajo

Para practicar las primeras herramientas de inspección, vamos a crear un dataset pequeño relacionado con ventas de una librería.

Cada fila representará la venta de un producto. Cada columna describirá una característica de esa venta: el producto vendido, la categoría, el precio unitario, la cantidad vendida, la sucursal y la fecha.

Este dataset todavía será simple, pero nos permitirá revisar varios elementos importantes de un `DataFrame`: filas, columnas, índice, tipos de datos y estadísticas iniciales.

Al igual que en el capítulo anterior, partiremos de un diccionario de Python y lo convertiremos en un `DataFrame` usando `pd.DataFrame(datos)`.

```
import pandas as pd

datos = {
    "producto": [
        "Cuaderno", "Lapicera", "Mochila", "Regla", "Cartuchera",
        "Cuaderno", "Marcadores", "Carpeta", "Goma", "Lapiz"
    ],
    "categoria": [
        "Libreria", "Libreria", "Accesorios", "Libreria", "Accesorios",
        "Libreria", "Libreria", "Libreria", "Libreria", "Libreria"
    ],
    "precio": [
        1200, 350, 8500, 500, 3200,
        1250, 1800, 950, 300, 250
    ],
    "cantidad_vendida": [
        15, 40, 5, 25, 8,
        12, 10, 18, 30, 50
    ],
    "sucursal": [
        "Centro", "Centro", "Norte", "Sur", "Norte",
        "Sur", "Centro", "Norte", "Sur", "Centro"
    ],
    "fecha": [
        "2024-03-01", "2024-03-01", "2024-03-02", "2024-03-02", "2024-03-03",
        "2024-03-03", "2024-03-04", "2024-03-04", "2024-03-05", "2024-03-05"
    ]
}

df = pd.DataFrame(datos)

df
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	fecha
0	Cuaderno	Libreria	1200	15	Centro	2024-03-01
1	Lapicera	Libreria	350	40	Centro	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	2024-03-02
3	Regla	Libreria	500	25	Sur	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	2024-03-03
5	Cuaderno	Libreria	1250	12	Sur	2024-03-03

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
6	Marcadores	Librería	1800	10	Centro	2024-03-04
7	Carpeta	Librería	950	18	Norte	2024-03-04
8	Goma	Librería	300	30	Sur	2024-03-05
9	Lapiz	Librería	250	50	Centro	2024-03-05

Ver las primeras filas con `head()`

Cuando un dataset tiene muchas filas, no siempre conviene mostrarlo completo. En esos casos, una de las primeras acciones que solemos realizar es mirar solo las primeras filas.

Para eso Pandas ofrece el método `head()`.

Este método nos permite obtener una vista rápida del comienzo del `DataFrame`.

```
df.head()
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
0	Cuaderno	Librería	1200	15	Centro	2024-03-01
1	Lapicera	Librería	350	40	Centro	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	2024-03-02
3	Regla	Librería	500	25	Sur	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	2024-03-03

Por defecto, `head()` muestra las primeras cinco filas del `DataFrame`.

Esto resulta útil para revisar rápidamente:

- si el dataset se cargó correctamente;
- si las columnas aparecen con los nombres esperados;
- si los valores tienen una forma razonable;
- si la tabla parece representar lo que esperábamos.

En nuestro caso, al usar `df.head()` vemos las primeras ventas registradas en el dataset.

También podemos indicar cuántas filas queremos ver. Por ejemplo, si queremos observar solo las primeras tres filas, podemos escribir:

```
df.head(3)
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
0	Cuaderno	Librería	1200	15	Centro	2024-03-01
1	Lapicera	Librería	350	40	Centro	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	2024-03-02

El número entre paréntesis indica cuántas filas queremos mostrar.

Entonces:

```
df.head(3)
```

significa:

```
mostrará las primeras 3 filas del DataFrame df
```

Esta instrucción no modifica el dataset. Solamente muestra una parte de la tabla para que podamos inspeccionarla.

Ver las últimas filas con `tail()`

Así como `head()` permite ver las primeras filas de un `DataFrame`, Pandas también ofrece el método `tail()`, que permite ver las últimas filas.

Esto es útil porque, a veces, los problemas de un dataset no aparecen al principio. Puede ocurrir que las primeras filas estén bien cargadas, pero que al final haya datos incompletos, totales agregados, notas aclaratorias o registros con otro formato.

Para revisar el final del `DataFrame`, usamos:

```
df.tail()
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
5	Cuaderno	Librería	1250	12	Sur	2024-03-03
6	Marcadores	Librería	1800	10	Centro	2024-03-04
7	Carpeta	Librería	950	18	Norte	2024-03-04
8	Goma	Librería	300	30	Sur	2024-03-05
9	Lapiz	Librería	250	50	Centro	2024-03-05

Por defecto, `tail()` muestra las últimas cinco filas.

En nuestro dataset, esto nos permite observar las ventas registradas hacia el final de la tabla.

También podemos indicar cuántas filas queremos ver. Por ejemplo:

```
df.tail(3)
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
7	Carpeta	Librería	950	18	Norte	2024-03-04
8	Goma	Librería	300	30	Sur	2024-03-05
9	Lapiz	Librería	250	50	Centro	2024-03-05

En este caso, Pandas muestra las últimas tres filas del `DataFrame`.

Podemos pensar estas dos herramientas como una primera mirada rápida:

```
head() → muestra el comienzo del DataFrame  
tail() → muestra el final del DataFrame
```

Ninguna de estas instrucciones modifica los datos. Solo nos ayudan a inspeccionarlos.

Conocer las dimensiones con `shape`

Después de mirar las primeras y las últimas filas, una pregunta muy común es:

```
¿Cuántas filas y cuántas columnas tiene este dataset?
```

En Pandas podemos responder esa pregunta usando `shape`.

A diferencia de `head()` y `tail()`, `shape` no se escribe con paréntesis porque no es un método, sino un atributo del `DataFrame`.

```
df.shape
```

La salida de `df.shape` tiene dos valores:

```
(filas, columnas)
```

En nuestro caso, el resultado es:

```
(10, 6)
```

Esto significa que el `DataFrame` tiene 10 filas y 6 columnas.

También podemos guardar esos dos valores en variables separadas:

```
cantidad_filas, cantidad_columnas = df.shape  
  
cantidad_filas  
cantidad_columnas
```

Esto puede ser útil cuando queremos usar esos valores más adelante en un análisis o en un mensaje.

Por ejemplo:

```
print(f"El dataset tiene {cantidad_filas} filas y {cantidad_columnas} columnas.")
```

Conocer las dimensiones del dataset es una de las primeras tareas de inspección.

Un dataset con 10 filas puede revisarse casi completo a simple vista. Pero uno con 10.000, 100.000 o varios millones de filas necesita herramientas de inspección, resumen y filtrado.

Por eso, `shape` nos da una primera idea del tamaño del problema con el que estamos trabajando.

Revisar los nombres de las columnas

Después de conocer las dimensiones del `DataFrame`, conviene revisar cómo se llaman sus columnas.

Los nombres de columnas son importantes porque los usamos para seleccionar datos, hacer cálculos, aplicar filtros y construir nuevas variables.

En Pandas podemos consultar los nombres de las columnas usando el atributo `columns`.

```
df.columns
```

```
Index(['producto', 'categoria', 'precio', 'cantidad_vendida', 'sucursal',  
      'fecha'],  
      dtype='object')
```


La salida muestra los nombres de todas las columnas del `DataFrame`.

En nuestro dataset aparecen estas columnas:

```
producto  
categoria  
precio  
cantidad_vendida  
sucursal  
fecha
```

Estos nombres nos dicen qué información contiene cada variable.

También podemos convertir los nombres de columnas en una lista de Python, si queremos verlos de una manera más simple:

```
list(df.columns)
```

```
['producto', 'categoria', 'precio', 'cantidad_vendida', 'sucursal', 'fecha']
```

Esto devuelve una lista con los nombres de las columnas.

Revisar los nombres de columnas es una tarea sencilla, pero muy importante. En datasets reales podemos encontrar problemas como:

```
nombres demasiado largos  
nombres poco claros  
espacios al principio o al final  
mayúsculas y minúsculas mezcladas  
caracteres especiales  
columnas sin nombre
```

Por ejemplo, una columna llamada `precio` es clara. En cambio, una columna llamada `columna_3`, `dato1` o `valor_final_2` puede dificultar la interpretación.

Más adelante vamos a aprender a limpiar y modificar nombres de columnas. Por ahora, nos interesa incorporar esta costumbre: antes de analizar un dataset, conviene revisar cómo están nombradas sus variables.

Reconocer el índice del DataFrame

Además de filas y columnas, un `DataFrame` tiene un elemento muy importante: el **índice**.

El índice sirve para identificar las filas. En muchos casos, Pandas crea automáticamente un índice numérico que empieza en 0.

Cuando mostramos nuestro dataset, vemos a la izquierda una serie de números:

```
0
1
2
3
4
...
```

Esos números no pertenecen a ninguna de las columnas originales del dataset. Son el índice que Pandas asignó a cada fila.

Podemos consultar el índice usando el atributo `index`.

```
df.index
```

```
RangeIndex(start=0, stop=10, step=1)
```

La salida indica cómo está construido el índice.

En este caso, Pandas nos muestra algo parecido a:

```
RangeIndex(start=0, stop=10, step=1)
```

Esto significa que el índice:

```
empieza en 0
termina antes de 10
avanza de 1 en 1
```

Como nuestro dataset tiene 10 filas, los índices van desde 0 hasta 9.

Es importante recordar que el índice no siempre coincide con una columna del dataset. En este ejemplo, el índice fue creado automáticamente por Pandas.

Más adelante vamos a trabajar con el índice con más detalle. Veremos cómo cambiarlo, cómo resetearlo y cómo usarlo para seleccionar filas. Por ahora, nos alcanza con reconocer que el índice identifica las filas del `DataFrame`.

Revisar los tipos de datos

Además de conocer las filas, las columnas y el índice, también necesitamos revisar qué tipo de dato tiene cada columna.

En Pandas, cada columna tiene un tipo de dato asociado. Ese tipo de dato le indica a Pandas cómo debe interpretar los valores de esa columna.

Por ejemplo, una columna puede contener:

```
números enteros
números decimales
textos
fechas
valores booleanos
```

Podemos consultar los tipos de datos usando `dtypes`.

```
df.dtypes
```

índice	0
producto	object
categoria	object
precio	int64
cantidad_vendida	int64
sucursal	object
fecha	object

```
dtype: object
```

La salida muestra cada columna junto con el tipo de dato que Pandas le asignó.

En nuestro dataset podríamos ver algo similar a esto:

```
producto      object
categoria     object
precio        int64
cantidad_vendida  int64
sucursal      object
fecha         object
dtype: object
```

Las columnas `precio` y `cantidad_vendida` aparecen como `int64`, porque contienen números enteros.

En cambio, las columnas `producto`, `categoria`, `sucursal` y `fecha` aparecen como `object`.

En Pandas, el tipo `object` suele usarse para columnas que contienen texto. Por eso es esperable que `producto`, `categoria` y `sucursal` aparezcan de esa manera.

Pero hay un detalle interesante: la columna `fecha` también aparece como `object`.

Eso ocurre porque, por ahora, las fechas fueron cargadas como texto. Pandas no las está interpretando todavía como fechas reales.

Esto no es un error grave en este momento, pero es una observación importante. Si más adelante queremos ordenar, filtrar o agrupar por fechas, probablemente necesitemos convertir esa columna a un tipo de dato temporal.

Este es un buen ejemplo de por qué conviene revisar los tipos de datos al inicio. A simple vista, una columna puede parecer una fecha, un número o una categoría, pero Pandas puede haberla interpretado de otra manera.

Obtener información general con `info()`

Pandas ofrece un método muy útil para obtener un resumen general del `DataFrame`: `info()`.

Este método muestra información sobre:

```
cantidad de filas
cantidad de columnas
nombre de cada columna
cantidad de valores no nulos
tipo de dato de cada columna
uso aproximado de memoria
```

Lo usamos así:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 6 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   producto              10 non-null    object
 1   categoria              10 non-null    object
 2   precio                 10 non-null    int64
 3   cantidad_vendida      10 non-null    int64
```

```
4  sucursal          10 non-null    object
5  fecha             10 non-null    object
dtypes: int64(2), object(4)
memory usage: 612.0+ bytes
```

La salida de `info()` puede parecer un poco más técnica al principio, pero contiene información muy importante.

En nuestro dataset, deberíamos observar que hay 10 entradas, es decir, 10 filas. También deberíamos ver las 6 columnas:

```
producto
categoria
precio
cantidad_vendida
sucursal
fecha
```

La columna `Non-Null Count` indica cuántos valores no nulos tiene cada columna.

Si una columna tiene 10 valores no nulos y el dataset tiene 10 filas, significa que esa columna no tiene valores faltantes.

Por ejemplo:

```
10 non-null
```

significa que hay 10 valores cargados en esa columna.

Si en una columna apareciera algo como:

```
8 non-null
```

eso indicaría que, de las 10 filas, solo 8 tienen un valor cargado en esa columna. Las otras 2 filas tendrían valores faltantes.

También vemos el tipo de dato de cada columna, igual que cuando usamos `dtypes`.

Por eso, `info()` suele ser una de las primeras instrucciones que usamos al recibir un dataset nuevo. Nos permite obtener rápidamente una primera imagen de su estructura y de posibles problemas.

Obtener estadísticas iniciales con `describe()`

Además de revisar la estructura del `DataFrame`, muchas veces queremos obtener un primer resumen estadístico de sus columnas numéricas.

Para eso podemos usar el método `describe()`.

Este método calcula automáticamente algunas medidas descriptivas básicas.

```
df.describe()
```

índice	precio	cantidad_vendida
count	10.000000	10.00000
mean	1830.000000	21.30000
std	2509.338115	14.79527
min	250.000000	5.00000
25%	387.500000	10.50000
50%	1075.000000	16.50000
75%	1662.500000	28.75000
max	8500.000000	50.00000

En nuestro dataset, `describe()` analiza las columnas numéricas:

```
precio  
cantidad_vendida
```

La salida incluye varias filas:

Medida	Significado
count	Cantidad de valores no nulos.
mean	Promedio.
std	Desvío estándar.
min	Valor mínimo.
25%	Primer cuartil.
50%	Mediana.
75%	Tercer cuartil.
max	Valor máximo.

No necesitamos interpretar todas estas medidas en profundidad todavía. Más adelante vamos a estudiar estadística descriptiva con más detalle.

Por ahora, lo importante es comprender que `describe()` nos ofrece una primera mirada general sobre las columnas numéricas.

Por ejemplo, podemos observar cuál es el precio mínimo, cuál es el precio máximo, cuál es el promedio de precios y cuántos valores hay cargados.

También podemos usar `describe()` sobre una sola columna:

```
df["precio"].describe()
```

índice	precio
count	10.000000
mean	1830.000000
std	2509.338115
min	250.000000
25%	387.500000
50%	1075.000000
75%	1662.500000
max	8500.000000

`dtype: float64`

En este caso, Pandas calcula el resumen estadístico solamente para la columna `precio`.

Esta herramienta es útil para empezar a detectar valores llamativos. Por ejemplo, si el precio mínimo fuera negativo o si el precio máximo fuera exageradamente alto, eso podría indicar un posible problema en los datos.

`describe()` no reemplaza una revisión cuidadosa, pero ayuda a construir una primera impresión del dataset.

Describir columnas no numéricas

Cuando usamos `df.describe()`, Pandas resume automáticamente las columnas numéricas.

Pero un dataset también puede tener columnas no numéricas, como `producto`, `categoría` o `sucursal`.

Si queremos obtener un resumen de todas las columnas, podemos usar `include="all"`.

```
df.describe(include="all")
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
count	10	10	10.000000	10.000000	10	10
unique	9	2	NaN	NaN	3	5
top	Cuaderno	Librería	NaN	NaN	Centro	2024-03-01
freq	2	8	NaN	NaN	4	2
mean	NaN	NaN	1830.000000	21.300000	NaN	NaN
std	NaN	NaN	2509.338115	14.79527	NaN	NaN
min	NaN	NaN	250.000000	5.000000	NaN	NaN
25%	NaN	NaN	387.500000	10.500000	NaN	NaN
50%	NaN	NaN	1075.000000	16.500000	NaN	NaN
75%	NaN	NaN	1662.500000	28.750000	NaN	NaN
max	NaN	NaN	8500.000000	50.000000	NaN	NaN

La salida ahora incluye también columnas de texto.

Para las columnas no numéricas, Pandas muestra otras medidas:

Medida	Significado
<code>count</code>	Cantidad de valores no nulos.
<code>unique</code>	Cantidad de valores distintos.
<code>top</code>	Valor que aparece con mayor frecuencia.
<code>freq</code>	Frecuencia del valor más repetido.

Por ejemplo, en la columna `sucursal`, `unique` nos indica cuántas sucursales diferentes aparecen en el dataset.

En la columna `producto`, `top` indica cuál es el producto que aparece más veces, y `freq` indica cuántas veces aparece.

Este tipo de resumen puede ser útil para comenzar a entender columnas categóricas o de texto.

Sin embargo, también debemos leerlo con cuidado. En columnas como `producto`, que pueden tener muchos valores distintos, a veces conviene usar otras herramientas, como `value_counts()`, para ver con más detalle la frecuencia de cada valor.

Y podemos agregar una primera comparación con `value_counts()`:

```
df["sucursal"].value_counts()
```

sucursal	count
Centro	4
Norte	3
Sur	3

`dtype: int64`

Esta instrucción cuenta cuántas veces aparece cada sucursal.

A diferencia de `describe(include="all")`, que nos da un resumen general, `value_counts()` nos muestra directamente la frecuencia de cada categoría.

En este caso, nos permite responder una pregunta simple:

¿Cuántas ventas se registraron en cada sucursal?

Construir una primera lectura del dataset

Hasta ahora usamos varias herramientas de inspección:

```
head()
tail()
shape
columns
index
dtypes
info()
describe()
value_counts()
```

Cada una responde una pregunta distinta sobre el `DataFrame`.

Podemos organizar esas herramientas de esta manera:

Pregunta	Herramienta posible
¿Cómo son las primeras filas?	<code>head()</code>
¿Cómo son las últimas filas?	<code>tail()</code>
¿Cuántas filas y columnas tiene?	<code>shape</code>
¿Cómo se llaman las columnas?	<code>columns</code>
¿Cómo está identificado cada registro?	<code>index</code>
¿Qué tipo de dato tiene cada columna?	<code>dtypes</code>
¿Hay valores faltantes visibles en el resumen?	<code>info()</code>
¿Qué resumen estadístico tienen las columnas numéricas?	<code>describe()</code>
¿Cuántas veces aparece cada categoría?	<code>value_counts()</code>

Estas herramientas no reemplazan el análisis profundo, pero nos permiten construir una primera lectura del dataset.

Por ejemplo, a partir de lo que vimos podemos decir:

El dataset tiene 10 filas y 6 columnas.
Cada fila representa una venta.
Las columnas incluyen producto, categoria, precio, cantidad vendida, sucursal y fecha.
Las columnas precio y cantidad_vendida son numéricas.
La columna fecha parece una fecha, pero Pandas la interpreta todavía como texto.
No se observan valores faltantes en este dataset.

Este tipo de lectura inicial es muy importante. Antes de limpiar, transformar, agrupar o visualizar datos, necesitamos saber qué tenemos delante.

Una buena práctica es no empezar directamente por los cálculos, sino dedicar unos minutos a revisar la estructura del dataset.

Resumen del capítulo

En este capítulo aprendimos a realizar una primera inspección de un `DataFrame`.

La inspección inicial es una etapa fundamental del trabajo con datos. Antes de limpiar, transformar, analizar o visualizar un dataset, necesitamos conocer su estructura general.

Para eso trabajamos con varias herramientas básicas de Pandas:

```
head()
tail()
shape
```

```
columns
index
dtypes
info()
describe()
value_counts()
```

Vimos que `head()` y `tail()` permiten observar el comienzo y el final del `DataFrame`, sin necesidad de mostrarlo completo.

También usamos `shape` para conocer la cantidad de filas y columnas, `columns` para revisar los nombres de las variables, e `index` para reconocer cómo Pandas identifica las filas.

Luego revisamos los tipos de datos con `dtypes`. Esta inspección nos permitió observar que algunas columnas fueron interpretadas como números enteros, mientras que otras fueron interpretadas como texto. En particular, vimos que la columna `fecha` parecía contener fechas, pero Pandas todavía la interpretaba como `object`.

Después usamos `info()` para obtener una vista general del dataset, incluyendo cantidad de columnas, cantidad de valores no nulos y tipos de datos.

Finalmente, usamos `describe()` para obtener estadísticas iniciales de las columnas numéricas, y `describe(include="all")` junto con `value_counts()` para observar también columnas categóricas o de texto.

La idea principal de este capítulo fue:

Antes de analizar un dataset, primero necesitamos inspeccionarlo.

Una buena inspección inicial nos ayuda a entender qué datos tenemos, detectar posibles problemas y decidir qué pasos conviene seguir.

Próximo paso

Ahora que sabemos inspeccionar la estructura general de un `DataFrame`, estamos en condiciones de empezar a trabajar con partes específicas del dataset.

Más adelante vamos a aprender a seleccionar columnas, seleccionar filas y construir subconjuntos de datos. Esto nos permitirá responder preguntas más precisas y preparar el camino para filtros, limpieza y análisis más detallados.

Capítulo 03 — Selección de columnas en un DataFrame

Breve repaso

En el capítulo anterior aprendimos a realizar una primera inspección de un `DataFrame`.

Vimos cómo observar sus primeras y últimas filas, revisar sus dimensiones, consultar los nombres de las columnas, reconocer el índice, mirar los tipos de datos y obtener resúmenes generales con herramientas como `info()` y `describe()`.

Ese primer recorrido nos permitió construir una idea importante: antes de analizar un dataset, necesitamos conocer su estructura.

En este capítulo vamos a dar un paso más. Vamos a aprender a seleccionar columnas de un `DataFrame`.

Seleccionar columnas es una operación básica, pero fundamental. Muchas veces no necesitamos trabajar con toda la tabla, sino con una o varias variables específicas. Por ejemplo, podemos querer analizar solamente los precios, revisar las fechas, observar las sucursales o comparar dos columnas entre sí.

A lo largo del capítulo vamos a trabajar con un dataset sencillo de ventas y vamos a ver cómo seleccionar una columna, cómo seleccionar varias columnas y qué diferencia hay entre obtener una `Series` y obtener otro `DataFrame`.

Al finalizar este capítulo deberías poder:

- Seleccionar una columna de un `DataFrame`.
- Seleccionar varias columnas al mismo tiempo.
- Reconocer cuándo Pandas devuelve una `Series`.
- Reconocer cuándo Pandas devuelve un `DataFrame`.
- Guardar una selección de columnas en una nueva variable.
- Evitar algunos errores frecuentes al escribir nombres de columnas.

Dataset de trabajo

En este capítulo vamos a seguir trabajando con un dataset pequeño de ventas de una librería.

Cada fila representa una venta registrada. Cada columna describe una característica de esa venta: el producto, la categoría, el precio unitario, la cantidad vendida, la sucursal y la fecha.

Usar un dataset ya conocido nos permite concentrarnos en el tema nuevo: la selección de columnas.

```
import pandas as pd

datos = {
    "producto": [
        "Cuaderno", "Lapicera", "Mochila", "Regla", "Cartuchera",
        "Cuaderno", "Marcadores", "Carpeta", "Goma", "Lapiz"
    ],
    "categoria": [
        "Libreria", "Libreria", "Accesorios", "Libreria", "Accesorios",
        "Libreria", "Libreria", "Libreria", "Libreria", "Libreria"
    ],
    "precio": [
        1200, 350, 8500, 500, 3200,
        1250, 1800, 950, 300, 250
    ],
    "cantidad_vendida": [
        15, 40, 5, 25, 8,
        12, 10, 18, 30, 50
    ],
    "sucursal": [
        "Centro", "Centro", "Norte", "Sur", "Norte",
        "Sur", "Centro", "Norte", "Sur", "Centro"
    ],
    "fecha": [
        "2024-03-01", "2024-03-01", "2024-03-02", "2024-03-02", "2024-03-03",
        "2024-03-03", "2024-03-04", "2024-03-04", "2024-03-05", "2024-03-05"
    ]
}

df = pd.DataFrame(datos)

df
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	fecha
0	Cuaderno	Libreria	1200	15	Centro	2024-03-01
1	Lapicera	Libreria	350	40	Centro	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	2024-03-02
3	Regla	Libreria	500	25	Sur	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	2024-03-03
5	Cuaderno	Libreria	1250	12	Sur	2024-03-03

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
6	Marcadores	Librería	1800	10	Centro	2024-03-04
7	Carpeta	Librería	950	18	Norte	2024-03-04
8	Goma	Librería	300	30	Sur	2024-03-05
9	Lapiz	Librería	250	50	Centro	2024-03-05

Antes de avanzar, recordemos brevemente qué contiene este `DataFrame`.

Tenemos 10 filas y 6 columnas. Las columnas son:

```
producto
categoria
precio
cantidad_vendida
sucursal
fecha
```

A partir de ahora vamos a aprender a quedarnos solamente con algunas de esas columnas, según la pregunta que queramos responder.

Qué significa seleccionar columnas

Un `DataFrame` puede tener muchas columnas. Algunas pueden ser necesarias para una pregunta determinada y otras no.

Seleccionar columnas significa quedarnos solamente con una parte vertical de la tabla.

Por ejemplo, si queremos analizar los precios, no necesitamos mirar todas las columnas del dataset. Podemos seleccionar solamente la columna `precio`.

Si queremos comparar productos y precios, podemos seleccionar dos columnas:

```
producto
precio
```

Y si queremos estudiar las ventas por sucursal, tal vez nos interesen estas columnas:

```
sucursal
producto
cantidad_vendida
```

Seleccionar columnas no elimina información del `DataFrame` original. Solamente nos permite observar o guardar aparte un subconjunto con las columnas que queremos analizar.

Esta operación es muy frecuente porque, en datasets reales, podemos tener decenas o cientos de columnas. En esos casos, seleccionar solo las variables relevantes hace que el análisis sea más claro y manejable.

Seleccionar una columna

Para seleccionar una columna de un `DataFrame`, escribimos el nombre del `DataFrame`, seguido del nombre de la columna entre corchetes.

Por ejemplo, si queremos seleccionar la columna `precio`, escribimos:

```
df["precio"]
```

índice	precio
0	1200
1	350
2	8500
3	500
4	3200
5	1250
6	1800
7	950
8	300
9	250

```
dtype: int64
```

La salida muestra todos los valores de la columna `precio`.

En este caso, Pandas nos devuelve una estructura de una sola dimensión llamada `Series`.

Una `Series` puede pensarse como una columna individual de datos. Tiene valores y también conserva el índice del `DataFrame` original.

Si observamos la salida anterior, vemos que a la izquierda aparece el índice de cada fila y a la derecha aparecen los valores de la columna `precio`.

Al final, Pandas indica el nombre de la `Series` y su tipo de dato:

```
Name: precio, dtype: int64
```

Esto nos dice que la columna se llama `precio` y que sus valores son números enteros.

Seleccionar otras columnas

La misma forma de selección sirve para cualquier columna del `DataFrame`.

Por ejemplo, podemos seleccionar la columna `producto`:

```
df["producto"]
```

índice	producto
0	Cuaderno
1	Lapicera
2	Mochila
3	Regla
4	Cartuchera
5	Cuaderno
6	Marcadores
7	Carpeta
8	Goma
9	Lapiz

```
dtype: object
```

En este caso, Pandas devuelve una `Series` con los nombres de los productos vendidos.

También podemos seleccionar la columna `sucursal`:

```
df["sucursal"]
```


índice	sucursal
0	Centro
1	Centro
2	Norte
3	Sur
4	Norte
5	Sur
6	Centro
7	Norte
8	Sur
9	Centro

dtype: object

O la columna `cantidad_vendida`:

```
df["cantidad_vendida"]
```

índice	cantidad_vendida
0	15
1	40
2	5
3	25
4	8
5	12
6	10
7	18
8	30
9	50

dtype: int64

La estructura general siempre es la misma:

```
df["nombre_de_la_columna"]
```

Donde `df` es el nombre del `DataFrame` y `"nombre_de_la_columna"` es el nombre exacto de la columna que queremos seleccionar.

Es importante escribir el nombre de la columna exactamente como aparece en el `DataFrame`. Pandas distingue entre mayúsculas y minúsculas, y también tiene en cuenta espacios, guiones bajos y otros caracteres.

Por ejemplo, en nuestro dataset la columna se llama:

```
cantidad_vendida
```

No se llama:

```
cantidad vendida
Cantidad_vendida
cantidad_Vendida
cantidad
```

Si escribimos mal el nombre de una columna, Pandas no podrá encontrarla y mostrará un error.

Errores frecuentes al seleccionar columnas

Cuando seleccionamos una columna, el nombre debe coincidir exactamente con el nombre que aparece en el `DataFrame`.

Si escribimos un nombre incorrecto, Pandas no puede encontrar la columna y muestra un error.

Por ejemplo, nuestro `DataFrame` tiene una columna llamada `precio`.

Si intentamos escribir:

```
# Esta celda produce un error porque la columna "Precio" no existe.
# df["Precio"]
```

Pandas mostrará un error, porque `Precio` con mayúscula inicial no es lo mismo que `precio`.

El error más frecuente en estos casos se llama `KeyError`.

Un `KeyError` significa que intentamos acceder a una clave, etiqueta o nombre que no existe. En este caso, Pandas está diciendo que no encuentra una columna con ese nombre.

También tendríamos un error si escribimos:

```
# Esta celda produce un error porque la columna se llama "cantidad_vendida".  
# df["cantidad vendida"]
```

porque en nuestro dataset la columna se llama `cantidad_vendida`, con guion bajo.

Para evitar este tipo de errores, conviene revisar los nombres de las columnas antes de seleccionarlas:

```
df.columns
```

```
Index(['producto', 'categoria', 'precio', 'cantidad_vendida', 'sucursal',  
      'fecha'],  
      dtype='object')
```

Esta instrucción nos muestra los nombres exactos de las columnas disponibles.

Una buena costumbre es copiar el nombre de la columna tal como aparece en `df.columns`, especialmente cuando el dataset tiene muchas variables o nombres largos.

Seleccionar una columna como DataFrame

Cuando escribimos:

```
df["precio"]
```

índice	precio
0	1200
1	350
2	8500
3	500
4	3200
5	1250
6	1800
7	950
8	300
9	250

```
dtype: int64
```

Pandas devuelve una `Series`.

Una `Series` es una estructura de una sola dimensión. Podemos imaginarla como una columna individual, acompañada por su índice.

Pero a veces queremos seleccionar una sola columna y conservar el formato de tabla, es decir, obtener como resultado un `DataFrame`.

Para eso usamos doble corchete:

```
df[["precio"]]
```

índice	precio
0	1200
1	350
2	8500
3	500
4	3200
5	1250
6	1800
7	950
8	300
9	250

La diferencia parece pequeña, pero es importante.

Comparemos las dos formas:

```
df["precio"]
```

devuelve una `Series`.

En cambio:

```
df[["precio"]]
```

devuelve un `DataFrame` con una sola columna.

Podemos verificarlo usando `type()`.

Nota: si estás usando Colab, al escribir `type(df["columna"])` como última línea de una celda, puede mostrarse una salida muy extensa con documentación interna de Pandas. Para ver solo el tipo de objeto de manera breve, usamos `print(type(df["columna"]))`.

```
print(type(df["precio"]))
```

```
<class 'pandas.core.series.Series'>
```

```
print(type(df[["precio"]]))
```

```
<class 'pandas.core.frame.DataFrame'>
```

En el primer caso, la salida indica que el objeto es una `Series`.

En el segundo caso, la salida indica que el objeto es un `DataFrame`.

La diferencia entre una `Series` y un `DataFrame` será importante en muchas operaciones de Pandas. Algunas funciones esperan recibir una columna individual, mientras que otras trabajan mejor con una tabla, aunque esa tabla tenga una sola columna.

Por ahora, podemos quedarnos con esta idea:

```
df["precio"]    → devuelve una Series  
df[["precio"]] → devuelve un DataFrame
```

El doble corchete no es un adorno. Cambia el tipo de objeto que obtenemos como resultado.

Seleccionar varias columnas

Muchas veces necesitamos trabajar con más de una columna al mismo tiempo.

Por ejemplo, si queremos ver qué producto se vendió y a qué precio, podemos seleccionar las columnas `producto` y `precio`.

Para seleccionar varias columnas, usamos una lista con los nombres de las columnas dentro de los corchetes del `DataFrame`.

```
df[["producto", "precio"]]
```

índice	producto	precio
0	Cuaderno	1200
1	Lapicera	350
2	Mochila	8500
3	Regla	500
4	Cartuchera	3200
5	Cuaderno	1250
6	Marcadores	1800
7	Carpeta	950
8	Goma	300
9	Lapiz	250

En este caso, Pandas devuelve un nuevo `DataFrame` formado solamente por las columnas seleccionadas.

La estructura general es:

```
df[["columna_1", "columna_2", "columna_3"]]
```

Es importante observar que usamos doble corchete porque estamos pasando una lista de columnas.

El corchete exterior pertenece a la selección del `DataFrame`.

El corchete interior pertenece a la lista de nombres de columnas.

Por eso escribimos:

```
df[["producto", "precio"]]
```

y no:

```
df["producto", "precio"]
```

La segunda forma produce un error, porque Pandas no interpreta esos dos nombres como una lista de columnas.

Veamos otro ejemplo. Si queremos analizar las ventas por sucursal, podríamos seleccionar:

```
df[["sucursal", "producto", "cantidad_vendida"]]
```

índice	sucursal	producto	cantidad_vendida
0	Centro	Cuaderno	15
1	Centro	Lapicera	40
2	Norte	Mochila	5
3	Sur	Regla	25
4	Norte	Cartuchera	8
5	Sur	Cuaderno	12
6	Centro	Marcadores	10
7	Norte	Carpeta	18
8	Sur	Goma	30
9	Centro	Lapiz	50

Esta selección nos permite concentrarnos en tres variables específicas del dataset:

```
sucursal  
producto  
cantidad_vendida
```

El `DataFrame` original sigue teniendo todas sus columnas. La selección solo muestra una parte de la tabla.

Guardar una selección en una nueva variable

Seleccionar columnas no modifica el `DataFrame` original.

Cuando escribimos una selección como:

```
df[["producto", "precio"]]
```

Pandas nos muestra una tabla con esas columnas, pero `df` sigue conservando todas las columnas originales.

Si queremos reutilizar esa selección más adelante, podemos guardarla en una nueva variable.

```
productos_y_precios = df[["producto", "precio"]]  
  
productos_y_precios
```

índice	producto	precio
0	Cuaderno	1200
1	Lapicera	350
2	Mochila	8500
3	Regla	500
4	Cartuchera	3200
5	Cuaderno	1250
6	Marcadores	1800
7	Carpeta	950
8	Goma	300
9	Lapiz	250

Ahora la variable `productos_y_precios` contiene un nuevo `DataFrame` con solamente dos columnas:

```
producto  
precio
```

Podemos comprobar que el `DataFrame` original sigue igual:

```
df
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	fecha
0	Cuaderno	Libreria	1200	15	Centro	2024-03-01
1	Lapicera	Libreria	350	40	Centro	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	2024-03-02
3	Regla	Libreria	500	25	Sur	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	2024-03-03
5	Cuaderno	Libreria	1250	12	Sur	2024-03-03
6	Marcadores	Libreria	1800	10	Centro	2024-03-04
7	Carpeta	Libreria	950	18	Norte	2024-03-04
8	Goma	Libreria	300	30	Sur	2024-03-05

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
9	Lapiz	Librería	250	50	Centro	2024-03-05

Esto es importante porque muchas veces vamos a crear subconjuntos de datos para trabajar de manera más cómoda.

Por ejemplo, podríamos crear un `DataFrame` solamente con las columnas relacionadas con la venta:

```
datos_venta = df[["producto", "precio", "cantidad_vendida"]]
```

```
datos_venta
```

índice	producto	precio	cantidad_vendida
0	Cuaderno	1200	15
1	Lapicera	350	40
2	Mochila	8500	5
3	Regla	500	25
4	Cartuchera	3200	8
5	Cuaderno	1250	12
6	Marcadores	1800	10
7	Carpeta	950	18
8	Goma	300	30
9	Lapiz	250	50

O podríamos crear otro `DataFrame` con información de ubicación y fecha:

```
datos_contexto = df[["sucursal", "fecha"]]
```

```
datos_contexto
```

índice	sucursal	fecha
0	Centro	2024-03-01
1	Centro	2024-03-01
2	Norte	2024-03-02
3	Sur	2024-03-02
4	Norte	2024-03-03
5	Sur	2024-03-03

índice	sucursal	fecha
6	Centro	2024-03-04
7	Norte	2024-03-04
8	Sur	2024-03-05
9	Centro	2024-03-05

La elección de las columnas que vamos a conservar depende de la pregunta que queramos responder con los datos. Si nuestro objetivo es analizar importes, probablemente necesitemos trabajar con las columnas `precio` y `cantidad_vendida`. Si nos interesa estudiar dónde se registraron las ventas, será importante conservar la columna `sucursal`. En cambio, si queremos observar cómo evolucionan las ventas a lo largo del tiempo, necesitaremos la columna `fecha`. Por eso, seleccionar columnas no es solo una operación técnica: también es una forma de enfocar el análisis.

Usar una columna seleccionada

Seleccionar una columna no sirve solamente para verla. También podemos usar esa columna para hacer operaciones. Por ejemplo, si seleccionamos la columna `precio`, podemos calcular algunos valores simples:

```
df["precio"].min()
```

```
250
```

```
df["precio"].max()
```

```
8500
```

```
df["precio"].mean()
```

```
np.float64(1830.0)
```

Nota: en Colab, Pandas puede mostrar el resultado como `np.float64(1830.0)`. Esto no cambia el valor obtenido: numéricamente equivale a `1830.0`. La parte `np.float64` indica que Pandas/NumPy lo representan internamente como un número decimal de 64 bits.

Estas instrucciones calculan, respectivamente:

```
el precio mínimo  
el precio máximo  
el precio promedio
```

También podemos hacer operaciones con la columna `cantidad_vendida`:

```
df["cantidad_vendida"].sum()
```

```
np.int64(213)
```

```
df["cantidad_vendida"].mean()
```

```
np.float64(21.3)
```

En este caso:

```
sum() calcula la suma total  
mean() calcula el promedio
```

Entonces, si escribimos:

```
df["cantidad_vendida"].sum()
```

```
np.int64(213)
```

estamos pidiendo la cantidad total de productos vendidos en todos los registros del dataset.

Esta forma de trabajo es muy común en Pandas:

```
seleccionamos una columna  
↓  
aplicamos una operación sobre esa columna  
↓  
interpretamos el resultado
```

Por eso es tan importante aprender a seleccionar columnas correctamente. Muchas operaciones de análisis comienzan eligiendo una o más variables del dataset.

Seleccionar columnas según la pregunta

Cuando trabajamos con datos, no seleccionamos columnas al azar. Elegimos las columnas que necesitamos según la pregunta que queremos responder.

Por ejemplo, si queremos saber cuáles son los productos más caros, necesitamos mirar la columna `producto` junto con la columna `precio`. En cambio, si queremos saber en qué sucursal se registraron más ventas, nos interesa observar la columna `sucursal`. Si queremos analizar la cantidad de unidades vendidas, necesitamos trabajar con `cantidad_vendida`.

Esta relación entre pregunta y selección es muy importante. Pandas nos da herramientas para seleccionar columnas, pero la decisión de qué columnas seleccionar depende del análisis que queremos hacer.

```
# Productos y precios  
  
df[["producto", "precio"]]
```

índice	producto	precio
0	Cuaderno	1200
1	Lapicera	350
2	Mochila	8500
3	Regla	500
4	Cartuchera	3200
5	Cuaderno	1250
6	Marcadores	1800
7	Carpeta	950
8	Goma	300
9	Lapiz	250

```
# Sucursales y cantidades vendidas  
  
df[["sucursal", "cantidad_vendida"]]
```

índice	sucursal	cantidad_vendida
0	Centro	15
1	Centro	40
2	Norte	5

índice	sucursal	cantidad_vendida
3	Sur	25
4	Norte	8
5	Sur	12
6	Centro	10
7	Norte	18
8	Sur	30
9	Centro	50

```
# Producto, sucursal y fecha de venta
```

```
df[["producto", "sucursal", "fecha"]]
```

índice	producto	sucursal	fecha
0	Cuaderno	Centro	2024-03-01
1	Lapicera	Centro	2024-03-01
2	Mochila	Norte	2024-03-02
3	Regla	Sur	2024-03-02
4	Cartuchera	Norte	2024-03-03
5	Cuaderno	Sur	2024-03-03
6	Marcadores	Centro	2024-03-04
7	Carpeta	Norte	2024-03-04
8	Goma	Sur	2024-03-05
9	Lapiz	Centro	2024-03-05

En cada caso, seleccionamos una parte diferente del `DataFrame`.

La primera selección nos ayuda a observar los productos junto con sus precios. La segunda permite relacionar cada sucursal con la cantidad vendida. La tercera combina información sobre qué producto se vendió, dónde se vendió y en qué fecha.

En un análisis real, esta decisión se repite constantemente. Primero formulamos una pregunta, luego identificamos qué columnas necesitamos y finalmente aplicamos las operaciones correspondientes.

Una buena forma de pensar el proceso es:

pregunta de análisis → columnas necesarias → operación en Pandas → interpretación

Seleccionar columnas es, entonces, una de las primeras formas de transformar una tabla completa en información más enfocada.

Selección con corchetes y selección con punto

En muchos ejemplos de Pandas es posible que veas otra forma de seleccionar columnas. En lugar de escribir:

```
df["precio"]
```

a veces se escribe:

```
df.precio
```

En este caso, ambas instrucciones devuelven la columna `precio`. Sin embargo, en esta serie vamos a preferir la selección con corchetes:

```
df["precio"]
```

índice	precio
0	1200
1	350
2	8500
3	500
4	3200
5	1250
6	1800
7	950
8	300
9	250

`dtype: int64`

Esta forma es más clara y funciona en más situaciones. Por ejemplo, si una columna tiene espacios en su nombre, caracteres especiales, o si su nombre coincide con algún método o atributo de Pandas, la notación con punto puede fallar o generar confusiones.

La notación con corchetes también deja más visible que estamos seleccionando una columna por su nombre. Por eso, aunque `df.precio` pueda funcionar en este caso, conviene acostumbrarse a escribir:

```
df["precio"]
```

Esta será la forma principal que usaremos para seleccionar columnas.

Seleccionar columnas como primer recorte del dataset

Cuando seleccionamos columnas, hacemos un primer recorte del dataset. No estamos filtrando filas todavía, ni eliminando datos del `DataFrame` original. Simplemente estamos eligiendo qué variables queremos observar o utilizar.

Esto es útil porque una tabla completa puede contener más información de la que necesitamos en un momento determinado. Si queremos analizar precios, podemos concentrarnos en `producto` y `precio`. Si queremos estudiar las ventas por sucursal, podemos mirar `sucursal` y `cantidad_vendida`. Si queremos trabajar con el momento en que ocurrió cada venta, necesitaremos la columna `fecha`.

En Pandas, este recorte puede producir dos tipos de objetos. Si seleccionamos una sola columna con un corchete, obtenemos una `Series`. Si seleccionamos una o varias columnas usando doble corchete, obtenemos un `DataFrame`.

Podemos resumirlo así:

<code>df["precio"]</code>	→ selecciona una columna como <code>Series</code>
<code>df[["precio"]]</code>	→ selecciona una columna como <code>DataFrame</code>
<code>df[["producto", "precio"]]</code>	→ selecciona varias columnas como <code>DataFrame</code>

Esta diferencia no es solamente visual. Una `Series` y un `DataFrame` son objetos distintos, y algunas operaciones de Pandas se comportan de manera diferente según el tipo de objeto con el que estemos trabajando.

Por ahora, lo más importante es incorporar esta idea: seleccionar columnas nos permite enfocar el análisis en las variables que necesitamos.

Resumen del capítulo

En este capítulo aprendimos a seleccionar columnas de un `DataFrame`.

Vimos que seleccionar columnas significa quedarnos con una o varias variables del dataset para observarlas, analizarlas o usarlas en operaciones posteriores. Esta es una de las acciones más frecuentes en Pandas, porque muchas veces no necesitamos trabajar con toda la tabla, sino con una parte específica de ella.

Primero seleccionamos una sola columna usando su nombre entre corchetes:

```
df["precio"]
```

Esa operación devuelve una `Series`, es decir, una estructura de una sola dimensión que conserva los valores de la columna y el índice del `DataFrame`.

Después vimos que también podemos seleccionar una columna como `DataFrame` usando doble corchete:

```
df[["precio"]]
```

Aunque la diferencia parezca pequeña, el resultado no es el mismo. En el primer caso obtenemos una `Series`; en el segundo, un `DataFrame` con una sola columna.

También aprendimos a seleccionar varias columnas al mismo tiempo:

```
df[["producto", "precio"]]
```

En ese caso, Pandas devuelve un nuevo `DataFrame` formado solo por las columnas seleccionadas.

A lo largo del capítulo también vimos que los nombres de columnas deben escribirse exactamente como aparecen en el `DataFrame`. Pandas distingue entre mayúsculas y minúsculas, espacios, guiones bajos y otros caracteres. Por eso, cuando tenemos dudas, conviene revisar los nombres disponibles con:

```
df.columns
```

Finalmente, vimos que seleccionar columnas no modifica el `DataFrame` original. Si queremos reutilizar una selección, podemos guardarla en una nueva variable.

La idea principal de este capítulo fue:

```
Seleccionar columnas nos permite enfocar el análisis en las variables que necesitamos.
```


Esta operación será la base para muchas tareas posteriores: calcular resúmenes, comparar variables, construir nuevos datasets, aplicar filtros y preparar datos para visualizaciones.

Próximo paso

Ya sabemos seleccionar columnas, es decir, elegir variables específicas de un `DataFrame`.

El siguiente paso será aprender a seleccionar filas. Esto nos permitirá observar registros particulares, trabajar con posiciones dentro de la tabla y empezar a preparar el camino para los filtros condicionales.

Más adelante vamos a combinar ambas ideas: seleccionar columnas y seleccionar filas al mismo tiempo. Esa combinación nos dará mucho más control sobre qué parte del dataset queremos analizar.

Capítulo 4 — Selección de filas en un DataFrame

Breve repaso

En el capítulo anterior aprendimos a seleccionar columnas de un `DataFrame`.

Vimos que una columna puede seleccionarse usando su nombre entre corchetes, por ejemplo `df["precio"]`, y que esa operación devuelve una `Series`. También vimos que, si usamos doble corchete, como en `df[["precio"]]`, Pandas devuelve un `DataFrame` con una sola columna.

Además, aprendimos a seleccionar varias columnas al mismo tiempo y a guardar esas selecciones en nuevas variables. Esa operación nos permitió hacer un primer recorte del dataset, eligiendo solamente las variables necesarias para una pregunta de análisis.

En este capítulo vamos a trabajar con otra forma de selección: la selección de filas.

Si las columnas representan variables, las filas representan observaciones, registros o casos. Por eso, seleccionar filas significa elegir qué registros del dataset queremos observar.

A lo largo del capítulo vamos a aprender a seleccionar filas por posición, a obtener una fila individual, a seleccionar rangos de filas y a reconocer la diferencia entre trabajar con una fila como `Series` o como `DataFrame`.

También vamos a empezar a preparar el camino para una herramienta muy importante de Pandas: `iloc`, que permite seleccionar datos usando posiciones numéricas.

Al finalizar este capítulo deberías poder:

- Seleccionar una fila de un `DataFrame` usando su posición.
- Seleccionar varias filas consecutivas.
- Comprender qué significa seleccionar por posición.
- Reconocer cuándo Pandas devuelve una `Series`.
- Reconocer cuándo Pandas devuelve un `DataFrame`.
- Usar `iloc` para seleccionar filas.
- Combinar una primera selección de filas y columnas usando posiciones.

Dataset de trabajo

En este capítulo vamos a seguir usando el dataset de ventas de una librería.

Cada fila representa una venta registrada y cada columna describe una característica de esa venta. Como ya conocemos la estructura general del dataset, podemos concentrarnos en una nueva pregunta: cómo seleccionar filas específicas.

Recordemos que, mientras las columnas representan variables, las filas representan registros u observaciones. En este caso, cada fila representa una venta.

```
import pandas as pd

datos = {
    "producto": [
        "Cuaderno", "Lapicera", "Mochila", "Regla", "Cartuchera",
        "Cuaderno", "Marcadores", "Carpeta", "Goma", "Lapiz"
    ],
    "categoria": [
        "Libreria", "Libreria", "Accesorios", "Libreria", "Accesorios",
        "Libreria", "Libreria", "Libreria", "Libreria", "Libreria"
    ],
    "precio": [
        1200, 350, 8500, 500, 3200,
        1250, 1800, 950, 300, 250
    ],
    "cantidad_vendida": [
        15, 40, 5, 25, 8,
        12, 10, 18, 30, 50
    ],
    "sucursal": [
        "Centro", "Centro", "Norte", "Sur", "Norte",
        "Sur", "Centro", "Norte", "Sur", "Centro"
    ],
    "fecha": [
        "2024-03-01", "2024-03-01", "2024-03-02", "2024-03-02", "2024-03-03",
        "2024-03-03", "2024-03-04", "2024-03-04", "2024-03-05", "2024-03-05"
    ]
}

df = pd.DataFrame(datos)

df
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	fecha
0	Cuaderno	Libreria	1200	15	Centro	2024-03-01
1	Lapicera	Libreria	350	40	Centro	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	2024-03-02

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
3	Regla	Librería	500	25	Sur	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	2024-03-03
5	Cuaderno	Librería	1250	12	Sur	2024-03-03
6	Marcadores	Librería	1800	10	Centro	2024-03-04
7	Carpeta	Librería	950	18	Norte	2024-03-04
8	Goma	Librería	300	30	Sur	2024-03-05
9	Lapiz	Librería	250	50	Centro	2024-03-05

Al mostrar el `DataFrame`, vemos nuevamente las 10 ventas registradas.

A la izquierda aparece el índice, que en este caso va de `0` a `9`. Ese índice nos permite identificar cada fila dentro del `DataFrame`. En este capítulo vamos a usarlo como referencia visual, pero el foco estará puesto en la selección por posición mediante `iloc`.

Qué significa seleccionar filas

En un `DataFrame`, las columnas representan variables y las filas representan observaciones, registros o casos.

En nuestro dataset, cada fila representa una venta. Por eso, seleccionar una fila significa elegir una venta específica del conjunto de datos.

Por ejemplo, podríamos querer observar solamente la primera venta registrada, revisar las primeras tres ventas, mirar una venta que aparece en una posición determinada o comparar algunos registros puntuales.

Seleccionar filas es diferente de seleccionar columnas. Cuando seleccionamos columnas, elegimos variables. Cuando seleccionamos filas, elegimos registros.

En este capítulo vamos a comenzar con una forma de selección muy importante: la selección por posición.

La posición indica el lugar que ocupa una fila dentro del `DataFrame`. Como ocurre muchas veces en Python, las posiciones empiezan a contarse desde `0`. Entonces, la primera fila está en la posición `0`, la segunda fila está en la posición `1`, la tercera fila está en la posición `2`, y así sucesivamente.

En nuestro dataset, que tiene 10 filas, las posiciones van desde `0` hasta `9`.

Para seleccionar filas por posición vamos a usar `iloc`.

Seleccionar una fila con `iloc`

`iloc` permite seleccionar filas y columnas usando posiciones numéricas. La palabra `iloc` puede pensarse como una abreviatura de *integer location*, es decir, ubicación mediante números enteros.

Para seleccionar una fila por su posición, escribimos el nombre del `DataFrame`, luego `.iloc[]`, y dentro de los corchetes indicamos la posición de la fila que queremos obtener. Por ejemplo, para seleccionar la primera fila del `DataFrame`, usamos la posición `0`.

```
df.iloc[0]
```

índice	0
producto	Cuaderno
categoría	Librería
precio	1200
cantidad_vendida	15
sucursal	Centro
fecha	2024-03-01

`dtype: object`

La salida muestra la información completa de la primera venta registrada. Como estamos seleccionando una sola fila, Pandas devuelve una `Series`.

En este caso, los nombres de las columnas aparecen a la izquierda y los valores de esa fila aparecen a la derecha. Es decir, la fila seleccionada se muestra como una estructura de una sola dimensión.

La posición `0` corresponde a la primera fila porque Pandas, al igual que Python, comienza a contar desde cero.

```
df.iloc[1]
```

índice	1
producto	Lapicera
categoría	Librería
precio	350
cantidad_vendida	40

índice	1
sucursal	Centro
fecha	2024-03-01

`dtype: object`

En este segundo ejemplo seleccionamos la fila ubicada en la posición `1`, que corresponde a la segunda fila del `DataFrame`.

Esto puede resultar extraño al principio si estamos acostumbrados a contar desde 1, pero es una convención muy frecuente en programación. La primera posición es `0`, la segunda es `1`, la tercera es `2`, y así sucesivamente.

Posición e índice no siempre son lo mismo

En nuestro dataset actual, el índice coincide visualmente con la posición de cada fila. La primera fila tiene índice `0`, la segunda tiene índice `1`, la tercera tiene índice `2`, y así sucesivamente.

Por eso, al principio puede parecer que índice y posición son lo mismo. Sin embargo, no siempre ocurre así.

La **posición** indica el lugar que ocupa una fila dentro del `DataFrame`: primera, segunda, tercera, etc. En cambio, el **índice** es una etiqueta que Pandas usa para identificar las filas. Muchas veces el índice es numérico y empieza en `0`, pero también puede estar formado por otros valores, como códigos, fechas, nombres o identificadores.

`iloc` trabaja con posiciones, no con etiquetas del índice.

Esto significa que cuando escribimos:

```
df.iloc[0]
```

estamos pidiendo la fila que está en la primera posición, no necesariamente la fila cuyo índice se llama `0`.

En este capítulo vamos a concentrarnos en la selección por posición. Más adelante vamos a trabajar con más detalle la selección por etiquetas y veremos por qué esta diferencia es importante.

Seleccionar la última fila

Así como en las listas de Python, `iloc` también permite usar posiciones negativas.

La posición `-1` representa la última fila del `DataFrame`.

```
df.iloc[-1]
```

índice	9
producto	Lapiz
categoria	Libreria
precio	250
cantidad_vendida	50
sucursal	Centro
fecha	2024-03-05

`dtype: object`

La salida muestra la última venta registrada en el dataset.

También podemos usar `-2` para seleccionar la anteúltima fila:

```
df.iloc[-2]
```

índice	8
producto	Goma
categoria	Libreria
precio	300
cantidad_vendida	30
sucursal	Sur
fecha	2024-03-05

`dtype: object`

Las posiciones negativas son útiles cuando queremos acceder al final de una tabla sin contar manualmente cuántas filas tiene.

Podemos resumirlo así:

```
df.iloc[0]    → primera fila
df.iloc[1]    → segunda fila
df.iloc[-1]   → última fila
df.iloc[-2]   → anteúltima fila
```

Seleccionar varias filas consecutivas

Hasta ahora seleccionamos una sola fila por vez. Pero muchas veces queremos obtener varias filas consecutivas. Para eso podemos usar un rango dentro de `iloc`.

Por ejemplo, si queremos seleccionar las primeras tres filas, escribimos:

```
df.iloc[0:3]
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
0	Cuaderno	Librería	1200	15	Centro	2024-03-01
1	Lapicera	Librería	350	40	Centro	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	2024-03-02

La notación `0:3` indica un rango de posiciones.

En Python, los rangos incluyen el primer valor, pero no incluyen el último. Por eso, `0:3` selecciona las posiciones `0`, `1` y `2`.

Dicho de otra manera:

```
df.iloc[0:3] → selecciona desde la posición 0 hasta antes de la posición 3
```

El resultado es un `DataFrame`, no una `Series`, porque estamos seleccionando varias filas.

Podemos probar otros rangos:

```
df.iloc[3:7]
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
3	Regla	Librería	500	25	Sur	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	2024-03-03
5	Cuaderno	Librería	1250	12	Sur	2024-03-03
6	Marcadores	Librería	1800	10	Centro	2024-03-04

En este caso seleccionamos las filas ubicadas en las posiciones 3, 4, 5 y 6.

También podemos omitir el primer valor del rango si queremos comenzar desde el principio:

```
df.iloc[:5]
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
0	Cuaderno	Librería	1200	15	Centro	2024-03-01
1	Lapicera	Librería	350	40	Centro	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	2024-03-02
3	Regla	Librería	500	25	Sur	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	2024-03-03

Esta instrucción selecciona desde el comienzo del `DataFrame` hasta antes de la posición 5. Es decir, devuelve las primeras cinco filas.

También podemos omitir el segundo valor si queremos seleccionar desde una posición hasta el final:

```
df.iloc[5:]
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
5	Cuaderno	Librería	1250	12	Sur	2024-03-03
6	Marcadores	Librería	1800	10	Centro	2024-03-04
7	Carpeta	Librería	950	18	Norte	2024-03-04
8	Goma	Librería	300	30	Sur	2024-03-05
9	Lapiz	Librería	250	50	Centro	2024-03-05

Esta instrucción selecciona desde la posición 5 hasta el final del `DataFrame`.

Esta forma de selección es muy útil cuando queremos revisar partes consecutivas de una tabla sin mostrar el dataset completo.

Seleccionar una fila como DataFrame

Cuando usamos `iloc` con una sola posición, Pandas devuelve una `Series`. Por ejemplo:

```
df.iloc[0]
```

índice	0
producto	Cuaderno
categoria	Libreria
precio	1200
cantidad_vendida	15
sucursal	Centro
fecha	2024-03-01

```
dtype: object
```

Esa salida representa una sola fila, pero no se muestra como tabla. Los nombres de las columnas aparecen como etiquetas y los valores de la fila aparecen a continuación.

A veces, sin embargo, queremos seleccionar una sola fila y conservar el formato de `DataFrame`. Para eso podemos usar una lista con una sola posición:

```
df.iloc[[0]]
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	fecha
0	Cuaderno	Libreria	1200	15	Centro	2024-03-01

La diferencia está en los corchetes. Cuando escribimos:

```
df.iloc[0]
```

obtenemos una `Series`. En cambio, cuando escribimos:

```
df.iloc[[0]]
```

obtenemos un `DataFrame` con una sola fila. Podemos verificarlo usando `type()`:

```
print(type(df.iloc[0]))
```

```
<class 'pandas.core.series.Series'>
```

```
print(type(df.iloc[[0]]))
```

```
<class 'pandas.core.frame.DataFrame'>
```

Esta diferencia es parecida a la que vimos al seleccionar columnas. En Pandas, la forma en que escribimos la selección puede cambiar el tipo de objeto que obtenemos.

Podemos resumirlo así:

```
df.iloc[0]    → selecciona una fila como Series  
df.iloc[[0]]  → selecciona una fila como DataFrame
```

Cuando queremos conservar la forma de tabla, usamos una lista de posiciones, aunque esa lista tenga un solo elemento.

Seleccionar filas no consecutivas

Con `iloc` también podemos seleccionar varias filas que no estén una al lado de la otra. Para eso usamos una lista de posiciones.

Por ejemplo, si queremos seleccionar la primera, la cuarta y la última fila del `DataFrame`, podemos escribir:

```
df.iloc[[0, 3, 9]]
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
0	Cuaderno	Librería	1200	15	Centro	2024-03-01
3	Regla	Librería	500	25	Sur	2024-03-02
9	Lapiz	Librería	250	50	Centro	2024-03-05

En este caso, Pandas devuelve un `DataFrame` con las filas ubicadas en las posiciones `0`, `3` y `9`.

La lista de posiciones se escribe entre corchetes:

```
[0, 3, 9]
```

y esa lista se coloca dentro de `iloc`:

```
df.iloc[[0, 3, 9]]
```

Esta forma de selección es útil cuando queremos observar registros puntuales sin seleccionar todo un rango.

También podemos seleccionar, por ejemplo, la segunda, la quinta y la octava fila:

```
df.iloc[[1, 4, 7]]
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
1	Lapicera	Librería	350	40	Centro	2024-03-01
4	Cartuchera	Accesorios	3200	8	Norte	2024-03-03
7	Carpeta	Librería	950	18	Norte	2024-03-04

Recordemos que las posiciones comienzan en `0`. Por eso, la segunda fila está en la posición `1`, la quinta fila está en la posición `4` y la octava fila está en la posición `7`.

Cuando usamos una lista de posiciones, el resultado conserva el formato de `DataFrame`, aunque seleccionemos una sola fila o varias filas separadas.

Seleccionar filas y columnas con `iloc`

Hasta ahora usamos `iloc` para seleccionar filas. Pero `iloc` también permite seleccionar filas y columnas al mismo tiempo usando posiciones numéricas.

La estructura general es:

```
df.iloc[filas, columnas]
```

El primer valor indica qué filas queremos seleccionar. El segundo valor indica qué columnas queremos seleccionar.

Por ejemplo, si queremos seleccionar la primera fila y la primera columna, podemos escribir:

```
df.iloc[0, 0]
```

```
'Cuaderno'
```

La salida es un único valor. En nuestro dataset, ese valor corresponde al producto de la primera venta.

También podemos seleccionar una fila completa y algunas columnas específicas. Por ejemplo:

```
df.iloc[0, [0, 2, 3]]
```

índice	0
producto	Cuaderno
precio	1200
cantidad_vendida	15

`dtype: object`

En este caso seleccionamos la fila ubicada en la posición `0`, y dentro de esa fila seleccionamos las columnas ubicadas en las posiciones `0`, `2` y `3`.

Para entender mejor esas posiciones, recordemos el orden de las columnas:

```
list(df.columns)
```

```
['producto', 'categoria', 'precio', 'cantidad_vendida', 'sucursal', 'fecha']
```

La lista de columnas nos muestra que:

```
posición 0 → producto
posición 1 → categoria
posición 2 → precio
posición 3 → cantidad_vendida
posición 4 → sucursal
posición 5 → fecha
```

Entonces:

```
df.iloc[0, [0, 2, 3]]
```

selecciona, para la primera fila, las columnas `producto`, `precio` y `cantidad_vendida`.

También podemos seleccionar varias filas y varias columnas:

```
df.iloc[0:4, [0, 2, 3]]
```

índice	producto	precio	cantidad_vendida
0	Cuaderno	1200	15
1	Lapicera	350	40
2	Mochila	8500	5
3	Regla	500	25

Esta instrucción selecciona las filas ubicadas desde la posición 0 hasta antes de la posición 4, y las columnas ubicadas en las posiciones 0, 2 y 3.

El resultado es un `DataFrame` con las primeras cuatro ventas y solo algunas columnas.

Esta forma de selección es potente, pero exige prestar atención a las posiciones. Por eso, cuando trabajamos con columnas, muchas veces resulta más claro seleccionarlas por nombre. Más adelante vamos a combinar distintas formas de selección para escribir código más legible.

Errores frecuentes al usar `iloc`

Cuando usamos `iloc`, debemos recordar que estamos trabajando con posiciones numéricas. Esto significa que no escribimos nombres de columnas ni etiquetas del índice, sino números que indican ubicaciones dentro del `DataFrame`.

Por ejemplo, esta instrucción es correcta:

```
df.iloc[0, 2]
```

```
np.int64(1200)
```

En este caso seleccionamos el valor ubicado en la primera fila y en la tercera columna. Recordemos que la primera posición es 0, por eso la columna en posición 2 es la tercera columna del `DataFrame`.

Un error frecuente sería intentar usar el nombre de una columna dentro de `iloc`:

```
# Esta instrucción produce un error porque iloc trabaja con posiciones, no con nombres.  
# df.iloc[0, "precio"]
```

Si queremos seleccionar usando nombres de columnas, necesitamos otra herramienta. Más adelante vamos a trabajar con `loc`, que permite seleccionar usando etiquetas o nombres.

Otro error frecuente es pedir una posición que no existe. Nuestro dataset tiene 10 filas, por lo tanto sus posiciones van desde 0 hasta 9. Si intentamos seleccionar la posición 10, Pandas mostrará un error porque esa fila no existe.

```
# Esta instrucción produce un error porque no existe una fila en la posición 10.  
# df.iloc[10]
```

Estos errores son normales al comenzar a trabajar con Pandas. La clave es recordar qué tipo de referencia espera cada herramienta.

Con `iloc`, usamos posiciones numéricas. Para evitar confusiones, puede ser útil revisar primero el tamaño del `DataFrame` con `shape` y el orden de las columnas con `list(df.columns)`.

Resumen del capítulo

En este capítulo aprendimos a seleccionar filas de un `DataFrame` usando `iloc`.

Mientras que en el capítulo anterior seleccionamos columnas, es decir, variables, ahora trabajamos con filas, que representan registros u observaciones. En nuestro dataset, cada fila corresponde a una venta registrada.

Vimos que `iloc` permite seleccionar datos usando posiciones numéricas. La primera fila está en la posición `0`, la segunda en la posición `1`, la tercera en la posición `2`, y así sucesivamente. Esta forma de contar desde cero es habitual en Python y también aparece en muchas operaciones de Pandas.

Primero seleccionamos una fila individual:

```
df.iloc[0]
```

Esa instrucción devuelve una `Series`, porque estamos seleccionando una sola fila.

Luego vimos que también podemos seleccionar una fila conservando el formato de tabla:

```
df.iloc[[0]]
```

En ese caso, Pandas devuelve un `DataFrame` con una sola fila.

También aprendimos a seleccionar varias filas consecutivas usando rangos:

```
df.iloc[0:3]
```

y filas no consecutivas usando listas de posiciones:

```
df.iloc[[0, 3, 9]]
```

Más adelante combinamos filas y columnas dentro de `iloc`, usando la estructura:

```
df.iloc[filas, columnas]
```

Por ejemplo:

```
df.iloc[0:4, [0, 2, 3]]
```

selecciona las primeras cuatro filas y las columnas ubicadas en las posiciones 0, 2 y 3.

Una idea importante de este capítulo fue distinguir entre posición e índice. En nuestro dataset actual parecen coincidir, pero no siempre será así. `iloc` trabaja con posiciones, no con etiquetas del índice.

La idea principal que conviene recordar es:

```
iloc permite seleccionar filas y columnas usando posiciones numéricas.
```

Esta herramienta es muy útil cuando queremos acceder a partes específicas de un `DataFrame` según su ubicación.

Próximo paso

Ya sabemos seleccionar columnas y también sabemos seleccionar filas usando posiciones numéricas.

El siguiente paso será aprender a seleccionar datos usando etiquetas, nombres de columnas e índices. Para eso vamos a trabajar con `loc`, una herramienta central de Pandas que permite escribir selecciones más expresivas cuando conocemos los nombres de las columnas o las etiquetas de las filas.

A medida que avancemos, vamos a combinar estas formas de selección para construir consultas cada vez más precisas sobre nuestros datasets.

Capítulo 5 — Selección con etiquetas usando loc

Breve repaso

En el capítulo anterior aprendimos a seleccionar filas de un `DataFrame` usando `iloc`.

Vimos que `iloc` trabaja con posiciones numéricas. Por ejemplo, `df.iloc[0]` selecciona la primera fila, `df.iloc[1]` selecciona la segunda, y `df.iloc[0:3]` selecciona las filas ubicadas desde la posición 0 hasta antes de la posición 3.

También vimos que `iloc` puede combinar filas y columnas usando posiciones, como en `df.iloc[0:4, [0, 2, 3]]`. Esa instrucción permite seleccionar un conjunto de filas y columnas indicando su ubicación dentro del `DataFrame`.

En este capítulo vamos a trabajar con otra herramienta fundamental de Pandas: `loc`.

A diferencia de `iloc`, que selecciona por posición, `loc` selecciona usando etiquetas. Esto significa que podemos indicar nombres de columnas y etiquetas del índice. En muchos casos, esta forma de selección resulta más clara porque permite escribir directamente qué columnas queremos observar.

Por ejemplo, en lugar de seleccionar la columna ubicada en la posición 2, podemos seleccionar la columna llamada "precio".

A lo largo del capítulo vamos a ver cómo usar `loc` para seleccionar filas, columnas y combinaciones de filas y columnas. También vamos a reforzar una idea importante: posición e índice pueden parecer lo mismo en algunos datasets, pero no siempre lo son.

Al finalizar este capítulo deberías poder:

- Comprender la diferencia entre `iloc` y `loc`.
- Seleccionar filas usando etiquetas del índice.
- Seleccionar columnas usando sus nombres.
- Combinar selección de filas y columnas con `loc`.
- Reconocer cuándo una selección devuelve una `Series` y cuándo devuelve un `DataFrame`.
- Entender por qué `loc` suele mejorar la legibilidad del código cuando trabajamos con nombres de columnas.

Dataset de trabajo

En este capítulo vamos a seguir usando el dataset de ventas de una librería.

Ya conocemos sus columnas principales: `producto`, `categoria`, `precio`, `cantidad_vendida`, `sucursal` y `fecha`. También sabemos que cada fila representa una venta registrada. Usar el mismo dataset nos permite concentrarnos en el nuevo tema: la selección con etiquetas usando `loc`.

Primero vamos a crear nuevamente el `DataFrame`.

```
import pandas as pd

datos = {
    "producto": [
        "Cuaderno", "Lapicera", "Mochila", "Regla", "Cartuchera",
        "Cuaderno", "Marcadores", "Carpeta", "Goma", "Lapiz"
    ],
    "categoria": [
        "Libreria", "Libreria", "Accesorios", "Libreria", "Accesorios",
        "Libreria", "Libreria", "Libreria", "Libreria", "Libreria"
    ],
    "precio": [
        1200, 350, 8500, 500, 3200,
        1250, 1800, 950, 300, 250
    ],
    "cantidad_vendida": [
        15, 40, 5, 25, 8,
        12, 10, 18, 30, 50
    ],
    "sucursal": [
        "Centro", "Centro", "Norte", "Sur", "Norte",
        "Sur", "Centro", "Norte", "Sur", "Centro"
    ],
    "fecha": [
        "2024-03-01", "2024-03-01", "2024-03-02", "2024-03-02", "2024-03-03",
        "2024-03-03", "2024-03-04", "2024-03-04", "2024-03-05", "2024-03-05"
    ]
}

df = pd.DataFrame(datos)

df
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	fecha
0	Cuaderno	Libreria	1200	15	Centro	2024-03-01
1	Lapicera	Libreria	350	40	Centro	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	2024-03-02

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
3	Regla	Librería	500	25	Sur	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	2024-03-03
5	Cuaderno	Librería	1250	12	Sur	2024-03-03
6	Marcadores	Librería	1800	10	Centro	2024-03-04
7	Carpeta	Librería	950	18	Norte	2024-03-04
8	Goma	Librería	300	30	Sur	2024-03-05
9	Lapiz	Librería	250	50	Centro	2024-03-05

Como el índice fue creado automáticamente por Pandas, las filas aparecen identificadas con etiquetas numéricas desde 0 hasta 9.

En este caso, esas etiquetas coinciden visualmente con las posiciones de las filas. Sin embargo, durante el capítulo vamos a ver que `loc` e `iloc` no funcionan de la misma manera. `iloc` usa posiciones; `loc` usa etiquetas.

Diferencia entre `iloc` y `loc`

En Pandas existen varias formas de seleccionar datos. Dos de las más importantes son `iloc` y `loc`.

La diferencia central es esta:

```
iloc selecciona por posición.
loc selecciona por etiqueta.
```

Cuando usamos `iloc`, indicamos posiciones numéricas. Por ejemplo, la posición 0, la posición 1 o el rango 0:3.

Cuando usamos `loc`, indicamos etiquetas. Esas etiquetas pueden ser los valores del índice para seleccionar filas, o los nombres de las columnas para seleccionar variables.

En nuestro `DataFrame` actual, el índice es numérico y va de 0 a 9. Por eso, al principio puede parecer que `loc` e `iloc` hacen lo mismo. Sin embargo, conceptualmente trabajan de manera distinta.

Por ejemplo:

```
df.iloc[0]
```

selecciona la fila que está en la primera posición.

En cambio:

```
df.loc[0]
```

selecciona la fila cuya etiqueta de índice es `0`.

En este dataset, ambas instrucciones devuelven la misma fila porque el índice coincide con las posiciones. Pero esto no siempre será así. Si el índice estuviera formado por códigos, fechas o nombres, `loc` usaría esas etiquetas, no la posición física de la fila.

Esta diferencia será cada vez más importante a medida que trabajemos con datasets más complejos.

```
df.iloc[0]
```

índice	0
producto	Cuaderno
categoria	Libreria
precio	1200
cantidad_vendida	15
sucursal	Centro
fecha	2024-03-01

`dtype: object`

```
df.loc[0]
```

índice	0
producto	Cuaderno
categoria	Libreria
precio	1200
cantidad_vendida	15
sucursal	Centro
fecha	2024-03-01

`dtype: object`

En este caso, las dos salidas son iguales porque el índice del `DataFrame` coincide con las posiciones. Aun así, conviene recordar la diferencia conceptual: `iloc` responde a la pregunta “¿qué fila está en esta posición?”, mientras que `loc` responde a la pregunta “¿qué fila tiene esta etiqueta?”.

Seleccionar filas con loc

`loc` permite seleccionar filas usando las etiquetas del índice.

En nuestro `DataFrame`, el índice fue creado automáticamente por Pandas y está formado por los números del `0` al `9`. Por eso, podemos seleccionar la fila cuya etiqueta es `0` escribiendo:

```
df.loc[0]
```

índice	0
producto	Cuaderno
categoria	Libreria
precio	1200
cantidad_vendida	15
sucursal	Centro
fecha	2024-03-01

`dtype: object`

La salida muestra la fila identificada con la etiqueta `0`.

En este caso, esa fila también es la primera fila del `DataFrame`, pero eso ocurre porque el índice automático coincide con las posiciones. Lo importante es recordar que `loc[0]` no significa necesariamente “primera fila”, sino “fila cuya etiqueta de índice es `0`”.

También podemos seleccionar otra fila por su etiqueta:

```
df.loc[3]
```

índice	3
producto	Regla
categoria	Libreria
precio	500

índice	3
cantidad_vendida	25
sucursal	Sur
fecha	2024-03-02

`dtype: object`

Esta instrucción selecciona la fila cuya etiqueta de índice es `3`.

Como el índice actual es simple y numérico, el resultado parece muy parecido al que obtendríamos con `iloc`. Sin embargo, la lógica no es la misma:

`df.iloc[3]` → selecciona la fila ubicada en la posición 3

`df.loc[3]` → selecciona la fila cuya etiqueta de índice es 3

En este dataset ambas instrucciones devuelven la misma fila, pero más adelante veremos ejemplos donde esto no ocurre.

Seleccionar varias filas con `loc`

También podemos usar `loc` para seleccionar varias filas. Si queremos seleccionar varias etiquetas puntuales del índice, podemos pasar una lista de etiquetas:

`df.loc[[0, 3, 5]]`

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
0	Cuaderno	Librería	1200	15	Centro	2024-03-01
3	Regla	Librería	500	25	Sur	2024-03-02
5	Cuaderno	Librería	1250	12	Sur	2024-03-03

En este caso, Pandas devuelve las filas cuyas etiquetas de índice son `0`, `3` y `5`. Como estamos seleccionando varias filas, el resultado es un `DataFrame`.

También podemos seleccionar un rango de etiquetas:

`df.loc[0:3]`

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
0	Cuaderno	Librería	1200	15	Centro	2024-03-01

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
1	Lapicera	Librería	350	40	Centro	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	2024-03-02
3	Regla	Librería	500	25	Sur	2024-03-02

Esta instrucción selecciona las filas cuyas etiquetas van desde 0 hasta 3. Aquí aparece una diferencia importante entre `loc` e `iloc`.

Con `iloc`, el final del rango no se incluye:

```
df.iloc[0:3]
```

selecciona las posiciones 0, 1 y 2. En cambio, con `loc`, cuando usamos etiquetas, el final del rango sí se incluye:

```
df.loc[0:3]
```

selecciona las etiquetas 0, 1, 2 y 3. Esta diferencia puede parecer pequeña, pero es muy importante para evitar errores al seleccionar datos.

Podemos resumirlo así:

```
df.iloc[0:3] → selecciona posiciones 0, 1 y 2
```

```
df.loc[0:3] → selecciona etiquetas 0, 1, 2 y 3
```

En nuestro dataset actual, como el índice coincide con las posiciones, esta diferencia se nota claramente en la cantidad de filas seleccionadas.

Seleccionar columnas con loc

Aunque `loc` suele presentarse como una herramienta para seleccionar por etiquetas del índice, también permite seleccionar columnas usando sus nombres.

La estructura general de `loc` es:

```
df.loc[filas, columnas]
```

El primer lugar corresponde a las filas y el segundo a las columnas.

Si queremos seleccionar todas las filas, usamos `:` en la parte de las filas. Luego, en la parte de columnas, indicamos el nombre de la columna que queremos seleccionar.

```
df.loc[:, "precio"]
```

índice	precio
0	1200
1	350
2	8500
3	500
4	3200
5	1250
6	1800
7	950
8	300
9	250

```
dtype: int64
```

```
df.loc[:, ["precio"]]
```

índice	precio
0	1200
1	350
2	8500
3	500
4	3200
5	1250
6	1800
7	950
8	300
9	250

En este caso, el resultado conserva el formato de tabla.

La diferencia es similar a la que vimos antes:


```
df.loc[:, "precio"]    → devuelve una Series  
df.loc[:, ["precio"]] → devuelve un DataFrame
```

Usar `loc` de esta manera puede parecer más largo que escribir simplemente `df["precio"]`, pero tiene una ventaja: permite combinar en una misma instrucción qué filas y qué columnas queremos seleccionar.

Seleccionar varias columnas con loc

Con `loc` también podemos seleccionar varias columnas por nombre. Para hacerlo, indicamos todas las filas con `:` y luego pasamos una lista con los nombres de las columnas que queremos conservar.

```
df.loc[:, ["producto", "precio"]]
```

índice	producto	precio
0	Cuaderno	1200
1	Lapicera	350
2	Mochila	8500
3	Regla	500
4	Cartuchera	3200
5	Cuaderno	1250
6	Marcadores	1800
7	Carpeta	950
8	Goma	300
9	Lapiz	250

Esta instrucción selecciona todas las filas del `DataFrame`, pero solamente las columnas `producto` y `precio`.

Podemos leerla de esta manera:

```
de todas las filas,  
mostrar solamente producto y precio
```

También podemos seleccionar otras combinaciones de columnas según la pregunta que queramos responder.

```
df.loc[:, ["sucursal", "producto", "cantidad_vendida"]]
```

índice	sucursal	producto	cantidad_vendida
0	Centro	Cuaderno	15
1	Centro	Lapicera	40
2	Norte	Mochila	5
3	Sur	Regla	25
4	Norte	Cartuchera	8
5	Sur	Cuaderno	12
6	Centro	Marcadores	10
7	Norte	Carpeta	18
8	Sur	Goma	30
9	Centro	Lapiz	50

En este caso seleccionamos tres columnas: `sucursal`, `producto` y `cantidad_vendida`. Esta selección podría servir para observar qué productos se vendieron, en qué sucursal y en qué cantidad.

La ventaja de `loc` es que la instrucción deja explícitas las dos dimensiones de la selección:

```
df.loc[filas, columnas]
```

Cuando escribimos:

```
df.loc[:, ["sucursal", "producto", "cantidad_vendida"]]
```

estamos diciendo que queremos conservar todas las filas, pero solo algunas columnas. Esta forma es muy clara cuando necesitamos seleccionar filas y columnas al mismo tiempo, como veremos a continuación.

Seleccionar filas y columnas con loc

La mayor ventaja de `loc` aparece cuando queremos seleccionar filas y columnas al mismo tiempo usando etiquetas.

La estructura general es:

```
df.loc[filas, columnas]
```

En la parte de filas indicamos qué etiquetas del índice queremos seleccionar. En la parte de columnas indicamos qué nombres de columnas queremos conservar.

Por ejemplo, podemos seleccionar las filas con etiquetas `0`, `1`, `2` y `3`, y quedarnos solamente con las columnas `producto` y `precio`.

```
df.loc[0:3, ["producto", "precio"]]
```

índice	producto	precio
0	Cuaderno	1200
1	Lapicera	350
2	Mochila	8500
3	Regla	500

Esta instrucción puede leerse así: selecciona las filas desde la etiqueta `0` hasta la etiqueta `3`, y de esas filas mostrará solamente las columnas `producto` y `precio`.

Recordemos que, con `loc`, el final del rango se incluye. Por eso `0:3` incluye las etiquetas `0`, `1`, `2` y `3`.

También podemos seleccionar filas puntuales y columnas puntuales:

```
df.loc[[0, 4, 8], ["producto", "sucursal", "fecha"]]
```

índice	producto	sucursal	fecha
0	Cuaderno	Centro	2024-03-01
4	Cartuchera	Norte	2024-03-03
8	Goma	Sur	2024-03-05

En este caso seleccionamos las filas con etiquetas `0`, `4` y `8`, y conservamos las columnas `producto`, `sucursal` y `fecha`.

Esta forma de selección es muy expresiva porque podemos leer con bastante claridad qué parte del dataset estamos pidiendo.

Por ejemplo:

```
df.loc[[0, 4, 8], ["producto", "sucursal", "fecha"]]
```

puede interpretarse como: “mostrá las ventas identificadas con las etiquetas 0, 4 y 8, pero solo con la información de producto, sucursal y fecha”.

A medida que los datasets crecen, esta claridad se vuelve cada vez más importante. Seleccionar por nombres de columnas suele ser más legible que seleccionar por posiciones numéricas, especialmente cuando no recordamos el orden exacto de las columnas.

Un ejemplo donde índice y posición no coinciden

Hasta ahora, nuestro `DataFrame` tiene un índice automático que va de 0 a 9. En ese caso, las etiquetas del índice coinciden visualmente con las posiciones de las filas. Pero esto no siempre ocurre.

Para verlo con más claridad, vamos a crear una copia del `DataFrame` y vamos a usar la columna `producto` como índice.

```
df_productos = df.set_index("producto")  
  
df_productos
```

producto	categoria	precio	cantidad_vendida	sucursal	fecha
Cuaderno	Libreria	1200	15	Centro	2024-03-01
Lapicera	Libreria	350	40	Centro	2024-03-01
Mochila	Accesorios	8500	5	Norte	2024-03-02
Regla	Libreria	500	25	Sur	2024-03-02
Cartuchera	Accesorios	3200	8	Norte	2024-03-03
Cuaderno	Libreria	1250	12	Sur	2024-03-03
Marcadores	Libreria	1800	10	Centro	2024-03-04
Carpeta	Libreria	950	18	Norte	2024-03-04
Goma	Libreria	300	30	Sur	2024-03-05
Lapiz	Libreria	250	50	Centro	2024-03-05

Ahora el índice ya no está formado por los números 0, 1, 2, 3, etc. El índice está formado por los nombres de los productos.

Esto cambia la forma en que usamos `loc`.

Por ejemplo, ahora podemos seleccionar las filas cuyo índice es "Cuaderno":

```
df_productos.loc["Cuaderno"]
```

producto	categoria	precio	cantidad_vendida	sucursal	fecha
Cuaderno	Libreria	1200	15	Centro	2024-03-01
Cuaderno	Libreria	1250	12	Sur	2024-03-03

La salida muestra las filas asociadas al producto "Cuaderno".

En este dataset, "Cuaderno" aparece más de una vez, por eso Pandas devuelve más de una fila. En cambio, si usamos `iloc`, seguimos seleccionando por posición:

```
df_productos.iloc[0]
```

índice	Cuaderno
categoria	Libreria
precio	1200
cantidad_vendida	15
sucursal	Centro
fecha	2024-03-01

`dtype: object`

Esta instrucción no busca el producto "Cuaderno" por su etiqueta. Simplemente selecciona la primera fila del `DataFrame`.

La diferencia central sigue siendo la misma:

```
loc → selecciona por etiqueta  
iloc → selecciona por posición
```

En este nuevo `DataFrame`, la diferencia se vuelve más visible porque las etiquetas del índice ya no son iguales a las posiciones numéricas.

Etiquetas repetidas en el índice

En el ejemplo anterior usamos `set_index("producto")` para crear un nuevo `DataFrame` llamado `df_productos`. Esto nos permitió usar los nombres de los productos como índice. Sin embargo, apareció una situación interesante: el producto "Cuaderno" aparece más de una vez en el dataset.

Eso significa que el índice tiene etiquetas repetidas. Cuando usamos:

```
df_productos.loc["Cuaderno"]
```

producto	categoria	precio	cantidad_vendida	sucursal	fecha
Cuaderno	Libreria	1200	15	Centro	2024-03-01
Cuaderno	Libreria	1250	12	Sur	2024-03-03

Pandas devuelve todas las filas cuyo índice tiene la etiqueta "Cuaderno".

Esto es importante porque `loc` no necesariamente devuelve una sola fila. Si la etiqueta buscada aparece una sola vez, puede devolver una fila. Si aparece varias veces, puede devolver varias filas.

Podemos probarlo con un producto que aparece una sola vez:

```
df_productos.loc["Mochila"]
```

índice	Mochila
categoria	Accesorios
precio	8500
cantidad_vendida	5
sucursal	Norte
fecha	2024-03-02

`dtype: object`

En este caso, como "Mochila" aparece una sola vez, Pandas devuelve una sola fila.

Este ejemplo nos ayuda a entender que el índice no siempre funciona como un identificador único. A veces puede tener valores repetidos. Eso no está necesariamente mal, pero debemos saberlo para interpretar correctamente el resultado de una selección.

Más adelante vamos a volver sobre el índice con más detalle. Por ahora, lo importante es recordar que `loc` busca etiquetas, y si una etiqueta aparece más de una vez, puede devolver más de una fila.

Errores frecuentes al usar loc

Cuando usamos `loc`, debemos recordar que estamos seleccionando por etiquetas. Esto significa que las filas se seleccionan usando etiquetas del índice y las columnas se seleccionan usando sus nombres.

Un error frecuente es intentar usar posiciones numéricas como si siempre fueran etiquetas. En nuestro primer `DataFrame`, esto parecía funcionar porque el índice estaba formado por números del 0 al 9. Pero cuando cambiamos el índice y usamos `producto` como etiqueta, la situación cambió.

Por ejemplo, en `df_productos`, el índice está formado por nombres de productos:

```
df_productos.head()
```

producto	categoria	precio	cantidad_vendida	sucursal	fecha
Cuaderno	Libreria	1200	15	Centro	2024-03-01
Lapicera	Libreria	350	40	Centro	2024-03-01
Mochila	Accesorios	8500	5	Norte	2024-03-02
Regla	Libreria	500	25	Sur	2024-03-02
Cartuchera	Accesorios	3200	8	Norte	2024-03-03

En ese `DataFrame`, esta instrucción no busca la primera fila:

```
# Esta instrucción produciría un error si no existe una etiqueta de índice llamada 0.  
# df_productos.loc[0]
```

Si queremos seleccionar la primera fila por posición, debemos usar `iloc`:

```
df_productos.iloc[0]
```

índice	Cuaderno
categoria	Libreria
precio	1200
cantidad_vendida	15
sucursal	Centro
fecha	2024-03-01

`dtype: object`

En cambio, si queremos seleccionar por etiqueta, usamos `loc` con una etiqueta existente del índice:

```
df_productos.loc["Cuaderno"]
```

producto	categoria	precio	cantidad_vendida	sucursal	fecha
Cuaderno	Libreria	1200	15	Centro	2024-03-01
Cuaderno	Libreria	1250	12	Sur	2024-03-03

Otro error frecuente es escribir mal el nombre de una columna. Como `loc` selecciona columnas por nombre, esos nombres deben coincidir exactamente con los del `DataFrame`.

Por ejemplo:

```
# Esta instrucción produciría un error porque la columna se llama "precio", no "Precio".
# df.loc[:, "Precio"]
```

Para evitar este tipo de problemas, conviene revisar los nombres disponibles antes de seleccionar:

```
df.columns
```

```
Index(['producto', 'categoria', 'precio', 'cantidad_vendida', 'sucursal',
      'fecha'],
      dtype='object')
```

La regla general es sencilla: si usamos `iloc`, pensamos en posiciones; si usamos `loc`, pensamos en etiquetas. Esa diferencia ayuda a decidir qué herramienta corresponde en cada situación.

Resumen del capítulo

En este capítulo aprendimos a seleccionar datos usando `loc`.

A diferencia de `iloc`, que trabaja con posiciones numéricas, `loc` trabaja con etiquetas. Esto significa que podemos seleccionar filas usando etiquetas del índice y columnas usando sus nombres.

Primero comparamos estas dos herramientas:

```
iloc selecciona por posición.
loc selecciona por etiqueta.
```

En nuestro `DataFrame` original, el índice estaba formado por números del 0 al 9, por eso al comienzo parecía que `iloc` y `loc` se comportaban de manera muy parecida. Sin embargo, la diferencia conceptual es importante: `df.iloc[0]` selecciona la

primera fila por posición, mientras que `df.loc[0]` selecciona la fila cuya etiqueta de índice es `0`.

También vimos que `loc` permite seleccionar varias filas usando listas de etiquetas o rangos. En ese punto apareció una diferencia clave con `iloc`: cuando usamos rangos con `loc`, el valor final del rango se incluye.

Por ejemplo:

```
df.loc[0:3]
```

selecciona las etiquetas `0`, `1`, `2` y `3`.

Después usamos `loc` para seleccionar columnas por nombre. Para eso trabajamos con la estructura:

```
df.loc[filas, columnas]
```

Cuando queremos conservar todas las filas y seleccionar una columna, podemos escribir:

```
df.loc[:, "precio"]
```

Y cuando queremos seleccionar varias columnas:

```
df.loc[:, ["producto", "precio"]]
```

Más adelante combinamos filas y columnas en una misma selección:

```
df.loc[0:3, ["producto", "precio"]]
```

Esta forma de trabajo es muy legible porque permite expresar directamente qué filas y qué columnas queremos observar.

También creamos un nuevo `DataFrame` usando `set_index("producto")` para mostrar un caso en el que índice y posición ya no coinciden. Ese ejemplo nos permitió ver con más claridad que `loc` busca etiquetas, mientras que `iloc` busca posiciones.

Una idea importante fue que el índice puede tener etiquetas repetidas. Cuando usamos `loc` con una etiqueta repetida, Pandas devuelve todas las filas que tienen esa etiqueta.

La idea principal de este capítulo fue:

`loc` permite seleccionar datos usando etiquetas del índice y nombres de columnas.

Esta herramienta será muy útil para construir selecciones más claras y expresivas, especialmente cuando trabajemos con datasets más grandes o con índices personalizados.

Próximo paso

Ya sabemos seleccionar columnas, seleccionar filas por posición con `iloc` y seleccionar datos por etiquetas con `loc`.

El siguiente paso será aprender a filtrar filas usando condiciones. Esto nos permitirá responder preguntas más específicas, como seleccionar solo las ventas de una sucursal, solo los productos de una categoría o solo los registros que superen cierto precio.

A partir de ahí, las selecciones dejarán de depender solamente de posiciones o etiquetas, y empezarán a depender del contenido real de los datos.

Capítulo 6 — Filtros con condiciones

Breve repaso

En los capítulos anteriores aprendimos distintas formas de seleccionar partes de un `DataFrame`.

Primero trabajamos con la selección de columnas. Vimos cómo elegir una columna individual, varias columnas al mismo tiempo y cómo distinguir entre una `Series` y un `DataFrame`.

Después aprendimos a seleccionar filas usando `iloc`, una herramienta que trabaja con posiciones numéricas. Con `iloc` pudimos seleccionar una fila, varias filas consecutivas, filas no consecutivas y combinaciones de filas y columnas según su ubicación dentro del `DataFrame`.

Más adelante incorporamos `loc`, que permite seleccionar datos usando etiquetas del índice y nombres de columnas. Esa herramienta nos permitió escribir selecciones más claras cuando conocemos los nombres de las variables que queremos observar.

En este capítulo vamos a dar un paso muy importante: aprender a filtrar filas usando condiciones.

Filtrar significa quedarse solamente con los registros que cumplen cierto criterio. Por ejemplo, podríamos querer ver solo las ventas realizadas en la sucursal "Centro", solo los productos de la categoría "Librería", solo las ventas con precio mayor a 1000, o solo los registros cuya cantidad vendida supere cierto valor.

Esta forma de selección es fundamental porque permite responder preguntas más específicas sobre los datos. Ya no elegimos filas por su posición ni por su etiqueta, sino por lo que esas filas contienen.

Al finalizar este capítulo deberías poder:

- Comprender qué significa filtrar filas en un `DataFrame`.
- Crear condiciones usando columnas de Pandas.
- Interpretar una máscara booleana.
- Filtrar filas que cumplen una condición.
- Filtrar valores numéricos.
- Filtrar valores categóricos o de texto.
- Combinar filtros con selección de columnas.

- Reconocer errores frecuentes al construir condiciones.

Dataset de trabajo

Para trabajar con filtros vamos a seguir usando el dataset de ventas de una librería. Cada fila representa una venta registrada y cada columna describe una característica de esa venta: el producto vendido, la categoría, el precio unitario, la cantidad vendida, la sucursal y la fecha.

Este dataset nos permite construir filtros simples y claros. Podemos seleccionar ventas según la sucursal, la categoría, el precio o la cantidad vendida. Así podremos concentrarnos en la lógica de los filtros sin cambiar todavía el contexto de trabajo.

```
import pandas as pd

datos = {
    "producto": [
        "Cuaderno", "Lapicera", "Mochila", "Regla", "Cartuchera",
        "Cuaderno", "Marcadores", "Carpeta", "Goma", "Lapiz"
    ],
    "categoria": [
        "Libreria", "Libreria", "Accesorios", "Libreria", "Accesorios",
        "Libreria", "Libreria", "Libreria", "Libreria", "Libreria"
    ],
    "precio": [
        1200, 350, 8500, 500, 3200,
        1250, 1800, 950, 300, 250
    ],
    "cantidad_vendida": [
        15, 40, 5, 25, 8,
        12, 10, 18, 30, 50
    ],
    "sucursal": [
        "Centro", "Centro", "Norte", "Sur", "Norte",
        "Sur", "Centro", "Norte", "Sur", "Centro"
    ],
    "fecha": [
        "2024-03-01", "2024-03-01", "2024-03-02", "2024-03-02", "2024-03-03",
        "2024-03-03", "2024-03-04", "2024-03-04", "2024-03-05", "2024-03-05"
    ]
}

df = pd.DataFrame(datos)

df
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	fecha
0	Cuaderno	Libreria	1200	15	Centro	2024-03-01
1	Lapicera	Libreria	350	40	Centro	2024-03-01

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
2	Mochila	Accesorios	8500	5	Norte	2024-03-02
3	Regla	Librería	500	25	Sur	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	2024-03-03
5	Cuaderno	Librería	1250	12	Sur	2024-03-03
6	Marcadores	Librería	1800	10	Centro	2024-03-04
7	Carpeta	Librería	950	18	Norte	2024-03-04
8	Goma	Librería	300	30	Sur	2024-03-05
9	Lapiz	Librería	250	50	Centro	2024-03-05

Al mostrar el `DataFrame`, podemos recordar su estructura general. Tenemos una tabla pequeña, con 10 ventas registradas y 6 columnas.

A partir de ahora, en lugar de seleccionar filas por posición o por etiqueta, vamos a seleccionar filas según condiciones. Es decir, vamos a pedirle a Pandas que nos muestre solamente los registros que cumplen cierto criterio.

Qué significa filtrar filas

Filtrar filas significa quedarnos solamente con los registros que cumplen una condición. En una tabla de datos, no siempre queremos trabajar con todas las filas. Muchas veces necesitamos observar un subconjunto del dataset. Por ejemplo, en nuestro caso podríamos querer ver solo las ventas de la sucursal "Centro", solo los productos de la categoría "Accesorios", solo las ventas con precio mayor a 1000, o solo los registros donde la cantidad vendida sea mayor que 20.

La idea general es sencilla: formulamos una condición y Pandas revisa fila por fila si esa condición se cumple. Las filas que cumplen la condición quedan seleccionadas. Las que no la cumplen quedan fuera del resultado.

Por ejemplo, una condición podría ser:

```
precio mayor que 1000
```

Otra condición podría ser:

```
sucursal igual a Centro
```

O también:

```
cantidad vendida mayor o igual que 20
```

Este tipo de selección es muy importante porque nos permite responder preguntas más específicas. En lugar de mirar toda la tabla, podemos concentrarnos en los casos que nos interesan.

Podemos pensar el filtrado de esta manera:

```
dataset completo → condición → filas que cumplen la condición
```

En Pandas, las condiciones se construyen a partir de columnas. Por eso, antes de filtrar necesitamos identificar qué columna contiene la información que queremos usar como criterio.

Construir una condición

Para filtrar filas en Pandas, primero necesitamos construir una condición.

Una condición es una expresión que puede evaluarse como verdadera o falsa para cada fila del `DataFrame`.

Por ejemplo, si queremos saber qué ventas tuvieron un precio mayor que `1000`, podemos comparar la columna `precio` con el valor `1000`.

```
df["precio"] > 1000
```

índice	precio
0	True
1	False
2	True
3	False
4	True
5	True
6	True
7	False
8	False
9	False

```
dtype: bool
```

La salida no muestra todavía las filas filtradas. Lo que vemos es una serie de valores `True` y `False`. Cada valor indica si la condición se cumple o no para una fila del `DataFrame`.

Cuando aparece `True`, significa que esa fila cumple la condición. Cuando aparece `False`, significa que no la cumple.

En este caso, Pandas revisa cada valor de la columna `precio` y pregunta:

```
¿este precio es mayor que 1000?
```

Si la respuesta es sí, devuelve `True`. Si la respuesta es no, devuelve `False`. Este resultado se llama **máscara booleana**.

Qué es una máscara booleana

Una **máscara booleana** es una serie de valores `True` y `False` que tiene la misma cantidad de filas que el `DataFrame`.

En el ejemplo anterior, la condición fue:

```
df["precio"] > 1000
```

índice	precio
0	True
1	False
2	True
3	False
4	True
5	True
6	True
7	False
8	False
9	False

```
dtype: bool
```

Pandas comparó cada valor de la columna `precio` con el número `1000` y produjo un resultado para cada fila.

Podemos guardar esa condición en una variable:

```
precios_mayores_a_1000 = df["precio"] > 1000  
  
precios_mayores_a_1000
```

índice	precio
0	True
1	False
2	True
3	False
4	True
5	True
6	True
7	False
8	False
9	False

`dtype: bool`

La variable `precios_mayores_a_1000` contiene la máscara booleana. Esta máscara no es todavía el dataset filtrado. Es solamente una serie que indica qué filas cumplen la condición.

Podemos leerla así:

```
True → la fila cumple la condición  
False → la fila no cumple la condición
```

Esta idea es fundamental para entender los filtros en Pandas. Primero construimos una condición. Esa condición genera una máscara booleana. Luego usamos esa máscara para seleccionar las filas que queremos conservar.

```
condición → máscara booleana → DataFrame filtrado
```

En la siguiente sección vamos a usar esta máscara para obtener solamente las ventas cuyo precio es mayor que `1000`.

Aplicar una máscara booleana

Una vez que tenemos una máscara booleana, podemos usarla para filtrar el `DataFrame`.

La idea es pedirle a Pandas que conserve solamente las filas donde la máscara tiene el valor `True`.

En nuestro caso, ya creamos una máscara llamada `precios_mayores_a_1000`. Ahora podemos usarla dentro de los corchetes del `DataFrame`.

```
df[precios_mayores_a_1000]
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
0	Cuaderno	Librería	1200	15	Centro	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	2024-03-03
5	Cuaderno	Librería	1250	12	Sur	2024-03-03
6	Marcadores	Librería	1800	10	Centro	2024-03-04

El resultado es un nuevo `DataFrame` que contiene solamente las ventas cuyo precio es mayor que `1000`. Las filas donde la condición era `True` quedaron incluidas. Las filas donde la condición era `False` no aparecen en el resultado.

También podemos escribir el filtro directamente, sin guardar antes la máscara en una variable:

```
df[df["precio"] > 1000]
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
0	Cuaderno	Librería	1200	15	Centro	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	2024-03-03
5	Cuaderno	Librería	1250	12	Sur	2024-03-03
6	Marcadores	Librería	1800	10	Centro	2024-03-04

Esta forma es muy frecuente en Pandas. Podemos leerla en dos partes. Primero, la condición:

```
df["precio"] > 1000
```

produce una máscara booleana. Luego, esa máscara se usa para filtrar el `DataFrame`:

```
df[ ... ]
```

Entonces, la expresión completa:

```
df[df["precio"] > 1000]
```

significa: mostrará solamente las filas de `df` donde la columna `precio` sea mayor que `1000`. Esta estructura será una de las más usadas cuando trabajemos con filtros.

Filtros con valores numéricos

Los filtros numéricos permiten seleccionar filas según el valor de una columna numérica. En nuestro dataset tenemos dos columnas numéricas principales: `precio` y `cantidad_vendida`. Podemos construir condiciones usando operadores de comparación como mayor que, menor que, igual, distinto, mayor o igual, y menor o igual.

Por ejemplo, podemos seleccionar las ventas con precio menor que `1000`.

```
df[df["precio"] < 1000]
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
1	Lapicera	Librería	350	40	Centro	2024-03-01
3	Regla	Librería	500	25	Sur	2024-03-02
7	Carpeta	Librería	950	18	Norte	2024-03-04
8	Goma	Librería	300	30	Sur	2024-03-05
9	Lapiz	Librería	250	50	Centro	2024-03-05

En este caso, Pandas conserva solamente las filas donde el valor de la columna `precio` es menor que `1000`.

También podemos seleccionar ventas con precio mayor o igual que `1000`:

```
df[df["precio"] >= 1000]
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
0	Cuaderno	Librería	1200	15	Centro	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	2024-03-03
5	Cuaderno	Librería	1250	12	Sur	2024-03-03
6	Marcadores	Librería	1800	10	Centro	2024-03-04

O ventas cuya cantidad vendida sea mayor o igual que 20 :

```
df[df["cantidad_vendida"] >= 20]
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
1	Lapicera	Librería	350	40	Centro	2024-03-01
3	Regla	Librería	500	25	Sur	2024-03-02
8	Goma	Librería	300	30	Sur	2024-03-05
9	Lapiz	Librería	250	50	Centro	2024-03-05

Los operadores de comparación más usados son:

Operador	Significado
>	Mayor que
<	Menor que
>=	Mayor o igual que
<=	Menor o igual que
==	Igual a
!=	Distinto de

Es importante prestar atención al operador que usamos. No es lo mismo preguntar por precios mayores que 1000 que por precios mayores o iguales que 1000. En el primer caso, el valor 1000 queda afuera. En el segundo, quedaría incluido si existiera en el dataset.

Los filtros numéricos son muy útiles para encontrar registros que superan cierto umbral, detectar valores bajos o altos, o concentrarnos en una parte específica del dataset.

Filtros con valores categóricos

Además de filtrar columnas numéricas, también podemos filtrar columnas categóricas o de texto. En nuestro dataset, algunas columnas de este tipo son `producto`, `categoría` y `sucursal`.

Por ejemplo, si queremos ver solamente las ventas realizadas en la sucursal "Centro", podemos construir una condición sobre la columna `sucursal`.

```
df[df["sucursal"] == "Centro"]
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
0	Cuaderno	Librería	1200	15	Centro	2024-03-01
1	Lapicera	Librería	350	40	Centro	2024-03-01
6	Marcadores	Librería	1800	10	Centro	2024-03-04
9	Lapiz	Librería	250	50	Centro	2024-03-05

En este caso usamos el operador `==`, que significa "igual a". La condición:

```
df["sucursal"] == "Centro"
```

revisa fila por fila si el valor de la columna `sucursal` es igual a "Centro". Las filas donde esa condición se cumple quedan incluidas en el resultado.

También podemos filtrar por categoría. Por ejemplo, podemos seleccionar solamente los productos de la categoría "Accesorios".

```
df[df["categoría"] == "Accesorios"]
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
2	Mochila	Accesorios	8500	5	Norte	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	2024-03-03

El resultado muestra únicamente las filas cuya categoría es "Accesorios".

Cuando filtramos textos, debemos escribir el valor exactamente como aparece en el `DataFrame`. Pandas distingue entre mayúsculas y minúsculas, y también tiene en cuenta espacios o diferencias de escritura.

Por ejemplo, "Centro" no es lo mismo que "centro", y "Accesorios" no es lo mismo que "accesorios".

Si no recordamos exactamente qué valores aparecen en una columna, podemos revisarlos con `unique()`:

```
df["sucursal"].unique()
```

```
array(['Centro', 'Norte', 'Sur'], dtype=object)
```

```
df["categoria"].unique()
```

```
array(['Libreria', 'Accesorios'], dtype=object)
```

Esta revisión es útil antes de construir un filtro, especialmente cuando trabajamos con datasets reales. A veces una misma categoría puede aparecer escrita de varias formas, y eso puede afectar los resultados del análisis.

Filtrar valores distintos

Además de seleccionar filas donde una columna sea igual a cierto valor, también podemos hacer lo contrario: seleccionar las filas donde una columna sea distinta de cierto valor. Para eso usamos el operador `!=`.

Por ejemplo, si queremos ver todas las ventas que no fueron realizadas en la sucursal "Centro", podemos escribir:

```
df[df["sucursal"] != "Centro"]
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	fecha
2	Mochila	Accesorios	8500	5	Norte	2024-03-02
3	Regla	Libreria	500	25	Sur	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	2024-03-03
5	Cuaderno	Libreria	1250	12	Sur	2024-03-03
7	Carpeta	Libreria	950	18	Norte	2024-03-04
8	Goma	Libreria	300	30	Sur	2024-03-05

La condición:

```
df["sucursal"] != "Centro"
```

significa: la columna `sucursal` debe ser distinta de `"Centro"`. El resultado conserva solamente las filas donde la sucursal no es `"Centro"`.

También podríamos seleccionar los registros que no pertenecen a la categoría `"Libreria"`:

```
df[df["categoria"] != "Libreria"]
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	fecha
2	Mochila	Accesorios	8500	5	Norte	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	2024-03-03

En nuestro dataset, esta selección devuelve las ventas de productos que pertenecen a otra categoría.

Este tipo de filtro es útil cuando queremos excluir un grupo del análisis. Por ejemplo, podríamos querer analizar todas las sucursales excepto una, todos los productos que no pertenecen a cierta categoría o todos los registros que no tienen determinado valor.

Como siempre, es importante revisar primero qué valores existen en la columna para evitar errores de escritura. Una diferencia de mayúsculas, minúsculas o espacios puede hacer que el filtro no devuelva el resultado esperado.

Filtrar filas y seleccionar columnas

Cuando aplicamos un filtro, Pandas devuelve todas las columnas de las filas que cumplen la condición.

Por ejemplo:

```
df[df["sucursal"] == "Centro"]
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	fecha
0	Cuaderno	Libreria	1200	15	Centro	2024-03-01
1	Lapicera	Libreria	350	40	Centro	2024-03-01
6	Marcadores	Libreria	1800	10	Centro	2024-03-04
9	Lapiz	Libreria	250	50	Centro	2024-03-05

Esta instrucción muestra todas las columnas de las ventas realizadas en la sucursal "Centro". Pero muchas veces no necesitamos ver todas las columnas. Tal vez queremos filtrar las filas y, al mismo tiempo, mostrar solamente algunas variables.

Para eso podemos combinar un filtro con selección de columnas usando `loc`.

La estructura general es:

```
df.loc[condición, columnas]
```

Por ejemplo, podemos seleccionar las ventas de la sucursal "Centro" y mostrar solo las columnas `producto`, `precio` y `cantidad_vendida`.

```
df.loc[df["sucursal"] == "Centro", ["producto", "precio", "cantidad_vendida"]]
```

índice	producto	precio	cantidad_vendida
0	Cuaderno	1200	15
1	Lapicera	350	40
6	Marcadores	1800	10
9	Lapiz	250	50

Esta instrucción puede leerse así: del `DataFrame df`, seleccioná las filas donde la columna `sucursal` sea igual a "Centro" y mostrará solamente las columnas `producto`, `precio` y `cantidad_vendida`.

También podemos filtrar por categoría y mostrar solo algunas columnas:

```
df.loc[df["categoria"] == "Accesorios", ["producto", "precio", "sucursal"]]
```

índice	producto	precio	sucursal
2	Mochila	8500	Norte
4	Cartuchera	3200	Norte

Esta forma de trabajo es muy útil porque nos permite obtener una tabla más enfocada. En lugar de ver todas las columnas del dataset, nos quedamos solamente con las que aportan información para la pregunta que estamos intentando responder.

La combinación entre filtros y selección de columnas será una herramienta central en el trabajo con Pandas.

Filtrar valores incluidos en una lista

A veces no queremos filtrar por un único valor, sino por varios valores posibles. Por ejemplo, podríamos querer seleccionar las ventas realizadas en las sucursales "Centro" o "Norte".

Una forma cómoda de hacerlo es usar `isin()`.

El método `isin()` permite verificar si cada valor de una columna pertenece a una lista de valores permitidos.

```
df["sucursal"].isin(["Centro", "Norte"])
```

índice	sucursal
0	True
1	True
2	True
3	False
4	True
5	False
6	True
7	True
8	False
9	True

`dtype: bool`

La salida es una máscara booleana. Cada fila recibe `True` si su sucursal está dentro de la lista `["Centro", "Norte"]`, y `False` si no lo está.

Luego podemos usar esa máscara para filtrar el `DataFrame`.

```
df[df["sucursal"].isin(["Centro", "Norte"])]
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	fecha
0	Cuaderno	Libreria	1200	15	Centro	2024-03-01
1	Lapicera	Libreria	350	40	Centro	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	2024-03-02

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
4	Cartuchera	Accesorios	3200	8	Norte	2024-03-03
6	Marcadores	Librería	1800	10	Centro	2024-03-04
7	Carpeta	Librería	950	18	Norte	2024-03-04
9	Lapiz	Librería	250	50	Centro	2024-03-05

El resultado conserva solamente las ventas realizadas en las sucursales "Centro" y "Norte".

También podemos aplicar `isin()` sobre otra columna. Por ejemplo, podemos seleccionar solamente algunos productos específicos:

```
df[df["producto"].isin(["Cuaderno", "Lapicera", "Lapiz"])]
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
0	Cuaderno	Librería	1200	15	Centro	2024-03-01
1	Lapicera	Librería	350	40	Centro	2024-03-01
5	Cuaderno	Librería	1250	12	Sur	2024-03-03
9	Lapiz	Librería	250	50	Centro	2024-03-05

Esta instrucción devuelve las filas cuyo producto es "Cuaderno", "Lapicera" o "Lapiz".

Usar `isin()` suele ser más claro que escribir muchas comparaciones separadas. En lugar de preguntar varias veces si una columna es igual a un valor, construimos una lista con los valores que nos interesan y le pedimos a Pandas que conserve las filas que pertenecen a esa lista.

Esta herramienta es especialmente útil cuando trabajamos con categorías, sucursales, provincias, productos, cursos, grupos o cualquier columna que pueda contener valores repetidos.

Filtrar textos que contienen una palabra

En algunas situaciones queremos seleccionar filas donde una columna de texto contenga cierta palabra o fragmento.

Para eso podemos usar `.str.contains()`.

Por ejemplo, si queremos buscar productos cuyo nombre contiene la letra "a", podemos escribir:

```
df[df["producto"].str.contains("a")]
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
0	Cuaderno	Librería	1200	15	Centro	2024-03-01
1	Lapicera	Librería	350	40	Centro	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	2024-03-02
3	Regla	Librería	500	25	Sur	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	2024-03-03
5	Cuaderno	Librería	1250	12	Sur	2024-03-03
6	Marcadores	Librería	1800	10	Centro	2024-03-04
7	Carpeta	Librería	950	18	Norte	2024-03-04
8	Goma	Librería	300	30	Sur	2024-03-05
9	Lapiz	Librería	250	50	Centro	2024-03-05

La condición:

```
df["producto"].str.contains("a")
```

revisa cada valor de la columna `producto` y devuelve `True` cuando encuentra la letra `"a"` dentro del texto.

También podemos buscar un fragmento más específico. Por ejemplo:

```
df[df["producto"].str.contains("Cuad")]
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
0	Cuaderno	Librería	1200	15	Centro	2024-03-01
5	Cuaderno	Librería	1250	12	Sur	2024-03-03

En este caso, Pandas devuelve las filas donde el nombre del producto contiene el fragmento `"Cuad"`.

Debemos tener en cuenta que, por defecto, esta búsqueda distingue entre mayúsculas y minúsculas. Es decir, buscar `"a"` no es lo mismo que buscar `"A"`.

Podemos comprobarlo con este ejemplo:

```
df[df["producto"].str.contains("A")]
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
--------	----------	-----------	--------	------------------	----------	-------

Como los nombres de productos de nuestro dataset comienzan con mayúscula, una búsqueda con "A" podría devolver resultados diferentes de una búsqueda con "a".

Si queremos hacer una búsqueda sin distinguir entre mayúsculas y minúsculas, podemos usar el parámetro `case=False`.

```
df[df["producto"].str.contains("a", case=False)]
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
0	Cuaderno	Librería	1200	15	Centro	2024-03-01
1	Lapicera	Librería	350	40	Centro	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	2024-03-02
3	Regla	Librería	500	25	Sur	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	2024-03-03
5	Cuaderno	Librería	1250	12	Sur	2024-03-03
6	Marcadores	Librería	1800	10	Centro	2024-03-04
7	Carpeta	Librería	950	18	Norte	2024-03-04
8	Goma	Librería	300	30	Sur	2024-03-05
9	Lapiz	Librería	250	50	Centro	2024-03-05

Este tipo de filtro es útil cuando queremos buscar palabras, fragmentos de texto, códigos o patrones simples dentro de una columna.

Más adelante vamos a trabajar con operaciones de texto con mayor detalle. Por ahora, nos alcanza con reconocer que Pandas también permite construir condiciones a partir de columnas textuales.

Errores frecuentes al construir filtros

Al construir filtros en Pandas, es común cometer algunos errores. La mayoría no son graves, pero pueden generar mensajes difíciles de interpretar al principio.

Un error frecuente es usar un solo signo igual `=` en lugar de doble signo igual `==`.

En Python, el signo `=` se usa para asignar un valor a una variable. En cambio, `==` se usa para comparar si dos valores son iguales.

Por eso, para filtrar las ventas de la sucursal "Centro", escribimos:

```
df[df["sucursal"] == "Centro"]
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
0	Cuaderno	Librería	1200	15	Centro	2024-03-01
1	Lapicera	Librería	350	40	Centro	2024-03-01
6	Marcadores	Librería	1800	10	Centro	2024-03-04
9	Lapiz	Librería	250	50	Centro	2024-03-05

y no:

```
# Esta instrucción produciría un error porque = no se usa para comparar.  
# df[df["sucursal"] = "Centro"]
```

Otro error frecuente es olvidar las comillas cuando filtramos valores de texto. Si queremos comparar con "Centro", ese valor debe escribirse como texto, entre comillas.

```
```python  
Esta instrucción produciría un error porque Centro no está escrito como texto.
df[df["sucursal"] == Centro]
```

También puede ocurrir que escribamos un valor de texto de una forma distinta a como aparece en el dataset. Por ejemplo:

```
df[df["sucursal"] == "centro"]
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	fecha
--------	----------	-----------	--------	------------------	----------	-------

Esta instrucción no produce un error, pero devuelve un resultado vacío, porque en nuestro dataset la sucursal está escrita como "Centro" con mayúscula inicial.

En estos casos, conviene revisar los valores disponibles antes de filtrar:

```
df["sucursal"].unique()
```

```
array(['Centro', 'Norte', 'Sur'], dtype=object)
```

También debemos prestar atención a los nombres de las columnas. Si escribimos mal el nombre de una columna, Pandas no podrá encontrarla.

```
Esta instrucción produciría un error porque la columna se llama "sucursal", no
"Sucursal".
df[df["Sucursal"] == "Centro"]
```

Una buena estrategia para evitar estos errores es revisar primero los nombres de las columnas y los valores disponibles en la columna que queremos usar para filtrar.

```
df.columns
```

```
Index(['producto', 'categoria', 'precio', 'cantidad_vendida', 'sucursal',
 'fecha'],
 dtype='object')
```

```
df["sucursal"].unique()
```

```
array(['Centro', 'Norte', 'Sur'], dtype=object)
```

Los filtros son una herramienta muy poderosa, pero requieren precisión. El nombre de la columna, el operador de comparación y el valor buscado deben estar escritos correctamente para que el resultado sea el esperado.

## Resumen del capítulo

---

En este capítulo aprendimos a filtrar filas de un `DataFrame` usando condiciones.

Hasta ahora habíamos seleccionado columnas, filas por posición con `iloc` y datos por etiquetas con `loc`. Con los filtros dimos un paso más: empezamos a seleccionar filas según el contenido real de los datos.

Vimos que una condición en Pandas produce una máscara booleana, es decir, una serie de valores `True` y `False`. Cada valor indica si una fila cumple o no cumple la condición planteada.

Por ejemplo:

```
df["precio"] > 1000
```

genera una máscara booleana que indica qué ventas tienen un precio mayor que `1000`.

Luego usamos esa máscara para filtrar el `DataFrame`:

```
df[df["precio"] > 1000]
```

Esta instrucción conserva solamente las filas donde la condición es verdadera.

También trabajamos con filtros numéricos usando operadores como:

```
>
<
>=
<=
==
!=
```

Estos operadores nos permitieron seleccionar ventas según precio o cantidad vendida.

Después aplicamos filtros sobre columnas categóricas o de texto, como `sucursal`, `categoria` y `producto`. Vimos que, al filtrar textos, debemos escribir los valores exactamente como aparecen en el dataset, respetando mayúsculas, minúsculas y espacios.

También usamos `isin()` para seleccionar filas cuyos valores pertenecen a una lista de opciones posibles:

```
df[df["sucursal"].isin(["Centro", "Norte"])]
```

Más adelante vimos que `.str.contains()` permite construir filtros a partir de fragmentos de texto:

```
df[df["producto"].str.contains("a", case=False)]
```

Finalmente, combinamos filtros con selección de columnas usando `loc`:

```
df.loc[df["sucursal"] == "Centro", ["producto", "precio", "cantidad_vendida"]]
```

Esta forma de trabajo nos permite obtener resultados más enfocados: no solo filtramos las filas que cumplen una condición, sino que también elegimos qué columnas queremos ver.

La idea principal de este capítulo fue:

```
Filtrar permite seleccionar filas según condiciones construidas a partir de los datos.
```

Los filtros son una de las herramientas más importantes de Pandas, porque permiten pasar de una tabla completa a un subconjunto específico de registros que cumplen ciertos criterios.

## Próximo paso

---

Ya sabemos seleccionar columnas, seleccionar filas por posición, seleccionar datos por etiquetas y filtrar filas usando condiciones simples.

El siguiente paso será aprender a combinar varias condiciones en un mismo filtro. Eso nos permitirá responder preguntas más precisas, como seleccionar ventas de una sucursal determinada cuyo precio supere cierto valor, o productos de una categoría específica con una cantidad vendida mayor que cierto límite.

Para eso vamos a trabajar con operadores lógicos, como `&`, `|` y `~`, y veremos por qué en Pandas es tan importante usar paréntesis al combinar condiciones.

## Capítulo 7 — Combinar condiciones en filtros

---

### Breve repaso

---

En el trabajo anterior sobre filtros aprendimos a seleccionar filas de un `DataFrame` usando condiciones simples.

Vimos que una condición, como `df["precio"] > 1000`, genera una máscara booleana: una serie de valores `True` y `False` que indica qué filas cumplen la condición y cuáles no. Luego usamos esa máscara para obtener solamente las filas que nos interesaban.

También filtramos columnas numéricas, categóricas y de texto. Por ejemplo, seleccionamos ventas según su precio, sucursal, categoría o producto. Además, combinamos filtros con selección de columnas usando `loc`, lo que nos permitió obtener resultados más enfocados.

En este capítulo vamos a avanzar un paso más: vamos a combinar varias condiciones dentro de un mismo filtro.

Esto nos permitirá responder preguntas más precisas, como:

```
¿Qué ventas de Tecnología tuvieron un precio mayor que 8000?
¿Qué ventas de la sucursal Centro se pagaron con crédito?
¿Qué productos que no son de Librería tuvieron pocas unidades vendidas?
¿Qué ventas corresponden a la sucursal Centro o a la sucursal Norte?
```

Para construir este tipo de filtros vamos a usar operadores lógicos:

```
& para expresar "y"
| para expresar "o"
~ para expresar "no"
```

También vamos a ver por qué, al combinar condiciones en Pandas, es necesario encerrar cada condición entre paréntesis.

Al finalizar este capítulo deberías poder:

- Combinar dos o más condiciones dentro de un filtro.
- Usar `&` para seleccionar filas que cumplen dos condiciones al mismo tiempo.
- Usar `|` para seleccionar filas que cumplen al menos una de varias condiciones.
- Usar `~` para negar una condición.
- Comprender por qué los paréntesis son necesarios al combinar condiciones.



- Guardar condiciones en variables para escribir filtros más claros.
- Combinar filtros con selección de columnas usando `loc`.
- Reconocer errores frecuentes al combinar condiciones.

## Dataset de trabajo

Para este capítulo vamos a usar un dataset de ventas de una tienda escolar. El contexto es parecido al que veníamos usando, pero ahora incorporamos más productos, más categorías y una nueva columna llamada `forma_pago`. Esto nos permitirá construir filtros combinados más interesantes.

Cada fila representa una venta registrada. Las columnas indican el producto vendido, su categoría, el precio unitario, la cantidad vendida, la sucursal, la forma de pago y la fecha de la venta.

```
import pandas as pd

datos = {
 "producto": [
 "Cuaderno", "Lapicera", "Mochila", "Regla", "Cartuchera",
 "Calculadora", "Auriculares", "Resaltadores", "Guardapolvo", "Compas",
 "Pendrive", "Carpeta", "Goma", "Lapiz", "Agenda"
],
 "categoria": [
 "Libreria", "Libreria", "Accesorios", "Libreria", "Accesorios",
 "Tecnologia", "Tecnologia", "Libreria", "Indumentaria", "Libreria",
 "Tecnologia", "Libreria", "Libreria", "Libreria", "Libreria"
],
 "precio": [
 1200, 350, 8500, 500, 3200,
 18500, 7600, 2100, 14500, 2600,
 9800, 950, 300, 250, 4300
],
 "cantidad_vendida": [
 15, 40, 5, 25, 8,
 3, 6, 12, 4, 10,
 7, 18, 30, 50, 9
],
 "sucursal": [
 "Centro", "Centro", "Norte", "Sur", "Norte",
 "Centro", "Sur", "Centro", "Norte", "Sur",
 "Centro", "Norte", "Sur", "Centro", "Sur"
],
 "forma_pago": [
 "Efectivo", "Debito", "Credito", "Efectivo", "Debito",
 "Credito", "Credito", "Debito", "Credito", "Efectivo",
 "Debito", "Efectivo", "Efectivo", "Debito", "Credito"
],
 "fecha": [
 "2024-03-01", "2024-03-01", "2024-03-02", "2024-03-02", "2024-03-03",

```

```

 "2024-03-03", "2024-03-04", "2024-03-04", "2024-03-05", "2024-03-05",
 "2024-03-06", "2024-03-06", "2024-03-07", "2024-03-07", "2024-03-08"
]
}

df = pd.DataFrame(datos)

df

```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
0	Cuaderno	Librería	1200	15	Centro	Efectivo	2024-03-01
1	Lapicera	Librería	350	40	Centro	Debito	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
3	Regla	Librería	500	25	Sur	Efectivo	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
5	Calculadora	Tecnología	18500	3	Centro	Credito	2024-03-03
6	Auriculares	Tecnología	7600	6	Sur	Credito	2024-03-04
7	Resaltadores	Librería	2100	12	Centro	Debito	2024-03-04
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
9	Compas	Librería	2600	10	Sur	Efectivo	2024-03-05
10	Pendrive	Tecnología	9800	7	Centro	Debito	2024-03-06
11	Carpeta	Librería	950	18	Norte	Efectivo	2024-03-06
12	Goma	Librería	300	30	Sur	Efectivo	2024-03-07
13	Lapiz	Librería	250	50	Centro	Debito	2024-03-07
14	Agenda	Librería	4300	9	Sur	Credito	2024-03-08

El dataset tiene 15 filas y 7 columnas. Antes de construir filtros combinados, conviene observar qué columnas tenemos disponibles y qué tipo de preguntas podríamos hacer con ellas. Por ejemplo, ahora podemos combinar condiciones sobre `categoría`, `precio`, `cantidad_vendida`, `sucursal` y `forma_pago`.

Esa combinación de criterios es lo que nos permitirá construir consultas más precisas.

## Recordar un filtro simple

Antes de combinar condiciones, recordemos cómo se construye un filtro simple.

Si queremos seleccionar solamente las ventas de la categoría `"Tecnología"`, podemos escribir:

```
df[df["categoria"] == "Tecnologia"]
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2024-03-04
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06

La condición:

```
df["categoria"] == "Tecnologia"
```

genera una máscara booleana. Esa máscara indica, fila por fila, si la venta pertenece o no a la categoría "Tecnologia".

También podemos filtrar por precio. Por ejemplo, si queremos seleccionar las ventas con precio mayor que 8000, escribimos:

```
df[df["precio"] > 8000]
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06

En ambos casos usamos una sola condición. El primer filtro selecciona filas según una categoría. El segundo selecciona filas según un valor numérico. Pero muchas preguntas reales requieren más de una condición. Por ejemplo, podríamos querer seleccionar solamente las ventas de "Tecnologia" cuyo precio sea mayor que 8000.

Para responder esa pregunta necesitamos combinar condiciones.

## Combinar condiciones con &

Cuando queremos que dos condiciones se cumplan al mismo tiempo, usamos el operador &. Por ejemplo, supongamos que queremos seleccionar las ventas que cumplen estas dos condiciones:

```
la categoría es Tecnologia
el precio es mayor que 8000
```

Primero podemos mirar cada condición por separado.

```
df["categoria"] == "Tecnologia"
```

índice	categoria
0	False
1	False
2	False
3	False
4	False
5	True
6	True
7	False
8	False
9	False
10	True
11	False
12	False
13	False
14	False

```
dtype: bool
```

```
df["precio"] > 8000
```

índice	precio
0	False
1	False
2	True
3	False
4	False
5	True
6	False
7	False

índice	precio
8	True
9	False
10	True
11	False
12	False
13	False
14	False

`dtype: bool`

Cada una de esas expresiones produce una máscara booleana. Para filtrar las filas que cumplen ambas condiciones al mismo tiempo, debemos combinar esas máscaras usando `&`.

En Pandas, cuando combinamos condiciones, cada condición debe ir encerrada entre paréntesis.

```
df[(df["categoria"] == "Tecnologia") & (df["precio"] > 8000)]
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06

El resultado muestra solamente las filas donde se cumplen las dos condiciones al mismo tiempo. Es decir, Pandas conserva las ventas que pertenecen a la categoría "Tecnologia" y que, además, tienen un precio mayor que 8000.

Podemos leer la instrucción de esta manera:

```
mostrá las filas donde
categoria sea igual a Tecnologia
y
precio sea mayor que 8000
```

El operador `&` funciona como un "y lógico". Para que una fila aparezca en el resultado, ambas condiciones deben ser verdaderas.

## Por qué necesitamos paréntesis

---

Al combinar condiciones en Pandas, cada condición debe escribirse entre paréntesis. La forma correcta es:

```
df[(df["categoria"] == "Tecnologia") & (df["precio"] > 8000)]
```

Los paréntesis le indican a Python qué comparación debe resolver primero. Sin ellos, la expresión puede interpretarse de una forma distinta a la que esperamos y producir un error.

Por ejemplo, esta forma no es recomendable:

```
Esta instrucción produciría un error porque faltan paréntesis.
df[df["categoria"] == "Tecnologia" & df["precio"] > 8000]
```

El problema es que Python no interpreta automáticamente la expresión como “categoría igual a Tecnología y precio mayor que 8000”. Al faltar los paréntesis, intenta resolver partes de la expresión en un orden que no corresponde para este tipo de filtro.

Por eso, cuando combinemos condiciones en Pandas, vamos a usar siempre esta estructura:

```
df[(condición_1) & (condición_2)]
```

Podemos verlo como una regla práctica:

```
cada condición va entre paréntesis
```

Esta costumbre evita muchos errores y hace que el filtro sea más claro de leer.

## Más ejemplos con &

---

El operador `&` sirve para construir filtros donde todas las condiciones deben cumplirse al mismo tiempo.

Por ejemplo, podemos seleccionar las ventas realizadas en la sucursal "Centro" que fueron pagadas con "Credito".

```
df[(df["sucursal"] == "Centro") & (df["forma_pago"] == "Credito")]
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
5	Calculadora	Tecnología	18500	3	Centro	Credito	2024-03-03

En este caso, una fila aparece en el resultado solamente si cumple las dos condiciones: la venta debe haberse realizado en la sucursal "Centro" y la forma de pago debe ser "Credito".

También podemos combinar una condición categórica con una condición numérica. Por ejemplo, podemos seleccionar los productos de "Librería" cuya cantidad vendida sea mayor o igual que 20.

```
df[(df["categoria"] == "Libreria") & (df["cantidad_vendida"] >= 20)]
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
1	Lapicera	Librería	350	40	Centro	Debito	2024-03-01
3	Regla	Librería	500	25	Sur	Efectivo	2024-03-02
12	Goma	Librería	300	30	Sur	Efectivo	2024-03-07
13	Lapiz	Librería	250	50	Centro	Debito	2024-03-07

Este filtro conserva únicamente las filas donde la categoría es "Librería" y la cantidad vendida es mayor o igual que 20.

Otro ejemplo posible sería seleccionar las ventas de la sucursal "Norte" con precio mayor que 5000:

```
df[(df["sucursal"] == "Norte") & (df["precio"] > 5000)]
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05

Estos ejemplos muestran que podemos combinar condiciones sobre columnas de distinto tipo. Una condición puede referirse a una categoría, otra a un número, otra a una forma de pago o a cualquier otra variable del dataset.

La clave es que, cuando usamos &, todas las condiciones deben cumplirse al mismo tiempo para que la fila quede incluida en el resultado.

## Combinar condiciones con |

Cuando queremos seleccionar filas que cumplan una condición u otra, usamos el operador `|`. Este operador funciona como un “o lógico”. Una fila queda incluida si cumple al menos una de las condiciones.

Por ejemplo, supongamos que queremos seleccionar las ventas realizadas en la sucursal "Centro" o en la sucursal "Norte".

Podemos escribir:

```
df[(df["sucursal"] == "Centro") | (df["sucursal"] == "Norte")]
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
0	Cuaderno	Librería	1200	15	Centro	Efectivo	2024-03-01
1	Lapicera	Librería	350	40	Centro	Debito	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
5	Calculadora	Tecnología	18500	3	Centro	Credito	2024-03-03
7	Resaltadores	Librería	2100	12	Centro	Debito	2024-03-04
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
10	Pendrive	Tecnología	9800	7	Centro	Debito	2024-03-06
11	Carpeta	Librería	950	18	Norte	Efectivo	2024-03-06
13	Lapiz	Librería	250	50	Centro	Debito	2024-03-07

El resultado incluye las filas donde la sucursal es "Centro" y también las filas donde la sucursal es "Norte".

En este caso no hace falta que una fila cumpla ambas condiciones al mismo tiempo. De hecho, una venta no puede pertenecer simultáneamente a la sucursal "Centro" y a la sucursal "Norte". Al usar `|`, alcanza con que se cumpla una de las dos condiciones.

Podemos leer la instrucción así:

```
mostrá las filas donde
sucursal sea Centro
o
sucursal sea Norte
```



También podemos usar `|` con otras columnas. Por ejemplo, podemos seleccionar las ventas pagadas con "Credito" o "Debito":

```
df[(df["forma_pago"] == "Credito") | (df["forma_pago"] == "Debito")]
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2024-03-04
7	Resaltadores	Libreria	2100	12	Centro	Debito	2024-03-04
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07
14	Agenda	Libreria	4300	9	Sur	Credito	2024-03-08

En este caso, el resultado excluye las ventas pagadas en efectivo y conserva solamente las que fueron pagadas con crédito o débito.

Al igual que con `&`, cada condición debe ir entre paréntesis.

## Diferencia entre `&` y `|`

Los operadores `&` y `|` permiten combinar condiciones, pero no significan lo mismo.

Usamos `&` cuando queremos que todas las condiciones se cumplan al mismo tiempo. En cambio, usamos `|` cuando alcanza con que se cumpla al menos una de las condiciones.

Por ejemplo, este filtro usa `&`:

```
df[(df["sucursal"] == "Centro") & (df["forma_pago"] == "Credito")]
```

El resultado incluye solamente las ventas que cumplen ambas condiciones: fueron realizadas en la sucursal "Centro" y fueron pagadas con "Credito".

En cambio, este filtro usa `|`:

```
df[(df["sucursal"] == "Centro") | (df["forma_pago"] == "Credito")]
```

El resultado incluye las ventas que cumplen al menos una de las dos condiciones: ventas realizadas en la sucursal "Centro", ventas pagadas con "Credito", o ventas que cumplen ambas condiciones al mismo tiempo.

La diferencia puede resumirse así:

```
& → deben cumplirse todas las condiciones
| → debe cumplirse al menos una condición
```

Esta diferencia es muy importante porque puede cambiar mucho la cantidad de filas obtenidas. En general, los filtros con & suelen ser más restrictivos, porque exigen que se cumplan varios criterios al mismo tiempo. Los filtros con | suelen ser más amplios, porque aceptan filas que cumplan cualquiera de las condiciones indicadas.

## Negar una condición con ~

A veces no queremos seleccionar las filas que cumplen una condición, sino justamente las que no la cumplen. Para eso usamos el operador ~, que permite negar una condición.

Por ejemplo, si queremos seleccionar todas las ventas que no corresponden a la sucursal "Sur", podemos escribir:

```
df[~(df["sucursal"] == "Sur")]
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
7	Resaltadores	Libreria	2100	12	Centro	Debito	2024-03-04
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06
11	Carpeta	Libreria	950	18	Norte	Efectivo	2024-03-06
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07

La condición original es:

```
df["sucursal"] == "Sur"
```

Esa condición devuelve `True` para las filas donde la sucursal es `"Sur"`.

Al colocar `~` delante de la condición, invertimos el resultado: los `True` pasan a ser `False` y los `False` pasan a ser `True`.

Entonces:

```
df[~(df["sucursal"] == "Sur")]
```

significa: mostrá las filas donde la sucursal no sea `"Sur"`.

En este caso, podríamos obtener el mismo resultado usando `!=`:

```
df[df["sucursal"] != "Sur"]
```

Ambas formas son válidas para este ejemplo. Sin embargo, `~` resulta especialmente útil cuando queremos negar condiciones más largas o cuando usamos métodos como `isin()`.

Por ejemplo, podemos seleccionar las ventas que no pertenecen a las categorías `"Tecnologia"` ni `"Accesorios"`:

```
df[~(df["categoria"].isin(["Tecnologia", "Accesorios"]))]
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
3	Regla	Libreria	500	25	Sur	Efectivo	2024-03-02
7	Resaltadores	Libreria	2100	12	Centro	Debito	2024-03-04
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
9	Compas	Libreria	2600	10	Sur	Efectivo	2024-03-05
11	Carpeta	Libreria	950	18	Norte	Efectivo	2024-03-06
12	Goma	Libreria	300	30	Sur	Efectivo	2024-03-07
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07
14	Agenda	Libreria	4300	9	Sur	Credito	2024-03-08

En este caso, `isin()` identifica las filas cuya categoría está dentro de la lista `["Tecnologia", "Accesorios"]`. Luego, el operador `~` invierte esa condición y conserva las filas que no pertenecen a esas categorías.

Podemos resumirlo así:

```
~ → niega una condición
```

El operador `~` debe colocarse delante de una condición entre paréntesis. De esa manera, Pandas sabe exactamente qué condición queremos invertir.

## Guardar condiciones en variables

Cuando combinamos condiciones, las instrucciones pueden volverse largas. Esto no está mal, pero a veces dificulta la lectura.

Por ejemplo, este filtro selecciona ventas de la categoría `"Tecnologia"` con precio mayor que `8000`:

```
df[(df["categoria"] == "Tecnologia") & (df["precio"] > 8000)]
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06

La instrucción funciona correctamente, pero podemos escribirla de una manera más clara guardando cada condición en una variable.

Primero creamos las condiciones:

```
es_tecnologia = df["categoria"] == "Tecnologia"
precio_mayor_a_8000 = df["precio"] > 8000
```

Luego combinamos esas variables dentro del filtro:

```
df[es_tecnologia & precio_mayor_a_8000]
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06

El resultado es el mismo que antes, pero el código puede leerse con más facilidad.

Ahora podemos interpretar el filtro casi como una frase:

```
mostrar las filas donde
es_tecnologia
y
precio_mayor_a_8000
```

Esta forma de trabajo es especialmente útil cuando las condiciones tienen nombres claros. Por ejemplo:

```
es_centro = df["sucursal"] == "Centro"
pago_con_credito = df["forma_pago"] == "Credito"

df[es_centro & pago_con_credito]
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03

Al guardar condiciones en variables, el código queda más ordenado y resulta más sencillo detectar errores. También se vuelve más fácil modificar una condición sin tener que reescribir todo el filtro.

Esta práctica será cada vez más útil a medida que construyamos consultas más complejas.

## Combinar condiciones y seleccionar columnas

Cuando combinamos condiciones, muchas veces no necesitamos mostrar todas las columnas del `DataFrame`. Tal vez queremos ver solamente las variables relevantes para la pregunta que estamos respondiendo.

Para eso podemos combinar condiciones dentro de `loc`.

La estructura general es:

```
df.loc[condición, columnas]
```

Por ejemplo, podemos seleccionar las ventas de "Tecnologia" con precio mayor que 8000, pero mostrar solamente el producto, el precio, la sucursal y la forma de pago.

```
df.loc[
 (df["categoria"] == "Tecnologia") & (df["precio"] > 8000),
 ["producto", "precio", "sucursal", "forma_pago"]]
```

```
["producto", "precio", "sucursal", "forma_pago"]
```

índice	producto	precio	sucursal	forma_pago
5	Calculadora	18500	Centro	Credito
10	Pendrive	9800	Centro	Debito

Esta instrucción tiene dos partes. En la primera parte indicamos la condición que deben cumplir las filas. En la segunda parte indicamos las columnas que queremos conservar en el resultado.

También podemos usar condiciones guardadas en variables:

```
es_tecnologia = df["categoria"] == "Tecnologia"
precio_mayor_a_8000 = df["precio"] > 8000

df.loc[
 es_tecnologia & precio_mayor_a_8000,
 ["producto", "precio", "sucursal", "forma_pago"]
]
```

índice	producto	precio	sucursal	forma_pago
5	Calculadora	18500	Centro	Credito
10	Pendrive	9800	Centro	Debito

El resultado es el mismo, pero el código puede resultar más fácil de leer porque las condiciones tienen nombres descriptivos.

Veamos otro ejemplo. Podemos seleccionar las ventas de la sucursal "Centro" o "Norte" que hayan sido pagadas con "Debito", y mostrar solamente algunas columnas.

```
es_centro_o_norte = df["sucursal"].isin(["Centro", "Norte"])
pago_con_debito = df["forma_pago"] == "Debito"

df.loc[
 es_centro_o_norte & pago_con_debito,
 ["producto", "sucursal", "forma_pago", "cantidad_vendida"]
]
```

Esta forma de trabajo permite construir resultados más claros. No solo filtramos las filas que cumplen ciertos criterios, sino que además elegimos qué información queremos mostrar de esas filas.

Cuando el filtro tiene varias condiciones, usar `loc` junto con variables descriptivas puede hacer que el código sea más legible y más fácil de revisar.

## Errores frecuentes al combinar condiciones

Al combinar condiciones en Pandas, hay algunos errores que aparecen con frecuencia. Conocerlos ayuda a interpretar mejor los mensajes de error y a corregir el código con más seguridad.

Un primer error común es usar `and` en lugar de `&`.

En Python, `and` sirve para combinar valores booleanos simples, como `True` y `False`. Pero cuando trabajamos con columnas de Pandas, cada condición produce una máscara booleana completa, no un único valor verdadero o falso. Por eso, para combinar condiciones sobre columnas usamos `&`.

```
Esta instrucción produciría un error porque en Pandas no usamos and
para combinar condiciones sobre columnas.

df[(df["categoria"] == "Tecnologia") and (df["precio"] > 8000)]
```

La forma correcta es:

```
df[(df["categoria"] == "Tecnologia") & (df["precio"] > 8000)]
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06

Otro error frecuente es usar `or` en lugar de `|`. Si queremos seleccionar filas que cumplan una condición u otra, debemos usar `|`.

```
Esta instrucción produciría un error porque en Pandas no usamos or
para combinar condiciones sobre columnas.

df[(df["sucursal"] == "Centro") or (df["sucursal"] == "Norte")]
```

La forma correcta es:

```
df[(df["sucursal"] == "Centro") | (df["sucursal"] == "Norte")]
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
7	Resaltadores	Libreria	2100	12	Centro	Debito	2024-03-04
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06
11	Carpeta	Libreria	950	18	Norte	Efectivo	2024-03-06
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07

También es muy común olvidar los paréntesis alrededor de cada condición. En Pandas, cuando combinamos condiciones, cada comparación debe quedar encerrada entre paréntesis.

```
Esta instrucción produciría un error porque faltan paréntesis.

df[df["categoria"] == "Tecnologia" & df["precio"] > 8000]
```

La forma correcta es:

```
df[(df["categoria"] == "Tecnologia") & (df["precio"] > 8000)]
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06

Finalmente, también debemos tener cuidado con el operador `~`. Este operador niega una condición, pero necesita saber exactamente qué condición debe invertir. Por eso conviene usarlo siempre junto con paréntesis.

```
df[~(df["sucursal"] == "Sur")]
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01



índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
5	Calculadora	Tecnología	18500	3	Centro	Credito	2024-03-03
7	Resaltadores	Librería	2100	12	Centro	Debito	2024-03-04
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
10	Pendrive	Tecnología	9800	7	Centro	Debito	2024-03-06
11	Carpeta	Librería	950	18	Norte	Efectivo	2024-03-06
13	Lapiz	Librería	250	50	Centro	Debito	2024-03-07

Una buena regla práctica es recordar estas equivalencias:

```
& reemplaza a "y" cuando combinamos condiciones de Pandas
| reemplaza a "o" cuando combinamos condiciones de Pandas
~ niega una condición
```

Y, junto con eso, mantener esta costumbre:

```
cada condición va entre paréntesis
```

Con estas dos ideas, la mayoría de los filtros combinados se vuelven mucho más fáciles de escribir y revisar.

## Resumen del capítulo

En este capítulo aprendimos a combinar condiciones dentro de un mismo filtro.

Hasta ahora habíamos construido filtros simples, usando una sola condición. Por ejemplo, podíamos seleccionar ventas de una categoría determinada o ventas cuyo precio superara cierto valor. En este capítulo dimos un paso más y comenzamos a construir filtros más precisos, capaces de combinar varios criterios al mismo tiempo.

Primero trabajamos con el operador `&`, que permite expresar una condición del tipo “esto y aquello”. Por ejemplo:

```
df[(df["categoria"] == "Tecnologia") & (df["precio"] > 8000)]
```

Esta instrucción conserva solamente las filas donde se cumplen ambas condiciones: la categoría debe ser `"Tecnologia"` y el precio debe ser mayor que `8000`.

Luego trabajamos con el operador `|`, que permite expresar una condición del tipo “esto o aquello”. Por ejemplo:

```
df[(df["sucursal"] == "Centro") | (df["sucursal"] == "Norte")]
```

En este caso, una fila queda incluida si pertenece a la sucursal `"Centro"` o si pertenece a la sucursal `"Norte"`. No hace falta que se cumplan ambas condiciones al mismo tiempo; alcanza con que se cumpla una de ellas.

Después incorporamos el operador `~`, que permite negar una condición. Por ejemplo:

```
df[~(df["sucursal"] == "Sur")]
```

Esta instrucción selecciona las ventas cuya sucursal no es `"Sur"`.

También vimos que, al combinar condiciones en Pandas, cada condición debe escribirse entre paréntesis. Esta regla es muy importante para evitar errores y para que Python interprete correctamente cada comparación.

Más adelante guardamos condiciones en variables para mejorar la legibilidad del código:

```
es_tecnologia = df["categoria"] == "Tecnologia"
precio_mayor_a_8000 = df["precio"] > 8000

df[es_tecnologia & precio_mayor_a_8000]
```

Esta forma de trabajo permite construir filtros más claros, especialmente cuando las condiciones empiezan a ser largas o cuando queremos reutilizarlas.

Finalmente combinamos filtros con selección de columnas usando `loc`:

```
df.loc[
 (df["categoria"] == "Tecnologia") & (df["precio"] > 8000),
 ["producto", "precio", "sucursal", "forma_pago"]
]
```

Esta estructura nos permite seleccionar las filas que cumplen ciertas condiciones y, al mismo tiempo, mostrar solamente las columnas que necesitamos.

La idea principal de este capítulo fue:

```
Podemos combinar condiciones para construir filtros más precisos sobre un DataFrame.
```

Los operadores lógicos `&`, `|` y `~` son herramientas fundamentales para transformar una pregunta de análisis en una selección concreta de datos.

## Próximo paso

---

Ya sabemos construir filtros simples y también combinar varias condiciones dentro de un mismo filtro.

El siguiente paso será aprender a ordenar datos. Muchas veces, después de filtrar un dataset, queremos ordenar los resultados: por precio, por cantidad vendida, por fecha, por categoría o por cualquier otra variable relevante.

Ordenar un `DataFrame` nos permitirá identificar valores altos o bajos, revisar rankings, encontrar registros extremos y presentar la información de manera más clara.

## Capítulo 8 — Ordenar datos en un DataFrame

---

### Breve repaso

---

En el trabajo anterior sobre filtros aprendimos a seleccionar filas usando condiciones. Primero construimos filtros simples, como seleccionar ventas de una categoría determinada o ventas con precio mayor que cierto valor. Después avanzamos hacia filtros combinados, usando operadores lógicos como `&`, `|` y `~`.

Eso nos permitió construir selecciones más precisas. Por ejemplo, pudimos pedir ventas de una categoría específica con precio alto, ventas de ciertas sucursales o registros que no pertenecieran a determinado grupo.

En este capítulo vamos a trabajar con otra operación fundamental: ordenar datos.

Ordenar un `DataFrame` significa reorganizar sus filas según los valores de una o más columnas. Esto permite leer mejor la información, encontrar rápidamente valores altos o bajos, comparar registros y construir primeros rankings.

Por ejemplo, podríamos ordenar las ventas por precio, por cantidad vendida, por categoría, por sucursal o por forma de pago. También podríamos combinar filtros y ordenamiento: primero seleccionar las ventas de una categoría y después ordenarlas de mayor a menor precio.

Al finalizar este capítulo deberías poder:

- Ordenar un `DataFrame` usando una columna.
- Ordenar datos en forma ascendente o descendente.
- Ordenar columnas numéricas y columnas de texto.
- Ordenar por más de una columna.
- Combinar filtros con ordenamiento.
- Seleccionar columnas después de ordenar.
- Guardar un resultado ordenado en una nueva variable.
- Reconocer que `sort_values()` no modifica el `DataFrame` original salvo que se indique explícitamente.
- Usar `sort_index()` para ordenar según el índice.

## Dataset de trabajo

Para estudiar cómo ordenar datos vamos a seguir trabajando con el dataset de ventas de una tienda escolar.

Recordemos: cada fila representa una venta registrada. Las columnas indican el producto vendido, la categoría, el precio unitario, la cantidad vendida, la sucursal, la forma de pago y la fecha. Es útil, porque tiene suficiente variedad para ordenar la información de distintas maneras: por precio, por cantidad vendida, por producto, por categoría, por sucursal o por forma de pago.

```
import pandas as pd

datos = {
 "producto": [
 "Cuaderno", "Lapicera", "Mochila", "Regla", "Cartuchera",
 "Calculadora", "Auriculares", "Resaltadores", "Guardapolvo", "Compas",
 "Pendrive", "Carpeta", "Goma", "Lapiz", "Agenda"
],
 "categoria": [
 "Libreria", "Libreria", "Accesorios", "Libreria", "Accesorios",
 "Tecnologia", "Tecnologia", "Libreria", "Indumentaria", "Libreria",
 "Tecnologia", "Libreria", "Libreria", "Libreria", "Libreria"
],
 "precio": [
 1200, 350, 8500, 500, 3200,
 18500, 7600, 2100, 14500, 2600,
 9800, 950, 300, 250, 4300
],
 "cantidad_vendida": [
 15, 40, 5, 25, 8,
 3, 6, 12, 4, 10,
 7, 18, 30, 50, 9
],
 "sucursal": [
 "Centro", "Centro", "Norte", "Sur", "Norte",
 "Centro", "Sur", "Centro", "Norte", "Sur",
 "Centro", "Norte", "Sur", "Centro", "Sur"
],
 "forma_pago": [
 "Efectivo", "Debito", "Credito", "Efectivo", "Debito",
 "Credito", "Credito", "Debito", "Credito", "Efectivo",
 "Debito", "Efectivo", "Efectivo", "Debito", "Credito"
],
 "fecha": [
 "2024-03-01", "2024-03-01", "2024-03-02", "2024-03-02", "2024-03-03",
 "2024-03-03", "2024-03-04", "2024-03-04", "2024-03-05", "2024-03-05",
 "2024-03-06", "2024-03-06", "2024-03-07", "2024-03-07", "2024-03-08"
]
}

df = pd.DataFrame(datos)
```

df

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
0	Cuaderno	Librería	1200	15	Centro	Efectivo	2024-03-01
1	Lapicera	Librería	350	40	Centro	Debito	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
3	Regla	Librería	500	25	Sur	Efectivo	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
5	Calculadora	Tecnología	18500	3	Centro	Credito	2024-03-03
6	Auriculares	Tecnología	7600	6	Sur	Credito	2024-03-04
7	Resaltadores	Librería	2100	12	Centro	Debito	2024-03-04
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
9	Compas	Librería	2600	10	Sur	Efectivo	2024-03-05
10	Pendrive	Tecnología	9800	7	Centro	Debito	2024-03-06
11	Carpeta	Librería	950	18	Norte	Efectivo	2024-03-06
12	Goma	Librería	300	30	Sur	Efectivo	2024-03-07
13	Lapiz	Librería	250	50	Centro	Debito	2024-03-07
14	Agenda	Librería	4300	9	Sur	Credito	2024-03-08

Antes de ordenar, conviene observar nuevamente la estructura general del `DataFrame`. Tenemos productos de distintas categorías, precios variados, cantidades vendidas diferentes y registros asociados a distintas sucursales y formas de pago.

Ese tipo de variedad permite que el ordenamiento sea útil. Si todos los precios o todas las cantidades fueran iguales, ordenar no aportaría demasiada información. En cambio, cuando los valores cambian de una fila a otra, ordenar ayuda a detectar rápidamente cuáles son los registros más altos, más bajos o más relevantes para una pregunta de análisis.

## Por qué ordenar datos

Ordenar datos significa reorganizar las filas de un `DataFrame` según los valores de una o más columnas.

Cuando miramos una tabla pequeña, tal vez podamos encontrar ciertos valores a simple vista. Pero a medida que el dataset crece, se vuelve mucho más difícil detectar cuáles son los precios más altos, cuáles son las cantidades más bajas, qué productos

aparecen primero alfabéticamente o cómo se distribuyen los registros dentro de una categoría.

El ordenamiento nos ayuda a leer mejor la información. Por ejemplo, si ordenamos las ventas por `precio`, podemos identificar rápidamente los productos más baratos o más caros. Si ordenamos por `cantidad_vendida`, podemos ver qué productos tuvieron mayor o menor cantidad de unidades vendidas. Si ordenamos por `categoria`, podemos agrupar visualmente productos similares.

Ordenar no cambia el significado de los datos. Las filas siguen representando las mismas ventas, pero aparecen en otro orden. Esa reorganización puede facilitar mucho la interpretación.

En Pandas, la herramienta principal para ordenar filas según los valores de una columna es `sort_values()`.

## Ordenar por una columna

Para ordenar un `DataFrame` según los valores de una columna, usamos el método `sort_values()`.

Dentro de los paréntesis indicamos el nombre de la columna que queremos usar como criterio de ordenamiento.

Por ejemplo, si queremos ordenar las ventas según el precio, podemos escribir:

```
df.sort_values("precio")
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07
12	Goma	Libreria	300	30	Sur	Efectivo	2024-03-07
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
3	Regla	Libreria	500	25	Sur	Efectivo	2024-03-02
11	Carpeta	Libreria	950	18	Norte	Efectivo	2024-03-06
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
7	Resaltadores	Libreria	2100	12	Centro	Debito	2024-03-04
9	Compas	Libreria	2600	10	Sur	Efectivo	2024-03-05
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
14	Agenda	Libreria	4300	9	Sur	Credito	2024-03-08

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2024-03-04
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
10	Pendrivel	Tecnologia	9800	7	Centro	Debito	2024-03-06
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03

El resultado muestra el `DataFrame` ordenado por la columna `precio`.

Por defecto, Pandas ordena de menor a mayor. Por eso, al ordenar por `precio`, las primeras filas corresponden a los productos más baratos y las últimas a los productos más caros.

Esta instrucción no elimina filas ni columnas. Tampoco cambia los valores del dataset. Solamente reorganiza el orden en que aparecen las filas.

Es importante observar que el índice original se conserva. Aunque las filas aparezcan en otro orden, cada una mantiene el índice que tenía en el `DataFrame` original. Esto nos recuerda que ordenar modifica la forma en que vemos la tabla, pero no reinventa los registros.

## Orden ascendente y descendente

Cuando usamos `sort_values()`, Pandas ordena de manera ascendente por defecto.

En una columna numérica, ordenar de forma ascendente significa ir de menor a mayor. Por eso, cuando escribimos:

```
df.sort_values("precio")
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07
12	Goma	Libreria	300	30	Sur	Efectivo	2024-03-07
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
3	Regla	Libreria	500	25	Sur	Efectivo	2024-03-02
11	Carpeta	Libreria	950	18	Norte	Efectivo	2024-03-06
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
7	Resaltadores	Libreria	2100	12	Centro	Debito	2024-03-04



índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
9	Compas	Libreria	2600	10	Sur	Efectivo	2024-03-05
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
14	Agenda	Libreria	4300	9	Sur	Credito	2024-03-08
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2024-03-04
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03

los productos aparecen ordenados desde el precio más bajo hasta el precio más alto.

Si queremos ordenar en sentido contrario, usamos el parámetro `ascending=False`.

```
df.sort_values("precio", ascending=False)
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2024-03-04
14	Agenda	Libreria	4300	9	Sur	Credito	2024-03-08
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
9	Compas	Libreria	2600	10	Sur	Efectivo	2024-03-05
7	Resaltadores	Libreria	2100	12	Centro	Debito	2024-03-04
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
11	Carpeta	Libreria	950	18	Norte	Efectivo	2024-03-06
3	Regla	Libreria	500	25	Sur	Efectivo	2024-03-02
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
12	Goma	Libreria	300	30	Sur	Efectivo	2024-03-07
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07

Ahora el `DataFrame` aparece ordenado de mayor a menor precio. Las primeras filas corresponden a los productos más caros y las últimas a los productos más baratos.

El parámetro `ascending` controla el sentido del ordenamiento:

```
ascending=True → orden ascendente
ascending=False → orden descendente
```

Como `ascending=True` es el valor por defecto, normalmente no hace falta escribirlo si queremos ordenar de menor a mayor. En cambio, sí debemos indicar `ascending=False` cuando queremos ordenar de mayor a menor.

Este detalle es muy útil cuando buscamos valores altos. Por ejemplo, si queremos ver rápidamente los productos más caros, conviene ordenar por precio en forma descendente.

## Ordenar por cantidad vendida

También podemos ordenar el `DataFrame` usando la columna `cantidad_vendida`.

Esto nos permite observar qué productos tuvieron menor o mayor cantidad de unidades vendidas.

Por ejemplo, podemos ordenar las ventas desde la menor cantidad vendida hasta la mayor:

```
df.sort_values("cantidad_vendida")
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2024-03-04
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
14	Agenda	Libreria	4300	9	Sur	Credito	2024-03-08
9	Compas	Libreria	2600	10	Sur	Efectivo	2024-03-05
7	Resaltadores	Libreria	2100	12	Centro	Debito	2024-03-04
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
11	Carpeta	Libreria	950	18	Norte	Efectivo	2024-03-06
3	Regla	Libreria	500	25	Sur	Efectivo	2024-03-02

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
12	Goma	Libreria	300	30	Sur	Efectivo	2024-03-07
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07

En este caso, las primeras filas corresponden a los registros con menos unidades vendidas.

Si queremos ver primero los productos con mayor cantidad vendida, usamos `ascending=False`:

```
df.sort_values("cantidad_vendida", ascending=False)
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
12	Goma	Libreria	300	30	Sur	Efectivo	2024-03-07
3	Regla	Libreria	500	25	Sur	Efectivo	2024-03-02
11	Carpeta	Libreria	950	18	Norte	Efectivo	2024-03-06
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
7	Resaltadores	Libreria	2100	12	Centro	Debito	2024-03-04
9	Compas	Libreria	2600	10	Sur	Efectivo	2024-03-05
14	Agenda	Libreria	4300	9	Sur	Credito	2024-03-08
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2024-03-04
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03

Ahora las primeras filas muestran las ventas con mayor cantidad de unidades.

Ordenar por cantidad vendida puede ayudarnos a detectar rápidamente qué productos se vendieron más y cuáles tuvieron menos movimiento. Sin embargo, todavía debemos interpretar estos resultados con cuidado: una cantidad vendida alta no siempre significa mayor ingreso total, porque también depende del precio de cada producto.

Por ejemplo, un producto barato puede vender muchas unidades, mientras que un producto caro puede vender pocas unidades y aun así representar un ingreso importante. Más adelante podremos crear nuevas columnas, como el importe total de cada venta, para analizar mejor ese tipo de situaciones.

## Ordenar columnas de texto

`sort_values()` también puede usarse con columnas de texto.

Cuando ordenamos una columna de texto, Pandas organiza los valores alfabéticamente. Por ejemplo, podemos ordenar el dataset según el nombre del producto.

```
df.sort_values("producto")
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
14	Agenda	Libreria	4300	9	Sur	Credito	2024-03-08
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2024-03-04
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
11	Carpeta	Libreria	950	18	Norte	Efectivo	2024-03-06
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
9	Compas	Libreria	2600	10	Sur	Efectivo	2024-03-05
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
12	Goma	Libreria	300	30	Sur	Efectivo	2024-03-07
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06
3	Regla	Libreria	500	25	Sur	Efectivo	2024-03-02
7	Resaltadores	Libreria	2100	12	Centro	Debito	2024-03-04

El resultado muestra las filas ordenadas alfabéticamente por la columna `producto`.

También podemos ordenar por `categoría`:

```
df.sort_values("categoria")
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
3	Regla	Libreria	500	25	Sur	Efectivo	2024-03-02
7	Resaltadores	Libreria	2100	12	Centro	Debito	2024-03-04
9	Compas	Libreria	2600	10	Sur	Efectivo	2024-03-05
11	Carpeta	Libreria	950	18	Norte	Efectivo	2024-03-06
12	Goma	Libreria	300	30	Sur	Efectivo	2024-03-07
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07
14	Agenda	Libreria	4300	9	Sur	Credito	2024-03-08
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2024-03-04
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06

En este caso, los registros quedan agrupados visualmente por categoría. Primero aparecen las filas de una categoría, luego las de otra, y así sucesivamente, siguiendo el orden alfabético.

Ordenar columnas de texto puede ser útil para revisar datos categóricos, detectar valores escritos de manera diferente o simplemente presentar una tabla de forma más clara. Por ejemplo, si una columna de categorías tuviera valores como "Libreria", "libreria" y "Librería", al ordenar podríamos notar más fácilmente que existen variantes de escritura que tal vez deberían limpiarse más adelante.

También podemos ordenar textos de manera descendente:

```
df.sort_values("producto", ascending=False)
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
7	Resaltadores	Libreria	2100	12	Centro	Debito	2024-03-04
3	Regla	Libreria	500	25	Sur	Efectivo	2024-03-02
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
12	Goma	Libreria	300	30	Sur	Efectivo	2024-03-07
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
9	Compas	Libreria	2600	10	Sur	Efectivo	2024-03-05
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
11	Carpeta	Libreria	950	18	Norte	Efectivo	2024-03-06
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2024-03-04
14	Agenda	Libreria	4300	9	Sur	Credito	2024-03-08

En este caso, el orden alfabético se invierte.

Aunque muchas veces usamos `sort_values()` con columnas numéricas, también es una herramienta útil para explorar columnas categóricas o textuales.

## Ordenar por más de una columna

A veces una sola columna no alcanza para organizar bien la información.

Por ejemplo, si ordenamos solamente por `categoria`, los productos quedan agrupados por categoría, pero dentro de cada categoría pueden aparecer en cualquier orden. Tal vez queremos que, dentro de cada categoría, los productos aparezcan ordenados por precio.

Para eso podemos ordenar usando más de una columna.

En `sort_values()`, en lugar de pasar un solo nombre de columna, pasamos una lista de columnas.

```
df.sort_values(["categoria", "precio"])
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07
12	Goma	Libreria	300	30	Sur	Efectivo	2024-03-07
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
3	Regla	Libreria	500	25	Sur	Efectivo	2024-03-02
11	Carpeta	Libreria	950	18	Norte	Efectivo	2024-03-06
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
7	Resaltadores	Libreria	2100	12	Centro	Debito	2024-03-04
9	Compas	Libreria	2600	10	Sur	Efectivo	2024-03-05
14	Agenda	Libreria	4300	9	Sur	Credito	2024-03-08
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2024-03-04
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03

Esta instrucción ordena primero por `categoría` y, dentro de cada categoría, ordena por `precio`.

Podemos leerlo así: *agrupá visualmente los registros por categoría y, dentro de cada grupo, mostrá los productos desde el precio más bajo hasta el precio más alto.*

También podemos ordenar por `sucursal` y luego por `cantidad_vendida`:

```
df.sort_values(["sucursal", "cantidad_vendida"])
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06
7	Resaltadores	Libreria	2100	12	Centro	Debito	2024-03-04
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
11	Carpeta	Libreria	950	18	Norte	Efectivo	2024-03-06

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
6	Auriculares	Tecnología	7600	6	Sur	Credito	2024-03-04
14	Agenda	Librería	4300	9	Sur	Credito	2024-03-08
9	Compas	Librería	2600	10	Sur	Efectivo	2024-03-05
3	Regla	Librería	500	25	Sur	Efectivo	2024-03-02
12	Goma	Librería	300	30	Sur	Efectivo	2024-03-07

En este caso, las filas se ordenan primero por `sucursal`. Luego, dentro de cada sucursal, se ordenan por `cantidad_vendida`.

Ordenar por más de una columna es útil cuando queremos organizar la información con distintos niveles de criterio. Primero definimos el criterio principal y después el criterio secundario.

## Ordenar varias columnas con distintos criterios

Cuando ordenamos por más de una columna, también podemos indicar si cada una debe ordenarse de forma ascendente o descendente.

Para eso usamos el parámetro `ascending`, pero en lugar de pasar un solo valor, pasamos una lista de valores.

Por ejemplo, podemos ordenar por `categoría` de manera ascendente y, dentro de cada categoría, ordenar por `precio` de manera descendente.

```
df.sort_values(
 ["categoría", "precio"],
 ascending=[True, False]
)
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
14	Agenda	Librería	4300	9	Sur	Credito	2024-03-08
9	Compas	Librería	2600	10	Sur	Efectivo	2024-03-05
7	Resaltadores	Librería	2100	12	Centro	Debito	2024-03-04
0	Cuaderno	Librería	1200	15	Centro	Efectivo	2024-03-01
11	Carpeta	Librería	950	18	Norte	Efectivo	2024-03-06



índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
3	Regla	Librería	500	25	Sur	Efectivo	2024-03-02
1	Lapicera	Librería	350	40	Centro	Debito	2024-03-01
12	Goma	Librería	300	30	Sur	Efectivo	2024-03-07
13	Lapiz	Librería	250	50	Centro	Debito	2024-03-07
5	Calculadora	Tecnología	18500	3	Centro	Credito	2024-03-03
10	Pendrive	Tecnología	9800	7	Centro	Debito	2024-03-06
6	Auriculares	Tecnología	7600	6	Sur	Credito	2024-03-04

En esta instrucción, la primera columna de ordenamiento es `categoría`, y se ordena alfabéticamente porque indicamos `True`. La segunda columna es `precio`, y se ordena de mayor a menor dentro de cada categoría porque indicamos `False`.

Esto permite construir ordenamientos más expresivos. Por ejemplo, si queremos revisar las ventas por sucursal y, dentro de cada sucursal, ver primero las mayores cantidades vendidas, podemos escribir:

```
df.sort_values(
 ["sucursal", "cantidad_vendida"],
 ascending=[True, False]
)
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
13	Lapiz	Librería	250	50	Centro	Debito	2024-03-07
1	Lapicera	Librería	350	40	Centro	Debito	2024-03-01
0	Cuaderno	Librería	1200	15	Centro	Efectivo	2024-03-01
7	Resaltadores	Librería	2100	12	Centro	Debito	2024-03-04
10	Pendrive	Tecnología	9800	7	Centro	Debito	2024-03-06
5	Calculadora	Tecnología	18500	3	Centro	Credito	2024-03-03
11	Carpeta	Librería	950	18	Norte	Efectivo	2024-03-06
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
12	Goma	Librería	300	30	Sur	Efectivo	2024-03-07
3	Regla	Librería	500	25	Sur	Efectivo	2024-03-02
9	Compas	Librería	2600	10	Sur	Efectivo	2024-03-05

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
14	Agenda	Libreria	4300	9	Sur	Credito	2024-03-08
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2024-03-04

En este caso, Pandas ordena primero por `sucursal` y luego, dentro de cada sucursal, muestra las ventas con mayor cantidad vendida al principio.

Cuando usamos varias columnas para ordenar, el orden de las listas importa. La primera columna es el criterio principal, la segunda columna es el criterio secundario, y así sucesivamente.

También debe coincidir la cantidad de columnas con la cantidad de valores en `ascending`. Si ordenamos por dos columnas, debemos indicar dos valores: uno para cada criterio.

## Combinar filtros y ordenamiento

En muchos análisis no ordenamos todo el dataset, sino un subconjunto de filas.

Por ejemplo, podemos querer ver solamente los productos de la categoría "Tecnologia" y ordenarlos por precio de mayor a menor.

Para hacerlo, primero filtramos y luego aplicamos `sort_values()` al resultado.

```
df[df["categoria"] == "Tecnologia"].sort_values("precio", ascending=False)
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2024-03-04

La primera parte de la instrucción filtra el `DataFrame`:

```
df[df["categoria"] == "Tecnologia"]
```

Luego, sobre ese resultado filtrado, aplicamos:

```
.sort_values("precio", ascending=False)
```

De esta manera obtenemos únicamente las ventas de la categoría "Tecnologia", ordenadas desde el precio más alto hasta el precio más bajo.

También podemos combinar un filtro más específico con ordenamiento. Por ejemplo, podemos seleccionar las ventas realizadas en la sucursal "Centro" y ordenarlas por `cantidad_vendida` de mayor a menor.

```
df[df["sucursal"] == "Centro"].sort_values("cantidad_vendida", ascending=False)
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
7	Resaltadores	Libreria	2100	12	Centro	Debito	2024-03-04
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03

Este encadenamiento es muy común en Pandas. Primero reducimos el dataset con un filtro y luego organizamos el resultado para leerlo mejor.

La estructura general es:

```
df[condición].sort_values("columna")
```

o, si queremos ordenar de mayor a menor:

```
df[condición].sort_values("columna", ascending=False)
```

Esta forma de trabajo permite responder preguntas más precisas. No solo preguntamos qué filas cumplen una condición, sino también cómo se ordenan esas filas según una variable relevante.

## Seleccionar columnas después de ordenar

Cuando ordenamos un `DataFrame`, Pandas muestra todas las columnas. Sin embargo, muchas veces nos interesa presentar solamente algunas.

Por ejemplo, si queremos ver los productos más caros, tal vez no necesitemos mostrar la fecha o la forma de pago. Podría alcanzarnos con ver el producto, la categoría y el precio.

Podemos ordenar primero y luego seleccionar algunas columnas:

```
df.sort_values("precio", ascending=False)[["producto", "categoria", "precio"]]
```

índice	producto	categoria	precio
5	Calculadora	Tecnologia	18500
8	Guardapolvo	Indumentaria	14500
10	Pendrive	Tecnologia	9800
2	Mochila	Accesorios	8500
6	Auriculares	Tecnologia	7600
14	Agenda	Libreria	4300
4	Cartuchera	Accesorios	3200
9	Compas	Libreria	2600
7	Resaltadores	Libreria	2100
0	Cuaderno	Libreria	1200
11	Carpeta	Libreria	950
3	Regla	Libreria	500
1	Lapicera	Libreria	350
12	Goma	Libreria	300
13	Lapiz	Libreria	250

Esta instrucción se lee de izquierda a derecha. Primero ordenamos el `DataFrame` por `precio` de mayor a menor. Luego, del resultado ordenado, seleccionamos solamente las columnas `producto`, `categoria` y `precio`.

También podemos hacer algo parecido con la cantidad vendida:

```
df.sort_values("cantidad_vendida", ascending=False)[
 ["producto", "categoria", "cantidad_vendida"]
]
```

índice	producto	categoria	cantidad_vendida
13	Lapiz	Libreria	50
1	Lapicera	Libreria	40
12	Goma	Libreria	30
3	Regla	Libreria	25
11	Carpeta	Libreria	18
0	Cuaderno	Libreria	15

índice	producto	categoría	cantidad_vendida
7	Resaltadores	Librería	12
9	Compas	Librería	10
14	Agenda	Librería	9
4	Cartuchera	Accesorios	8
10	Pendrive	Tecnología	7
6	Auriculares	Tecnología	6
2	Mochila	Accesorios	5
8	Guardapolvo	Indumentaria	4
5	Calculadora	Tecnología	3

En este caso, la tabla resultante permite identificar rápidamente qué productos tuvieron mayor cantidad de unidades vendidas.

Esta forma de trabajo es útil cuando queremos construir salidas más limpias. En lugar de mostrar toda la tabla, podemos quedarnos con las columnas necesarias para responder una pregunta concreta.

También podríamos combinar filtro, ordenamiento y selección de columnas:

```
df[df["categoria"] == "Librería"].sort_values(
 "cantidad_vendida",
 ascending=False
)[["producto", "categoria", "cantidad_vendida", "sucursal"]]
```

índice	producto	categoría	cantidad_vendida	sucursal
13	Lapiz	Librería	50	Centro
1	Lapicera	Librería	40	Centro
12	Goma	Librería	30	Sur
3	Regla	Librería	25	Sur
11	Carpeta	Librería	18	Norte
0	Cuaderno	Librería	15	Centro
7	Resaltadores	Librería	12	Centro
9	Compas	Librería	10	Sur
14	Agenda	Librería	9	Sur

Esta instrucción selecciona solamente las ventas de la categoría "Libreria", las ordena por cantidad\_vendida de mayor a menor y muestra algunas columnas relevantes.

Cuando las operaciones se encadenan, el orden importa. En este ejemplo, primero filtramos, después ordenamos y finalmente seleccionamos columnas para presentar el resultado de manera más clara.

## Guardar un resultado ordenado

Cuando usamos `sort_values()`, Pandas muestra el `DataFrame` ordenado, pero no modifica el `DataFrame` original.

Por ejemplo, podemos ordenar por precio de mayor a menor:

```
df.sort_values("precio", ascending=False)
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2024-03-04
14	Agenda	Libreria	4300	9	Sur	Credito	2024-03-08
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
9	Compas	Libreria	2600	10	Sur	Efectivo	2024-03-05
7	Resaltadores	Libreria	2100	12	Centro	Debito	2024-03-04
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
11	Carpeta	Libreria	950	18	Norte	Efectivo	2024-03-06
3	Regla	Libreria	500	25	Sur	Efectivo	2024-03-02
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
12	Goma	Libreria	300	30	Sur	Efectivo	2024-03-07
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07

Si después mostramos nuevamente `df`, veremos que el orden original se conserva.

```
df
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
3	Regla	Libreria	500	25	Sur	Efectivo	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2024-03-04
7	Resaltadores	Libreria	2100	12	Centro	Debito	2024-03-04
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
9	Compas	Libreria	2600	10	Sur	Efectivo	2024-03-05
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06
11	Carpeta	Libreria	950	18	Norte	Efectivo	2024-03-06
12	Goma	Libreria	300	30	Sur	Efectivo	2024-03-07
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07
14	Agenda	Libreria	4300	9	Sur	Credito	2024-03-08

Esto ocurre porque `sort_values()` devuelve un nuevo resultado ordenado, pero no cambia automáticamente el `DataFrame` sobre el que se aplicó.

Si queremos reutilizar el resultado ordenado más adelante, podemos guardarlo en una nueva variable.

```
ventas_por_precio = df.sort_values("precio", ascending=False)

ventas_por_precio
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2024-03-04
14	Agenda	Libreria	4300	9	Sur	Credito	2024-03-08
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
9	Compas	Libreria	2600	10	Sur	Efectivo	2024-03-05
7	Resaltadores	Libreria	2100	12	Centro	Debito	2024-03-04
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
11	Carpeta	Libreria	950	18	Norte	Efectivo	2024-03-06
3	Regla	Libreria	500	25	Sur	Efectivo	2024-03-02
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
12	Goma	Libreria	300	30	Sur	Efectivo	2024-03-07
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07

Ahora `ventas_por_precio` contiene el dataset ordenado por precio de mayor a menor, mientras que `df` sigue conservando el orden original.

Esta forma de trabajo es recomendable porque evita modificar accidentalmente los datos originales. Podemos tener distintas versiones ordenadas del mismo dataset, según la pregunta que queramos responder.

Por ejemplo:

```
ventas_por_cantidad = df.sort_values("cantidad_vendida", ascending=False)

ventas_por_cantidad
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
12	Goma	Libreria	300	30	Sur	Efectivo	2024-03-07
3	Regla	Libreria	500	25	Sur	Efectivo	2024-03-02
11	Carpeta	Libreria	950	18	Norte	Efectivo	2024-03-06
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
7	Resaltadores	Libreria	2100	12	Centro	Debito	2024-03-04
9	Compas	Libreria	2600	10	Sur	Efectivo	2024-03-05
14	Agenda	Libreria	4300	9	Sur	Credito	2024-03-08
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2024-03-04



índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
5	Calculadora	Tecnología	18500	3	Centro	Credito	2024-03-03

En este caso, la variable `ventas_por_cantidad` contiene las ventas ordenadas según la cantidad vendida, de mayor a menor.

Guardar resultados en variables con nombres claros ayuda a que el análisis sea más legible. Un nombre como `ventas_por_precio` o `ventas_por_cantidad` permite entender rápidamente qué contiene cada objeto.

## Ordenar por índice

Hasta ahora ordenamos el `DataFrame` usando los valores de una columna. Para eso usamos `sort_values()`.

Pero Pandas también permite ordenar un `DataFrame` según su índice. Para eso usamos `sort_index()`.

Primero veamos un ejemplo. Podemos crear un `DataFrame` ordenado por precio de mayor a menor:

```
ventas_por_precio = df.sort_values("precio", ascending=False)

ventas_por_precio
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
5	Calculadora	Tecnología	18500	3	Centro	Credito	2024-03-03
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
10	Pendrive	Tecnología	9800	7	Centro	Debito	2024-03-06
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
6	Auriculares	Tecnología	7600	6	Sur	Credito	2024-03-04
14	Agenda	Librería	4300	9	Sur	Credito	2024-03-08
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
9	Compas	Librería	2600	10	Sur	Efectivo	2024-03-05
7	Resaltadores	Librería	2100	12	Centro	Debito	2024-03-04
0	Cuaderno	Librería	1200	15	Centro	Efectivo	2024-03-01

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
11	Carpeta	Libreria	950	18	Norte	Efectivo	2024-03-06
3	Regla	Libreria	500	25	Sur	Efectivo	2024-03-02
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
12	Goma	Libreria	300	30	Sur	Efectivo	2024-03-07
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07

Observemos que las filas aparecen ordenadas por precio, pero el índice conserva las etiquetas originales. Por eso, al mirar la columna del índice, los valores pueden aparecer en un orden como 5, 8, 10, 2, etc.

Si queremos ordenar nuevamente ese resultado según el índice, podemos usar `sort_index()`.

```
ventas_por_precio.sort_index()
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
3	Regla	Libreria	500	25	Sur	Efectivo	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2024-03-04
7	Resaltadores	Libreria	2100	12	Centro	Debito	2024-03-04
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
9	Compas	Libreria	2600	10	Sur	Efectivo	2024-03-05
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06
11	Carpeta	Libreria	950	18	Norte	Efectivo	2024-03-06
12	Goma	Libreria	300	30	Sur	Efectivo	2024-03-07
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07
14	Agenda	Libreria	4300	9	Sur	Credito	2024-03-08

Ahora las filas vuelven a aparecer ordenadas según las etiquetas del índice.

La diferencia principal es esta:

```
sort_values() → ordena según los valores de una o más columnas
sort_index() → ordena según el índice
```

En este punto no necesitamos profundizar demasiado en el índice. Más adelante lo vamos a estudiar con más detalle. Por ahora, alcanza con reconocer que Pandas puede ordenar tanto por valores de columnas como por etiquetas del índice.

## Errores frecuentes al ordenar datos

Al ordenar datos con Pandas, pueden aparecer algunos errores o confusiones frecuentes.

Uno de los errores más comunes es escribir mal el nombre de la columna. Por ejemplo, nuestro `DataFrame` tiene una columna llamada `precio`, con minúscula. Si intentamos ordenar usando `"Precio"` con mayúscula inicial, Pandas no podrá encontrar esa columna.

```
Esta instrucción produciría un error porque la columna se llama "precio", no "Precio".
df.sort_values("Precio")
```

Para evitar este problema, conviene revisar los nombres de las columnas antes de ordenar.

```
df.columns
```

```
Index(['producto', 'categoria', 'precio', 'cantidad_vendida', 'sucursal',
 'forma_pago', 'fecha'],
 dtype='object')
```

Otro error frecuente es olvidar `ascending=False` cuando queremos ordenar de mayor a menor. Si escribimos:

```
df.sort_values("precio")
```

Pandas ordena de menor a mayor, porque ese es el comportamiento por defecto. Si queremos ver primero los precios más altos, debemos escribir:

```
df.sort_values("precio", ascending=False)
```

También puede ocurrir que pensemos que `sort_values()` modifica el `DataFrame` original. Sin embargo, como vimos antes, el método devuelve un resultado ordenado, pero no cambia automáticamente `df`.

Por eso, si queremos conservar el resultado ordenado, conviene guardarlo en una nueva variable:

```
ventas_ordenadas = df.sort_values("precio", ascending=False)
```

Finalmente, cuando ordenamos por varias columnas, debemos cuidar que la lista de columnas y la lista de valores de ascending tengan la misma longitud.

```
Esta instrucción produciría un error porque se indican dos columnas,
pero solo un valor para ascending dentro de una lista.

df.sort_values(["categoria", "precio"], ascending=[True])
```

La forma correcta sería:

```
df.sort_values(["categoria", "precio"], ascending=[True, False])
```

En general, para ordenar datos de manera segura conviene revisar tres cosas: que el nombre de la columna exista, que el sentido del ordenamiento sea el que necesitamos y que sepamos si queremos solamente ver el resultado ordenado o guardarlo para usarlo más adelante.

## Resumen del capítulo

---

En este capítulo aprendimos a ordenar datos en un `DataFrame`.

Ordenar datos significa reorganizar las filas según los valores de una o más columnas. Esta operación no cambia el significado de los datos, pero puede facilitar mucho su lectura. Al ordenar una tabla podemos detectar rápidamente valores altos, valores bajos, rankings, agrupamientos visuales o registros que conviene revisar con más atención.

La herramienta principal que usamos fue `sort_values()`.

Primero ordenamos el `DataFrame` según una sola columna:

```
df.sort_values("precio")
```

Por defecto, Pandas ordena de manera ascendente. En una columna numérica, eso significa de menor a mayor. Para ordenar en sentido descendente usamos `ascending=False`:

```
df.sort_values("precio", ascending=False)
```

También vimos que podemos ordenar por otras columnas numéricas, como `cantidad_vendida`, y por columnas de texto, como `producto`, `categoria` o `sucursal`. En las columnas textuales, Pandas ordena alfabéticamente.

Luego aprendimos a ordenar por más de una columna. Para eso pasamos una lista de columnas a `sort_values()`:

```
df.sort_values(["categoria", "precio"])
```

En este caso, Pandas ordena primero por `categoria` y, dentro de cada categoría, ordena por `precio`.

También vimos que podemos indicar distintos sentidos de ordenamiento para cada columna:

```
df.sort_values(
 ["categoria", "precio"],
 ascending=[True, False]
)
```

Después combinamos filtros y ordenamiento. Esta forma de trabajo permite seleccionar primero un subconjunto del dataset y luego organizar ese resultado:

```
df[df["categoria"] == "Tecnologia"].sort_values("precio", ascending=False)
```

Más adelante vimos que también podemos ordenar y seleccionar solamente algunas columnas para construir salidas más limpias:

```
df.sort_values("precio", ascending=False)[["producto", "categoria", "precio"]]
```

Otro punto importante fue que `sort_values()` no modifica automáticamente el `DataFrame` original. Si queremos conservar un resultado ordenado, conviene guardarlo en una nueva variable:

```
ventas_por_precio = df.sort_values("precio", ascending=False)
```

Finalmente presentamos `sort_index()`, que permite ordenar un `DataFrame` según su índice:

```
ventas_por_precio.sort_index()
```

La idea principal de este capítulo fue:

Ordenar datos nos ayuda a leer, comparar y detectar valores importantes dentro de un DataFrame.

Ordenar no reemplaza el análisis, pero muchas veces es un paso necesario para observar mejor la información y preparar resultados más claros.

## Próximo paso

---

Ya sabemos seleccionar columnas, seleccionar filas, filtrar registros usando condiciones y ordenar un DataFrame.

El siguiente paso será aprender a identificar valores extremos y primeros rankings de manera más directa. Para eso vamos a trabajar con herramientas como `head()`, `tail()`, `nlargest()` y `nsmallest()`.

Estas operaciones nos permitirán responder preguntas como cuáles son los productos más caros, cuáles son los más baratos, cuáles tienen mayor cantidad vendida o cuáles aparecen al final de un ordenamiento determinado.

## Capítulo 9 — Valores extremos y primeros rankings

---

### Breve repaso

---

En el trabajo anterior sobre ordenamiento aprendimos a reorganizar las filas de un `DataFrame` usando `sort_values()`.

Vimos que ordenar datos permite leer mejor una tabla, identificar valores altos o bajos, agrupar visualmente registros relacionados y presentar la información de una manera más clara. También aprendimos a ordenar de forma ascendente o descendente, a ordenar por más de una columna y a combinar filtros con ordenamiento.

En este capítulo vamos a usar esas ideas para resolver un tipo de pregunta muy frecuente en el análisis de datos: la búsqueda de valores extremos y primeros rankings.

Muchas veces no necesitamos ver todo el dataset ordenado. Tal vez solo queremos conocer los productos más caros, los más baratos, las ventas con mayor cantidad de unidades o los registros que aparecen al final de cierto ordenamiento.

Para eso vamos a trabajar con herramientas como `head()`, `tail()`, `nlargest()` y `nsmallest()`.

Estas herramientas nos permiten construir respuestas más directas a preguntas como:

```
¿Cuáles son los productos más caros?
¿Cuáles son los productos más baratos?
¿Qué ventas tuvieron mayor cantidad vendida?
¿Qué ventas tuvieron menor cantidad vendida?
¿Cuáles son los primeros registros después de ordenar?
¿Cuáles son los últimos registros después de ordenar?
```

Al finalizar este capítulo deberías poder:

- Usar `head()` para ver los primeros registros de un `DataFrame`.
- Usar `tail()` para ver los últimos registros de un `DataFrame`.
- Combinar `sort_values()` con `head()` y `tail()`.
- Usar `nlargest()` para obtener los mayores valores de una columna.
- Usar `nsmallest()` para obtener los menores valores de una columna.
- Construir rankings simples a partir de distintas variables.

- Combinar filtros, ordenamiento y selección de columnas para obtener resultados más claros.
- Reconocer cuándo conviene usar `sort_values()` y cuándo conviene usar `nlargest()` o `nsmallest()`.

## Dataset de trabajo

Para estudiar valores extremos y primeros rankings vamos a seguir trabajando con el dataset de ventas de una tienda escolar. Cada fila representa una venta registrada. Las columnas indican el producto vendido, la categoría, el precio unitario, la cantidad vendida, la sucursal, la forma de pago y la fecha.

Este dataset nos permite buscar distintos tipos de extremos: productos más caros, productos más baratos, ventas con mayor cantidad vendida y ventas con menor cantidad vendida.

```
import pandas as pd

datos = {
 "producto": [
 "Cuaderno", "Lapicera", "Mochila", "Regla", "Cartuchera",
 "Calculadora", "Auriculares", "Resaltadores", "Guardapolvo", "Compas",
 "Pendrive", "Carpeta", "Goma", "Lapiz", "Agenda"
],
 "categoria": [
 "Libreria", "Libreria", "Accesorios", "Libreria", "Accesorios",
 "Tecnologia", "Tecnologia", "Libreria", "Indumentaria", "Libreria",
 "Tecnologia", "Libreria", "Libreria", "Libreria", "Libreria"
],
 "precio": [
 1200, 350, 8500, 500, 3200,
 18500, 7600, 2100, 14500, 2600,
 9800, 950, 300, 250, 4300
],
 "cantidad_vendida": [
 15, 40, 5, 25, 8,
 3, 6, 12, 4, 10,
 7, 18, 30, 50, 9
],
 "sucursal": [
 "Centro", "Centro", "Norte", "Sur", "Norte",
 "Centro", "Sur", "Centro", "Norte", "Sur",
 "Centro", "Norte", "Sur", "Centro", "Sur"
],
 "forma_pago": [
 "Efectivo", "Debito", "Credito", "Efectivo", "Debito",
 "Credito", "Credito", "Debito", "Credito", "Efectivo",
 "Debito", "Efectivo", "Efectivo", "Debito", "Credito"
],
 "fecha": [
```



```

 "2024-03-01", "2024-03-01", "2024-03-02", "2024-03-02", "2024-03-03",
 "2024-03-03", "2024-03-04", "2024-03-04", "2024-03-05", "2024-03-05",
 "2024-03-06", "2024-03-06", "2024-03-07", "2024-03-07", "2024-03-08"
]
}

df = pd.DataFrame(datos)

df

```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
0	Cuaderno	Librería	1200	15	Centro	Efectivo	2024-03-01
1	Lapicera	Librería	350	40	Centro	Debito	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
3	Regla	Librería	500	25	Sur	Efectivo	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
5	Calculadora	Tecnología	18500	3	Centro	Credito	2024-03-03
6	Auriculares	Tecnología	7600	6	Sur	Credito	2024-03-04
7	Resaltadores	Librería	2100	12	Centro	Debito	2024-03-04
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
9	Compas	Librería	2600	10	Sur	Efectivo	2024-03-05
10	Pendrive	Tecnología	9800	7	Centro	Debito	2024-03-06
11	Carpeta	Librería	950	18	Norte	Efectivo	2024-03-06
12	Goma	Librería	300	30	Sur	Efectivo	2024-03-07
13	Lapiz	Librería	250	50	Centro	Debito	2024-03-07
14	Agenda	Librería	4300	9	Sur	Credito	2024-03-08

Al observar el `DataFrame`, vemos que hay productos con precios muy diferentes y cantidades vendidas también variadas. Esa diversidad hace que tenga sentido buscar valores extremos.

Por ejemplo, podemos preguntarnos qué productos tienen los precios más altos, cuáles tienen los precios más bajos, qué registros muestran mayor cantidad vendida o cuáles aparecen al final de un determinado ordenamiento.

## Qué significa buscar valores extremos

Cuando analizamos datos, muchas veces nos interesan especialmente los valores más altos o más bajos de una variable.

Por ejemplo, en nuestro dataset podríamos buscar el producto con mayor precio, el producto con menor precio, la venta con mayor cantidad de unidades o la venta con menor cantidad de unidades. Estos casos se conocen como valores extremos porque se ubican en los límites de una variable: los más grandes o los más pequeños.

Buscar valores extremos no significa necesariamente que esos valores sean errores. Un precio muy alto puede ser perfectamente válido si corresponde a un producto caro, como una calculadora o un guardapolvo. Una cantidad vendida muy baja también puede ser razonable si se trata de un producto de mayor precio o de menor demanda.

Lo importante es que los valores extremos llaman nuestra atención y nos invitan a observar con más cuidado. A veces nos ayudan a construir rankings, a detectar productos destacados o a identificar registros que conviene revisar.

En Pandas podemos encontrar estos valores de varias maneras. Una estrategia es ordenar el `DataFrame` y mirar las primeras o últimas filas. Otra posibilidad es usar métodos específicos como `nlargest()` y `smallest()`, que permiten obtener directamente los mayores o menores valores de una columna.

## Primeros registros con `head()`

El método `head()` permite ver las primeras filas de un `DataFrame`.

Ya lo usamos antes para inspeccionar rápidamente una tabla. En este capítulo lo vamos a retomar con otro propósito: observar los primeros registros de un resultado ordenado.

Primero recordemos su uso básico:

```
df.head()
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
3	Regla	Libreria	500	25	Sur	Efectivo	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03

Por defecto, `head()` muestra las primeras cinco filas.

También podemos indicar cuántas filas queremos ver:

```
df.head(3)
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02

En este caso, Pandas muestra las primeras tres filas del `DataFrame`.

Por sí solo, `head()` no busca los valores más altos ni los más bajos. Simplemente muestra las primeras filas según el orden actual de la tabla.

Por eso, si queremos usar `head()` para construir un ranking, primero necesitamos ordenar los datos.

## Últimos registros con `tail()`

El método `tail()` permite ver las últimas filas de un `DataFrame`.

Su uso es similar al de `head()`, pero en lugar de mostrar el comienzo de la tabla, muestra el final.

```
df.tail()
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06
11	Carpeta	Libreria	950	18	Norte	Efectivo	2024-03-06
12	Goma	Libreria	300	30	Sur	Efectivo	2024-03-07
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07
14	Agenda	Libreria	4300	9	Sur	Credito	2024-03-08

Por defecto, `tail()` muestra las últimas cinco filas.

También podemos indicar una cantidad específica:

```
df.tail(3)
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
12	Goma	Libreria	300	30	Sur	Efectivo	2024-03-07

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07
14	Agenda	Libreria	4300	9	Sur	Credito	2024-03-08

Al igual que `head()`, `tail()` no calcula por sí mismo valores extremos. Solo muestra las últimas filas según el orden actual del `DataFrame`.

Esto significa que `head()` y `tail()` se vuelven especialmente útiles cuando los combinamos con un ordenamiento previo.

## Ordenar y luego usar head()

Si queremos obtener los valores más altos de una columna, podemos ordenar el `DataFrame` de forma descendente y luego usar `head()`.

Por ejemplo, para ver los cinco productos con mayor precio, podemos escribir:

```
df.sort_values("precio", ascending=False).head()
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2024-03-04

Esta instrucción combina dos pasos.

Primero, `sort_values("precio", ascending=False)` ordena el `DataFrame` por precio de mayor a menor. Luego, `head()` muestra las primeras cinco filas de ese resultado ordenado.

De esta manera obtenemos un primer ranking de productos según su precio.

Si queremos ver solo los tres productos más caros, podemos indicar ese número dentro de `head()`.

```
df.sort_values("precio", ascending=False).head(3)
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
5	Calculadora	Tecnología	18500	3	Centro	Credito	2024-03-03
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
10	Pendrive	Tecnología	9800	7	Centro	Debito	2024-03-06

Ahora el resultado muestra solamente las tres ventas con precios más altos.

También podemos aplicar la misma idea con la columna `cantidad_vendida`. Si queremos ver las ventas con mayor cantidad de unidades, ordenamos por `cantidad_vendida` en forma descendente y luego usamos `head()`.

```
df.sort_values("cantidad_vendida", ascending=False).head()
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
13	Lapiz	Librería	250	50	Centro	Debito	2024-03-07
1	Lapicera	Librería	350	40	Centro	Debito	2024-03-01
12	Goma	Librería	300	30	Sur	Efectivo	2024-03-07
3	Regla	Librería	500	25	Sur	Efectivo	2024-03-02
11	Carpeta	Librería	950	18	Norte	Efectivo	2024-03-06

Esta forma de trabajo es muy frecuente: primero ordenamos según la variable que nos interesa y después mostramos los primeros registros del resultado.

## Ordenar y luego usar `tail()`

También podemos usar `tail()` después de ordenar.

Si ordenamos una columna de menor a mayor, las últimas filas serán las que tienen los valores más altos.

Por ejemplo:

```
df.sort_values("precio").tail()
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
6	Auriculares	Tecnología	7600	6	Sur	Credito	2024-03-04
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
10	Pendrive	Tecnología	9800	7	Centro	Debito	2024-03-06
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
5	Calculadora	Tecnología	18500	3	Centro	Credito	2024-03-03

En este caso, `sort_values("precio")` ordena los precios de menor a mayor. Luego, `tail()` muestra las últimas cinco filas, que corresponden a los precios más altos.

Esto produce un resultado parecido a ordenar de mayor a menor y usar `head()` :

```
df.sort_values("precio", ascending=False).head()
```

Ambas estrategias pueden servir para encontrar valores altos, aunque suele ser más directo ordenar en forma descendente y usar `head()` .

También podemos usar `tail()` para encontrar valores bajos si ordenamos de mayor a menor:

```
df.sort_values("precio", ascending=False).tail()
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
11	Carpeta	Librería	950	18	Norte	Efectivo	2024-03-06
3	Regla	Librería	500	25	Sur	Efectivo	2024-03-02
1	Lapicera	Librería	350	40	Centro	Debito	2024-03-01
12	Goma	Librería	300	30	Sur	Efectivo	2024-03-07
13	Lapiz	Librería	250	50	Centro	Debito	2024-03-07

En este caso, primero ordenamos los precios de mayor a menor. Luego, `tail()` muestra las últimas filas, que corresponden a los precios más bajos.

Esta combinación es válida, pero debemos leerla con cuidado. Cuando usamos `head()` o `tail()` después de ordenar, siempre conviene preguntarnos:

```
¿En qué sentido ordené los datos?
¿Estoy mirando el comienzo o el final del resultado?
```

Esa interpretación es necesaria para saber si estamos observando los valores más altos o los más bajos.

## Obtener los mayores valores con `nlargest()`

Pandas ofrece un método específico para obtener las filas con los mayores valores de una columna: `nlargest()` .

Este método permite pedir directamente los registros más grandes según una variable numérica.

Por ejemplo, si queremos obtener las tres ventas con mayor precio, podemos escribir:

```
df.nlargest(3, "precio")
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
5	Calculadora	Tecnología	18500	3	Centro	Credito	2024-03-03
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
10	Pendrive	Tecnología	9800	7	Centro	Debito	2024-03-06

La instrucción anterior devuelve las tres filas con los valores más altos en la columna `precio`.

Esta forma es más directa que ordenar todo el `DataFrame` y luego usar `head()`.

Podemos comparar estas dos instrucciones:

```
df.sort_values("precio", ascending=False).head(3)
```

y

```
df.nlargest(3, "precio")
```

En este caso, ambas permiten obtener las tres ventas con mayor precio.

También podemos usar `nlargest()` con otra columna numérica. Por ejemplo, para obtener las cinco ventas con mayor cantidad vendida:

```
df.nlargest(5, "cantidad_vendida")
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
13	Lapiz	Librería	250	50	Centro	Debito	2024-03-07
1	Lapicera	Librería	350	40	Centro	Debito	2024-03-01
12	Goma	Librería	300	30	Sur	Efectivo	2024-03-07
3	Regla	Librería	500	25	Sur	Efectivo	2024-03-02
11	Carpeta	Librería	950	18	Norte	Efectivo	2024-03-06

`nlargest()` es útil cuando queremos obtener rápidamente los registros con valores más altos en una columna numérica.

## Obtener los menores valores con `nsmallest()`

Así como `nlargest()` permite obtener los valores más altos, Pandas también ofrece `nsmallest()` para obtener los valores más bajos de una columna numérica.

Por ejemplo, si queremos ver las tres ventas con menor precio, podemos escribir:

```
df.nsmallest(3, "precio")
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07
12	Goma	Libreria	300	30	Sur	Efectivo	2024-03-07
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01

La instrucción anterior devuelve las tres filas con los valores más bajos en la columna `precio`.

Esto nos permite responder rápidamente una pregunta como:

¿Cuáles son los productos más baratos del dataset?

También podemos usar `nsmallest()` con la columna `cantidad_vendida`:

```
df.nsmallest(5, "cantidad_vendida")
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2024-03-04
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06

En este caso obtenemos las cinco ventas con menor cantidad de unidades vendidas.

Al igual que `nlargest()`, `nsmallest()` trabaja con columnas numéricas. Si queremos ordenar columnas de texto, debemos usar `sort_values()`.



Podemos resumir la diferencia así:

```
nlargest() → obtiene los registros con valores más altos
nsmallest() → obtiene los registros con valores más bajos
```

Estos métodos son muy útiles cuando queremos encontrar extremos sin ordenar visualmente todo el `DataFrame`.

## Comparar formas de buscar extremos

Para buscar valores extremos en Pandas tenemos varias alternativas.

Una posibilidad es ordenar el `DataFrame` y luego mirar las primeras o últimas filas. Por ejemplo:

```
df.sort_values("precio", ascending=False).head(3)
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
5	Calculadora	Tecnología	18500	3	Centro	Credito	2024-03-03
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
10	Pendrive	Tecnología	9800	7	Centro	Debito	2024-03-06

Esta instrucción ordena todo el `DataFrame` por `precio`, de mayor a menor, y luego muestra las primeras tres filas.

Otra posibilidad es usar directamente `nlargest()`:

```
df.nlargest(3, "precio")
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
5	Calculadora	Tecnología	18500	3	Centro	Credito	2024-03-03
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
10	Pendrive	Tecnología	9800	7	Centro	Debito	2024-03-06

En este caso, Pandas devuelve directamente las tres filas con mayor precio.

Para los valores más bajos ocurre algo similar. Podemos ordenar de menor a mayor y usar `head()`:

```
df.sort_values("precio").head(3)
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07
12	Goma	Libreria	300	30	Sur	Efectivo	2024-03-07
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01

O podemos usar `nsmallest()` :

```
df.nsmallest(3, "precio")
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07
12	Goma	Libreria	300	30	Sur	Efectivo	2024-03-07
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01

Ambas estrategias pueden llegar al mismo resultado, pero no siempre se usan con la misma intención.

`sort_values()` es más general. Sirve para ordenar todo el `DataFrame`, ordenar por columnas de texto, ordenar por varias columnas o preparar una tabla completa en cierto orden.

En cambio, `nlargest()` y `nsmallest()` son más directos cuando queremos obtener los mayores o menores valores de una columna numérica.

Podemos pensarlo así:

Queremos...	Herramienta posible
Ordenar toda la tabla	<code>sort_values()</code>
Ver los primeros registros de una tabla	<code>head()</code>
Ver los últimos registros de una tabla	<code>tail()</code>
Obtener los mayores valores de una columna numérica	<code>nlargest()</code>
Obtener los menores valores de una columna numérica	<code>nsmallest()</code>

La elección depende de la pregunta que estemos intentando responder. Si queremos explorar el orden completo de los datos, `sort_values()` es una buena opción. Si solo queremos obtener los primeros puestos de un ranking numérico, `nlargest()` o `nsmallest()` pueden ser más directos.

## Rankings según distintas columnas

Un ranking es una lista ordenada según algún criterio.

En análisis de datos, no existe un único ranking posible. Todo depende de la variable que usemos para ordenar o seleccionar valores extremos.

Por ejemplo, si usamos la columna `precio`, podemos construir un ranking de productos más caros:

```
df.nlargest(5, "precio")[["producto", "categoria", "precio"]]
```

índice	producto	categoria	precio
5	Calculadora	Tecnologia	18500
8	Guardapolvo	Indumentaria	14500
10	Pendrive	Tecnologia	9800
2	Mochila	Accesorios	8500
6	Auriculares	Tecnologia	7600

Esta tabla muestra las cinco ventas con mayor precio.

Pero si usamos la columna `cantidad_vendida`, el ranking cambia:

```
df.nlargest(5, "cantidad_vendida")[["producto", "categoria", "cantidad_vendida"]]
```

índice	producto	categoria	cantidad_vendida
13	Lapiz	Libreria	50
1	Lapicera	Libreria	40
12	Goma	Libreria	30
3	Regla	Libreria	25
11	Carpeta	Libreria	18

Ahora la tabla muestra las cinco ventas con mayor cantidad de unidades vendidas.

Esto nos permite observar una diferencia importante: un producto puede estar entre los más caros, pero no necesariamente entre los más vendidos. Del mismo modo, un producto barato puede vender muchas unidades.

También podemos construir rankings de valores bajos. Por ejemplo, los cinco productos con menor precio:

```
df.nsmallest(5, "precio")[["producto", "categoria", "precio"]]
```

índice	producto	categoria	precio
13	Lapiz	Libreria	250
12	Goma	Libreria	300
1	Lapicera	Libreria	350
3	Regla	Libreria	500
11	Carpeta	Libreria	950

O las cinco ventas con menor cantidad vendida:

```
df.nsmallest(5, "cantidad_vendida")[["producto", "categoria", "cantidad_vendida"]]
```

índice	producto	categoria	cantidad_vendida
5	Calculadora	Tecnologia	3
8	Guardapolvo	Indumentaria	4
2	Mochila	Accesorios	5
6	Auriculares	Tecnologia	6
10	Pendrive	Tecnologia	7

Cada ranking responde una pregunta distinta.

Ordenar por `precio` nos ayuda a observar productos más caros o más baratos. Ordenar por `cantidad_vendida` nos ayuda a observar productos con mayor o menor movimiento en unidades. Más adelante, cuando aprendamos a crear nuevas columnas, podremos construir rankings más ricos, por ejemplo calculando el importe total de cada venta.

Por ahora, lo importante es reconocer que el criterio elegido cambia completamente la lectura del resultado.

## Combinar filtros y rankings

Muchas veces no queremos construir un ranking sobre todo el dataset, sino sobre un subconjunto de filas.

Por ejemplo, podemos querer conocer los productos más caros dentro de una categoría determinada. Para eso, primero filtramos las filas y luego buscamos los valores más altos.

Si queremos obtener las ventas más caras de la categoría "Tecnología", podemos escribir:

```
df[df["categoria"] == "Tecnologia"].nlargest(3, "precio")
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2024-03-04

La primera parte de la instrucción filtra el dataset:

```
df[df["categoria"] == "Tecnologia"]
```

Después, sobre ese resultado filtrado, aplicamos:

```
.nlargest(3, "precio")
```

De esta manera obtenemos las tres ventas con mayor precio dentro de la categoría "Tecnología".

También podemos buscar las ventas con mayor cantidad vendida en una sucursal determinada:

```
df[df["sucursal"] == "Centro"].nlargest(5, "cantidad_vendida")
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
7	Resaltadores	Libreria	2100	12	Centro	Debito	2024-03-04
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06

En este caso, primero seleccionamos las ventas de la sucursal "Centro" y luego obtenemos las cinco filas con mayor cantidad vendida dentro de ese subconjunto.

También podemos combinar filtros más complejos. Por ejemplo, podemos seleccionar las ventas pagadas con "Credito" y buscar las de menor cantidad vendida:

```
df[df["forma_pago"] == "Credito"].nsmallest(3, "cantidad_vendida")
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
5	Calculadora	Tecnología	18500	3	Centro	Credito	2024-03-03
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02

Esta forma de trabajo permite construir preguntas más precisas. No solo preguntamos cuáles son los valores más altos o más bajos del dataset completo, sino cuáles son los extremos dentro de un grupo específico.

La estructura general es:

```
df[condición].nlargest(n, "columna")
```

O:

```
df[condición].nsmallest(n, "columna")
```

Primero reducimos el dataset con un filtro. Luego buscamos los mayores o menores valores dentro de ese resultado.

## Mostrar rankings más claros

Cuando usamos `nlargest()` o `nsmallest()`, Pandas devuelve todas las columnas del `DataFrame`.

Eso puede ser útil si queremos revisar el registro completo, pero muchas veces un ranking se lee mejor si mostramos solamente las columnas necesarias.

Por ejemplo, si queremos ver los productos más caros, probablemente nos interese mostrar el producto, la categoría y el precio.

```
df.nlargest(5, "precio")[["producto", "categoria", "precio"]]
```

índice	producto	categoría	precio
5	Calculadora	Tecnología	18500
8	Guardapolvo	Indumentaria	14500
10	Pendrive	Tecnología	9800
2	Mochila	Accesorios	8500

índice	producto	categoría	precio
6	Auriculares	Tecnología	7600

Esta tabla es más directa que mostrar todas las columnas. Permite concentrarse en la pregunta que estamos respondiendo: cuáles son los productos con mayor precio.

También podemos construir un ranking de productos con mayor cantidad vendida y mostrar solo las columnas relevantes:

```
df.nlargest(5, "cantidad_vendida")[["producto", "categoría", "cantidad_vendida"]]
```

índice	producto	categoría	cantidad_vendida
13	Lapiz	Librería	50
1	Lapicera	Librería	40
12	Goma	Librería	30
3	Regla	Librería	25
11	Carpeta	Librería	18

En este caso, el resultado se enfoca en el movimiento de unidades vendidas.

Si combinamos filtros y rankings, la selección de columnas también ayuda a que la salida sea más clara:

```
df[df["sucursal"] == "Centro"].nlargest(
 5,
 "cantidad_vendida"
)[["producto", "sucursal", "cantidad_vendida"]]
```

índice	producto	sucursal	cantidad_vendida
13	Lapiz	Centro	50
1	Lapicera	Centro	40
0	Cuaderno	Centro	15
7	Resaltadores	Centro	12
10	Pendrive	Centro	7

Esta instrucción selecciona las ventas de la sucursal "Centro", obtiene las cinco con mayor cantidad vendida y muestra solamente el producto, la sucursal y la cantidad.

Cuando construimos rankings, conviene pensar no solo en el cálculo, sino también en la presentación del resultado. Una tabla más pequeña y enfocada suele ser más fácil de interpretar que una tabla con muchas columnas.

## Errores frecuentes al buscar valores extremos

Cuando buscamos valores extremos o construimos rankings, pueden aparecer algunas confusiones.

Una primera confusión frecuente es pensar que `head()` siempre muestra los valores más altos. En realidad, `head()` solo muestra las primeras filas del `DataFrame` según el orden actual. Si el `DataFrame` no está ordenado, `head()` no necesariamente muestra un ranking.

```
df.head()
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
3	Regla	Libreria	500	25	Sur	Efectivo	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03

Para que `head()` muestre los valores más altos de una columna, primero debemos ordenar el `DataFrame` en forma descendente.

```
df.sort_values("precio", ascending=False).head()
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2024-03-04

Otra confusión posible es usar `tail()` sin revisar previamente el sentido del ordenamiento. Si ordenamos de menor a mayor y luego usamos `tail()`, veremos los valores más altos. Pero si ordenamos de mayor a menor y luego usamos `tail()`, veremos los valores más bajos.



```
df.sort_values("precio").tail()
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2024-03-04
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03

```
df.sort_values("precio", ascending=False).tail()
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
11	Carpeta	Libreria	950	18	Norte	Efectivo	2024-03-06
3	Regla	Libreria	500	25	Sur	Efectivo	2024-03-02
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
12	Goma	Libreria	300	30	Sur	Efectivo	2024-03-07
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07

También debemos recordar que `nlargest()` y `nsmallest()` trabajan con columnas numéricas. Si queremos ordenar una columna de texto, como `producto` o `categoria`, debemos usar `sort_values()`.

Por ejemplo, esta instrucción permite ordenar alfabéticamente por producto:

```
df.sort_values("producto")
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
14	Agenda	Libreria	4300	9	Sur	Credito	2024-03-08
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2024-03-04
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
11	Carpeta	Libreria	950	18	Norte	Efectivo	2024-03-06
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
9	Compas	Libreria	2600	10	Sur	Efectivo	2024-03-05
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
12	Goma	Libreria	300	30	Sur	Efectivo	2024-03-07

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
1	Lapicera	Librería	350	40	Centro	Debito	2024-03-01
13	Lapiz	Librería	250	50	Centro	Debito	2024-03-07
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
10	Pendrive	Tecnología	9800	7	Centro	Debito	2024-03-06
3	Regla	Librería	500	25	Sur	Efectivo	2024-03-02
7	Resaltadores	Librería	2100	12	Centro	Debito	2024-03-04

En cambio, no tendría sentido buscar “los productos más grandes” usando `nlargest()` sobre una columna de texto.

Otro punto importante es no confundir precio alto con mayor venta total. En nuestro dataset, `precio` indica el precio unitario y `cantidad_vendida` indica cuántas unidades se vendieron. Un producto puede tener precio alto y pocas unidades vendidas, o precio bajo y muchas unidades vendidas.

Por eso, si queremos analizar ingresos, todavía nos falta una columna que calcule el importe total de cada venta. Más adelante vamos a crear nuevas columnas para responder ese tipo de preguntas.

La idea principal es que un ranking siempre depende del criterio elegido. Antes de interpretar los primeros puestos, debemos preguntarnos qué variable usamos para construir ese ranking.

## Resumen del capítulo

En este capítulo trabajamos con valores extremos y primeros rankings dentro de un `DataFrame`.

Primero retomamos `head()` y `tail()`. Vimos que `head()` muestra los primeros registros y `tail()` muestra los últimos registros según el orden actual de la tabla. Por sí solos, estos métodos no buscan valores máximos ni mínimos; simplemente muestran el comienzo o el final del `DataFrame`.

Después combinamos `sort_values()` con `head()` y `tail()`. Esta combinación nos permitió construir rankings simples. Por ejemplo, al ordenar por `precio` de mayor a menor y luego usar `head()`, pudimos obtener las ventas con precios más altos:

```
df.sort_values("precio", ascending=False).head()
```

También vimos que el sentido del ordenamiento es fundamental. Si ordenamos de menor a mayor, los valores más bajos aparecen al principio y los más altos al final. Si ordenamos de mayor a menor, ocurre lo contrario.

Luego incorporamos dos métodos específicos para buscar valores extremos en columnas numéricas: `nlargest()` y `smallest()`.

Con `nlargest()` podemos obtener directamente los mayores valores de una columna:

```
df.nlargest(3, "precio")
```

Con `smallest()` podemos obtener directamente los menores valores:

```
df.smallest(3, "precio")
```

Estos métodos son útiles cuando queremos construir rankings numéricos de manera directa, sin ordenar visualmente todo el `DataFrame`.

También comparamos distintas estrategias. `sort_values()` es más general y sirve para ordenar tablas completas, columnas de texto o varias columnas al mismo tiempo. En cambio, `nlargest()` y `smallest()` son más directos cuando buscamos los mayores o menores valores de una columna numérica.

Después construimos rankings según distintas variables. Vimos que un ranking por `precio` no responde la misma pregunta que un ranking por `cantidad_vendida`. Un producto puede estar entre los más caros, pero no necesariamente entre los más vendidos. Por eso, siempre debemos interpretar un ranking según la variable usada como criterio.

Más adelante combinamos filtros y rankings. Esto nos permitió buscar valores extremos dentro de un subconjunto del dataset, por ejemplo las ventas más caras de una categoría o las ventas con mayor cantidad vendida dentro de una sucursal.

Finalmente, vimos que conviene seleccionar solamente algunas columnas para que los rankings sean más claros:

```
df.nlargest(5, "precio")[["producto", "categoria", "precio"]]
```

La idea principal de este capítulo fue:

```
Los rankings dependen del criterio elegido para ordenar o seleccionar valores extremos.
```

Buscar valores extremos nos ayuda a identificar casos destacados, pero la interpretación siempre debe hacerse con cuidado. Antes de sacar conclusiones, necesitamos preguntarnos qué variable usamos, qué representa esa variable y qué información todavía falta para responder mejor la pregunta de análisis.

## Próximo paso

---

Ya sabemos seleccionar, filtrar, ordenar y construir primeros rankings.

El siguiente paso será aprender a crear nuevas columnas dentro de un `DataFrame`. Esto nos permitirá generar información nueva a partir de datos existentes.

Por ejemplo, en nuestro dataset tenemos el precio unitario y la cantidad vendida. A partir de esas dos columnas podremos crear una nueva columna llamada `importe_total`, que represente el valor total de cada venta.

Crear columnas nuevas será un paso muy importante, porque muchas veces los datos originales no contienen directamente la variable que necesitamos analizar. En esos casos, debemos construirla.

## Capítulo 10 — Crear nuevas columnas en un DataFrame

---

### Breve repaso

---

En el capítulo anterior sobre valores extremos y rankings aprendimos a identificar registros destacados dentro de un `DataFrame`.

Usamos `head()` y `tail()` para observar los primeros y últimos registros, combinamos `sort_values()` con esas herramientas y luego incorporamos métodos más directos como `nlargest()` y `nsmallest()`.

Ese recorrido nos permitió responder preguntas como cuáles son los productos más caros, cuáles son los más baratos, qué ventas tuvieron mayor cantidad vendida y cuáles tuvieron menor cantidad vendida.

Sin embargo, también apareció una idea importante: no siempre las columnas originales del dataset contienen directamente toda la información que necesitamos analizar.

En nuestro dataset tenemos una columna `precio`, que indica el precio unitario de cada producto, y una columna `cantidad_vendida`, que indica cuántas unidades se vendieron. Pero si queremos saber cuánto dinero generó cada venta, necesitamos combinar esas dos columnas. Para eso necesitamos crear una nueva columna.

En este capítulo vamos a aprender a generar nuevas variables dentro de un `DataFrame` a partir de columnas existentes. Empezaremos con operaciones matemáticas simples y luego veremos cómo crear columnas usando condiciones.

Al finalizar este capítulo deberías poder:

- Comprender qué significa crear una nueva columna en un `DataFrame`.
- Crear columnas a partir de operaciones entre columnas existentes.
- Crear una columna de importe total usando `precio` y `cantidad_vendida`.
- Crear columnas con valores constantes.
- Crear columnas a partir de condiciones simples.
- Usar `loc` para asignar valores según una condición.
- Verificar que la nueva columna se haya creado correctamente.
- Reconocer errores frecuentes al crear o modificar columnas.

## Dataset de trabajo

---

Para aprender a crear nuevas columnas vamos a seguir usando el dataset de ventas de una tienda escolar.

Cada fila representa una venta registrada. Las columnas actuales indican el producto vendido, la categoría, el precio unitario, la cantidad vendida, la sucursal, la forma de pago y la fecha.

Este dataset es especialmente útil para este capítulo porque contiene dos columnas que podemos combinar de manera natural:

```
precio
cantidad_vendida
```

A partir de esas dos columnas podremos crear una nueva variable: el importe total de cada venta.

```
import pandas as pd

datos = {
 "producto": [
 "Cuaderno", "Lapicera", "Mochila", "Regla", "Cartuchera",
 "Calculadora", "Auriculares", "Resaltadores", "Guardapolvo", "Compas",
 "Pendrive", "Carpeta", "Goma", "Lapiz", "Agenda"
],
 "categoria": [
 "Libreria", "Libreria", "Accesorios", "Libreria", "Accesorios",
 "Tecnologia", "Tecnologia", "Libreria", "Indumentaria", "Libreria",
 "Tecnologia", "Libreria", "Libreria", "Libreria", "Libreria"
],
 "precio": [
 1200, 350, 8500, 500, 3200,
 18500, 7600, 2100, 14500, 2600,
 9800, 950, 300, 250, 4300
],
 "cantidad_vendida": [
 15, 40, 5, 25, 8,
 3, 6, 12, 4, 10,
 7, 18, 30, 50, 9
],
 "sucursal": [
 "Centro", "Centro", "Norte", "Sur", "Norte",
 "Centro", "Sur", "Centro", "Norte", "Sur",
 "Centro", "Norte", "Sur", "Centro", "Sur"
],
 "forma_pago": [
 "Efectivo", "Debito", "Credito", "Efectivo", "Debito",
 "Credito", "Credito", "Debito", "Credito", "Efectivo",
 "Debito", "Efectivo", "Efectivo", "Debito", "Credito"
],
}
```

```

 "fecha": [
 "2024-03-01", "2024-03-01", "2024-03-02", "2024-03-02", "2024-03-03",
 "2024-03-03", "2024-03-04", "2024-03-04", "2024-03-05", "2024-03-05",
 "2024-03-06", "2024-03-06", "2024-03-07", "2024-03-07", "2024-03-08"
]
}

df = pd.DataFrame(datos)

df

```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
3	Regla	Libreria	500	25	Sur	Efectivo	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2024-03-04
7	Resaltadores	Libreria	2100	12	Centro	Debito	2024-03-04
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
9	Compas	Libreria	2600	10	Sur	Efectivo	2024-03-05
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06
11	Carpeta	Libreria	950	18	Norte	Efectivo	2024-03-06
12	Goma	Libreria	300	30	Sur	Efectivo	2024-03-07
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07
14	Agenda	Libreria	4300	9	Sur	Credito	2024-03-08

Al observar el `DataFrame`, vemos que todavía no existe una columna que indique el importe total de cada venta.

Sabemos cuánto cuesta cada unidad y cuántas unidades se vendieron, pero el dataset no trae directamente calculado el resultado de multiplicar esos dos valores.

Ese es un caso muy común en el trabajo con datos. Muchas veces necesitamos crear nuevas columnas para representar información que no aparece explícitamente en el dataset original, pero que puede obtenerse a partir de las columnas disponibles.

## Qué significa crear una nueva columna

---

Crear una nueva columna significa agregar una variable al `DataFrame`.

Esa nueva variable puede construirse de distintas maneras. A veces surge de una operación matemática entre columnas existentes. Otras veces se crea a partir de una condición, una clasificación, una transformación de texto, una fecha o una regla definida por quien analiza los datos.

En nuestro caso, tenemos dos columnas numéricas:

```
precio
cantidad_vendida
```

La columna `precio` indica el precio unitario del producto. La columna `cantidad_vendida` indica cuántas unidades se vendieron. Si multiplicamos ambos valores, obtenemos el importe total de esa venta.

Podemos pensarlo así:

```
importe_total = precio × cantidad_vendida
```

Esa nueva columna no estaba en el dataset original, pero podemos construirla a partir de información que ya tenemos.

Crear columnas nuevas es una tarea muy importante en análisis de datos, porque muchas preguntas no se responden directamente con las variables originales. A veces necesitamos construir una variable más útil para el análisis.

Por ejemplo, con el dataset actual podríamos preguntarnos:

```
¿Qué venta generó más dinero?
¿Qué categoría produjo mayor importe total?
¿Qué sucursal registró ventas de mayor valor?
```

Para responder esas preguntas, no alcanza con mirar solo el precio o solo la cantidad vendida. Necesitamos combinar ambas columnas en una nueva variable.

## Crear una columna a partir de otras columnas

---

Para crear una nueva columna en Pandas, escribimos el nombre del `DataFrame`, seguido del nombre de la nueva columna entre corchetes. Por ejemplo, si queremos crear una columna llamada `importe_total`, podemos escribir:



```
df["importe_total"]
```

Pero además necesitamos indicar qué valores tendrá esa columna. En este caso, el importe total de cada venta se obtiene multiplicando el precio unitario por la cantidad vendida.

```
df["importe_total"] = df["precio"] * df["cantidad_vendida"]
```

```
df
```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
3	Regla	Libreria	500	25	Sur	Efectivo	2024-03-02
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2024-03-04
7	Resaltadores	Libreria	2100	12	Centro	Debito	2024-03-04
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
9	Compas	Libreria	2600	10	Sur	Efectivo	2024-03-05
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06
11	Carpeta	Libreria	950	18	Norte	Efectivo	2024-03-06
12	Goma	Libreria	300	30	Sur	Efectivo	2024-03-07
13	Lapiz	Libreria	250	50	Centro	Debito	2024-03-07
14	Agenda	Libreria	4300	9	Sur	Credito	2024-03-08

Ahora el `DataFrame` tiene una nueva columna llamada `importe_total`.

Pandas realizó la multiplicación fila por fila. Es decir, tomó el precio de cada producto y lo multiplicó por la cantidad vendida en esa misma fila. Por ejemplo, si una fila tiene:

```
precio = 1200
cantidad_vendida = 15
```

entonces el importe total será:

```
1200 × 15 = 18000
```

Esta forma de trabajo es muy común en Pandas. No necesitamos recorrer las filas una por una con un bucle. Cuando escribimos una operación entre columnas, Pandas aplica esa operación de manera vectorizada, es decir, sobre todos los valores de la columna al mismo tiempo.

La nueva columna queda incorporada al `DataFrame`. A partir de este momento podemos seleccionarla, ordenarla, filtrarla o usarla en nuevos cálculos.

## Verificar la nueva columna

Después de crear una columna nueva, conviene verificar que se haya creado correctamente.

Una forma sencilla de hacerlo es observar solamente las columnas involucradas en el cálculo. En este caso, nos interesa revisar `producto`, `precio`, `cantidad_vendida` e `importe_total`.

```
df[["producto", "precio", "cantidad_vendida", "importe_total"]]
```

índice	producto	precio	cantidad_vendida	importe_total
0	Cuaderno	1200	15	18000
1	Lapicera	350	40	14000
2	Mochila	8500	5	42500
3	Regla	500	25	12500
4	Cartuchera	3200	8	25600
5	Calculadora	18500	3	55500
6	Auriculares	7600	6	45600
7	Resaltadores	2100	12	25200
8	Guardapolvo	14500	4	58000
9	Compas	2600	10	26000
10	Pendrive	9800	7	68600
11	Carpeta	950	18	17100
12	Goma	300	30	9000
13	Lapiz	250	50	12500
14	Agenda	4300	9	38700

Esta vista nos permite comprobar si el cálculo tiene sentido.

Por ejemplo, para cada fila podemos revisar mentalmente que el importe total corresponda a la multiplicación entre el precio unitario y la cantidad vendida.

También podemos usar algunas herramientas que ya conocemos. Por ejemplo, podemos ordenar el `DataFrame` según la nueva columna `importe_total` para ver cuáles fueron las ventas de mayor valor.

```
df.sort_values("importe_total", ascending=False)
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
10	Pendrive	Tecnología	9800	7	Centro	Debito	2024-03-06
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
5	Calculadora	Tecnología	18500	3	Centro	Credito	2024-03-03
6	Auriculares	Tecnología	7600	6	Sur	Credito	2024-03-04
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
14	Agenda	Librería	4300	9	Sur	Credito	2024-03-08
9	Compas	Librería	2600	10	Sur	Efectivo	2024-03-05
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
7	Resaltadores	Librería	2100	12	Centro	Debito	2024-03-04
0	Cuaderno	Librería	1200	15	Centro	Efectivo	2024-03-01
11	Carpeta	Librería	950	18	Norte	Efectivo	2024-03-06
1	Lapicera	Librería	350	40	Centro	Debito	2024-03-01
3	Regla	Librería	500	25	Sur	Efectivo	2024-03-02
13	Lapiz	Librería	250	50	Centro	Debito	2024-03-07
12	Goma	Librería	300	30	Sur	Efectivo	2024-03-07

Ahora el ordenamiento se realiza usando una columna que no estaba en el dataset original, sino que fue creada por nosotros.

Esto muestra una idea importante: una vez que agregamos una columna al `DataFrame`, Pandas puede trabajar con ella igual que con cualquier otra columna. Podemos seleccionarla, ordenarla, filtrarla, resumirla o combinarla con otras variables.

Por ejemplo, podemos mostrar solamente algunas columnas para leer mejor el resultado:

```
df.sort_values("importe_total", ascending=False)[
 ["producto", "categoria", "precio", "cantidad_vendida", "importe_total"]
]
```

índice	producto	categoria	precio	cantidad_vendida	importe_total
10	Pendrive	Tecnologia	9800	7	68600
8	Guardapolvo	Indumentaria	14500	4	58000
5	Calculadora	Tecnologia	18500	3	55500
6	Auriculares	Tecnologia	7600	6	45600
2	Mochila	Accesorios	8500	5	42500
14	Agenda	Libreria	4300	9	38700
9	Compas	Libreria	2600	10	26000
4	Cartuchera	Accesorios	3200	8	25600
7	Resaltadores	Libreria	2100	12	25200
0	Cuaderno	Libreria	1200	15	18000
11	Carpeta	Libreria	950	18	17100
1	Lapicera	Libreria	350	40	14000
3	Regla	Libreria	500	25	12500
13	Lapiz	Libreria	250	50	12500
12	Goma	Libreria	300	30	9000

Esta tabla permite comparar el precio unitario, la cantidad vendida y el importe total de cada venta.

Al observarla, podemos notar que la venta de mayor importe total no necesariamente corresponde al producto con mayor precio unitario ni al producto con mayor cantidad vendida por separado. El importe total depende de la combinación de ambas variables.

## Comparar precio, cantidad e importe total

La nueva columna `importe_total` nos permite analizar las ventas desde una perspectiva más completa.

Antes podíamos construir rankings usando `precio` o `cantidad_vendida`. Pero cada una de esas columnas muestra solo una parte del problema.

El precio indica cuánto cuesta una unidad del producto. La cantidad vendida indica cuántas unidades se vendieron. El importe total combina ambas cosas. Comparemos algunos rankings.

```
Ventas con mayor precio unitario

df.nlargest(5, "precio")[
 ["producto", "categoria", "precio", "cantidad_vendida", "importe_total"]
]
```

índice	producto	categoria	precio	cantidad_vendida	importe_total
5	Calculadora	Tecnologia	18500	3	55500
8	Guardapolvo	Indumentaria	14500	4	58000
10	Pendrive	Tecnologia	9800	7	68600
2	Mochila	Accesorios	8500	5	42500
6	Auriculares	Tecnologia	7600	6	45600

```
Ventas con mayor cantidad vendida

df.nlargest(5, "cantidad_vendida")[
 ["producto", "categoria", "precio", "cantidad_vendida", "importe_total"]
]
```

índice	producto	categoria	precio	cantidad_vendida	importe_total
13	Lapiz	Libreria	250	50	12500
1	Lapicera	Libreria	350	40	14000
12	Goma	Libreria	300	30	9000
3	Regla	Libreria	500	25	12500
11	Carpeta	Libreria	950	18	17100

```
Ventas con mayor importe total

df.nlargest(5, "importe_total")[
 ["producto", "categoria", "precio", "cantidad_vendida", "importe_total"]
]
```

índice	producto	categoria	precio	cantidad_vendida	importe_total
10	Pendrive	Tecnologia	9800	7	68600
8	Guardapolvo	Indumentaria	14500	4	58000
5	Calculadora	Tecnologia	18500	3	55500

índice	producto	categoría	precio	cantidad_vendida	importe_total
6	Auriculares	Tecnología	7600	6	45600
2	Mochila	Accesorios	8500	5	42500

Al comparar estos tres rankings, podemos observar que no responden la misma pregunta.

El primer ranking muestra los productos con mayor precio unitario. El segundo muestra las ventas con mayor cantidad de unidades. El tercero muestra las ventas que generaron mayor importe total.

Esta comparación es muy importante porque nos recuerda que la variable elegida cambia la interpretación del análisis. Un producto caro puede vender pocas unidades. Un producto barato puede vender muchas unidades. El importe total permite combinar esas dos dimensiones y observar el valor económico de cada venta.

Crear nuevas columnas nos permite construir variables más cercanas a la pregunta que realmente queremos responder.

## Crear una columna con un valor constante

No siempre creamos columnas nuevas a partir de cálculos. A veces necesitamos agregar una columna que tenga el mismo valor para todas las filas. Por ejemplo, podríamos querer registrar que todos los datos de este `DataFrame` pertenecen a una misma tienda o a una misma fuente de información.

Supongamos que queremos agregar una columna llamada `origen` con el valor "Tienda escolar".

```
df["origen"] = "Tienda escolar"

df
```

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
0	Cuaderno	Librería	1200	15	Centro	Efectivo	2024-03-01
1	Lapicera	Librería	350	40	Centro	Debito	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
3	Regla	Librería	500	25	Sur	Efectivo	2024-03-02

índice	producto	categoría	precio	cantidad_vendida	sucursal	forma_pago	fecha
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03
5	Calculadora	Tecnología	18500	3	Centro	Credito	2024-03-03
6	Auriculares	Tecnología	7600	6	Sur	Credito	2024-03-04
7	Resaltadores	Librería	2100	12	Centro	Debito	2024-03-04
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05
9	Compas	Librería	2600	10	Sur	Efectivo	2024-03-05
10	Pendrive	Tecnología	9800	7	Centro	Debito	2024-03-06
11	Carpeta	Librería	950	18	Norte	Efectivo	2024-03-06
12	Goma	Librería	300	30	Sur	Efectivo	2024-03-07
13	Lapiz	Librería	250	50	Centro	Debito	2024-03-07
14	Agenda	Librería	4300	9	Sur	Credito	2024-03-08

Pandas agrega la columna `origen` y coloca el mismo valor en todas las filas.

Este tipo de columna puede ser útil cuando combinamos datos de distintas fuentes. Por ejemplo, si más adelante tuviéramos ventas de varias tiendas, podríamos agregar una columna que indique de dónde proviene cada registro.

Aunque este ejemplo es simple, muestra una idea importante: para crear una columna nueva, solo necesitamos asignar valores a un nombre de columna que todavía no existe.

La estructura general es:

```
df["nueva_columna"] = valores
```

Esos valores pueden ser un único valor constante, una operación entre columnas, una condición o una transformación más elaborada.

## Crear una columna a partir de una condición

También podemos crear columnas nuevas usando condiciones.

Por ejemplo, ahora que tenemos la columna `importe_total`, podríamos clasificar cada venta según su valor. Una forma simple sería marcar si una venta tuvo un importe alto o no.

Supongamos que queremos considerar como venta de importe alto a toda venta cuyo `importe_total` sea mayor o igual que `50000`.

Primero podemos construir la condición:

```
df["importe_total"] >= 50000
```

índice	importe_total
0	False
1	False
2	False
3	False
4	False
5	True
6	False
7	False
8	True
9	False
10	True
11	False
12	False
13	False
14	False

```
dtype: bool
```

La salida es una máscara booleana. Cada fila recibe `True` si su importe total es mayor o igual que `50000`, y `False` si no lo es.

Ahora podemos usar esa condición para crear una nueva columna llamada `venta_alta`.



```
df["venta_alta"] = df["importe_total"] >= 50000

df[["producto", "precio", "cantidad_vendida", "importe_total", "venta_alta"]]
```

índice	producto	precio	cantidad_vendida	importe_total	venta_alta
0	Cuaderno	1200	15	18000	False
1	Lapicera	350	40	14000	False
2	Mochila	8500	5	42500	False
3	Regla	500	25	12500	False
4	Cartuchera	3200	8	25600	False
5	Calculadora	18500	3	55500	True
6	Auriculares	7600	6	45600	False
7	Resaltadores	2100	12	25200	False
8	Guardapolvo	14500	4	58000	True
9	Compas	2600	10	26000	False
10	Pendrive	9800	7	68600	True
11	Carpeta	950	18	17100	False
12	Goma	300	30	9000	False
13	Lapiz	250	50	12500	False
14	Agenda	4300	9	38700	False

La nueva columna `venta_alta` contiene valores booleanos: `True` o `False`.

Esto significa que cada fila quedó clasificada según una condición. Las ventas con importe total mayor o igual que `50000` aparecen como `True`. Las demás aparecen como `False`.

Este tipo de columna puede ser útil cuando queremos marcar registros que cumplen cierto criterio. Más adelante podremos usar columnas como esta para filtrar, contar o comparar grupos.

## Crear una columna con etiquetas de texto

La columna `venta_alta` que creamos recién contiene valores booleanos: `True` o `False`.

Eso puede ser útil, pero a veces queremos que la nueva columna tenga etiquetas más descriptivas. Por ejemplo, en lugar de mostrar `True` o `False`, podríamos crear una columna que diga `"Alta"` o `"Normal"`.

Para resolver este tipo de situación podemos usar `np.where()`.

`np.where()` es una función de NumPy que permite elegir entre dos valores según se cumpla o no una condición. En este caso, la vamos a usar para decir:

```
si la venta cumple la condición → escribir "Alta"
si no cumple la condición → escribir "Normal"
```

NumPy es una biblioteca muy usada en Python para trabajar con datos numéricos y operaciones sobre conjuntos de valores. Pandas se apoya muchas veces en NumPy, y por eso es habitual usarlas juntas.

Para poder usar `np.where()`, primero importamos NumPy con el alias `np`.

```
import numpy as np
```

Ahora podemos crear una columna llamada `tipo_venta`.

La regla será:

```
si importe_total es mayor o igual que 50000 → "Alta"
si no → "Normal"
```

Esa regla tiene tres partes: una condición, un valor para los casos donde la condición se cumple y otro valor para los casos donde la condición no se cumple.

```
df["tipo_venta"] = np.where(
 df["importe_total"] >= 50000,
 "Alta",
 "Normal"
)

df[["producto", "importe_total", "venta_alta", "tipo_venta"]]
```

índice	producto	importe_total	venta_alta	tipo_venta
0	Cuaderno	18000	False	Normal
1	Lapicera	14000	False	Normal
2	Mochila	42500	False	Normal
3	Regla	12500	False	Normal

índice	producto	importe_total	venta_alta	tipo_venta
4	Cartuchera	25600	False	Normal
5	Calculadora	55500	True	Alta
6	Auriculares	45600	False	Normal
7	Resaltadores	25200	False	Normal
8	Guardapolvo	58000	True	Alta
9	Compas	26000	False	Normal
10	Pendrive	68600	True	Alta
11	Carpeta	17100	False	Normal
12	Goma	9000	False	Normal
13	Lapiz	12500	False	Normal
14	Agenda	38700	False	Normal

La nueva columna `tipo_venta` clasifica cada registro usando una etiqueta de texto.

Cuando la condición `df["importe_total"] >= 50000` se cumple, `np.where()` devuelve el valor `"Alta"`. Cuando la condición no se cumple, devuelve el valor `"Normal"`. Ese resultado se guarda en la nueva columna `tipo_venta`.

Esta forma de trabajo es útil cuando queremos transformar una condición en una categoría más fácil de leer.

Podemos pensar la estructura de `np.where()` de esta manera:

```
np.where(condición, valor_si_verdadero, valor_si_falso)
```

En nuestro ejemplo:

```
np.where(
 df["importe_total"] >= 50000,
 "Alta",
 "Normal"
)
```

significa: si el importe total es mayor o igual que `50000`, escribí `"Alta"`; en caso contrario, escribí `"Normal"`.

Este tipo de clasificación aparece con frecuencia en el trabajo con datos. Nos permite crear categorías nuevas a partir de reglas definidas por el análisis.

## Asignar valores usando loc

Otra forma de crear o modificar columnas según una condición es usar `loc`.

Esta opción resulta muy útil cuando queremos asignar valores solo a las filas que cumplen cierto criterio.

Por ejemplo, podríamos crear una columna llamada `observacion` y colocar inicialmente el texto `"Sin observaciones"` para todas las filas.

```
df["observacion"] = "Sin observaciones"

df[["producto", "importe_total", "observacion"]]
```

índice	producto	importe_total	observacion
0	Cuaderno	18000	Sin observaciones
1	Lapicera	14000	Sin observaciones
2	Mochila	42500	Sin observaciones
3	Regla	12500	Sin observaciones
4	Cartuchera	25600	Sin observaciones
5	Calculadora	55500	Sin observaciones
6	Auriculares	45600	Sin observaciones
7	Resaltadores	25200	Sin observaciones
8	Guardapolvo	58000	Sin observaciones
9	Compas	26000	Sin observaciones
10	Pendrive	68600	Sin observaciones
11	Carpeta	17100	Sin observaciones
12	Goma	9000	Sin observaciones
13	Lapiz	12500	Sin observaciones
14	Agenda	38700	Sin observaciones

Ahora todas las filas tienen el mismo valor en la columna `observacion`.

Supongamos que queremos cambiar ese texto solamente para las ventas cuyo importe total sea mayor o igual que `50000`. En esas filas vamos a escribir `"Revisar venta alta"`.

```
df.loc[
 df["importe_total"] >= 50000,
```

```

 "observacion"
] = "Revisar venta alta"

df[["producto", "importe_total", "tipo_venta", "observacion"]]

```

índice	producto	importe_total	tipo_venta	observacion
0	Cuaderno	18000	Normal	Sin observaciones
1	Lapicera	14000	Normal	Sin observaciones
2	Mochila	42500	Normal	Sin observaciones
3	Regla	12500	Normal	Sin observaciones
4	Cartuchera	25600	Normal	Sin observaciones
5	Calculadora	55500	Alta	Revisar venta alta
6	Auriculares	45600	Normal	Sin observaciones
7	Resaltadores	25200	Normal	Sin observaciones
8	Guardapolvo	58000	Alta	Revisar venta alta
9	Compas	26000	Normal	Sin observaciones
10	Pendrive	68600	Alta	Revisar venta alta
11	Carpeta	17100	Normal	Sin observaciones
12	Goma	9000	Normal	Sin observaciones
13	Lapiz	12500	Normal	Sin observaciones
14	Agenda	38700	Normal	Sin observaciones

En este caso usamos `loc` para indicar dos cosas al mismo tiempo.

Primero indicamos qué filas queremos modificar:

```
df["importe_total"] >= 50000
```

Después indicamos qué columna queremos modificar:

```
"observacion"
```

La estructura general es:

```
df.loc[condición, "columna"] = nuevo_valor
```

Esta forma de trabajo permite modificar solo una parte del `DataFrame`. Las filas que cumplen la condición reciben el nuevo valor. Las filas que no cumplen la condición conservan el valor anterior.

Usar `loc` para asignar valores es una práctica muy frecuente en Pandas, especialmente cuando queremos crear categorías, marcar casos especiales o corregir valores en una columna.

## Revisar las columnas creadas

---

Después de crear varias columnas nuevas, conviene revisar cómo quedó el `DataFrame`.

Hasta ahora agregamos estas columnas:

```
importe_total
origen
venta_alta
tipo_venta
observacion
```

Podemos consultar los nombres de todas las columnas usando `columns`.

```
df.columns
```

```
Index(['producto', 'categoria', 'precio', 'cantidad_vendida', 'sucursal',
 'forma_pago', 'fecha', 'importe_total', 'origen', 'venta_alta',
 'tipo_venta', 'observacion'],
 dtype='object')
```

También podemos observar una vista reducida del `DataFrame` para concentrarnos solamente en las columnas más importantes para este capítulo.

```
df[
 [
 "producto",
 "precio",
 "cantidad_vendida",
 "importe_total",
 "tipo_venta",
 "observacion"
]
]
```

índice	producto	precio	cantidad_vendida	importe_total	tipo_venta	observacion
0	Cuaderno	1200	15	18000	Normal	Sin observaciones
1	Lapicera	350	40	14000	Normal	Sin observaciones
2	Mochila	8500	5	42500	Normal	Sin observaciones
3	Regla	500	25	12500	Normal	Sin observaciones
4	Cartuchera	3200	8	25600	Normal	Sin observaciones
5	Calculadora	18500	3	55500	Alta	Revisar venta alta
6	Auriculares	7600	6	45600	Normal	Sin observaciones
7	Resaltadores	2100	12	25200	Normal	Sin observaciones
8	Guardapolvo	14500	4	58000	Alta	Revisar venta alta
9	Compas	2600	10	26000	Normal	Sin observaciones
10	Pendrive	9800	7	68600	Alta	Revisar venta alta
11	Carpeta	950	18	17100	Normal	Sin observaciones
12	Goma	300	30	9000	Normal	Sin observaciones
13	Lapiz	250	50	12500	Normal	Sin observaciones
14	Agenda	4300	9	38700	Normal	Sin observaciones

Esta vista nos permite revisar si las nuevas columnas tienen sentido.

La columna `importe_total` surge de multiplicar `precio` por `cantidad_vendida`. La columna `venta_alta` indica con `True` o `False` si una venta supera cierto umbral. La columna `tipo_venta` expresa una clasificación similar, pero con etiquetas de texto. La columna `observacion` agrega una descripción para las ventas que queremos destacar.

También podemos usar `info()` para revisar la estructura general del `DataFrame` después de agregar columnas.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 12 columns):
Column Non-Null Count Dtype
--- -
0 producto 15 non-null object
1 categoria 15 non-null object
2 precio 15 non-null int64
3 cantidad_vendida 15 non-null int64
4 sucursal 15 non-null object
5 forma_pago 15 non-null object
6 fecha 15 non-null object
7 importe_total 15 non-null int64
8 origen 15 non-null object
9 venta_alta 15 non-null bool
10 tipo_venta 15 non-null object
11 observacion 15 non-null object
dtypes: bool(1), int64(3), object(8)
memory usage: 1.4+ KB
```

Al mirar la salida de `info()`, veremos que el `DataFrame` ahora tiene más columnas que al comienzo.

Esto muestra que crear columnas modifica la estructura del `DataFrame`. Por eso es importante revisar el resultado después de cada transformación relevante.

En análisis de datos, no alcanza con escribir una instrucción y seguir adelante. Cada vez que modificamos el dataset, conviene verificar que el cambio haya producido el resultado esperado.

## Errores frecuentes al crear columnas

Al crear nuevas columnas en Pandas, pueden aparecer algunos errores o confusiones frecuentes.

Una primera confusión es escribir mal el nombre de una columna existente. Por ejemplo, para crear `importe_total` usamos las columnas `precio` y `cantidad_vendida`. Si escribimos mal uno de esos nombres, Pandas no podrá encontrar la columna.



```
Esta instrucción produciría un error porque la columna se llama "precio", no "Precio".
df["importe_total_error"] = df["Precio"] * df["cantidad_vendida"]
```

Para evitar este problema, conviene revisar los nombres disponibles con:

```
df.columns
```

```
Index(['producto', 'categoria', 'precio', 'cantidad_vendida', 'sucursal',
 'forma_pago', 'fecha', 'importe_total', 'origen', 'venta_alta',
 'tipo_venta', 'observacion'],
 dtype='object')
```

Otra confusión frecuente es creer que una nueva columna se crea de manera temporal. En realidad, cuando escribimos una asignación como:

```
df["importe_total"] = df["precio"] * df["cantidad_vendida"]
```

la columna queda agregada al `DataFrame`. A partir de ese momento, forma parte de `df` y puede usarse en selecciones, filtros, ordenamientos o nuevos cálculos.

También debemos tener cuidado al crear columnas usando condiciones. Por ejemplo, si definimos una venta alta como una venta con importe total mayor o igual que `50000`, esa regla depende de una decisión nuestra. Otro análisis podría usar un umbral diferente, como `30000`, `75000` o el promedio de los importes.

Por eso, cuando creamos columnas a partir de reglas, conviene que el nombre de la columna sea claro y que el criterio usado esté explicado.

Otro punto importante es no crear demasiadas columnas sin necesidad. Agregar columnas puede enriquecer el análisis, pero también puede volver el `DataFrame` más difícil de leer si no tenemos claro para qué sirve cada nueva variable.

Una buena práctica es revisar el `DataFrame` después de cada transformación importante y conservar nombres de columnas descriptivos.

Por ejemplo, un nombre como `importe_total` es más claro que un nombre como `total`, `valor`, `dato_nuevo` o `columna_1`.

Al crear columnas conviene verificar tres cosas:

```
que los nombres de columnas usados existan
que la operación o condición represente lo que queremos calcular
que la nueva columna tenga un nombre claro
```

Crear columnas es una herramienta muy poderosa, pero también implica tomar decisiones sobre cómo queremos representar la información.

## Resumen del capítulo

---

En este capítulo aprendimos a crear nuevas columnas dentro de un `DataFrame`.

La idea central fue que muchas veces los datos originales no contienen directamente toda la información que necesitamos analizar. En esos casos, podemos construir nuevas variables a partir de columnas existentes, operaciones matemáticas, condiciones o reglas definidas durante el análisis.

Primero creamos la columna `importe_total`, multiplicando el precio unitario por la cantidad vendida:

```
df["importe_total"] = df["precio"] * df["cantidad_vendida"]
```

Esta nueva columna nos permitió responder preguntas que no podían resolverse mirando solamente `precio` o `cantidad_vendida` por separado. A partir de `importe_total`, pudimos ordenar ventas, construir rankings y observar cuáles generaron mayor valor económico.

También vimos que, una vez creada, una columna nueva puede usarse igual que cualquier otra columna del `DataFrame`. Podemos seleccionarla, ordenarla, filtrarla, mostrarla junto con otras variables o usarla para nuevos cálculos.

Después creamos una columna con un valor constante:

```
df["origen"] = "Tienda escolar"
```

Este tipo de columna puede ser útil cuando queremos registrar una fuente, una etiqueta general o un dato común a todas las filas.

Más adelante creamos columnas a partir de condiciones. Primero generamos una columna booleana:

```
df["venta_alta"] = df["importe_total"] >= 50000
```

Luego usamos `np.where()` para crear una columna con etiquetas de texto:

```
df["tipo_venta"] = np.where(
 df["importe_total"] >= 50000,
 "Alta",
```

```
"Normal"
)
```

También usamos `loc` para asignar valores solamente a las filas que cumplan una condición:

```
df.loc[
 df["importe_total"] >= 50000,
 "observacion"
] = "Revisar venta alta"
```

Esta forma de trabajo permite modificar una parte específica del `DataFrame` sin afectar todas las filas.

Finalmente revisamos las columnas creadas usando herramientas como `columns`, selección de columnas e `info()`. Esa verificación es importante porque cada nueva columna modifica la estructura del dataset.

La idea principal de este capítulo fue:

```
Crear columnas nuevas permite construir información que no estaba explícita en el
dataset original.
```

Crear columnas es una de las herramientas más importantes del trabajo con datos. Nos permite transformar un dataset inicial en una versión más útil para responder preguntas de análisis.

## Próximo paso

---

Ya sabemos seleccionar, filtrar, ordenar, construir rankings y crear nuevas columnas.

El siguiente paso será aprender a modificar columnas existentes. Muchas veces no necesitamos agregar una variable nueva, sino corregir, transformar o ajustar una columna que ya existe.

Por ejemplo, podemos querer cambiar nombres de categorías, normalizar textos, reemplazar valores, modificar formatos o preparar una columna para que sea más fácil de analizar.

Ese trabajo nos acercará cada vez más a una etapa fundamental del análisis de datos: la limpieza y preparación del dataset.

## Capítulo 11 — Modificar columnas existentes en un DataFrame

---

### Breve repaso

---

En el trabajo anterior sobre creación de columnas aprendimos que un dataset no siempre contiene directamente todas las variables que necesitamos para analizarlo.

A partir de columnas existentes pudimos construir nueva información. Por ejemplo, usamos `precio` y `cantidad_vendida` para crear una columna llamada `importe_total`, que representaba el valor total de cada venta. También creamos columnas con valores constantes, columnas booleanas y columnas con etiquetas de texto generadas a partir de condiciones.

En este capítulo vamos a trabajar con una idea complementaria: modificar columnas que ya existen.

Muchas veces no necesitamos agregar una columna nueva, sino corregir o transformar una columna del dataset. Por ejemplo, podríamos querer cambiar la forma en que aparecen escritas las categorías, convertir textos a minúsculas, reemplazar valores, corregir nombres inconsistentes o ajustar una columna para que sea más clara.

Este tipo de transformación es muy frecuente en el trabajo con datos. Los datasets reales suelen tener valores escritos de distintas maneras, categorías poco claras, textos con espacios innecesarios o datos que necesitan ser normalizados antes de analizarlos.

Al finalizar este capítulo deberías poder:

- Comprender qué significa modificar una columna existente.
- Reemplazar valores dentro de una columna.
- Usar `replace()` para corregir valores.
- Aplicar transformaciones simples sobre columnas de texto.
- Usar métodos de `.str` para modificar textos.
- Crear versiones corregidas de una columna.
- Decidir cuándo conviene modificar una columna y cuándo conviene crear una nueva.
- Verificar los cambios realizados sobre el `DataFrame`.
- Reconocer errores frecuentes al modificar columnas.

## Dataset de trabajo

---

Para trabajar con modificación de columnas vamos a usar una nueva versión del dataset de ventas de una tienda escolar.

Esta vez el dataset tendrá algunos problemas intencionales. Por ejemplo, algunas categorías aparecerán escritas de distintas maneras, algunas formas de pago tendrán mayúsculas o minúsculas mezcladas, y algunos textos tendrán espacios innecesarios.

Estos problemas son muy habituales en datasets reales. A veces los datos provienen de formularios cargados manualmente, de archivos combinados o de sistemas que no usan siempre los mismos criterios de escritura.

En este capítulo nos vamos a concentrar en modificar los valores de columnas existentes. Más adelante trabajaremos con otras operaciones estructurales, como renombrar, eliminar o reordenar columnas.

```
import pandas as pd

datos = {
 "producto": [
 "Cuaderno", "Lapicera", "Mochila", "Regla", "Cartuchera",
 "Calculadora", "Auriculares", "Resaltadores", "Guardapolvo", "Compas",
 "Pendrive", "Carpeta", "Goma", "Lapiz", "Agenda"
],
 "categoria": [
 "Libreria", "libreria", "Accesorios", " Libreria", "accesorios",
 "Tecnologia", "tecnologia", "Libreria ", "Indumentaria", "Libreria",
 "Tecnologia", " libreria", "Libreria", "libreria ", "Libreria"
],
 "precio": [
 1200, 350, 8500, 500, 3200,
 18500, 7600, 2100, 14500, 2600,
 9800, 950, 300, 250, 4300
],
 "cantidad_vendida": [
 15, 40, 5, 25, 8,
 3, 6, 12, 4, 10,
 7, 18, 30, 50, 9
],
 "sucursal": [
 "Centro", "Centro", "Norte", "Sur", "Norte",
 "Centro", "Sur", "Centro", "Norte", "Sur",
 "Centro", "Norte", "Sur", "Centro", "Sur"
],
 "forma_pago": [
 "Efectivo", "debito", "Credito", "Efectivo ", "Debito",
 "credito", "Credito", "Debito", "credito ", "Efectivo",
 "Debito", "efectivo", "Efectivo", "debito ", "Credito"
],
 "fecha": [
```

```

 "2024-03-01", "2024-03-01", "2024-03-02", "2024-03-02", "2024-03-03",
 "2024-03-03", "2024-03-04", "2024-03-04", "2024-03-05", "2024-03-05",
 "2024-03-06", "2024-03-06", "2024-03-07", "2024-03-07", "2024-03-08"
]
}

df = pd.DataFrame(datos)

df

```

índice	producto	categoria	precio	cantidad_vendida	sucursal	forma_pago	fecha
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01
1	Lapicera	libreria	350	40	Centro	debito	2024-03-01
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02
3	Regla	Libreria	500	25	Sur	Efectivo	2024-03-02
4	Cartuchera	accesorios	3200	8	Norte	Debito	2024-03-03
5	Calculadora	Tecnologia	18500	3	Centro	credito	2024-03-03
6	Auriculares	tecnologia	7600	6	Sur	Credito	2024-03-04
7	Resaltadores	Libreria	2100	12	Centro	Debito	2024-03-04
8	Guardapolvo	Indumentaria	14500	4	Norte	credito	2024-03-05
9	Compas	Libreria	2600	10	Sur	Efectivo	2024-03-05
10	Pendrive	Tecnologia	9800	7	Centro	Debito	2024-03-06
11	Carpeta	libreria	950	18	Norte	efectivo	2024-03-06
12	Goma	Libreria	300	30	Sur	Efectivo	2024-03-07
13	Lapiz	libreria	250	50	Centro	debito	2024-03-07
14	Agenda	Libreria	4300	9	Sur	Credito	2024-03-08

Al observar el `DataFrame`, algunos problemas pueden pasar desapercibidos a simple vista. Por ejemplo, `"Libreria"`, `"libreria"`, `" Libreria"` y `"Libreria "` parecen valores muy parecidos, pero Pandas los interpreta como valores distintos.

Lo mismo ocurre con algunas formas de pago, como `"Credito"`, `"credito"` o `"credito "`.

Este tipo de inconsistencias puede afectar filtros, conteos, agrupamientos y gráficos. Por eso, antes de analizar datos categóricos o textuales, muchas veces necesitamos revisar y modificar los valores de algunas columnas.

## Qué significa modificar una columna existente

Modificar una columna existente significa cambiar los valores que ya están dentro de esa columna.

Esto puede hacerse por distintos motivos. A veces necesitamos corregir errores de escritura. Otras veces queremos unificar categorías que aparecen escritas de formas distintas. También puede ocurrir que necesitemos quitar espacios innecesarios, pasar textos a minúsculas o mayúsculas, reemplazar abreviaturas o transformar una columna para que sea más fácil de analizar.

Por ejemplo, en nuestro dataset la columna `categoria` contiene valores como:

```
Libreria
libreria
 Libreria
Libreria
```

Para una persona, todos esos valores probablemente representen la misma categoría. Sin embargo, para Pandas son valores diferentes, porque algunos tienen mayúsculas distintas y otros tienen espacios al principio o al final.

Esto puede generar resultados engañosos. Si contamos cuántas ventas hay por categoría, Pandas separará esos valores como si fueran categorías distintas.

Veamos qué ocurre:

```
df["categoria"].value_counts()
```

categoria	count
Libreria	4
Tecnologia	2
libreria	1
Accesorios	1
Libreria	1
accesorios	1
tecnologia	1
Libreria	1
Indumentaria	1
libreria	1

categoria	count
libreria	1

`dtype: int64`

La salida muestra que la columna `categoria` no está unificada. Aparecen varias versiones de una misma categoría.

Este es un problema frecuente en datos reales. Antes de analizar una columna categórica, conviene revisar si sus valores están escritos de manera consistente.

Modificar una columna, en este contexto, no significa cambiar su nombre ni eliminarla. Significa transformar los valores que contiene para que sean más claros, coherentes o útiles para el análisis.

## Revisar valores únicos antes de modificar

Antes de modificar una columna, conviene revisar qué valores contiene.

Para eso podemos usar `unique()`, que muestra los valores distintos que aparecen en una columna.

```
df["categoria"].unique()
```

```
array(['Libreria', 'libreria', 'Accesorios', ' Libreria', 'accesorios',
 'Tecnologia', 'tecnologia', 'Libreria ', 'Indumentaria',
 ' libreria', 'libreria '], dtype=object)
```

La salida nos permite ver las distintas formas en que aparece escrita la categoría.

También podemos revisar la columna `forma_pago`:

```
df["forma_pago"].unique()
```

```
array(['Efectivo', 'debito', 'Credito', 'Efectivo ', 'Debito', 'credito',
 'credito ', 'efectivo', 'debito '], dtype=object)
```

Esta revisión es importante porque nos ayuda a decidir qué transformación necesitamos aplicar.

En este dataset aparecen principalmente dos tipos de problemas:



```
diferencias entre mayúsculas y minúsculas
espacios innecesarios al comienzo o al final del texto
```

Por ejemplo, "Credito" y "credito" representan la misma forma de pago, pero Pandas los considera valores diferentes. Lo mismo ocurre con "Efectivo" y "Efectivo ".

Antes de analizar columnas categóricas, conviene revisar sus valores únicos y detectar si hay categorías que deberían unificarse.

## Quitar espacios innecesarios

Uno de los problemas más comunes en columnas de texto es la presencia de espacios innecesarios.

Por ejemplo, estos valores parecen iguales a simple vista:

```
"Libreria"
" Libreria"
"Libreria "
```

Pero no son iguales para Pandas. El segundo tiene un espacio al comienzo y el tercero tiene un espacio al final.

Para eliminar espacios al principio y al final de cada texto podemos usar `.str.strip()`.

Primero vamos a aplicarlo sobre la columna `categoria`.

```
df["categoria"] = df["categoria"].str.strip()

df["categoria"].unique()
```

```
array(['Libreria', 'libreria', 'Accesorios', 'accesorios', 'Tecnologia',
 'tecnologia', 'Indumentaria'], dtype=object)
```

Ahora desaparecieron las variantes que se diferenciaban solamente por espacios al comienzo o al final.

El método `.str.strip()` no elimina espacios internos entre palabras. Su función es quitar espacios sobrantes a los costados del texto.

También podemos aplicar la misma transformación sobre la columna `forma_pago`.

```
df["forma_pago"] = df["forma_pago"].str.strip()

df["forma_pago"].unique()
```

```
array(['Efectivo', 'debito', 'Credito', 'Debito', 'credito', 'efectivo'],
 dtype=object)
```

Después de esta transformación, valores como "Efectivo " o "credito " quedan sin el espacio final.

Esta operación modifica las columnas existentes. No estamos creando columnas nuevas, sino reemplazando los valores originales por versiones corregidas.

## Unificar mayúsculas y minúsculas

Después de quitar espacios innecesarios, todavía puede quedar otro problema: valores escritos con distintas combinaciones de mayúsculas y minúsculas.

Por ejemplo, en la columna `categoria` podemos encontrar:

```
Libreria
libreria
Tecnologia
tecnologia
```

Para Pandas, "Libreria" y "libreria" son valores diferentes. Si queremos analizarlos como una misma categoría, necesitamos unificarlos.

Una estrategia posible es convertir todos los textos a minúsculas usando `.str.lower()`.

```
df["categoria"] = df["categoria"].str.lower()

df["categoria"].unique()
```

```
array(['libreria', 'accesorios', 'tecnologia', 'indumentaria'],
 dtype=object)
```

Ahora todas las categorías quedaron escritas en minúsculas.

También podemos aplicar la misma transformación a la columna `forma_pago`.

```
df["forma_pago"] = df["forma_pago"].str.lower()
```

```
df["forma_pago"].unique()
```

```
array(['efectivo', 'debito', 'credito'], dtype=object)
```

Después de esta transformación, valores como "Credito", "credito" y "credito " quedan unificados como "credito".

Esto mejora la consistencia de los datos. Si ahora contamos los valores de `categoria` o `forma_pago`, Pandas ya no separará categorías que en realidad representan lo mismo.

```
df["categoria"].value_counts()
```

categoria	count
libreria	9
tecnologia	3
accesorios	2
indumentaria	1

```
dtype: int64
```

```
df["forma_pago"].value_counts()
```

forma_pago	count
efectivo	5
debito	5
credito	5

```
dtype: int64
```

Unificar mayúsculas y minúsculas es una operación frecuente cuando trabajamos con textos o categorías.

No siempre es obligatorio convertir todo a minúsculas. En algunos casos podríamos preferir mayúsculas iniciales o nombres con cierto formato. Lo importante es que los valores sean consistentes.

## Reemplazar valores con replace()

Otra forma de modificar una columna existente es reemplazar algunos valores por otros.

Para eso podemos usar `replace()`.

Este método es útil cuando queremos corregir valores específicos. Por ejemplo, después de unificar la columna `categoria`, tenemos valores como:

```
libreria
accesorios
tecnologia
indumentaria
```

Podríamos decidir que queremos mostrar esas categorías con mayúscula inicial para que resulten más claras al leer la tabla.

Una forma de hacerlo es usar un diccionario de reemplazos.

```
reemplazos_categoria = {
 "libreria": "Libreria",
 "accesorios": "Accesorios",
 "tecnologia": "Tecnologia",
 "indumentaria": "Indumentaria"
}

df["categoria"] = df["categoria"].replace(reemplazos_categoria)

df["categoria"].unique()
```

```
array(['Libreria', 'Accesorios', 'Tecnologia', 'Indumentaria'],
 dtype=object)
```

En este caso, el diccionario indica qué valor debe ser reemplazado por qué nuevo valor.

Por ejemplo:

```
"libreria" → "Libreria"
"accesorios" → "Accesorios"
"tecnologia" → "Tecnologia"
```

También podemos aplicar una lógica similar sobre la columna `forma_pago`.

```
reemplazos_pago = {
 "efectivo": "Efectivo",
 "debito": "Debito",
 "credito": "Credito"
}

df["forma_pago"] = df["forma_pago"].replace(reemplazos_pago)

df["forma_pago"].unique()
```

```
array(['Efectivo', 'Debito', 'Credito'], dtype=object)
```

Ahora los valores de `forma_pago` también quedaron unificados con un formato más claro.

El método `replace()` es muy útil cuando conocemos exactamente qué valores queremos cambiar. A diferencia de `.str.lower()` o `.str.strip()`, que aplican una transformación general sobre todos los textos, `replace()` permite indicar reemplazos puntuales.

Después de aplicar estos reemplazos, podemos volver a revisar los conteos.

```
df["categoria"].value_counts()
```

categoria	count
Libreria	9
Tecnologia	3
Accesorios	2
Indumentaria	1

```
dtype: int64
```

```
df["forma_pago"].value_counts()
```

forma_pago	count
Efectivo	5
Debito	5
Credito	5

```
dtype: int64
```

Esta verificación nos permite confirmar que las categorías quedaron unificadas.

Antes de la limpieza, Pandas separaba valores que representaban lo mismo pero estaban escritos de manera diferente. Después de aplicar `strip()`, `lower()` y `replace()`, las columnas categóricas quedan mucho más consistentes.

## Modificar una columna o crear una nueva

---

En los ejemplos anteriores modificamos directamente las columnas `categoria` y `forma_pago`.

Por ejemplo, escribimos:

```
df["categoria"] = df["categoria"].str.strip()
```

y luego:

```
df["categoria"] = df["categoria"].str.lower()
```

Eso significa que los valores originales de la columna fueron reemplazados por los valores corregidos.

Esta forma de trabajo es válida cuando estamos seguros de que la transformación es necesaria y no necesitamos conservar la versión original. Por ejemplo, quitar espacios al comienzo o al final de un texto suele ser una corrección segura, porque esos espacios normalmente no tienen significado.

Sin embargo, hay situaciones en las que conviene crear una nueva columna en lugar de sobrescribir la original.

Por ejemplo, podríamos conservar `categoria` tal como vino en el dataset y crear una nueva columna llamada `categoria_limpia`.

```
Volvemos a crear el DataFrame original para mostrar el ejemplo desde cero.

df = pd.DataFrame(datos)

df["categoria_limpia"] = (
 df["categoria"]
 .str.strip()
 .str.lower()
 .replace({
 "libreria": "Libreria",
 "accesorios": "Accesorios",
 "tecnologia": "Tecnologia",
 "indumentaria": "Indumentaria"
 })
)
```

```
df[["categoria", "categoria_limpia"]]
```

índice	categoria	categoria_limpia
0	Libreria	Libreria
1	libreria	Libreria
2	Accesorios	Accesorios
3	Libreria	Libreria
4	accesorios	Accesorios
5	Tecnologia	Tecnologia
6	tecnologia	Tecnologia
7	Libreria	Libreria
8	Indumentaria	Indumentaria
9	Libreria	Libreria
10	Tecnologia	Tecnologia
11	libreria	Libreria
12	Libreria	Libreria
13	libreria	Libreria
14	Libreria	Libreria

Ahora tenemos dos columnas:

```
categoria → conserva los valores originales
categoria_limpia → contiene los valores corregidos
```

Esta estrategia puede ser útil cuando queremos comparar la versión original con la versión modificada, documentar el proceso de limpieza o evitar perder información.

No hay una única regla para todos los casos. En general, podemos pensar así:

Si la corrección es simple y segura, podemos modificar la columna original.

Si la transformación cambia la interpretación de los datos o queremos conservar evidencia del valor original, conviene crear una nueva columna.

En trabajos reales, conservar una versión original puede ser muy útil, especialmente durante las primeras etapas de limpieza y revisión.

## Encadenar transformaciones

En Pandas es común aplicar varias transformaciones seguidas sobre una misma columna.

Por ejemplo, para limpiar la columna `categoria`, podemos querer hacer tres cosas:

```
quitar espacios al comienzo y al final
pasar el texto a minúsculas
reemplazar valores por una versión con formato uniforme
```

Podemos hacer esas transformaciones paso a paso, guardando el resultado después de cada operación. Pero también podemos encadenarlas en una misma instrucción.

```
df["categoria_limpia"] = (
 df["categoria"]
 .str.strip()
 .str.lower()
 .replace({
 "libreria": "Libreria",
 "accesorios": "Accesorios",
 "tecnologia": "Tecnologia",
 "indumentaria": "Indumentaria"
 })
)

df[["categoria", "categoria_limpia"]]
```

índice	categoria	categoria_limpia
0	Libreria	Libreria
1	libreria	Libreria
2	Accesorios	Accesorios
3	Libreria	Libreria
4	accesorios	Accesorios
5	Tecnologia	Tecnologia
6	tecnologia	Tecnologia
7	Libreria	Libreria
8	Indumentaria	Indumentaria
9	Libreria	Libreria
10	Tecnologia	Tecnologia
11	libreria	Libreria



índice	categoría	categoría_limpia
12	Librería	Librería
13	librería	Librería
14	Librería	Librería

Esta forma de escribir el código puede parecer más extensa al principio, pero tiene una ventaja: muestra con claridad la secuencia de transformaciones aplicadas.

Primero partimos de la columna original:

```
df["categoría"]
```

Luego quitamos espacios innecesarios:

```
.str.strip()
```

Después pasamos el texto a minúsculas:

```
.str.lower()
```

Finalmente reemplazamos los valores por una versión más uniforme:

```
.replace({...})
```

El uso de paréntesis permite escribir la instrucción en varias líneas. Esto mejora la legibilidad, especialmente cuando la transformación tiene varios pasos.

Encadenar transformaciones es una práctica muy habitual en Pandas. Permite construir procesos de limpieza más claros y evita crear muchas variables intermedias que solo usaríamos una vez.

## Modificar columnas numéricas

No solo podemos modificar columnas de texto. También podemos transformar columnas numéricas.

Por ejemplo, supongamos que queremos expresar los precios con un aumento del 10%. Podemos modificar la columna `precio` multiplicando sus valores por `1.10`.

Antes de hacerlo, conviene observar algunos valores actuales.

```
df[["producto", "precio"]].head()
```

índice	producto	precio
0	Cuaderno	1200
1	Lapicera	350
2	Mochila	8500
3	Regla	500
4	Cartuchera	3200

Ahora aplicamos el aumento:

```
df["precio"] = df["precio"] * 1.10

df[["producto", "precio"]].head()
```

índice	producto	precio
0	Cuaderno	1320.0
1	Lapicera	385.0
2	Mochila	9350.0
3	Regla	550.0
4	Cartuchera	3520.0

La columna `precio` fue modificada. Cada valor quedó multiplicado por `1.10`, es decir, aumentó un 10%.

Este ejemplo muestra que una columna numérica puede transformarse mediante operaciones matemáticas. Podemos sumar, restar, multiplicar, dividir o aplicar cálculos más complejos.

Sin embargo, debemos tener cuidado. En este caso modificamos directamente la columna original. Si necesitáramos conservar los precios originales, sería mejor crear una columna nueva, por ejemplo `precio_actualizado`.

Veamos esa alternativa recreando nuevamente el `DataFrame` original.

Como al recrear `df` desde cero se pierden las columnas que habíamos creado antes, vamos a volver a generar también `categoria_limpia`. De esa manera, más adelante podremos verificar tanto la transformación de texto como la nueva columna numérica `precio_actualizado`.

```
df = pd.DataFrame(datos)

df["categoria_limpia"] = (
 df["categoria"]
 .str.strip()
 .str.lower()
 .replace({
 "libreria": "Libreria",
 "accesorios": "Accesorios",
 "tecnologia": "Tecnologia",
 "indumentaria": "Indumentaria"
 })
)

df["precio_actualizado"] = df["precio"] * 1.10

df[["producto", "categoria", "categoria_limpia", "precio", "precio_actualizado"]].head()
```

índice	producto	categoria	categoria_limpia	precio	precio_actualizado
0	Cuaderno	Libreria	Libreria	1200	1320.0
1	Lapicera	libreria	Libreria	350	385.0
2	Mochila	Accesorios	Accesorios	8500	9350.0
3	Regla	Libreria	Libreria	500	550.0
4	Cartuchera	accesorios	Accesorios	3200	3520.0

Ahora conservamos la columna `precio` original y agregamos una nueva columna con el precio actualizado. Además, volvimos a crear `categoria_limpia` para mantener disponible la versión normalizada de la categoría.

Esta decisión depende del objetivo del análisis. Si estamos corrigiendo un error, puede tener sentido modificar la columna original. Si estamos simulando un escenario, comparando versiones o aplicando una transformación que queremos documentar, suele ser mejor crear una nueva columna.

## Verificar los cambios realizados

Cada vez que modificamos una columna, conviene revisar el resultado.

Modificar datos sin verificar puede llevarnos a errores difíciles de detectar más adelante. Por ejemplo, podríamos pensar que unificamos correctamente una categoría, pero dejar todavía valores escritos de distintas maneras. También podríamos modificar una columna numérica y no advertir que el resultado quedó con más decimales de los esperados.

Después de transformar columnas de texto, podemos revisar los valores únicos o los conteos:

```
df["categoria_limpia"].unique()
```

```
array(['Libreria', 'Accesorios', 'Tecnologia', 'Indumentaria'],
 dtype=object)
```

```
df["categoria_limpia"].value_counts()
```

categoria_limpia	count
Libreria	9
Tecnologia	3
Accesorios	2
Indumentaria	1

dtype: int64

Después de modificar o crear columnas numéricas, podemos observar algunas filas y calcular estadísticas básicas:

```
df[["producto", "precio", "precio_actualizado"]].head()
```

índice	producto	precio	precio_actualizado
0	Cuaderno	1200	1320.0
1	Lapicera	350	385.0
2	Mochila	8500	9350.0
3	Regla	500	550.0
4	Cartuchera	3200	3520.0

```
df[["precio", "precio_actualizado"]].describe()
```

índice	precio	precio_actualizado
count	15.000000	15.000000
mean	4976.666667	5474.333333
std	5668.265881	6235.092469
min	250.000000	275.000000

índice	precio	precio_actualizado
25%	725.000000	797.500000
50%	2600.000000	2860.000000
75%	8050.000000	8855.000000
max	18500.000000	20350.000000

Estas verificaciones no son pasos accesorios. Forman parte del proceso de trabajo con datos. Una transformación debería poder responder dos preguntas:

¿Qué cambio hice?  
¿Cómo sé que el cambio quedó bien?

La primera pregunta apunta a la intención de la transformación. La segunda apunta a la verificación del resultado.

En Pandas, algunas herramientas útiles para revisar cambios son:

Herramienta	Para qué puede servir
<code>head()</code>	Ver las primeras filas después de un cambio.
<code>unique()</code>	Revisar valores distintos en una columna.
<code>value_counts()</code>	Contar cuántas veces aparece cada valor.
<code>describe()</code>	Obtener un resumen de columnas numéricas.
<code>info()</code>	Revisar estructura general y tipos de datos.
Selección de columnas	Comparar columnas originales y transformadas.

La idea principal es no avanzar a ciegas. Cada transformación importante debería ir acompañada por alguna forma de comprobación.

## Errores frecuentes al modificar columnas

Modificar columnas es una tarea muy común, pero también puede generar errores si no revisamos bien lo que estamos haciendo.

Un primer error frecuente es escribir mal el nombre de una columna. Si queremos transformar `categoria`, pero escribimos `"Categoría"` o `"categoria "` con un espacio al final, Pandas no encontrará la columna.

```
Esta instrucción produciría un error porque la columna se llama "categoria", no
"Categoría".
df["Categoría"].str.lower()
```

Para evitar este problema, conviene revisar los nombres disponibles:

```
df.columns
```

```
Index(['producto', 'categoria', 'precio', 'cantidad_vendida', 'sucursal',
 'forma_pago', 'fecha', 'categoria_limpia', 'precio_actualizado'],
 dtype='object')
```

Otro error posible es aplicar métodos de texto sobre columnas que no contienen texto. Por ejemplo, `.str.lower()` tiene sentido en una columna textual como `categoria`, pero no en una columna numérica como `precio`.

```
Esta instrucción produciría un error porque precio es una columna numérica.
df["precio"].str.lower()
```

También debemos tener cuidado cuando modificamos directamente una columna. Si escribimos:

```
df["precio"] = df["precio"] * 1.10
```

la columna `precio` queda reemplazada por los nuevos valores. Si necesitábamos conservar los precios originales, debimos crear una columna nueva antes de sobrescribirla.

Por eso, antes de modificar una columna existente, conviene preguntarnos:

```
¿quiero reemplazar los valores originales?
¿necesito conservar una versión original?
¿sería mejor crear una nueva columna?
```

Otro error frecuente es creer que una transformación quedó bien sin verificarla. Por ejemplo, después de limpiar categorías, conviene usar `unique()` o `value_counts()` para comprobar si los valores quedaron unificados.

En general, antes de modificar columnas conviene seguir esta rutina:

```
revisar los valores originales
aplicar la transformación
verificar el resultado
```

Esta forma de trabajo reduce errores y ayuda a construir un análisis más confiable.

## Resumen del capítulo

---

En este capítulo aprendimos a modificar columnas existentes dentro de un `DataFrame`.

Hasta ahora habíamos creado columnas nuevas a partir de información ya disponible. En este trabajo dimos un paso complementario: transformar valores que ya estaban presentes en el dataset.

Comenzamos con un dataset que tenía inconsistencias intencionales en columnas de texto. Algunas categorías aparecían con mayúsculas y minúsculas mezcladas, y otras tenían espacios innecesarios al comienzo o al final. Vimos que, aunque para una persona esos valores pueden parecer equivalentes, Pandas los interpreta como valores distintos.

Para revisar ese problema usamos herramientas como:

```
df["categoria"].unique()
```

y:

```
df["categoria"].value_counts()
```

Después aplicamos transformaciones de texto. Usamos `.str.strip()` para quitar espacios innecesarios al comienzo y al final de los valores:

```
df["categoria"] = df["categoria"].str.strip()
```

Luego usamos `.str.lower()` para unificar mayúsculas y minúsculas:

```
df["categoria"] = df["categoria"].str.lower()
```

También trabajamos con `replace()`, que permite reemplazar valores específicos dentro de una columna. Esta herramienta resultó útil para transformar valores normalizados en etiquetas más claras:

```
df["categoria"] = df["categoria"].replace(reemplazos_categoria)
```

Más adelante discutimos una decisión importante: a veces conviene modificar directamente la columna original, pero otras veces es preferible crear una nueva versión corregida. Por ejemplo, podemos conservar `categoria` y crear

`categoria_limpia` para comparar los valores originales con los valores transformados.

También vimos que las transformaciones pueden encadenarse:

```
df["categoria_limpia"] = (
 df["categoria"]
 .str.strip()
 .str.lower()
 .replace({
 "libreria": "Libreria",
 "accesorios": "Accesorios",
 "tecnologia": "Tecnologia",
 "indumentaria": "Indumentaria"
 })
)
```

Esta forma de escritura permite mostrar con claridad la secuencia de pasos aplicada sobre una columna.

Luego trabajamos con columnas numéricas. Vimos que también podemos modificar valores numéricos mediante operaciones matemáticas, por ejemplo aplicando un aumento del 10% sobre los precios. En ese punto volvimos a distinguir entre sobrescribir una columna existente o crear una nueva columna, como `precio_actualizado`.

Finalmente, insistimos en la importancia de verificar cada transformación. Después de modificar columnas, conviene revisar valores únicos, conteos, primeras filas, resúmenes estadísticos o la estructura general del `DataFrame`.

La idea principal de este capítulo fue:

Modificar columnas existentes permite corregir, normalizar o transformar datos que ya forman parte del `DataFrame`.

Modificar datos es una tarea poderosa, pero requiere cuidado. Cada transformación debería tener un propósito claro y una verificación posterior.

## Próximo paso

---

Ya sabemos crear nuevas columnas y modificar columnas existentes.

El siguiente paso será trabajar con la organización general del `DataFrame`: renombrar columnas, eliminar columnas que no necesitamos y reordenar variables para que la tabla sea más clara.



Estas operaciones no cambian solamente valores individuales, sino la estructura del dataset. Por eso conviene tratarlas como una etapa propia dentro de la preparación de datos.

## Capítulo 12 — Organizar columnas en un DataFrame

---

### Breve repaso

---

En el trabajo anterior sobre modificación de columnas aprendimos a transformar valores que ya estaban dentro de un `DataFrame`.

Vimos cómo corregir textos con espacios innecesarios, unificar mayúsculas y minúsculas, reemplazar valores específicos con `replace()`, encadenar transformaciones y modificar columnas numéricas. También discutimos cuándo conviene modificar una columna existente y cuándo es mejor crear una nueva versión para conservar los datos originales.

En este capítulo vamos a trabajar con otro tipo de transformación: la organización de columnas.

Hasta ahora nos concentramos principalmente en los valores de las columnas. Ahora vamos a mirar la estructura del `DataFrame`: cómo se llaman sus columnas, cuáles queremos conservar, cuáles podemos eliminar y en qué orden conviene presentarlas.

Estas operaciones son importantes porque un dataset no solo debe contener datos correctos. También debe estar organizado de manera clara. Un `DataFrame` con nombres de columnas confusos, variables innecesarias o columnas desordenadas puede dificultar el análisis, aunque sus valores sean correctos.

A lo largo del capítulo vamos a trabajar con operaciones como renombrar columnas, eliminar columnas, reordenar variables y construir una versión más clara del dataset.

Al finalizar este capítulo deberías poder:

- Revisar los nombres de las columnas de un `DataFrame`.
- Renombrar columnas usando `rename()`.
- Eliminar columnas usando `drop()`.
- Comprender cuándo conviene eliminar una columna y cuándo conservarla.
- Reordenar columnas para mejorar la lectura del dataset.
- Crear una versión organizada del `DataFrame`.
- Verificar la estructura final luego de modificar columnas.
- Reconocer errores frecuentes al organizar columnas.

## Dataset de trabajo

Para trabajar con organización de columnas vamos a usar una nueva versión del dataset de ventas de una tienda escolar.

Esta vez, el problema principal no estará en los valores de las filas, sino en la estructura de la tabla. Algunas columnas tendrán nombres poco claros, abreviaturas, espacios o estilos de escritura diferentes.

Esto es muy frecuente en datasets reales. A veces recibimos archivos donde las columnas fueron nombradas por distintas personas, exportadas desde sistemas externos o armadas rápidamente sin seguir un criterio uniforme.

Nuestro objetivo será transformar esa estructura inicial en un `DataFrame` más claro y cómodo para analizar.

```
import pandas as pd

datos = {
 "Producto vendido": [
 "Cuaderno", "Lapicera", "Mochila", "Regla", "Cartuchera",
 "Calculadora", "Auriculares", "Resaltadores", "Guardapolvo", "Compas"
],
 "Cat": [
 "Libreria", "Libreria", "Accesorios", "Libreria", "Accesorios",
 "Tecnologia", "Tecnologia", "Libreria", "Indumentaria", "Libreria"
],
 "Precio Unitario": [
 1200, 350, 8500, 500, 3200,
 18500, 7600, 2100, 14500, 2600
],
 "Cant.": [
 15, 40, 5, 25, 8,
 3, 6, 12, 4, 10
],
 "Sucursal Venta": [
 "Centro", "Centro", "Norte", "Sur", "Norte",
 "Centro", "Sur", "Centro", "Norte", "Sur"
],
 "Pago": [
 "Efectivo", "Debito", "Credito", "Efectivo", "Debito",
 "Credito", "Credito", "Debito", "Credito", "Efectivo"
],
 "Fecha venta": [
 "2024-03-01", "2024-03-01", "2024-03-02", "2024-03-02", "2024-03-03",
 "2024-03-03", "2024-03-04", "2024-03-04", "2024-03-05", "2024-03-05"
],
 "Código interno": [
 "A001", "A002", "B010", "A003", "B011",
 "T020", "T021", "A004", "I030", "A005"
]
}
```

```
}

df = pd.DataFrame(datos)

df
```

índice	Producto vendido	Cat	Precio Unitario	Cant.	Sucursal Venta	Pago	Fecha venta	Código interno
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01	A001
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01	A002
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02	B010
3	Regla	Libreria	500	25	Sur	Efectivo	2024-03-02	A003
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03	B011
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2024-03-03	T020
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2024-03-04	T021
7	Resaltadores	Libreria	2100	12	Centro	Debito	2024-03-04	A004
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2024-03-05	I030
9	Compas	Libreria	2600	10	Sur	Efectivo	2024-03-05	A005

Al observar el `DataFrame`, vemos que los datos son comprensibles, pero los nombres de las columnas no siguen un criterio uniforme.

Algunas columnas tienen espacios, como `Producto vendido` o `Precio Unitario`. Otras usan abreviaturas, como `Cat` o `Cant.`. También hay nombres que podrían ser más descriptivos, como `Pago`, que en realidad representa la forma de pago.

Estos detalles pueden parecer menores, pero afectan la claridad del código. Es más cómodo y menos propenso a errores trabajar con nombres de columnas consistentes, descriptivos y fáciles de escribir.

## Qué significa organizar columnas

Organizar columnas significa revisar y ajustar la estructura del `DataFrame` para que sea más clara y fácil de usar.

Esto puede incluir varias tareas. Por ejemplo, podemos renombrar columnas para que tengan nombres más descriptivos, eliminar columnas que no necesitamos, reordenar variables para que la tabla sea más legible o construir una versión final del dataset con las columnas que vamos a usar en el análisis.

En este capítulo no nos vamos a concentrar en corregir los valores dentro de las columnas, sino en la estructura de la tabla.

Por ejemplo, una columna llamada `Cat` puede entenderse si conocemos el contexto, pero no es tan clara como `categoria`. Una columna llamada `Cant.` puede ser breve, pero un nombre como `cantidad_vendida` describe mejor qué representa la variable.

También es importante evitar nombres incómodos para escribir código. Los nombres con espacios, puntos o mayúsculas mezcladas pueden funcionar, pero suelen ser menos prácticos. Por ejemplo:

```
df["Producto vendido"]
```

funciona correctamente, pero es más cómodo trabajar con:

```
df["producto"]
```

o con:

```
df["producto_vendido"]
```

Una buena organización de columnas no cambia los datos en sí mismos, pero mejora la claridad del trabajo. Hace que el código sea más fácil de leer, reduce errores de escritura y ayuda a que otras personas comprendan mejor el dataset.

## Revisar los nombres actuales

---

Antes de renombrar, eliminar o reordenar columnas, conviene revisar qué nombres tiene actualmente el `DataFrame`.

Para eso podemos usar el atributo `columns`.

```
df.columns
```

```
Index(['Producto vendido', 'Cat', 'Precio Unitario', 'Cant.', 'Sucursal Venta',
 'Pago', 'Fecha venta', 'Código interno'],
 dtype='object')
```

La salida muestra todos los nombres de columnas del `DataFrame`.

En nuestro caso, vemos nombres como:

```
Producto vendido
Cat
Precio Unitario
Cant.
Sucursal Venta
Pago
Fecha venta
Código interno
```

Algunos de estos nombres son comprensibles, pero no todos son igualmente claros o cómodos para trabajar. Por ejemplo, `Cat` probablemente significa categoría, pero el nombre completo `categoría` sería más explícito. Algo parecido ocurre con `Cant.`, que podemos interpretar como cantidad, aunque `cantidad_vendida` sería más descriptivo.

También podemos convertir los nombres de columnas en una lista de Python:

```
list(df.columns)
```

```
['Producto vendido',
 'Cat',
 'Precio Unitario',
 'Cant.',
 'Sucursal Venta',
 'Pago',
 'Fecha venta',
 'Código interno']
```

Esta forma suele ser más cómoda para revisar los nombres, copiarlos o planificar cambios.

Antes de organizar columnas, es útil hacernos algunas preguntas:

```
¿Los nombres son claros?
¿Hay abreviaturas poco descriptivas?
¿Hay espacios o signos que puedan incomodar al escribir código?
¿Los nombres siguen un mismo estilo?
¿Hay columnas que no vamos a necesitar?
¿Conviene reordenar las columnas para que la tabla se lea mejor?
```

Estas preguntas nos ayudan a tomar decisiones antes de modificar la estructura del `DataFrame`.

## Renombrar columnas con rename()

Una de las formas más claras de cambiar nombres de columnas en Pandas es usar el método `rename()`.

Este método permite indicar qué columnas queremos renombrar y qué nuevo nombre debe recibir cada una.

Para eso usamos un diccionario. En ese diccionario, cada clave es el nombre actual de una columna, y cada valor es el nuevo nombre que queremos asignarle.

```
df = df.rename(columns={
 "Producto vendido": "producto",
 "Cat": "categoria",
 "Precio Unitario": "precio_unitario",
 "Cant.": "cantidad_vendida",
 "Sucursal Venta": "sucursal",
 "Pago": "forma_pago",
 "Fecha venta": "fecha"
})

df
```

índice	producto	categoria	precio_unitario	cantidad_vendida	sucursal	forma_pago	fecha
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2020-01-01
1	Lapicera	Libreria	350	40	Centro	Debito	2020-01-02
2	Mochila	Accesorios	8500	5	Norte	Credito	2020-01-03
3	Regla	Libreria	500	25	Sur	Efectivo	2020-01-04
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2020-01-05
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2020-01-06
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2020-01-07
7	Resaltadores	Libreria	2100	12	Centro	Debito	2020-01-08
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2020-01-09
9	Compas	Libreria	2600	10	Sur	Efectivo	2020-01-10

Ahora el `DataFrame` tiene nombres de columnas más claros y consistentes.

Podemos revisar nuevamente los nombres:

```
df.columns
```

```
Index(['producto', 'categoria', 'precio_unitario', 'cantidad_vendida',
 'sucursal', 'forma_pago', 'fecha', 'Código interno'],
 dtype='object')
```

Los nombres nuevos siguen un criterio común:

```
producto
categoria
precio_unitario
cantidad_vendida
sucursal
forma_pago
fecha
Código interno
```

La columna `Código interno` todavía conserva su nombre original. Esto ocurrió porque no la incluimos en el diccionario de reemplazos.

Con `rename()`, Pandas solo cambia los nombres de las columnas indicadas. Las columnas que no aparecen en el diccionario quedan como estaban.

Esto puede ser útil cuando queremos modificar solo algunas columnas sin afectar el resto del `DataFrame`.

## Guardar el resultado de `rename()`

En la celda anterior escribimos:

```
df = df.rename(columns={
 "Producto vendido": "producto",
 "Cat": "categoria",
 "Precio Unitario": "precio_unitario",
 "Cant.": "cantidad_vendida",
 "Sucursal Venta": "sucursal",
 "Pago": "forma_pago",
 "Fecha venta": "fecha"
})
```

Es importante observar que asignamos el resultado nuevamente a la variable `df`.

Esto significa que reemplazamos el `DataFrame` anterior por una nueva versión con los nombres de columnas modificados.

Si usáramos `rename()` sin guardar el resultado, Pandas mostraría una versión renombrada, pero el `DataFrame` original seguiría igual.

Veamos un ejemplo pequeño para entenderlo mejor.



```
df_ejemplo = pd.DataFrame({
 "Nombre Producto": ["Cuaderno", "Lapicera"],
 "Precio": [1200, 350]
})

df_ejemplo
```

índice	Nombre Producto	Precio
0	Cuaderno	1200
1	Lapicera	350

```
df_ejemplo.rename(columns={
 "Nombre Producto": "producto",
 "Precio": "precio"
})
```

índice	producto	precio
0	Cuaderno	1200
1	Lapicera	350

La salida muestra una versión con nombres modificados. Sin embargo, si volvemos a mostrar `df_ejemplo`, veremos que conserva los nombres originales.

```
df_ejemplo
```

índice	Nombre Producto	Precio
0	Cuaderno	1200
1	Lapicera	350

Para conservar el cambio, debemos guardar el resultado:

```
df_ejemplo = df_ejemplo.rename(columns={
 "Nombre Producto": "producto",
 "Precio": "precio"
})

df_ejemplo
```

índice	producto	precio
0	Cuaderno	1200
1	Lapicera	350

Esta idea aparece muchas veces en Pandas. Muchos métodos devuelven un nuevo resultado, pero no modifican automáticamente el objeto original. Por eso, cuando queremos conservar una transformación, debemos asignarla a una variable.

## Eliminar columnas con drop()

Además de renombrar columnas, a veces necesitamos eliminar columnas que no vamos a utilizar.

En nuestro `DataFrame` hay una columna llamada `Código interno`. Esa columna podría ser útil para un sistema de gestión, pero tal vez no sea necesaria para el análisis que queremos realizar.

Si decidimos quitarla, podemos usar el método `drop()`.

Para eliminar columnas, usamos el parámetro `columns`.

```
df = df.drop(columns=["Código interno"])
```

```
df
```

índice	producto	categoría	precio_unitario	cantidad_vendida	sucursal	forma_pago	fecha
0	Cuaderno	Librería	1200	15	Centro	Efectivo	2020-01-01
1	Lapicera	Librería	350	40	Centro	Debito	2020-01-02
2	Mochila	Accesorios	8500	5	Norte	Credito	2020-01-03
3	Regla	Librería	500	25	Sur	Efectivo	2020-01-04
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2020-01-05
5	Calculadora	Tecnología	18500	3	Centro	Credito	2020-01-06
6	Auriculares	Tecnología	7600	6	Sur	Credito	2020-01-07
7	Resaltadores	Librería	2100	12	Centro	Debito	2020-01-08
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2020-01-09
9	Compas	Librería	2600	10	Sur	Efectivo	2020-01-10

Ahora el `DataFrame` ya no contiene la columna `Código interno`.

Al igual que ocurría con `rename()`, usamos una asignación:

```
df = df.drop(columns=["Código interno"])
```

Esto significa que guardamos en `df` una nueva versión del `DataFrame`, pero sin esa columna.

Eliminar columnas puede ser útil cuando una variable no aporta información para el análisis, cuando contiene datos redundantes o cuando queremos construir una versión más simple del dataset. Sin embargo, conviene ser cuidadosos. Una columna que parece innecesaria al comienzo puede servir más adelante para verificar información, cruzar datos o reconstruir decisiones.

Por eso, antes de eliminar una columna conviene preguntarnos:

```
¿Esta columna aporta información útil para el análisis?
¿La información aparece repetida en otra columna?
¿Podría necesitarla más adelante para verificar o cruzar datos?
¿Estoy trabajando sobre una copia o sobre el único DataFrame disponible?
```

En general, si no estamos seguros, puede ser mejor conservar la columna o crear una versión organizada del `DataFrame` en una nueva variable.

## Eliminar más de una columna

`drop()` también permite eliminar varias columnas al mismo tiempo.

Para hacerlo, pasamos una lista con los nombres de las columnas que queremos quitar.

Veamos un ejemplo con un `DataFrame` auxiliar. Vamos a recrear una versión pequeña del dataset para mostrar la eliminación de varias columnas sin afectar el `DataFrame` principal que ya venimos organizando.

```
df_auxiliar = pd.DataFrame(datos)

df_auxiliar.head()
```

índice	Producto vendido	Cat	Precio Unitario	Cant.	Sucursal Venta	Pago	Fecha venta	Código interno
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-03-01	A001
1	Lapicera	Libreria	350	40	Centro	Debito	2024-03-01	A002
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-03-02	B010
3	Regla	Libreria	500	25	Sur	Efectivo	2024-03-02	A003
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-03-03	B011

En este `DataFrame` auxiliar todavía aparecen los nombres originales de las columnas.

Supongamos que queremos eliminar las columnas `Código interno` y `Fecha venta`.

```
df_auxiliar = df_auxiliar.drop(columns=["Código interno", "Fecha venta"])

df_auxiliar.head()
```

índice	Producto vendido	Cat	Precio Unitario	Cant.	Sucursal Venta	Pago
0	Cuaderno	Libreria	1200	15	Centro	Efectivo
1	Lapicera	Libreria	350	40	Centro	Debito
2	Mochila	Accesorios	8500	5	Norte	Credito
3	Regla	Libreria	500	25	Sur	Efectivo
4	Cartuchera	Accesorios	3200	8	Norte	Debito

Ahora `df_auxiliar` ya no contiene esas dos columnas.

La estructura general es:

```
df.drop(columns=["columna_1", "columna_2"])
```

Eliminar varias columnas puede ser útil cuando queremos simplificar un dataset y quedarnos solamente con las variables necesarias. Sin embargo, conviene hacerlo con cuidado. Si eliminamos columnas demasiado pronto, podríamos perder información que luego necesitábamos para revisar el origen de los datos, controlar errores o enriquecer el análisis.

Por eso, muchas veces es preferible trabajar con una copia o con una nueva variable, especialmente durante las primeras etapas de exploración y preparación.

## Reordenar columnas

Además de renombrar o eliminar columnas, también podemos cambiar el orden en que aparecen dentro del `DataFrame`.

Reordenar columnas no modifica los valores. Solo cambia la forma en que se presenta la tabla.

Esto puede ser útil cuando queremos que las variables sigan un orden más lógico. Por ejemplo, podríamos preferir que primero aparezcan las columnas que identifican la venta, luego las variables numéricas y finalmente la información contextual.

En nuestro `DataFrame` actual tenemos estas columnas:

```
df.columns
```

```
Index(['producto', 'categoria', 'precio_unitario', 'cantidad_vendida',
 'sucursal', 'forma_pago', 'fecha'],
 dtype='object')
```

Podemos crear una lista con el orden deseado:

```
columnas_ordenadas = [
 "producto",
 "categoria",
 "precio_unitario",
 "cantidad_vendida",
 "sucursal",
 "forma_pago",
 "fecha"
]
```

Luego usamos esa lista para reorganizar el `DataFrame`:

```
df = df[columnas_ordenadas]

df
```

índice	producto	categoria	precio_unitario	cantidad_vendida	sucursal	forma_pago	fecha
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2020-01-01
1	Lapicera	Libreria	350	40	Centro	Debito	2020-01-02
2	Mochila	Accesorios	8500	5	Norte	Credito	2020-01-03
3	Regla	Libreria	500	25	Sur	Efectivo	2020-01-04
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2020-01-05
5	Calculadora	Tecnologia	18500	3	Centro	Credito	2020-01-06
6	Auriculares	Tecnologia	7600	6	Sur	Credito	2020-01-07
7	Resaltadores	Libreria	2100	12	Centro	Debito	2020-01-08
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	2020-01-09
9	Compas	Libreria	2600	10	Sur	Efectivo	2020-01-10

Ahora las columnas aparecen en el orden que indicamos en la lista.

Esta técnica es muy útil porque nos permite construir una versión final del dataset con una estructura más clara.

La idea general es:

```
df = df[lista_de_columnas_en_el_orden_deseado]
```

Es importante que todos los nombres de la lista existan en el `DataFrame`. Si escribimos mal un nombre de columna o incluimos una columna que no existe, Pandas mostrará un error.

## Construir una versión organizada del DataFrame

Hasta ahora aplicamos varias operaciones por separado: renombramos columnas, eliminamos una columna que no necesitábamos y reordenamos las variables.

En un trabajo real, muchas veces queremos construir una versión organizada del dataset para usarla en el análisis posterior. Esa versión puede conservar solamente las columnas necesarias, con nombres claros y en un orden lógico.

Una forma de hacerlo es crear una nueva variable, por ejemplo `df_organizado`.

Para mostrar el proceso completo, vamos a recrear el `DataFrame` original y aplicar las transformaciones principales.

```
df_original = pd.DataFrame(datos)

df_organizado = df_original.rename(columns={
 "Producto vendido": "producto",
 "Cat": "categoria",
 "Precio Unitario": "precio_unitario",
 "Cant.": "cantidad_vendida",
 "Sucursal Venta": "sucursal",
 "Pago": "forma_pago",
 "Fecha venta": "fecha"
})

df_organizado = df_organizado.drop(columns=["Código interno"])

df_organizado = df_organizado[
 [
 "producto",
 "categoria",
 "precio_unitario",
 "cantidad_vendida",
 "sucursal",
 "forma_pago",
 "fecha"
]
]
```

```
df_organizado
```

índice	producto	categoría	precio_unitario	cantidad_vendida	sucursal	forma_pago	fe
0	Cuaderno	Librería	1200	15	Centro	Efectivo	20
1	Lapicera	Librería	350	40	Centro	Debito	20
2	Mochila	Accesorios	8500	5	Norte	Credito	20
3	Regla	Librería	500	25	Sur	Efectivo	20
4	Cartuchera	Accesorios	3200	8	Norte	Debito	20
5	Calculadora	Tecnología	18500	3	Centro	Credito	20
6	Auriculares	Tecnología	7600	6	Sur	Credito	20
7	Resaltadores	Librería	2100	12	Centro	Debito	20
8	Guardapolvo	Indumentaria	14500	4	Norte	Credito	20
9	Compas	Librería	2600	10	Sur	Efectivo	20

En este bloque realizamos tres pasos.

Primero renombramos las columnas para que tengan nombres más claros y consistentes. Luego eliminamos la columna `Código interno`, porque decidimos que no era necesaria para este análisis. Finalmente reordenamos las columnas para que la tabla quede presentada de una manera más cómoda.

El `DataFrame` original sigue estando disponible en la variable `df_original`, mientras que la versión organizada quedó guardada en `df_organizado`.

Esta forma de trabajo es recomendable porque conserva una versión inicial de los datos y, al mismo tiempo, nos permite construir una versión más limpia y clara para continuar el análisis.

## Verificar la estructura final

Después de organizar las columnas, conviene revisar cómo quedó el `DataFrame`.

Podemos empezar consultando sus nombres de columnas:

```
df_organizado.columns
```

```
Index(['producto', 'categoria', 'precio_unitario', 'cantidad_vendida',
 'sucursal', 'forma_pago', 'fecha'],
 dtype='object')
```

También podemos revisar sus dimensiones:

```
df_organizado.shape
```

```
(10, 7)
```

Y podemos observar las primeras filas:

```
df_organizado.head()
```

índice	producto	categoria	precio_unitario	cantidad_vendida	sucursal	forma_pago	fecha
0	Cuaderno	Libreria	1200	15	Centro	Efectivo	2024-0
1	Lapicera	Libreria	350	40	Centro	Debito	2024-0
2	Mochila	Accesorios	8500	5	Norte	Credito	2024-0
3	Regla	Libreria	500	25	Sur	Efectivo	2024-0
4	Cartuchera	Accesorios	3200	8	Norte	Debito	2024-0

Estas verificaciones nos permiten confirmar que la organización se aplicó correctamente.

El `DataFrame` organizado debería tener nombres de columnas claros, no debería incluir columnas que decidimos eliminar y debería presentar las variables en un orden lógico.

También podemos comparar las columnas originales con las columnas organizadas:

```
print("Columnas originales:")
print(list(df_original.columns))

print()

print("Columnas organizadas:")
print(list(df_organizado.columns))
```

```
Columnas originales:
['Producto vendido', 'Cat', 'Precio Unitario', 'Cant.', 'Sucursal Venta', 'Pago', 'Fecha
venta', 'Código interno']
```

```
Columnas organizadas:
```



```
['producto', 'categoria', 'precio_unitario', 'cantidad_vendida', 'sucursal',
'forma_pago', 'fecha']
```

Esta comparación muestra con claridad qué cambió.

El dataset original conservaba nombres como `Producto vendido`, `Cat`, `Precio Unitario`, `Cant.`, `Sucursal Venta`, `Pago`, `Fecha venta` y `Código interno`.

La versión organizada, en cambio, tiene nombres más consistentes y ya no incluye la columna que decidimos quitar.

Verificar la estructura final es una buena práctica. Cada vez que renombramos, eliminamos o reordenamos columnas, conviene revisar que el resultado coincida con lo que esperábamos.

## Errores frecuentes al organizar columnas

Al organizar columnas en Pandas pueden aparecer algunos errores comunes.

Uno de los más frecuentes es escribir mal el nombre de una columna. Esto puede ocurrir al renombrar, eliminar o reordenar.

Por ejemplo, si intentamos eliminar una columna llamada `"Codigo interno"` sin tilde, pero en el `DataFrame` la columna se llama `"Código interno"` con tilde, Pandas no encontrará esa columna.

```
Esta instrucción produciría un error porque la columna se llama "Código interno".
df_original.drop(columns=["Codigo interno"])
```

Para evitar este tipo de problemas, conviene revisar los nombres exactos con:

```
df_original.columns
```

```
Index(['Producto vendido', 'Cat', 'Precio Unitario', 'Cant.', 'Sucursal Venta',
 'Pago', 'Fecha venta', 'Código interno'],
 dtype='object')
```

Otro error frecuente es olvidar guardar el resultado de una operación. Métodos como `rename()` y `drop()` devuelven una nueva versión del `DataFrame`, pero no modifican automáticamente el objeto original si no guardamos el resultado.

Por ejemplo:

```
df_ejemplo = pd.DataFrame({
 "Producto vendido": ["Cuaderno", "Lapicera"],
 "Precio Unitario": [1200, 350]
})

df_ejemplo.rename(columns={
 "Producto vendido": "producto",
 "Precio Unitario": "precio_unitario"
})
```

índice	producto	precio_unitario
0	Cuaderno	1200
1	Lapicera	350

La salida parece mostrar columnas renombradas, pero si volvemos a revisar `df_ejemplo`, veremos que conserva los nombres originales:

```
df_ejemplo
```

índice	Producto vendido	Precio Unitario
0	Cuaderno	1200
1	Lapicera	350

Para conservar el cambio, debemos guardar el resultado:

```
df_ejemplo = df_ejemplo.rename(columns={
 "Producto vendido": "producto",
 "Precio Unitario": "precio_unitario"
})

df_ejemplo
```

índice	producto	precio_unitario
0	Cuaderno	1200
1	Lapicera	350

También puede aparecer un error al reordenar columnas si la lista contiene un nombre que no existe o si olvidamos incluir una columna que queríamos conservar.

```
Esta instrucción produciría un error porque "precio" no existe en df_organizado.
La columna se llama "precio_unitario".

df_organizado[["producto", "categoria", "precio"]]
```

Finalmente, debemos tener cuidado al eliminar columnas demasiado pronto. Eliminar una columna puede simplificar el dataset, pero también puede hacernos perder información útil para controles posteriores.

Una buena rutina para organizar columnas podría ser:

```
revisar nombres actuales
renombrar columnas necesarias
decidir qué columnas conservar o eliminar
reordenar variables
verificar la estructura final
```

Organizar columnas no es solo una cuestión estética. Una estructura clara ayuda a escribir código más legible, reduce errores y facilita el análisis posterior.

## Resumen del capítulo

---

En este capítulo aprendimos a organizar las columnas de un `DataFrame`.

Hasta ahora habíamos trabajado principalmente con valores: crear columnas nuevas, modificar valores existentes, corregir textos o transformar datos numéricos. En este capítulo, en cambio, nos concentramos en la estructura de la tabla.

Comenzamos con un dataset que tenía nombres de columnas poco uniformes. Algunas columnas tenían espacios, otras usaban abreviaturas y otras tenían nombres que podían ser más claros. Vimos que esos detalles pueden dificultar la escritura del código y hacer menos comprensible el análisis.

Primero revisamos los nombres actuales usando:

```
df.columns
```

y también convertimos esos nombres en una lista para leerlos con mayor comodidad:

```
list(df.columns)
```

Después usamos `rename()` para cambiar nombres de columnas. Para eso construimos un diccionario donde cada clave era el nombre original de la columna y cada valor era el nuevo nombre:

```
df = df.rename(columns={
 "Producto vendido": "producto",
 "Cat": "categoria",
 "Precio Unitario": "precio_unitario",
```

```
"Cant.": "cantidad_vendida",
"Sucursal Venta": "sucursal",
"Pago": "forma_pago",
"Fecha venta": "fecha"
})
```

También vimos que muchos métodos de Pandas no modifican automáticamente el `DataFrame` original. Por eso, si queremos conservar el resultado de una transformación, debemos asignarlo a una variable.

Luego aprendimos a eliminar columnas usando `drop()`:

```
df = df.drop(columns=["Código interno"])
```

Esta operación puede ser útil cuando una columna no aporta información al análisis, pero también requiere cuidado. Eliminar columnas demasiado pronto puede hacernos perder datos útiles para revisar, verificar o cruzar información más adelante.

Después trabajamos con el reordenamiento de columnas. Para eso creamos una lista con el orden deseado y la usamos para reorganizar el `DataFrame`:

```
columnas_ordenadas = [
 "producto",
 "categoria",
 "precio_unitario",
 "cantidad_vendida",
 "sucursal",
 "forma_pago",
 "fecha"
]

df = df[columnas_ordenadas]
```

Más adelante integramos estas operaciones en una versión organizada del dataset. Creamos un `DataFrame` original y luego construimos una nueva variable llamada `df_organizado`, con columnas renombradas, una columna eliminada y un orden más claro.

Finalmente, verificamos la estructura final revisando los nombres de columnas, las dimensiones, las primeras filas y comparando las columnas originales con las columnas organizadas.

La idea principal de este capítulo fue:

```
Organizar columnas permite construir un DataFrame más claro, legible y cómodo para analizar.
```

Una buena estructura de columnas no cambia los datos por sí misma, pero mejora el trabajo posterior. Nombres claros, columnas necesarias y un orden lógico ayudan a escribir código más comprensible y reducen errores.

## Próximo paso

---

Ya sabemos seleccionar, filtrar, ordenar, crear columnas, modificar valores y organizar la estructura de un `DataFrame`.

El siguiente paso será comenzar a trabajar con valores faltantes. En los datasets reales es muy común encontrar celdas vacías, datos incompletos o valores que Pandas representa como `NaN`.

Aprender a detectar y tratar valores faltantes será una parte central de la limpieza de datos, porque esos valores pueden afectar conteos, promedios, filtros, gráficos y conclusiones.

## Capítulo 13 — Qué significa limpiar datos

---

### Breve repaso

---

Hasta ahora aprendimos varias operaciones fundamentales para trabajar con `DataFrames` en Pandas. Seleccionamos columnas, seleccionamos filas, usamos `loc` e `iloc`, filtramos registros, combinamos condiciones, ordenamos datos, construimos rankings, creamos nuevas columnas, modificamos valores existentes y organizamos la estructura de una tabla.

Ese recorrido nos dio herramientas importantes. Sin embargo, todavía trabajamos principalmente con datasets pequeños y controlados, creados especialmente para aprender una operación por vez.

A partir de este capítulo vamos a iniciar una nueva etapa: la preparación y limpieza de datos.

En situaciones reales, los datasets rara vez llegan listos para analizar. Pueden tener valores faltantes, columnas con tipos de datos incorrectos, fechas cargadas como texto, números escritos de manera inconsistente, categorías mezcladas, duplicados, errores de carga o valores que no tienen sentido dentro del contexto del problema.

Limpiar datos no significa simplemente “borrar lo que está mal”. Significa revisar el dataset, comprender sus problemas, tomar decisiones justificadas, aplicar transformaciones y verificar que el resultado sea más confiable para el análisis.

En este capítulo vamos a trabajar con una mirada principalmente conceptual y metodológica. Vamos a presentar qué significa limpiar datos, por qué es necesario hacerlo, qué tipos de problemas pueden aparecer y por qué cada decisión de limpieza debe tomarse con cuidado.

Al finalizar este capítulo deberías poder:

- Explicar qué significa limpiar datos.
- Reconocer que la limpieza de datos es una etapa de análisis, no solo de programación.
- Identificar problemas frecuentes en datasets reales.
- Comprender la diferencia entre detectar un problema y decidir cómo tratarlo.
- Reconocer que eliminar datos no siempre es la mejor solución.
- Comprender la importancia de validar el dataset después de limpiarlo.
- Entender el flujo general: diagnosticar, decidir, transformar y verificar.

## Un dataset real para trabajar la limpieza

---

Para estudiar limpieza de datos vamos a usar un dataset real disponible en Kaggle: **Cafe Sales — Dirty Data for Cleaning Training**.

Este dataset contiene registros de ventas de una cafetería. Cada fila representa una transacción, y las columnas describen aspectos de esa venta, como el producto vendido, la cantidad, el precio unitario, el total gastado, el método de pago, la ubicación y la fecha de la transacción.

A diferencia de los datasets pequeños que usamos hasta ahora, este archivo está pensado para practicar limpieza de datos. Eso significa que no todos sus valores están listos para ser analizados directamente. Algunas columnas pueden tener datos faltantes, valores inconsistentes, tipos de datos incorrectos o registros que requieren revisión.

Esto lo vuelve muy útil para esta etapa del recorrido.

A partir de ahora, el objetivo no será solamente aprender una instrucción nueva de Pandas. Vamos a trabajar sobre un problema más realista: recibimos un dataset imperfecto y necesitamos prepararlo para que pueda ser analizado con mayor confianza.

Esto implica hacernos preguntas como:

```
¿Qué problemas tiene el dataset?
¿Qué columnas necesitan revisión?
¿Qué valores faltan?
¿Qué valores parecen inconsistentes?
¿Qué tipos de datos fueron interpretados correctamente?
¿Qué columnas se pueden usar para verificar otras?
¿Qué decisiones de limpieza son razonables?
¿Qué riesgos tiene cada decisión?
¿Cómo sabemos si el dataset quedó mejor preparado?
```

La limpieza de datos no es una etapa mecánica. No alcanza con aplicar comandos como `dropna()`, `fillna()` o `replace()` sin pensar. Cada transformación debe tener una justificación y debe ser revisada después de aplicarse.

Por eso, en esta parte vamos a avanzar con más atención conceptual. El código seguirá siendo importante, pero estará al servicio de un proceso: comprender, diagnosticar, limpiar y validar.

## Limpiar datos no es solo aplicar comandos

Cuando empezamos a trabajar con Pandas, puede parecer que limpiar datos consiste simplemente en conocer algunos métodos:

```
dropna()
fillna()
replace()
astype()
to_datetime()
drop_duplicates()
```

Esos métodos son importantes, y vamos a usarlos más adelante. Pero la limpieza de datos no empieza por el comando. Empieza por una pregunta:

¿Qué problema estoy tratando de resolver?

Por ejemplo, si una columna tiene valores faltantes, podríamos pensar rápidamente en eliminarlos o completarlos. Sin embargo, antes de decidir qué hacer necesitamos entender qué representan esos valores faltantes.

No es lo mismo que falte el método de pago en una venta, que falte la cantidad vendida, que falte la fecha de la transacción o que falte el nombre del producto. Cada caso puede tener consecuencias diferentes para el análisis.

También necesitamos preguntarnos por qué falta ese dato. Tal vez hubo un error de carga. Tal vez el dato no correspondía. Tal vez el sistema no lo registró. Tal vez aparece como faltante porque fue escrito de una forma que Pandas no interpretó correctamente.

Por eso, limpiar datos implica tomar decisiones. Algunas decisiones pueden ser simples, como quitar espacios innecesarios en una columna de texto. Otras pueden ser más delicadas, como eliminar filas, completar valores faltantes o modificar tipos de datos.

Un buen proceso de limpieza suele seguir este recorrido:

```
diagnosticar el problema
 ↓
decidir una estrategia
 ↓
aplicar una transformación
 ↓
verificar el resultado
```



Si omitimos la etapa de diagnóstico, podemos aplicar soluciones incorrectas. Si omitimos la etapa de verificación, podemos creer que el problema fue resuelto cuando en realidad sigue presente o fue reemplazado por otro.

Por eso, a partir de ahora vamos a trabajar con una idea central: cada transformación sobre un dataset debe tener un motivo y debe ser revisada después de aplicarse.

## Problemas frecuentes en datasets reales

---

Un dataset puede tener muchos tipos de problemas. Algunos son visibles rápidamente, mientras que otros aparecen recién cuando intentamos analizar, filtrar, calcular o graficar.

Uno de los problemas más conocidos son los **valores faltantes**. Un valor faltante aparece cuando una celda no tiene información disponible. En Pandas, muchas veces estos valores se representan como `NaN`. Los valores faltantes pueden afectar conteos, promedios, filtros y gráficos, pero no siempre deben tratarse de la misma manera. A veces conviene eliminarlos, otras veces completarlos, y en algunos casos puede ser mejor conservarlos como evidencia de que la información no está disponible.

También pueden aparecer **duplicados**. Un duplicado ocurre cuando una misma observación aparece más de una vez. En algunos datasets, una fila duplicada puede ser un error de carga. En otros casos, dos filas parecidas pueden representar eventos reales distintos. Por eso, antes de eliminar duplicados, necesitamos entender qué representa cada fila.

Otro problema frecuente son las **inconsistencias en categorías o textos**. Por ejemplo, una misma categoría puede aparecer escrita como `"Credito"`, `"credito"`, `"Crédito"` o `" credito "`. Para una persona, esos valores pueden representar lo mismo. Para Pandas, en cambio, son valores diferentes. Si no corregimos esas diferencias, los conteos y agrupamientos pueden quedar fragmentados.

También pueden aparecer **tipos de datos incorrectos**. Una columna que debería ser numérica puede estar cargada como texto. Una fecha puede aparecer como `object` en lugar de tener un tipo temporal. Una columna con importes puede incluir símbolos, comas, espacios o palabras que impiden convertirla directamente a número.

Además, pueden existir **valores imposibles o sospechosos**. Por ejemplo, una cantidad vendida negativa, un precio igual a cero, una fecha futura en un registro histórico o un total gastado que no coincide con la cantidad multiplicada por el precio unitario. Estos casos no siempre son errores, pero deben llamar nuestra atención.

Finalmente, puede haber **problemas estructurales**, como nombres de columnas poco claros, columnas redundantes, variables mal separadas o información mezclada en una misma columna.

Podemos resumir algunos problemas frecuentes así:

Problema	Ejemplo	Posible impacto
Valores faltantes	Falta el método de pago de una venta	Puede afectar conteos o análisis por categoría
Duplicados	La misma transacción aparece dos veces	Puede inflar ventas o cantidades
Categorías inconsistentes	"Credito" y "credito"	Fragmenta conteos y agrupamientos
Tipos incorrectos	Un precio cargado como texto	Impide cálculos numéricos confiables
Fechas mal interpretadas	Una fecha leída como texto	Dificulta ordenar o agrupar temporalmente
Valores sospechosos	Cantidad negativa o precio cero	Puede indicar errores de carga
Columnas poco claras	Nombres como <code>col1</code> o <code>dato</code>	Dificulta comprender el dataset

Detectar estos problemas es solo el primer paso. La parte más importante es decidir qué hacer con ellos de manera razonada.

## Diagnosticar antes de limpiar

Cuando encontramos un problema en un dataset, es tentador corregirlo de inmediato. Si vemos valores faltantes, podemos querer eliminarlos. Si vemos categorías mal escritas, podemos querer reemplazarlas. Si vemos una columna con tipo incorrecto, podemos querer convertirla rápidamente.

Pero antes de modificar el dataset conviene diagnosticar.

Diagnosticar significa observar el problema con más detalle para entender su alcance, su posible causa y su impacto sobre el análisis.

Por ejemplo, si una columna tiene valores faltantes, no alcanza con saber que existen. También necesitamos saber cuántos faltan, qué porcentaje representan, en qué columnas aparecen, si se concentran en cierto grupo, si afectan variables importantes o si pueden reconstruirse a partir de otras columnas.

No es lo mismo que falte el 1% de los valores de una columna secundaria que el 60% de los valores de una columna central para el análisis. Tampoco es lo mismo que los

valores faltantes estén distribuidos al azar que concentrados en una sucursal, una fecha o un tipo de producto.

Algo parecido ocurre con los duplicados. Antes de eliminarlos, necesitamos preguntarnos si realmente son duplicados o si representan transacciones distintas que comparten algunos datos. Dos ventas del mismo producto, con el mismo precio y la misma fecha, pueden parecer repetidas, pero tal vez sean ventas reales diferentes.

El diagnóstico nos ayuda a evitar decisiones automáticas.

Una buena pregunta inicial no es:

¿Cómo borro este problema?

sino:

¿Qué representa este problema dentro del dataset?

A partir de esa respuesta podemos decidir mejor qué hacer. En limpieza de datos, la calidad de la decisión suele ser más importante que la rapidez con la que aplicamos una instrucción.

## Decidir una estrategia de limpieza

---

Después de diagnosticar un problema, necesitamos decidir qué hacer.

En limpieza de datos, una misma situación puede tener distintas estrategias posibles. Por ejemplo, si encontramos valores faltantes, podríamos eliminar las filas incompletas, eliminar una columna con demasiados faltantes, completar los valores con una medida estadística, reconstruirlos a partir de otras columnas o conservarlos si la ausencia de información también es relevante.

Ninguna de esas opciones es correcta en todos los casos. La decisión depende del contexto, del objetivo del análisis, de la cantidad de datos afectados y de la importancia de la columna.

Por ejemplo, si en un dataset de ventas falta el método de pago en algunas filas, tal vez podamos conservar esas ventas y simplemente registrar que el método de pago es desconocido. Pero si falta la cantidad vendida o el precio unitario, el problema puede ser más serio, porque esas columnas son necesarias para calcular importes.

También puede ocurrir que un valor faltante pueda reconstruirse. Si tenemos `Quantity`, `Price Per Unit` y `Total Spent`, y falta solo uno de esos tres valores, quizá

podamos deducirlo usando los otros dos. En ese caso, completar el dato no sería una simple imputación arbitraria, sino una reconstrucción basada en una relación interna del dataset.

Con los duplicados ocurre algo parecido. Si dos filas son exactamente iguales, tal vez una de ellas sea una copia accidental. Pero si dos filas tienen el mismo producto, el mismo precio y la misma fecha, eso no alcanza para decir que son duplicadas: podrían representar dos ventas reales distintas.

Por eso, antes de limpiar conviene formular una estrategia. Esa estrategia debería responder preguntas como:

¿Qué problema detectamos?  
¿Qué columnas están afectadas?  
¿Cuántas filas están involucradas?  
¿Qué impacto tiene el problema sobre el análisis?  
¿Qué opciones de tratamiento tenemos?  
¿Qué opción conserva mejor la información?  
¿Qué riesgos tiene la decisión elegida?  
¿Cómo vamos a verificar el resultado?

La limpieza de datos no debería ser un conjunto de acciones automáticas. Es un proceso de toma de decisiones documentadas.

## Verificar después de transformar

---

Una limpieza no termina cuando aplicamos una transformación. Termina cuando verificamos que esa transformación produjo el resultado esperado.

Este punto es fundamental. Podemos escribir una instrucción correcta desde el punto de vista sintáctico y, aun así, obtener un resultado que no resuelve el problema. También puede ocurrir que corrijamos una columna, pero sin darnos cuenta generemos un problema nuevo en otra parte del dataset.

Por ejemplo, si unificamos categorías, deberíamos revisar nuevamente los valores únicos o los conteos para confirmar que las variantes quedaron agrupadas. Si convertimos una columna a tipo numérico, deberíamos revisar cuántos valores no pudieron convertirse. Si eliminamos duplicados, deberíamos comprobar cuántas filas quedaron antes y después. Si imputamos valores faltantes, deberíamos verificar que la cantidad de faltantes disminuyó y que la imputación no generó valores incoherentes.

La verificación posterior responde una pregunta muy simple:

¿Cómo sé que la limpieza funcionó?

En Pandas, algunas herramientas útiles para verificar transformaciones son:

Herramienta	Para qué puede servir
<code>head()</code>	Observar las primeras filas después de un cambio
<code>info()</code>	Revisar tipos de datos y valores no nulos
<code>isna().sum()</code>	Contar valores faltantes por columna
<code>value_counts()</code>	Revisar categorías y frecuencias
<code>unique()</code>	Observar valores distintos en una columna
<code>duplicated().sum()</code>	Contar filas duplicadas
<code>describe()</code>	Revisar estadísticas de columnas numéricas
<code>shape</code>	Comparar cantidad de filas y columnas antes y después

La validación posterior es una forma de control de calidad. Nos permite comprobar que el dataset quedó más preparado para el análisis, y no simplemente diferente.

Por eso, una buena limpieza debería dejar algún tipo de evidencia: una comparación, una tabla de revisión, un conteo antes y después, o una comprobación clara de que el problema fue tratado correctamente.

## Un flujo general para limpiar datos

Aunque cada dataset tiene sus propios problemas, podemos pensar la limpieza de datos como un proceso ordenado.

No siempre vamos a seguir exactamente los mismos pasos ni en el mismo orden, pero sí conviene tener una guía general. Esa guía nos ayuda a no limpiar datos de manera improvisada.

Una posible secuencia de trabajo es:

```
cargar el dataset
↓
inspeccionar su estructura
↓
detectar problemas
↓
priorizar qué problemas tratar
↓
aplicar transformaciones
↓
```

```
verificar los cambios
 ↓
documentar decisiones
 ↓
obtener una versión preparada para analizar
```

En la práctica, este recorrido no siempre es lineal. Muchas veces detectamos un problema, lo corregimos, verificamos el resultado y eso nos permite encontrar otro problema que antes no era visible. La limpieza puede requerir varias idas y vueltas.

Por ejemplo, al convertir una columna a tipo numérico podemos descubrir que algunos valores contenían texto. Al limpiar una columna categórica podemos encontrar categorías que no esperábamos. Al revisar duplicados podemos descubrir que necesitamos mirar más de una columna para decidir si dos registros son realmente iguales.

Por eso, más que una receta cerrada, este flujo debe entenderse como una forma de trabajo:

```
observar → decidir → transformar → verificar
```

A medida que avancemos, vamos a aplicar esta lógica sobre un dataset real. No vamos a limpiar todo de una vez. Vamos a separar los problemas, analizarlos y tomar decisiones paso a paso.

## Cargar el dataset real

Para comenzar a trabajar con un dataset real, vamos a descargarlo desde Kaggle usando la biblioteca `kagglehub`.

Este paso puede tardar unos segundos la primera vez que se ejecuta. Una vez descargado el dataset, vamos a localizar el archivo CSV y cargarlo con Pandas.

En este capítulo no vamos a resolver todos los problemas del dataset. La idea es observarlo por primera vez y comenzar a pensar qué tipo de limpieza podría necesitar.

```
Instalamos kagglehub si no está disponible en el entorno.
En Google Colab, esta instalación suele ser necesaria.

!pip install kagglehub
```

```
import kagglehub
import pandas as pd
import os
```

```
Descargamos el dataset desde Kaggle.
KaggleHub devuelve la ruta local donde quedó almacenado.

ruta_dataset = kagglehub.dataset_download(
 "ahmedmohamed2003/cafe-sales-dirty-data-for-cleaning-training"
)

ruta_dataset
```

```
Revisamos qué archivos se descargaron dentro de la carpeta del dataset.

archivos = os.listdir(ruta_dataset)

archivos
```

```
['dirty_cafe_sales.csv']
```

La celda anterior muestra los archivos disponibles dentro de la carpeta descargada.

Como el dataset puede venir acompañado por más de un archivo, primero revisamos el contenido de la carpeta y luego elegimos el archivo CSV que vamos a cargar.

```
Buscamos archivos CSV dentro de la carpeta descargada.

archivos_csv = [archivo for archivo in archivos if archivo.endswith(".csv")]

archivos_csv
```

```
['dirty_cafe_sales.csv']
```

```
Cargamos el primer archivo CSV encontrado.

ruta_csv = os.path.join(ruta_dataset, archivos_csv[0])

df = pd.read_csv(ruta_csv)

df.head()
```

índice	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date
0	TXN_1961373	Coffee	2	2.0	4.0	Credit Card	Takeaway	2023-09-08
1	TXN_4977031	Cake	4	3.0	12.0	Cash	In-store	2023-05-16
2	TXN_4271903	Cookie	4	1.0	ERROR	Credit Card	In-store	2023-07-19

índice	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date
3	TXN_7034554	Salad	2	5.0	10.0	UNKNOWN	UNKNOWN	2023-04-27
4	TXN_3160411	Coffee	2	2.0	4.0	Digital Wallet	In-store	2023-06-11

Ahora tenemos el dataset cargado en un `DataFrame` llamado `df`.

La salida de `head()` nos muestra las primeras filas. Esta primera vista no alcanza para diagnosticar todos los problemas, pero nos permite confirmar que el archivo fue cargado correctamente y empezar a reconocer sus columnas.

A partir de este punto, el trabajo de limpieza comienza con una pregunta simple: ¿qué tenemos delante?

## Primera inspección del dataset

Una vez cargado el archivo, necesitamos hacer una primera inspección general.

En esta etapa no vamos a limpiar todavía. Solo queremos observar la estructura del dataset: cuántas filas y columnas tiene, cómo se llaman las columnas, qué tipos de datos interpretó Pandas y qué primeras señales de problemas aparecen.

Vamos a comenzar revisando las dimensiones del `DataFrame`.

```
df.shape
```

```
(10000, 8)
```

La salida de `shape` nos indica cuántas filas y cuántas columnas tiene el dataset.

Como estamos trabajando con un archivo real, ya no se trata de una tabla pequeña creada manualmente. Este tamaño nos obliga a trabajar con herramientas de inspección y resumen, porque no podemos revisar cada fila una por una.

```
df.columns
```

```
Index(['Transaction ID', 'Item', 'Quantity', 'Price Per Unit', 'Total Spent',
 'Payment Method', 'Location', 'Transaction Date'],
 dtype='object')
```

La salida de `columns` muestra los nombres de las columnas disponibles.



A partir de esos nombres podemos empezar a comprender qué información contiene el dataset. En este caso, las columnas describen transacciones de venta: identificador de transacción, producto, cantidad, precio unitario, total gastado, método de pago, ubicación y fecha.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 8 columns):
Column Non-Null Count Dtype
--- -
0 Transaction ID 10000 non-null object
1 Item 9667 non-null object
2 Quantity 9862 non-null object
3 Price Per Unit 9821 non-null object
4 Total Spent 9827 non-null object
5 Payment Method 7421 non-null object
6 Location 6735 non-null object
7 Transaction Date 9841 non-null object
dtypes: object(8)
memory usage: 625.1+ KB
```

La salida de `info()` nos da una primera imagen de la estructura del dataset.

Nos permite observar cuántos valores no nulos tiene cada columna y qué tipo de dato interpretó Pandas. Esta información será importante para detectar posibles problemas, como columnas con valores faltantes o columnas numéricas que fueron cargadas como texto.

En esta primera lectura todavía no tomamos decisiones. Solo estamos reuniendo evidencia para entender qué tipo de limpieza puede necesitar el dataset.

## Primeras señales de problemas

Después de revisar la estructura general del dataset, podemos empezar a buscar señales de posibles problemas.

En esta etapa todavía no vamos a resolverlos. Solo queremos detectarlos y registrar qué aspectos conviene analizar con más detalle.

Algunas preguntas iniciales pueden ser:

```
¿Hay valores faltantes?
¿Hay columnas que parecen numéricas pero fueron cargadas como texto?
¿Hay fechas interpretadas correctamente?
```

¿Aparecen valores extraños en columnas categóricas?  
¿Hay columnas que se puedan usar para verificar otras?

Para empezar, podemos observar algunas filas del dataset.

```
df.head(10)
```

índice	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date
0	TXN_1961373	Coffee	2	2.0	4.0	Credit Card	Takeaway	2023-09-08
1	TXN_4977031	Cake	4	3.0	12.0	Cash	In-store	2023-05-16
2	TXN_4271903	Cookie	4	1.0	ERROR	Credit Card	In-store	2023-07-19
3	TXN_7034554	Salad	2	5.0	10.0	UNKNOWN	UNKNOWN	2023-04-27
4	TXN_3160411	Coffee	2	2.0	4.0	Digital Wallet	In-store	2023-06-11
5	TXN_2602893	Smoothie	5	4.0	20.0	Credit Card	NaN	2023-03-31
6	TXN_4433211	UNKNOWN	3	3.0	9.0	ERROR	Takeaway	2023-10-06
7	TXN_6699534	Sandwich	4	4.0	16.0	Cash	UNKNOWN	2023-10-28
8	TXN_4717867	NaN	5	3.0	15.0	NaN	Takeaway	2023-07-28
9	TXN_2064365	Sandwich	5	4.0	20.0	NaN	In-store	2023-12-31

Esta vista inicial permite reconocer la forma general de los datos.

Sin embargo, mirar unas pocas filas no alcanza para diagnosticar un dataset completo. Algunos problemas pueden aparecer recién más adelante, o afectar solo a una parte pequeña de las filas.

Por eso, además de observar ejemplos, necesitamos usar herramientas de resumen.

```
df.describe(include="all")
```

índice	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date
count	10000	9667	9862	9821	9827	7421	6735	9841
unique	10000	10	7	8	19	5	4	367
top	TXN_9226047	Juice	5	3.0	6.0	Digital Wallet	Takeaway	UNKNOWN

índice	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date
freq	1	1171	2013	2429	979	2291	3022	159

El método `describe(include="all")` nos da un resumen amplio del dataset, incluyendo columnas numéricas y no numéricas.

En este punto, no necesitamos interpretar cada valor en detalle. Lo importante es empezar a detectar señales: columnas con muchos valores distintos, valores frecuentes, posibles datos faltantes, tipos de datos inesperados o columnas que merecen una revisión más cuidadosa.

La limpieza de datos suele comenzar así: no con una transformación inmediata, sino con una lectura atenta de las señales que el dataset nos ofrece.

## Detectar valores faltantes como diagnóstico inicial

Uno de los primeros problemas que conviene revisar en un dataset real es la presencia de valores faltantes.

Un valor faltante indica que una celda no tiene información disponible. En Pandas, estos valores suelen aparecer como `NaN`.

En este capítulo no vamos a decidir todavía cómo tratar esos valores. Por ahora, solo queremos detectar en qué columnas aparecen y tener una primera idea de su magnitud.

Para eso podemos usar:

```
df.isna().sum()
```

índice	0
Transaction ID	0
Item	333
Quantity	138
Price Per Unit	179
Total Spent	173
Payment Method	2579
Location	3265

índice	0
Transaction Date	159

`dtype: int64`

La instrucción `isna()` identifica los valores faltantes del `DataFrame`, y `sum()` permite contar cuántos hay en cada columna.

La salida nos muestra, columna por columna, cuántos valores faltantes fueron detectados por Pandas.

Este conteo es una primera señal importante. No alcanza para decidir qué hacer, pero nos ayuda a saber qué columnas necesitan revisión.

También podemos calcular el porcentaje de valores faltantes por columna:

```
porcentaje_faltantes = df.isna().mean() * 100
porcentaje_faltantes
```

índice	0
Transaction ID	0.00
Item	3.33
Quantity	1.38
Price Per Unit	1.79
Total Spent	1.73
Payment Method	25.79
Location	32.65
Transaction Date	1.59

`dtype: float64`

Este porcentaje ayuda a interpretar mejor el problema.

No es lo mismo que una columna tenga 5 valores faltantes en un dataset de 10.000 filas que en un dataset de 20 filas. Por eso, además del conteo absoluto, suele ser útil observar la proporción de datos faltantes.

En los próximos pasos vamos a estudiar los valores faltantes con más profundidad. Por ahora, lo importante es registrar que forman parte del diagnóstico inicial del dataset.

## Valores que parecen faltantes pero no siempre son NaN

Cuando usamos `isna()`, Pandas detecta valores faltantes reales, como `NaN`.

Sin embargo, en muchos datasets aparecen valores que representan ausencia de información, pero que fueron escritos como texto. Por ejemplo:

```
UNKNOWN
ERROR
N/A
Sin dato
No informado
```

Para Pandas, esos valores no siempre son faltantes. Si aparecen como texto dentro de una columna, Pandas puede tratarlos como valores comunes.

Por eso, además de contar `NaN`, conviene revisar los valores únicos de algunas columnas categóricas.

```
df["Item"].value_counts(dropna=False)
```

Item	count
Juice	1171
Coffee	1165
Salad	1148
Cake	1139
Sandwich	1131
Smoothie	1096
Cookie	1092
Tea	1089
UNKNOWN	344
NaN	333
ERROR	292

```
dtype: int64
```

```
df["Payment Method"].value_counts(dropna=False)
```

Payment Method	count
NaN	2579
Digital Wallet	2291
Credit Card	2273
Cash	2258
ERROR	306
UNKNOWN	293

dtype: int64

```
df["Location"].value_counts(dropna=False)
```

Location	count
NaN	3265
Takeaway	3022
In-store	3017
ERROR	358
UNKNOWN	338

dtype: int64

Usamos `value_counts(dropna=False)` para contar todos los valores, incluyendo posibles valores faltantes.

Esta revisión puede mostrar valores como `"UNKNOWN"` o `"ERROR"`, que no necesariamente aparecen en el conteo de `isna()`, pero que también pueden indicar problemas de calidad de datos.

Esto nos enseña una idea importante: detectar valores faltantes no siempre alcanza con buscar `NaN`. A veces necesitamos conocer cómo fue cargado el dataset y revisar qué valores especiales se usaron para representar datos desconocidos, errores o información no disponible.

En capítulos posteriores vamos a decidir cómo tratar estos casos. Por ahora, los registramos como parte del diagnóstico inicial.

## Tipos de datos que necesitan revisión

Además de los valores faltantes, otro aspecto importante del diagnóstico inicial es revisar los tipos de datos.

Pandas asigna un tipo de dato a cada columna cuando carga el archivo. Sin embargo, esa interpretación no siempre coincide con lo que esperamos.

Por ejemplo, en un dataset de ventas, columnas como `Quantity`, `Price Per Unit` o `Total Spent` deberían poder analizarse como valores numéricos. La columna `Transaction Date`, en cambio, debería poder interpretarse como una fecha.

Podemos revisar los tipos de datos con:

```
df.dtypes
```

índice	0
Transaction ID	object
Item	object
Quantity	object
Price Per Unit	object
Total Spent	object
Payment Method	object
Location	object
Transaction Date	object

```
dtype: object
```

Si una columna que debería ser numérica aparece como `object`, eso puede indicar que contiene textos, símbolos, valores especiales o errores de carga.

Algo parecido ocurre con las fechas. Una columna de fecha puede aparecer inicialmente como `object` porque Pandas la cargó como texto. Eso no significa necesariamente que los datos estén mal, pero sí indica que necesitaremos convertirlos antes de hacer análisis temporal.

En este punto no vamos a convertir todavía las columnas. Solo vamos a registrar qué columnas necesitan revisión.

Podemos observar algunas columnas específicas:

```
df[["Quantity", "Price Per Unit", "Total Spent", "Transaction Date"]].head()
```

índice	Quantity	Price Per Unit	Total Spent	Transaction Date
0	2	2.0	4.0	2023-09-08
1	4	3.0	12.0	2023-05-16
2	4	1.0	ERROR	2023-07-19
3	2	5.0	10.0	2023-04-27
4	2	2.0	4.0	2023-06-11

Esta vista nos permite mirar columnas que probablemente requieran tratamiento posterior.

Si aparecen valores como textos, errores, símbolos o datos faltantes, eso puede explicar por qué Pandas no interpretó correctamente el tipo de dato.

Más adelante vamos a trabajar con conversiones usando herramientas como `pd.to_numeric()` y `pd.to_datetime()`. Por ahora, nos alcanza con reconocer que revisar los tipos de datos forma parte del diagnóstico inicial.

## Columnas que pueden verificarse entre sí

En algunos datasets, ciertas columnas están relacionadas entre sí.

En este caso, tenemos columnas que deberían guardar una relación matemática:

```
Total Spent = Quantity × Price Per Unit
```

Esto significa que, si conocemos la cantidad vendida y el precio unitario, podemos calcular el total gastado de una transacción.

Esa relación interna puede ser muy útil durante la limpieza. Por ejemplo, si falta `Total Spent`, pero tenemos `Quantity` y `Price Per Unit`, tal vez podamos reconstruir el valor faltante. Si `Total Spent` está cargado, pero no coincide con la multiplicación entre cantidad y precio unitario, podríamos haber encontrado una inconsistencia.

Todavía no vamos a corregir esos problemas. En esta etapa solo queremos reconocer que el dataset contiene columnas que pueden ayudarnos a validar otras columnas.

Podemos observar algunas filas con esas variables:

```
df[["Quantity", "Price Per Unit", "Total Spent"]].head(10)
```



índice	Quantity	Price Per Unit	Total Spent
0	2	2.0	4.0
1	4	3.0	12.0
2	4	1.0	ERROR
3	2	5.0	10.0
4	2	2.0	4.0
5	5	4.0	20.0
6	3	3.0	9.0
7	4	4.0	16.0
8	5	3.0	15.0
9	5	4.0	20.0

Esta vista permite empezar a pensar en una validación posterior.

En limpieza de datos, no siempre dependemos de reglas externas. A veces el propio dataset contiene relaciones que nos ayudan a detectar errores, reconstruir valores o comprobar si una transformación fue razonable.

Este punto será muy importante más adelante, cuando trabajemos con valores faltantes, conversiones numéricas y validación posterior a la limpieza.

## Registrar los problemas detectados

Después de una primera inspección, conviene registrar qué problemas o señales encontramos.

Este registro no tiene que ser definitivo. Es una primera lista de aspectos que vamos a revisar con más detalle más adelante.

A partir de las inspecciones realizadas, podríamos anotar problemas como estos:

Aspecto observado	Posible problema	Qué deberíamos revisar después
Valores faltantes detectados con <code>isna()</code>	Hay celdas sin información disponible	Cantidad, porcentaje e importancia de los faltantes
Valores como UNKNOWN o ERROR	Pueden representar datos desconocidos o errores de carga	En qué columnas aparecen y cómo tratarlos
Columnas numéricas cargadas como <code>object</code>	Puede haber textos o valores no numéricos mezclados	Conversión con control de errores

Aspecto observado	Posible problema	Qué deberíamos revisar después
Fechas cargadas como texto	No se pueden analizar temporalmente de forma adecuada	Conversión a tipo fecha
Relación entre Quantity, Price Per Unit y Total Spent	Puede servir para detectar o reconstruir valores	Validar si el total coincide con cantidad por precio

Este tipo de registro es importante porque evita que la limpieza se convierta en una serie de acciones improvisadas.

Antes de transformar el dataset, necesitamos tener una idea de qué problemas existen y en qué orden conviene tratarlos.

En los próximos capítulos vamos a tomar estos problemas uno por uno. Primero vamos a profundizar en los valores faltantes, luego veremos estrategias para tratarlos, y más adelante trabajaremos con duplicados, textos inconsistentes, tipos de datos, fechas y validación posterior.

## Resumen del capítulo

En este capítulo iniciamos una nueva etapa del recorrido: la preparación y limpieza de datos.

Hasta ahora habíamos trabajado principalmente con operaciones puntuales sobre `DataFrames`: seleccionar, filtrar, ordenar, crear columnas, modificar valores y organizar la estructura de una tabla. En este capítulo cambiamos el enfoque. Empezamos a pensar la limpieza de datos como un proceso completo, no como una lista de comandos aislados.

La idea principal fue que limpiar datos no significa simplemente borrar lo que está mal. Limpiar datos implica diagnosticar problemas, comprender su posible causa, decidir una estrategia, aplicar transformaciones y verificar los resultados.

También vimos que los datasets reales pueden presentar muchos tipos de problemas: valores faltantes, duplicados, categorías inconsistentes, tipos de datos incorrectos, fechas mal interpretadas, valores sospechosos o columnas poco claras.

Un punto importante fue distinguir entre detectar un problema y decidir cómo tratarlo. Saber que una columna tiene valores faltantes, por ejemplo, no nos dice automáticamente si debemos eliminarlos, imputarlos, reconstruirlos o conservarlos.

Esa decisión depende del contexto, del objetivo del análisis y del impacto que tenga el problema sobre los resultados.

También presentamos un flujo general de limpieza:

```
graph TD; A[cargar el dataset] --> B[inspeccionar su estructura]; B --> C[detectar problemas]; C --> D[priorizar qué problemas tratar]; D --> E[aplicar transformaciones]; E --> F[verificar los cambios]; F --> G[documentar decisiones]; G --> H[obtener una versión preparada para analizar];
```

Luego cargamos un dataset real desde Kaggle: **Cafe Sales — Dirty Data for Cleaning Training**. A partir de ese dataset hicimos una primera inspección general usando herramientas ya conocidas como `shape`, `columns`, `info()`, `head()` y `describe(include="all")`.

También hicimos un diagnóstico inicial. Revisamos valores faltantes con `isna().sum()`, calculamos porcentajes de faltantes, observamos valores categóricos con `value_counts(dropna=False)`, revisamos tipos de datos con `dtypes` y reconocimos que algunas columnas pueden servir para validar otras. En particular, vimos que en este dataset existe una relación esperada entre `Quantity`, `Price Per Unit` y `Total Spent`.

La idea principal de este capítulo fue:

Limpiar datos es un proceso de diagnóstico, decisión, transformación y validación.

A partir de ahora vamos a trabajar sobre problemas concretos del dataset, pero manteniendo esta lógica: antes de modificar los datos, debemos comprender qué problema estamos tratando de resolver.

## Próximo paso

El siguiente paso será estudiar en profundidad los valores faltantes.

Vamos a analizar qué significa que un dato esté ausente, cómo representa Pandas esos valores, cómo detectarlos, cómo contarlos, cómo calcular su proporción y por qué no todos los valores faltantes deben tratarse de la misma manera.

También vamos a empezar a distinguir entre distintos tipos de ausencia: datos realmente faltantes, valores desconocidos escritos como texto y errores de carga que necesitan ser interpretados antes de decidir una estrategia de limpieza.

## Capítulo 14 — Valores faltantes

---

### Breve repaso

---

En el trabajo anterior sobre limpieza de datos comenzamos una nueva etapa del recorrido. Dejamos de pensar en Pandas solo como un conjunto de instrucciones aisladas y empezamos a pensar en el proceso completo de preparación de un dataset.

Vimos que limpiar datos implica diagnosticar problemas, tomar decisiones, aplicar transformaciones y verificar resultados. También presentamos un dataset real de ventas de cafetería y realizamos una primera inspección general para detectar señales de posibles problemas.

En este capítulo vamos a concentrarnos en uno de los problemas más frecuentes en datasets reales: los valores faltantes.

Un valor faltante aparece cuando una celda no tiene información disponible. Esto puede ocurrir por muchos motivos: errores de carga, datos que no fueron registrados, campos que no correspondían para cierta observación, problemas al exportar un archivo o decisiones tomadas por el sistema que generó los datos.

El objetivo de este capítulo no será eliminar ni completar valores faltantes todavía. Antes de decidir qué hacer con ellos, necesitamos entenderlos. Vamos a aprender a detectarlos, contarlos, calcular su proporción y observar en qué columnas aparecen.

También vamos a distinguir entre valores realmente faltantes, como `NaN`, y valores escritos como texto, como `"UNKNOWN"` o `"ERROR"`, que pueden representar información desconocida o problemas de carga, pero que Pandas no siempre interpreta automáticamente como faltantes.

Al finalizar este capítulo deberías poder:

- Explicar qué significa que falte un dato.
- Reconocer cómo representa Pandas los valores faltantes.
- Detectar valores faltantes con `isna()`.
- Contar valores faltantes por columna.
- Calcular porcentajes de valores faltantes.
- Ver filas que contienen datos faltantes.
- Distinguir entre `NaN` y valores textuales como `"UNKNOWN"` o `"ERROR"`.
- Construir un primer diagnóstico de faltantes sin tomar decisiones apresuradas.

## Dataset de trabajo

---

En este capítulo vamos a seguir trabajando con el dataset **Cafe Sales — Dirty Data for Cleaning Training**.

Cada fila representa una transacción de venta en una cafetería. Las columnas describen información como el producto vendido, la cantidad, el precio unitario, el total gastado, el método de pago, la ubicación y la fecha de la transacción.

Este dataset es especialmente útil para estudiar valores faltantes porque contiene datos incompletos y valores problemáticos que necesitaremos diagnosticar antes de decidir cómo tratarlos.

## Qué significa que falte un dato

---

Un valor faltante aparece cuando una celda del dataset no contiene información disponible.

Esto puede parecer un problema simple, pero no siempre significa lo mismo. Un dato puede faltar por muchas razones. Tal vez la información no fue registrada, tal vez hubo un error al cargar el archivo, tal vez el campo no correspondía para esa transacción, o tal vez el dato existe pero fue escrito con un valor especial que Pandas no reconoce automáticamente como faltante.

Por ejemplo, en un dataset de ventas de cafetería no es lo mismo que falte el método de pago, que falte el producto vendido o que falte el total gastado. Cada columna cumple una función distinta dentro del análisis, y por eso la ausencia de datos puede tener distintos niveles de gravedad.

Si falta el método de pago, quizás todavía podamos conservar la transacción para analizar productos vendidos o importes. En cambio, si falta la cantidad o el precio unitario, tal vez no podamos calcular correctamente el total de la venta. Si falta la fecha, podríamos perder la posibilidad de analizar la evolución temporal de las ventas.

Por eso, antes de eliminar o completar valores faltantes, necesitamos entender qué representan.

Una buena pregunta inicial no es:

¿Cómo saco los valores faltantes?

sino:

¿Qué significa que falte este dato en esta columna?

Ese cambio de enfoque es importante. Los valores faltantes no son solamente un problema técnico. También son una señal sobre la calidad, el origen y la confiabilidad del dataset.

## Cómo representa Pandas los valores faltantes

---

En Pandas, los valores faltantes suelen representarse como `NaN`.

`NaN` significa *Not a Number*, pero en Pandas se usa de manera más amplia para indicar ausencia de información. Puede aparecer en columnas numéricas, columnas de texto, fechas u otros tipos de datos.

Cuando vemos `NaN` en una celda, significa que Pandas reconoce que allí falta un valor.

Es importante distinguir entre un valor faltante real y otros valores que pueden parecer faltantes, pero que Pandas interpreta como texto común.

Por ejemplo, estos valores pueden aparecer en un dataset:

```
NaN
UNKNOWN
ERROR
N/A
Sin dato
```

Para Pandas, `NaN` es un valor faltante reconocido. En cambio, valores como `"UNKNOWN"` o `"ERROR"` pueden estar escritos como texto. Eso significa que Pandas no necesariamente los va a contar como faltantes cuando usemos herramientas como `isna()`.

Esta diferencia será importante en este capítulo. Primero vamos a detectar los valores que Pandas reconoce como faltantes. Luego vamos a revisar valores textuales que pueden representar información desconocida o errores de carga.

## Detectar valores faltantes con `isna()`

---

Para detectar valores faltantes en Pandas podemos usar `isna()`.

Este método revisa cada celda del `DataFrame` y devuelve `True` cuando encuentra un valor faltante, y `False` cuando la celda contiene algún valor.

Veamos qué ocurre al aplicarlo sobre el dataset.

```
df.isna()
```

índice	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date
0	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...
9995	False	False	False	False	False	True	False	False
9996	False	True	False	True	False	False	True	False
9997	False	False	False	False	False	False	True	False
9998	False	False	False	True	False	False	True	False
9999	False	False	False	False	False	False	False	False

La salida tiene la misma forma que el `DataFrame` original, pero en lugar de mostrar los datos muestra valores booleanos.

Cada `True` indica que en esa posición hay un valor faltante. Cada `False` indica que en esa posición hay un valor presente.

Esta vista puede ser útil para entender la idea, pero en un dataset grande no resulta práctica para leer directamente. Si tenemos muchas filas y columnas, una tabla llena de `True` y `False` no alcanza para saber rápidamente dónde está el problema.

Por eso, el siguiente paso será contar cuántos valores faltantes hay en cada columna.

## Contar valores faltantes por columna

Para que la detección de valores faltantes sea más útil, podemos combinar `isna()` con `sum()`.

Primero, `isna()` identifica las celdas faltantes. Luego, `sum()` cuenta cuántos valores `True` hay en cada columna.



En Pandas, los valores `True` se cuentan como `1` y los valores `False` como `0`. Por eso, al sumar una máscara booleana obtenemos la cantidad de valores faltantes.

```
df.isna().sum()
```

índice	0
Transaction ID	0
Item	333
Quantity	138
Price Per Unit	179
Total Spent	173
Payment Method	2579
Location	3265
Transaction Date	159

```
dtype: int64
```

La salida muestra cuántos valores faltantes hay en cada columna.

Este conteo nos permite identificar rápidamente qué columnas tienen datos incompletos y cuáles parecen estar completas.

Sin embargo, todavía necesitamos interpretar esos números. Una columna con 100 valores faltantes puede parecer problemática, pero su importancia depende del tamaño total del dataset. No es lo mismo tener 100 faltantes en un dataset de 200 filas que en un dataset de 100.000 filas.

Por eso, además de contar valores faltantes, conviene calcular qué porcentaje representan.

## Calcular porcentajes de valores faltantes

El conteo absoluto de valores faltantes es útil, pero no siempre alcanza para interpretar la magnitud del problema.

Para comprender mejor el impacto de los faltantes, conviene calcular qué porcentaje representan sobre el total de filas del dataset.

Podemos hacerlo usando `isna().mean()`. Como `True` se interpreta como `1` y `False` como `0`, el promedio de una máscara booleana indica la proporción de valores faltantes.

```
porcentaje_faltantes = df.isna().mean() * 100

porcentaje_faltantes
```

índice	0
Transaction ID	0.00
Item	3.33
Quantity	1.38
Price Per Unit	1.79
Total Spent	1.73
Payment Method	25.79
Location	32.65
Transaction Date	1.59

```
dtype: float64
```

La salida muestra el porcentaje de valores faltantes en cada columna.

Por ejemplo, si una columna muestra un valor cercano a **5**, significa que aproximadamente el 5% de sus filas tienen valores faltantes. Si una columna muestra **0**, significa que Pandas no detectó valores faltantes en esa columna.

También podemos combinar el conteo y el porcentaje en una misma tabla para leer el diagnóstico con mayor claridad.

```
diagnostico_faltantes = pd.DataFrame({
 "faltantes": df.isna().sum(),
 "porcentaje": df.isna().mean() * 100
})

diagnostico_faltantes
```

índice	faltantes	porcentaje
Transaction ID	0	0.00
Item	333	3.33
Quantity	138	1.38
Price Per Unit	179	1.79
Total Spent	173	1.73
Payment Method	2579	25.79

índice	faltantes	porcentaje
Location	3265	32.65
Transaction Date	159	1.59

Esta tabla resume, para cada columna, cuántos valores faltantes hay y qué porcentaje representan.

Este tipo de diagnóstico es muy útil porque nos permite priorizar. Una columna con pocos faltantes puede requerir una estrategia diferente de una columna con muchos faltantes. Además, no todas las columnas tienen la misma importancia para el análisis.

Por ahora seguimos en la etapa de diagnóstico. Todavía no vamos a eliminar ni completar valores faltantes.

## Ordenar el diagnóstico de faltantes

Cuando un dataset tiene muchas columnas, la tabla de diagnóstico puede ser más fácil de leer si la ordenamos según la cantidad de valores faltantes.

Podemos ordenar la tabla `diagnostico_faltantes` usando `sort_values()`.

```
diagnostico_faltantes.sort_values("faltantes", ascending=False)
```

índice	faltantes	porcentaje
Location	3265	32.65
Payment Method	2579	25.79
Item	333	3.33
Price Per Unit	179	1.79
Total Spent	173	1.73
Transaction Date	159	1.59
Quantity	138	1.38
Transaction ID	0	0.00

Ahora las columnas con más valores faltantes aparecen primero.

Esta forma de presentación ayuda a identificar rápidamente dónde se concentra el problema. En una limpieza real, muchas veces conviene empezar revisando las columnas con mayor cantidad de faltantes o aquellas que son más importantes para el análisis.

También podemos ordenar por porcentaje:

```
diagnostico_faltantes.sort_values("porcentaje", ascending=False)
```

índice	faltantes	porcentaje
Location	3265	32.65
Payment Method	2579	25.79
Item	333	3.33
Price Per Unit	179	1.79
Total Spent	173	1.73
Transaction Date	159	1.59
Quantity	138	1.38
Transaction ID	0	0.00

En este caso, el resultado es similar porque el porcentaje se calcula a partir del mismo total de filas. Sin embargo, incluir el porcentaje facilita la interpretación.

Un número absoluto puede parecer grande o pequeño según el tamaño del dataset. En cambio, el porcentaje permite evaluar mejor el peso relativo del problema.

Por ejemplo, una columna con 500 valores faltantes puede parecer muy problemática. Pero si el dataset tiene 100.000 filas, esos 500 faltantes representan solo el 0,5%. En cambio, si el dataset tiene 1.000 filas, representan el 50%.

Por eso, cuando analizamos faltantes, conviene mirar siempre dos cosas:

```
cantidad de valores faltantes
porcentaje sobre el total de filas
```

Ambas medidas se complementan y ayudan a tomar mejores decisiones.

## Ver filas con valores faltantes

Después de contar los valores faltantes por columna, puede ser útil observar algunas filas donde esos faltantes aparecen.

Esto nos permite pasar del diagnóstico general a una revisión más concreta. No alcanza con saber que una columna tiene valores faltantes; también conviene mirar ejemplos reales para entender cómo aparecen dentro del dataset.

Podemos construir una máscara que indique qué filas tienen al menos un valor faltante.

```
filas_con_faltantes = df.isna().any(axis=1)

filas_con_faltantes
```

índice	0
0	False
1	False
2	False
3	False
4	False
...	...
9995	True
9996	True
9997	True
9998	True
9999	False

`dtype: bool`

La instrucción `df.isna()` detecta los valores faltantes en todo el `DataFrame`.

Luego usamos `.any(axis=1)` para preguntar, fila por fila, si aparece al menos un valor faltante.

El parámetro `axis=1` indica que queremos revisar horizontalmente, es decir, a lo largo de las columnas de cada fila.

El resultado es una máscara booleana: `True` para las filas que tienen al menos un valor faltante, y `False` para las filas que no tienen faltantes detectados por Pandas.

Ahora podemos usar esa máscara para ver solamente las filas incompletas.

```
df[filas_con_faltantes].head(10)
```

índice	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date
5	TXN_2602893	Smoothie	5	4.0	20.0	Credit Card	NaN	2023-03-31
8	TXN_4717867	NaN	5	3.0	15.0	NaN	Takeaway	2023-07-28
9	TXN_2064365	Sandwich	5	4.0	20.0	NaN	In-store	2023-12-31
13	TXN_9437049	Cookie	5	1.0	5.0	NaN	Takeaway	2023-06-01
14	TXN_8915701	ERROR	2	1.5	3.0	NaN	In-store	2023-03-21
16	TXN_3765707	Sandwich	1	4.0	4.0	NaN	NaN	2023-06-10
23	TXN_2616390	Sandwich	2	4.0	8.0	NaN	NaN	2023-09-18
25	TXN_7958992	Smoothie	3	4.0	NaN	UNKNOWN	UNKNOWN	2023-12-13
28	TXN_8467949	Smoothie	5	4.0	20.0	Credit Card	NaN	2023-03-11
30	TXN_1736287	NaN	5	2.0	10.0	Digital Wallet	NaN	2023-06-02

Esta vista nos muestra algunos ejemplos de filas con valores faltantes.

Observar casos concretos ayuda a interpretar el problema. Podemos revisar en qué columnas faltan datos, si faltan varios valores en una misma fila o si los faltantes parecen concentrarse en ciertos tipos de registros.

Como el dataset puede tener muchas filas incompletas, usamos `head(10)` para ver solo las primeras diez. Esta muestra no reemplaza el diagnóstico general, pero nos ayuda a comprender mejor cómo se ven los faltantes dentro de la tabla.

## Analizar faltantes en columnas importantes

No todas las columnas tienen la misma importancia para el análisis.

En un dataset de ventas, algunas columnas son centrales porque permiten entender qué se vendió, cuánto se vendió, cuánto costó y cuándo ocurrió la transacción. Otras columnas pueden ser importantes para ciertos análisis, pero no para todos.

Por ejemplo, en este dataset hay columnas especialmente relevantes:

```
Item
Quantity
Price Per Unit
Total Spent
Transaction Date
```

Si faltan valores en `Item`, no sabemos qué producto se vendió. Si faltan valores en `Quantity` o `Price Per Unit`, se vuelve difícil calcular el importe de la venta. Si falta `Total Spent`, tal vez podamos reconstruirlo usando otras columnas, pero primero necesitamos revisar el caso. Si falta `Transaction Date`, perdemos información temporal.

Podemos observar el diagnóstico solamente para algunas columnas importantes.

```
columnas_importantes = [
 "Item",
 "Quantity",
 "Price Per Unit",
 "Total Spent",
 "Transaction Date"
]

diagnostico_faltantes.loc[columnas_importantes]
```

índice	faltantes	porcentaje
Item	333	3.33
Quantity	138	1.38
Price Per Unit	179	1.79
Total Spent	173	1.73
Transaction Date	159	1.59

Esta selección nos permite concentrarnos en columnas que probablemente tengan un impacto fuerte sobre el análisis.

La importancia de un valor faltante no depende solamente de cuántos faltantes hay, sino también de qué representa la columna. Un faltante en una columna secundaria puede tener poco impacto. Un faltante en una columna central puede impedir cálculos importantes o limitar las conclusiones posibles.

Por eso, al diagnosticar valores faltantes conviene combinar dos miradas:

```
magnitud del problema → cuántos faltantes hay
importancia del problema → qué columna está afectada
```

Esa combinación nos ayuda a priorizar qué problemas revisar primero.

## Valores textuales que pueden representar ausencia o error

Hasta ahora usamos `isna()` para detectar valores faltantes reconocidos por Pandas.

Sin embargo, algunos datasets usan palabras o códigos especiales para representar información desconocida o errores de carga. Por ejemplo, pueden aparecer valores como:

```
UNKNOWN
ERROR
N/A
Sin dato
No informado
```

El problema es que, si esos valores están escritos como texto, Pandas no los cuenta necesariamente como valores faltantes.

Por eso, además de buscar `NaN`, debemos revisar los valores que aparecen dentro de algunas columnas categóricas.

```
df["Item"].value_counts(dropna=False)
```

Item	count
Juice	1171
Coffee	1165
Salad	1148
Cake	1139
Sandwich	1131
Smoothie	1096
Cookie	1092
Tea	1089
UNKNOWN	344
NaN	333
ERROR	292

```
dtype: int64
```

```
df["Payment Method"].value_counts(dropna=False)
```

Payment Method	count
NaN	2579
Digital Wallet	2291



Payment Method	count
Credit Card	2273
Cash	2258
ERROR	306
UNKNOWN	293

dtype: int64

```
df["Location"].value_counts(dropna=False)
```

Location	count
NaN	3265
Takeaway	3022
In-store	3017
ERROR	358
UNKNOWN	338

dtype: int64

El parámetro `dropna=False` permite que `value_counts()` incluya también los valores faltantes reales, si los hay.

Al revisar estas salidas, debemos prestar atención a valores como `"UNKNOWN"` o `"ERROR"`. Esos valores no son exactamente iguales a `NaN`, pero pueden indicar que el dato no está disponible, que fue cargado con un código especial o que hubo algún problema en el registro.

Esta diferencia es importante. Si una columna tiene pocos `NaN`, podríamos pensar que está casi completa. Pero si además tiene muchos valores `"UNKNOWN"` o `"ERROR"`, entonces el problema de calidad puede ser mayor de lo que parecía al mirar solo `isna()`.

Por eso, cuando diagnosticamos faltantes, conviene mirar dos cosas:

```
valores faltantes reconocidos por Pandas
valores textuales que podrían representar ausencia o error
```

Todavía no vamos a decidir cómo tratar esos valores. Por ahora, los estamos identificando para entender mejor el estado real del dataset.

## Comparar NaN con UNKNOWN y ERROR

---

En este punto ya podemos distinguir dos situaciones diferentes.

Por un lado, están los valores faltantes que Pandas reconoce como `NaN`. Estos aparecen cuando una celda está realmente vacía o cuando Pandas interpreta que allí falta un valor.

Por otro lado, están los valores escritos como texto, como `"UNKNOWN"` o `"ERROR"`. Estos valores pueden indicar ausencia de información o problemas de carga, pero Pandas los trata como textos comunes.

Esto significa que una columna puede parecer bastante completa si miramos solo `isna()`, pero tener muchos valores problemáticos si también contamos `"UNKNOWN"` y `"ERROR"`.

Podemos revisar cuántas veces aparecen esos valores en algunas columnas.

```
valores_problematicos = ["UNKNOWN", "ERROR"]

for columna in ["Item", "Payment Method", "Location"]:
 print(f"Columna: {columna}")
 print(df[columna].isin(valores_problematicos).sum())
 print()
```

```
Columna: Item
636
```

```
Columna: Payment Method
599
```

```
Columna: Location
696
```

En esta celda usamos `isin()` para revisar si los valores de cada columna pertenecen a la lista `["UNKNOWN", "ERROR"]`.

La salida muestra cuántos valores problemáticos aparecen en cada columna revisada.

Esta información complementa el diagnóstico de valores faltantes. No reemplaza a `isna()`, porque `NaN`, `"UNKNOWN"` y `"ERROR"` no son exactamente lo mismo. Pero nos ayuda a ver que el problema de calidad de datos puede ser más amplio que el conteo de celdas vacías.

Podemos construir una pequeña tabla para resumir esta revisión.

```

diagnostico_textos_problematicos = pd.DataFrame({
 "UNKNOWN_o_ERROR": {
 columna: df[columna].isin(valores_problematicos).sum()
 for columna in ["Item", "Payment Method", "Location"]
 }
})

diagnostico_textos_problematicos

```

índice	UNKNOWN_o_ERROR
Item	636
Payment Method	599
Location	696

Esta tabla nos permite registrar cuántos valores "UNKNOWN" o "ERROR" aparecen en algunas columnas categóricas.

Más adelante tendremos que decidir qué hacer con esos valores. Podríamos tratarlos como datos faltantes, reemplazarlos por una categoría como "Desconocido", excluirllos de ciertos análisis o revisarlos junto con otras columnas.

Por ahora, lo importante es reconocer que no todos los problemas de ausencia aparecen automáticamente como NaN.

## Registrar un diagnóstico de valores faltantes

Después de revisar NaN, porcentajes de faltantes y valores textuales como "UNKNOWN" o "ERROR", conviene registrar un diagnóstico inicial.

Este diagnóstico no es todavía una limpieza. Es una síntesis del problema.

La idea es reunir evidencia para poder decidir, más adelante, qué estrategia de tratamiento conviene aplicar en cada caso.

Podemos empezar armando una tabla con los faltantes reconocidos por Pandas.

```

diagnostico_faltantes = pd.DataFrame({
 "faltantes_nan": df.isna().sum(),
 "porcentaje_nan": df.isna().mean() * 100
})

diagnostico_faltantes.sort_values("faltantes_nan", ascending=False)

```

índice	faltantes_nan	porcentaje_nan
Location	3265	32.65
Payment Method	2579	25.79
Item	333	3.33
Price Per Unit	179	1.79
Total Spent	173	1.73
Transaction Date	159	1.59
Quantity	138	1.38
Transaction ID	0	0.00

Esta tabla resume los valores que Pandas reconoce como faltantes.

Pero, como vimos, también pueden existir valores problemáticos escritos como texto. Para algunas columnas categóricas, podemos complementar el diagnóstico contando "UNKNOWN" y "ERROR".

```
columnas_categoricas_revisar = ["Item", "Payment Method", "Location"]

diagnostico_textos = pd.DataFrame({
 "UNKNOWN_o_ERROR": {
 columna: df[columna].isin(["UNKNOWN", "ERROR"]).sum()
 for columna in columnas_categoricas_revisar
 }
})

diagnostico_textos
```

índice	UNKNOWN_o_ERROR
Item	636
Payment Method	599
Location	696

Estas dos tablas nos muestran dos tipos de problemas distintos.

Por un lado, tenemos los faltantes reconocidos como NaN. Por otro lado, tenemos valores escritos como texto que pueden representar ausencia de información o errores de carga.

En un análisis real, este diagnóstico debería acompañarse con una interpretación. Por ejemplo:

Algunas columnas tienen valores faltantes reconocidos por Pandas. Además, ciertas columnas categóricas contienen valores como UNKNOWN o ERROR. Estos valores no aparecen necesariamente en el conteo de NaN, pero también requieren revisión. Antes de eliminar o imputar datos, conviene decidir qué representa cada caso.

Registrar el diagnóstico nos ayuda a no actuar de manera automática. La limpieza de datos no consiste solamente en aplicar una función, sino en tomar decisiones a partir de evidencia.

## Por qué todavía no vamos a eliminar ni imputar

Después de detectar valores faltantes, puede parecer natural querer resolver el problema de inmediato.

Podríamos pensar en eliminar las filas incompletas, completar valores con algún dato fijo, reemplazar valores desconocidos o reconstruir información faltante a partir de otras columnas.

Sin embargo, todavía no conviene hacerlo.

Antes de eliminar o imputar valores necesitamos comprender mejor el problema. Una decisión apresurada puede afectar el análisis. Por ejemplo, si eliminamos todas las filas con algún valor faltante, podríamos perder muchas transacciones útiles. Si completamos valores sin criterio, podríamos introducir información artificial que modifique los resultados. Si tratamos "UNKNOWN" y "ERROR" como si fueran exactamente lo mismo, tal vez estemos mezclando ausencia de información con errores de carga.

También debemos recordar que no todas las columnas tienen la misma importancia. Un faltante en `Payment Method` no tiene el mismo impacto que un faltante en `Quantity`, `Price Per Unit` o `Total Spent`. Algunas ausencias pueden limitar ciertos análisis, mientras que otras pueden impedir cálculos centrales.

Por eso, en este capítulo nos concentramos en diagnosticar. El objetivo fue responder preguntas como:

¿Dónde hay valores faltantes?  
¿Cuántos hay?  
¿Qué porcentaje representan?  
¿Qué columnas están afectadas?  
¿Existen valores textuales que parecen representar ausencia o error?  
¿Qué columnas requieren una revisión más cuidadosa?

Una vez que entendemos mejor el problema, estamos en mejores condiciones de decidir una estrategia.

La limpieza de datos responsable no empieza por eliminar o completar, sino por comprender.

## Resumen del capítulo

---

En este capítulo estudiamos los valores faltantes desde una perspectiva de diagnóstico.

Primero vimos que un valor faltante aparece cuando una celda del dataset no contiene información disponible. También aclaramos que la ausencia de un dato puede tener distintos significados según la columna afectada y el contexto del análisis. No es lo mismo que falte el método de pago, el producto vendido, la cantidad, el precio unitario o la fecha de una transacción.

Luego vimos que Pandas suele representar los valores faltantes como `NaN`. Para detectarlos usamos `isna()`, que devuelve `True` cuando encuentra un valor faltante y `False` cuando la celda contiene algún valor.

Después combinamos `isna()` con `sum()` para contar valores faltantes por columna:

```
df.isna().sum()
```

También calculamos porcentajes de faltantes usando:

```
df.isna().mean() * 100
```

Esto nos permitió interpretar mejor la magnitud del problema. Un conteo absoluto puede ser útil, pero el porcentaje ayuda a entender qué peso tienen los faltantes en relación con el tamaño total del dataset.

Más adelante construimos una tabla de diagnóstico:

```
diagnostico_faltantes = pd.DataFrame({
 "faltantes": df.isna().sum(),
 "porcentaje": df.isna().mean() * 100
})
```

Esa tabla nos permitió observar, columna por columna, cuántos valores faltantes había y qué proporción representaban.

También aprendimos a ver filas con al menos un valor faltante usando:

```
filas_con_faltantes = df.isna().any(axis=1)

df[filas_con_faltantes].head(10)
```

Este paso fue importante porque nos permitió pasar del resumen general a la observación de casos concretos.

Luego analizamos columnas importantes para el contexto del dataset, como `Item`, `Quantity`, `Price Per Unit`, `Total Spent` y `Transaction Date`. Esto nos ayudó a recordar que no todos los valores faltantes tienen el mismo impacto: la importancia del problema depende tanto de la cantidad de faltantes como del significado de la columna afectada.

También distinguimos entre valores faltantes reconocidos por Pandas y valores textuales que pueden representar ausencia o error, como `"UNKNOWN"` o `"ERROR"`. Estos valores no siempre aparecen en el conteo de `isna()`, porque Pandas puede interpretarlos como texto común.

Por eso revisamos algunas columnas categóricas con:

```
value_counts(dropna=False)
```

y también contamos apariciones de `"UNKNOWN"` y `"ERROR"` usando `isin()`.

La idea principal de este capítulo fue:

```
Antes de tratar valores faltantes, necesitamos diagnosticarlos.
```

Detectar faltantes no significa decidir automáticamente qué hacer con ellos. Primero necesitamos saber dónde están, cuántos son, qué porcentaje representan, qué columnas afectan y si existen valores textuales que también deban considerarse problemáticos.

## Próximo paso

---

El siguiente paso será estudiar estrategias para tratar valores faltantes.

Vamos a analizar cuándo puede tener sentido eliminar filas, cuándo conviene eliminar columnas, cuándo es razonable completar valores faltantes y cuándo podemos reconstruir información usando relaciones internas del dataset.

También vamos a discutir los riesgos de cada decisión. Eliminar datos puede reducir el tamaño del dataset o sesgar el análisis. Imputar valores puede introducir

información artificial. Reconstruir datos puede ser útil, pero solo si la relación entre columnas es confiable.

El objetivo será pasar del diagnóstico a la toma de decisiones.



## Capítulo 15 — Eliminar, imputar o reconstruir valores faltantes

---

### Breve repaso

---

En el trabajo anterior sobre valores faltantes aprendimos a diagnosticar la presencia de datos incompletos dentro de un `DataFrame`.

Vimos que Pandas suele representar los valores faltantes como `NaN`, y usamos herramientas como `isna()`, `sum()` y `mean()` para detectar cuántos faltantes había en cada columna y qué porcentaje representaban sobre el total de filas.

También observamos que no todos los problemas de ausencia aparecen como `NaN`. Algunos datasets usan valores textuales como `"UNKNOWN"` o `"ERROR"` para indicar información desconocida, errores de carga o datos que no pudieron registrarse correctamente. Por eso, además de contar valores faltantes reconocidos por Pandas, también revisamos valores únicos y frecuencias en columnas categóricas.

En este capítulo vamos a avanzar un paso más: vamos a estudiar distintas estrategias para tratar valores faltantes.

La idea no será aplicar una solución automática, sino analizar distintas posibilidades. A veces puede tener sentido eliminar filas incompletas. En otros casos, puede ser mejor conservarlas. Algunas columnas pueden completarse con un valor razonable, mientras que otras pueden reconstruirse usando información disponible en el propio dataset.

Trabajaremos con tres grandes estrategias:

```
eliminar datos faltantes
imputar valores faltantes
reconstruir valores faltantes a partir de otras columnas
```

Cada estrategia tiene ventajas, riesgos y condiciones de uso. Por eso, el objetivo de este capítulo no es memorizar comandos, sino aprender a decidir con criterio.

Al finalizar este capítulo deberías poder:

- Comprender la diferencia entre eliminar, imputar y reconstruir valores faltantes.
- Usar `dropna()` para eliminar filas o columnas con valores faltantes.
- Usar `fillna()` para completar valores faltantes en algunos casos.

- Evaluar los riesgos de eliminar datos.
- Evaluar los riesgos de imputar datos.
- Reconstruir valores faltantes cuando existe una relación confiable entre columnas.
- Verificar el resultado después de tratar valores faltantes.
- Comprender por qué no hay una única estrategia correcta para todos los datasets.

En este capítulo vamos a seguir usando el dataset **Cafe Sales — Dirty Data for Cleaning Training**.

Antes de decidir cómo tratar los valores faltantes, conviene recordar el estado general del dataset.

Vamos a revisar nuevamente la cantidad de filas y columnas, los valores faltantes reconocidos por Pandas y algunos valores textuales problemáticos.

```
df.shape
```

```
(10000, 8)
```

```
df.isna().sum()
```

índice	0
Transaction ID	0
Item	333
Quantity	138
Price Per Unit	179
Total Spent	173
Payment Method	2579
Location	3265
Transaction Date	159

```
dtype: int64
```

```
df[["Item", "Payment Method", "Location"]].apply(
 lambda columna: columna.isin(["UNKNOWN", "ERROR"]).sum()
)
```

índice	0
Item	636
Payment Method	599
Location	696

`dtype: int64`

Estas primeras revisiones nos permiten recuperar el diagnóstico inicial.

Tenemos, por un lado, valores faltantes reconocidos por Pandas como `NaN`. Por otro lado, tenemos valores escritos como `"UNKNOWN"` o `"ERROR"` que también pueden representar ausencia de información o errores de carga, aunque Pandas los trate como texto común.

En este capítulo vamos a trabajar principalmente con estrategias de tratamiento. Antes de aplicar cualquier transformación, mantendremos una idea central: cada decisión debe justificarse según el tipo de columna, el problema detectado y el análisis que queremos realizar.

## Tres estrategias posibles

---

Cuando encontramos valores faltantes, podemos seguir distintas estrategias.

No existe una única solución correcta para todos los casos. La decisión depende del contexto, de la columna afectada, de la cantidad de datos faltantes y del objetivo del análisis.

En este capítulo vamos a trabajar con tres estrategias generales:

```
eliminar
imputar
reconstruir
```

**Eliminar** significa quitar filas o columnas que contienen valores faltantes. Es una estrategia simple, pero puede hacernos perder información. Si eliminamos demasiadas filas, el dataset puede quedar reducido o sesgado. Si eliminamos una columna completa, podemos perder una variable que tal vez era importante.

**Imputar** significa completar valores faltantes con algún valor elegido. Ese valor puede ser una constante, una categoría como `"Desconocido"`, una medida estadística como la media o la mediana, o algún valor definido según el contexto. Imputar permite conservar filas, pero también introduce información que no estaba originalmente en el dataset.

**Reconstruir** significa calcular un valor faltante a partir de otras columnas disponibles. Esta suele ser una estrategia más sólida que imputar de manera genérica, siempre que exista una relación confiable entre las columnas. En este dataset, por ejemplo, podemos usar la relación entre `Quantity`, `Price Per Unit` y `Total Spent`.

Podemos pensar estas estrategias de esta manera:

Estrategia	Qué hace	Riesgo principal
Eliminar	Quita filas o columnas incompletas	Perder información útil
Imputar	Completa valores faltantes con un criterio	Introducir información artificial
Reconstruir	Calcula valores usando otras columnas	Depender de una relación que debe ser válida

Antes de elegir una estrategia, necesitamos hacernos una pregunta fundamental:

¿Qué representa este valor faltante y qué impacto tiene sobre el análisis?

La respuesta puede cambiar de una columna a otra. Por eso, en limpieza de datos, no conviene aplicar la misma solución a todo el dataset sin revisar primero el contexto.

## Eliminar filas con valores faltantes

Una primera estrategia posible es eliminar las filas que tienen valores faltantes.

En Pandas, podemos hacerlo con `dropna()`.

Si usamos `dropna()` sin parámetros adicionales, Pandas elimina todas las filas que tengan al menos un valor faltante.

```
df_sin_faltantes = df.dropna()

df_sin_faltantes.shape
```

```
(4550, 8)
```

La salida muestra cuántas filas y columnas quedan después de eliminar las filas incompletas.

Para interpretar mejor el impacto de esta decisión, conviene comparar el tamaño del dataset original con el tamaño del dataset resultante.

```
print("Tamaño original:", df.shape)
print("Tamaño después de dropna():", df_sin_faltantes.shape)
```

```
Tamaño original: (10000, 8)
Tamaño después de dropna(): (4550, 8)
```

Esta comparación es fundamental.

Eliminar filas con valores faltantes puede parecer una solución rápida, pero también puede reducir mucho el dataset. Si se eliminan muchas filas, podríamos perder información importante o modificar la composición de los datos.

Por eso, `dropna()` debe usarse con cuidado. No deberíamos aplicarlo automáticamente sin revisar cuántas filas se pierden y qué información contienen esas filas.

También podemos calcular cuántas filas fueron eliminadas:

```
filas_eliminadas = df.shape[0] - df_sin_faltantes.shape[0]

filas_eliminadas
```

```
5450
```

Y podemos calcular qué porcentaje del dataset original representa esa pérdida:

```
porcentaje_eliminado = filas_eliminadas / df.shape[0] * 100

porcentaje_eliminado
```

```
54.500000000000001
```

Si el porcentaje eliminado es alto, usar `dropna()` de manera general puede ser una mala decisión.

En algunos casos, eliminar filas incompletas puede ser razonable. Por ejemplo, si son muy pocas filas y los valores faltantes afectan columnas centrales que no pueden reconstruirse. Pero en otros casos puede ser preferible conservar las filas y aplicar otra estrategia.

La idea principal es que eliminar datos siempre tiene un costo: perdemos registros. Por eso, antes de hacerlo, debemos medir el impacto de esa decisión.

## Eliminar filas según columnas específicas

Usar `dropna()` sobre todo el `DataFrame` puede ser una decisión demasiado amplia. Tal vez una fila tenga un valor faltante en una columna secundaria, pero conserve información útil en columnas importantes.

Por eso, muchas veces conviene aplicar la eliminación solo considerando ciertas columnas.

Por ejemplo, podríamos decidir que para algunos análisis necesitamos conservar únicamente las filas que tienen información válida en estas columnas:

```
Item
Quantity
Price Per Unit
Total Spent
```

Estas columnas son importantes porque permiten saber qué producto se vendió, cuántas unidades se vendieron, cuál fue el precio unitario y cuánto se gastó en total.

Podemos usar `dropna()` con el parámetro `subset` para indicar qué columnas deben revisarse.

```
columnas_ventas = [
 "Item",
 "Quantity",
 "Price Per Unit",
 "Total Spent"
]

df_ventas_completas = df.dropna(subset=columnas_ventas)

df_ventas_completas.shape
```

```
(9197, 8)
```

En este caso, Pandas elimina solamente las filas que tienen valores faltantes en alguna de las columnas indicadas en `subset`.

Si una fila tiene un faltante en otra columna, como `Payment Method` o `Location`, no será eliminada por esta instrucción.

Comparemos nuevamente los tamaños:

```
print("Tamaño original:", df.shape)
print("Tamaño con ventas completas:", df_ventas_completas.shape)
```

```
Tamaño original: (10000, 8)
Tamaño con ventas completas: (9197, 8)
```

Esta estrategia es más controlada que usar `dropna()` sobre todo el `DataFrame`.

La diferencia es importante: no estamos diciendo “eliminar cualquier fila con cualquier dato faltante”, sino “eliminar las filas que no tienen información suficiente en las columnas que consideramos centrales para este análisis”.

En limpieza de datos, este tipo de decisión suele ser más razonable porque se apoya en el significado de las columnas y no solo en la presencia general de valores faltantes.

## Eliminar columnas con demasiados valores faltantes

Además de eliminar filas, también podríamos eliminar columnas.

Esta decisión puede tener sentido cuando una columna tiene una proporción muy alta de valores faltantes y no resulta central para el análisis. Sin embargo, eliminar una columna completa es una decisión importante, porque implica descartar una variable del dataset.

Antes de eliminar columnas, conviene revisar el porcentaje de faltantes.

```
diagnostico_faltantes = pd.DataFrame({
 "faltantes": df.isna().sum(),
 "porcentaje": df.isna().mean() * 100
})

diagnostico_faltantes.sort_values("porcentaje", ascending=False)
```

índice	faltantes	porcentaje
Location	3265	32.65
Payment Method	2579	25.79
Item	333	3.33
Price Per Unit	179	1.79
Total Spent	173	1.73
Transaction Date	159	1.59
Quantity	138	1.38
Transaction ID	0	0.00

Supongamos que decidimos eliminar columnas con más del 50% de valores faltantes.

Para hacerlo, podemos usar `dropna()` con el parámetro `axis=1`.

Cuando `axis=0`, Pandas trabaja sobre filas. Cuando `axis=1`, trabaja sobre columnas.

También podemos usar el parámetro `thresh`, que indica cuántos valores no faltantes debe tener una columna para conservarse.

Por ejemplo, si queremos conservar solamente las columnas que tienen al menos la mitad de sus valores presentes, podemos calcular ese mínimo de esta manera:

```
minimo_valores_presentes = len(df) * 0.5

minimo_valores_presentes
```

```
5000.0
```

```
df_columnas_suficientes = df.dropna(
 axis=1,
 thresh=minimo_valores_presentes
)

df_columnas_suficientes.shape
```

```
(10000, 8)
```

Esta instrucción conserva solamente las columnas que tienen al menos la cantidad mínima de valores no faltantes indicada en `thresh`.

En este ejemplo usamos un umbral del 50%, pero ese número no debe tomarse como una regla universal. En algunos análisis podríamos tolerar más faltantes si la columna es muy importante. En otros casos, podríamos exigir columnas casi completas.

También podemos comparar qué columnas quedaron:

```
print("Columnas originales:")
print(list(df.columns))

print()

print("Columnas conservadas:")
print(list(df_columnas_suficientes.columns))
```



Columnas originales:

```
['Transaction ID', 'Item', 'Quantity', 'Price Per Unit', 'Total Spent', 'Payment Method', 'Location', 'Transaction Date']
```

Columnas conservadas:

```
['Transaction ID', 'Item', 'Quantity', 'Price Per Unit', 'Total Spent', 'Payment Method', 'Location', 'Transaction Date']
```

Eliminar columnas puede simplificar el dataset, pero también puede hacernos perder información valiosa. Por eso, esta decisión debe tomarse con cuidado.

Antes de eliminar una columna, conviene preguntarnos:

¿Qué representa esta columna?

¿Cuántos valores faltan?

¿La columna es importante para el análisis?

¿Existe alguna forma razonable de completar o reconstruir sus valores?

¿Qué perdemos si la eliminamos?

La cantidad de faltantes es un dato importante, pero no debería ser el único criterio para decidir.

## Imputar valores faltantes con un valor fijo

Otra estrategia posible es completar valores faltantes con algún valor definido. A esta acción se la suele llamar **imputar**.

Imputar significa reemplazar un valor faltante por otro valor. Ese valor puede ser una constante, una categoría, una medida estadística o un valor calculado a partir de otras columnas.

Una forma simple de imputación es completar valores faltantes en una columna categórica con una etiqueta como "Desconocido".

Por ejemplo, si faltan valores en la columna `Payment Method`, podríamos decidir conservar esas transacciones y registrar que el método de pago es desconocido.

Antes de aplicar el cambio, revisemos cuántos faltantes tiene esa columna.

```
df["Payment Method"].isna().sum()
```

```
np.int64(2579)
```

Ahora podemos crear una copia del `DataFrame` y completar los valores faltantes de `Payment Method` con "Desconocido".

Usamos una copia para no modificar todavía el dataset original.

```
df_imputado = df.copy()

df_imputado["Payment Method"] = df_imputado["Payment Method"].fillna("Desconocido")

df_imputado["Payment Method"].value_counts(dropna=False)
```

Payment Method	count
Desconocido	2579
Digital Wallet	2291
Credit Card	2273
Cash	2258
ERROR	306
UNKNOWN	293

dtype: int64

El método `fillna()` permite reemplazar valores faltantes por un valor determinado.

En este caso, los `NaN` de la columna `Payment Method` fueron reemplazados por `"Desconocido"`.

Esta estrategia puede ser razonable cuando el dato faltante representa una categoría desconocida y queremos conservar las filas para otros análisis.

Sin embargo, también tiene un riesgo: al completar los faltantes con una etiqueta, estamos creando una categoría nueva que no estaba explícitamente registrada en los datos originales. Eso puede ser útil, pero debe interpretarse correctamente.

No significa que el método de pago haya sido realmente `"Desconocido"` como una opción elegida por el cliente. Significa que nosotros decidimos etiquetar así las transacciones donde esa información no estaba disponible.

Antes de imputar valores numéricos, necesitamos revisar un detalle importante de este dataset. Algunas columnas que deberían ser numéricas fueron cargadas como texto, porque contienen valores como `"ERROR"` o `"UNKNOWN"` mezclados con números.

Más adelante vamos a estudiar la conversión de tipos de datos con mayor profundidad. Por ahora, para poder trabajar con imputación y reconstrucción de valores, vamos a crear una copia del dataset y convertir algunas columnas a formato numérico.

Usaremos `pd.to_numeric()` con `errors="coerce"`. Esto intenta convertir los valores a número y, cuando encuentra un valor que no puede convertir, lo transforma en `NaN`.

```
df_tratamiento = df.copy()

columnas_numericas = [
 "Quantity",
 "Price Per Unit",
 "Total Spent"
]

for columna in columnas_numericas:
 df_tratamiento[columna] = pd.to_numeric(
 df_tratamiento[columna],
 errors="coerce"
)

df_tratamiento[columnas_numericas].info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 3 columns):
Column Non-Null Count Dtype
--- -
0 Quantity 9521 non-null float64
1 Price Per Unit 9467 non-null float64
2 Total Spent 9498 non-null float64
dtypes: float64(3)
memory usage: 234.5 KB
```

## Imputar valores numéricos con una medida estadística

Cuando una columna numérica tiene valores faltantes, una estrategia posible es completarlos con una medida estadística.

Por ejemplo, podríamos usar la media, la mediana o algún otro valor representativo de la columna.

Esta estrategia puede parecer sencilla, pero debe usarse con cuidado. Completar valores faltantes con una medida estadística introduce información artificial en el dataset. No estamos recuperando el dato real, sino reemplazándolo por una estimación.

Por eso, antes de imputar una columna numérica conviene revisar su distribución y pensar si la medida elegida tiene sentido.

Veamos un ejemplo con la columna `Price Per Unit`.

```
df_tratamiento["Price Per Unit"].isna().sum()
```

```
np.int64(533)
```

Primero revisamos cuántos valores faltantes tiene la columna.

Ahora podemos observar algunas estadísticas básicas:

```
df_tratamiento["Price Per Unit"].describe()
```

índice	Price Per Unit
count	9467.000000
mean	2.949984
std	1.278450
min	1.000000
25%	2.000000
50%	3.000000
75%	4.000000
max	5.000000

```
dtype: float64
```

Si decidiéramos completar los valores faltantes con la mediana, podríamos hacerlo así:

```
df_precio_imputado = df_tratamiento.copy()

mediana_precio = df_precio_imputado["Price Per Unit"].median()

df_precio_imputado["Price Per Unit"] = (
 df_precio_imputado["Price Per Unit"].fillna(mediana_precio)
)

df_precio_imputado["Price Per Unit"].isna().sum()
```

```
np.int64(0)
```

Después de aplicar `fillna()`, la columna ya no tiene valores faltantes reconocidos por Pandas.

Sin embargo, esto no significa que hayamos recuperado los precios reales que faltaban. Lo que hicimos fue reemplazar esos valores por la mediana de la columna.

La mediana puede ser una opción razonable cuando queremos evitar que valores muy altos o muy bajos influyan demasiado en la imputación. A diferencia de la media, la mediana suele ser más resistente a valores extremos.

Aun así, esta decisión debe justificarse. No deberíamos completar valores numéricos automáticamente solo porque Pandas nos permite hacerlo.

En algunos casos, imputar con una medida estadística puede ser útil. En otros, puede ocultar problemas importantes o distorsionar el análisis.

## Reconstruir valores usando otras columnas

En algunos casos, no necesitamos imputar un valor faltante con una media, una mediana o una etiqueta fija. A veces podemos reconstruirlo usando información disponible en otras columnas.

Este dataset tiene una relación muy importante:

```
Total Spent = Quantity × Price Per Unit
```

Esto significa que, si conocemos la cantidad y el precio unitario, podemos calcular el total gastado.

Esta estrategia suele ser más razonable que imputar con una medida general, porque utiliza una relación interna del propio dataset.

Primero observemos cuántos valores faltantes tiene cada una de estas columnas.

```
df_tratamiento[["Quantity", "Price Per Unit", "Total Spent"]].isna().sum()
```

índice	0
Quantity	479
Price Per Unit	533
Total Spent	502

```
dtype: int64
```

Ahora vamos a buscar filas donde falta `Total Spent`, pero sí están presentes `Quantity` y `Price Per Unit`.

En esos casos, podríamos reconstruir el total.

```
condicion_reconstruir_total = (
 df_tratamiento["Total Spent"].isna()
 & df_tratamiento["Quantity"].notna()
 & df_tratamiento["Price Per Unit"].notna()
)

df_tratamiento[condicion_reconstruir_total][
 ["Quantity", "Price Per Unit", "Total Spent"]
].head()
```

índice	Quantity	Price Per Unit	Total Spent
2	4.0	1.0	NaN
25	3.0	4.0	NaN
31	2.0	1.0	NaN
42	2.0	1.5	NaN
94	3.0	3.0	NaN

La condición anterior identifica filas donde **Total Spent** está faltante, pero las otras dos columnas necesarias para calcularlo están disponibles.

Para reconstruir esos valores, vamos a trabajar sobre una copia del dataset.

```
df_reconstruido = df_tratamiento.copy()

df_reconstruido.loc[
 condicion_reconstruir_total,
 "Total Spent"
] = (
 df_reconstruido.loc[condicion_reconstruir_total, "Quantity"]
 * df_reconstruido.loc[condicion_reconstruir_total, "Price Per Unit"]
)

df_reconstruido[condicion_reconstruir_total][
 ["Quantity", "Price Per Unit", "Total Spent"]
].head()
```

índice	Quantity	Price Per Unit	Total Spent
2	4.0	1.0	4.0
25	3.0	4.0	12.0
31	2.0	1.0	2.0
42	2.0	1.5	3.0

índice	Quantity	Price Per Unit	Total Spent
94	3.0	3.0	9.0

Ahora las filas seleccionadas tienen un valor reconstruido en `Total Spent`.

Esta estrategia no consiste en inventar un valor promedio, sino en aprovechar una relación esperada entre columnas. Por eso puede ser más defendible que una imputación estadística genérica.

De todos modos, también requiere cuidado. Solo podemos reconstruir valores si estamos seguros de que la relación entre columnas es válida y si las columnas usadas para el cálculo son confiables.

Por eso, después de reconstruir valores, siempre debemos verificar el resultado.

## Verificar valores reconstruidos

Después de reconstruir valores faltantes, necesitamos verificar que la transformación haya producido resultados coherentes.

En este caso reconstruimos algunos valores de `Total Spent` usando la relación:

$$\text{Total Spent} = \text{Quantity} \times \text{Price Per Unit}$$

Podemos comprobar si, para las filas reconstruidas, el valor de `Total Spent` coincide con la multiplicación entre `Quantity` y `Price Per Unit`.

```
df_reconstruido.loc[
 condicion_reconstruir_total,
 ["Quantity", "Price Per Unit", "Total Spent"]
].head()
```

índice	Quantity	Price Per Unit	Total Spent
2	4.0	1.0	4.0
25	3.0	4.0	12.0
31	2.0	1.0	2.0
42	2.0	1.5	3.0
94	3.0	3.0	9.0

También podemos crear una comparación explícita para esas filas.

```

verificacion_total = (
 df_reconstruido.loc[condicion_reconstruir_total, "Quantity"]
 * df_reconstruido.loc[condicion_reconstruir_total, "Price Per Unit"]
)

verificacion_total.head()

```

índice	0
2	4.0
25	12.0
31	2.0
42	3.0
94	9.0

`dtype: float64`

Ahora podemos comparar ese cálculo con la columna `Total Spent` reconstruida.

```

(
 df_reconstruido.loc[condicion_reconstruir_total, "Total Spent"]
 == verificacion_total
).value_counts()

```

índice	count
True	462

`dtype: int64`

Si la salida muestra `True` para todas las filas reconstruidas, significa que los valores cargados en `Total Spent` coinciden con el cálculo esperado.

Esta verificación es importante porque la reconstrucción no termina en la asignación. También necesitamos comprobar que el resultado respete la relación que usamos para completarlo.

En un proceso real de limpieza, cada transformación importante debería tener una verificación posterior. Esa verificación puede ser un conteo, una comparación, una revisión de filas, un resumen estadístico o cualquier otra comprobación que nos permita confiar en el cambio realizado.

## Comparar estrategias de tratamiento

Hasta ahora vimos tres formas posibles de tratar valores faltantes:



eliminar  
imputar  
reconstruir

Cada una responde a una lógica diferente.

Eliminar datos puede ser útil cuando los registros incompletos son pocos o cuando no hay una forma razonable de recuperar la información. Pero también puede reducir el tamaño del dataset y modificar su composición.

Imputar permite conservar registros, pero introduce un valor que no estaba originalmente en los datos. Puede ser una etiqueta como "Desconocido" o una medida estadística como la mediana. Esta estrategia debe interpretarse con cuidado, porque no recupera el dato real.

Reconstruir valores suele ser una alternativa más sólida cuando existe una relación confiable entre columnas. En este dataset, por ejemplo, **Total Spent** puede reconstruirse si conocemos **Quantity** y **Price Per Unit**.

Podemos resumirlo así:

Estrategia	Cuándo puede servir	Qué debemos verificar
Eliminar filas	Cuando los faltantes son pocos o afectan columnas críticas imposibles de reconstruir	Cuántas filas se pierden y si la pérdida puede sesgar el análisis
Eliminar columnas	Cuando una columna tiene demasiados faltantes y no es central	Qué información se pierde al quitar esa variable
Imputar con valor fijo	Cuando queremos conservar registros y marcar ausencia de información	Que el nuevo valor no se interprete como dato real observado
Imputar con medida estadística	Cuando una estimación resulta razonable para una columna numérica	Que la imputación no distorsione la distribución
Reconstruir	Cuando otras columnas permiten calcular el valor faltante	Que la relación usada para reconstruir sea válida

La decisión depende del contexto. No deberíamos aplicar una misma estrategia a todas las columnas solo porque es rápida o sencilla.

En limpieza de datos, una buena decisión no es necesariamente la que elimina más problemas visibles, sino la que conserva mejor la información útil y deja el dataset más confiable para el análisis.

## Trabajar con copias durante la limpieza

Cuando estamos probando estrategias de tratamiento, conviene evitar modificar el dataset original de manera apresurada.

Por eso, en varios ejemplos usamos copias:

```
df_imputado = df.copy()
```

O:

```
df_reconstruido = df.copy()
```

Esta práctica nos permite experimentar con una transformación sin perder el punto de partida. Si la decisión no resulta adecuada, podemos volver al dataset original y probar otra estrategia.

Trabajar con copias también ayuda a comparar resultados. Por ejemplo, podemos revisar cuántos valores faltantes tenía el dataset original y cuántos quedan después de una imputación o reconstrucción.

Veamos un ejemplo comparando `df` y `df_reconstruido` en la columna `Total Spent`.

```
print("Faltantes originales en Total Spent:")
print(df["Total Spent"].isna().sum())

print()

print("Faltantes después de reconstruir Total Spent:")
print(df_reconstruido["Total Spent"].isna().sum())
```

```
Faltantes originales en Total Spent:
173

Faltantes después de reconstruir Total Spent:
40
```

Esta comparación muestra si la reconstrucción redujo la cantidad de valores faltantes en esa columna.

También podemos comparar el tamaño del dataset original con el de una versión donde eliminamos filas incompletas:

```
print("Tamaño original:")
print(df.shape)
```

```
print()

print("Tamaño después de eliminar filas con faltantes:")
print(df_sin_faltantes.shape)
```

```
Tamaño original:
(10000, 8)

Tamaño después de eliminar filas con faltantes:
(4550, 8)
```

Estas comparaciones permiten documentar el impacto de cada decisión.

En limpieza de datos, no alcanza con transformar el dataset. También necesitamos poder explicar qué hicimos, por qué lo hicimos y qué cambió después de hacerlo.

Una buena práctica es conservar evidencia del proceso:

```
conteos antes y después
porcentajes antes y después
cantidad de filas eliminadas
cantidad de valores imputados
cantidad de valores reconstruidos
```

Esto vuelve el proceso más transparente y facilita revisar las decisiones tomadas.

## Errores frecuentes al tratar valores faltantes

Al tratar valores faltantes, uno de los errores más comunes es aplicar una solución automática sin diagnóstico previo.

Por ejemplo, podríamos usar `dropna()` sobre todo el `DataFrame` y eliminar todas las filas incompletas. Técnicamente es una instrucción válida, pero puede ser una mala decisión si elimina una parte importante del dataset o si descarta registros que todavía eran útiles para otros análisis.

Otro error frecuente es imputar valores sin justificar el criterio. Completar una columna numérica con la media o la mediana puede ser útil en algunos casos, pero no debe hacerse solo porque es rápido. Esa decisión puede modificar la distribución de los datos y afectar los resultados posteriores.

También debemos evitar tratar todos los faltantes como si fueran iguales. Un valor faltante en `Payment Method` no tiene el mismo impacto que un faltante en `Quantity`, `Price Per Unit` o `Total Spent`. Cada columna debe analizarse según lo que representa.

Un caso especialmente importante es confundir imputación con reconstrucción. Si completamos un valor usando una mediana, estamos estimando. Si lo calculamos a partir de una relación confiable entre columnas, estamos reconstruyendo. Ambas acciones completan un faltante, pero no tienen el mismo significado.

También puede ocurrir que tratemos solamente los `NaN` y olvidemos valores como `"UNKNOWN"` o `"ERROR"`. Esos valores pueden representar ausencia de información o errores de carga, aunque Pandas los interprete como texto común. Por eso, el tratamiento de faltantes no debe limitarse siempre a `isna()`.

Finalmente, otro error frecuente es no verificar después de transformar. Después de eliminar, imputar o reconstruir datos, debemos revisar qué cambió.

```
Ejemplo de verificación posterior para valores faltantes reconocidos por Pandas.

df_reconstruido.isna().sum()
```

índice	0
Transaction ID	0
Item	333
Quantity	479
Price Per Unit	533
Total Spent	40
Payment Method	2579
Location	3265
Transaction Date	159

```
dtype: int64
```

Esta revisión permite ver cuántos valores faltantes quedan después de la reconstrucción realizada.

En general, al tratar valores faltantes conviene seguir esta rutina:

```
diagnosticar
decidir una estrategia
aplicar la transformación
verificar el resultado
documentar la decisión
```

El objetivo no es dejar el dataset “sin ningún faltante” a cualquier costo. El objetivo es tomar decisiones razonables para que el dataset quede más preparado y más confiable para el análisis.

## Resumen del capítulo

---

En este capítulo trabajamos con distintas estrategias para tratar valores faltantes.

Partimos de una idea central: detectar valores faltantes no alcanza. Después del diagnóstico necesitamos decidir qué hacer con ellos. Esa decisión depende del contexto, de la columna afectada, de la cantidad de datos incompletos y del tipo de análisis que queremos realizar.

Primero distinguimos tres estrategias generales:

```
eliminar
imputar
reconstruir
```

Eliminar significa quitar filas o columnas con valores faltantes. Usamos `dropna()` para eliminar filas incompletas:

```
df_sin_faltantes = df.dropna()
```

Después comparamos el tamaño del dataset original con el tamaño del dataset resultante para medir cuántas filas se perdían. Esta comparación fue importante porque eliminar datos siempre tiene un costo: reduce el tamaño del dataset y puede modificar su composición.

También vimos una forma más controlada de eliminar filas usando `subset` :

```
df_ventas_completas = df.dropna(subset=[
 "Item",
 "Quantity",
 "Price Per Unit",
 "Total Spent"
)
```

En ese caso, no eliminamos cualquier fila con cualquier faltante, sino solamente aquellas que tenían datos incompletos en columnas consideradas centrales para cierto análisis.

Luego analizamos la posibilidad de eliminar columnas con muchos faltantes usando `dropna(axis=1, thresh=...)`. Vimos que esta decisión puede simplificar el dataset, pero también puede hacernos perder variables importantes.

Más adelante trabajamos con imputación. Usamos `fillna()` para completar valores faltantes con un valor fijo:

```
df_imputado["Payment Method"] = (
 df_imputado["Payment Method"].fillna("Desconocido")
)
```

También vimos un ejemplo de imputación numérica usando la mediana:

```
mediana_precio = df_imputado["Price Per Unit"].median()

df_imputado["Price Per Unit"] = (
 df_imputado["Price Per Unit"].fillna(mediana_precio)
)
```

En ambos casos aclaramos que imputar no significa recuperar el dato real. Significa completar un faltante con un criterio elegido. Por eso, la imputación debe interpretarse con cuidado.

Después trabajamos con una estrategia más sólida cuando el dataset lo permite: reconstruir valores usando otras columnas. En este caso usamos la relación:

```
Total Spent = Quantity × Price Per Unit
```

A partir de esa relación, reconstruimos valores faltantes de `Total Spent` cuando `Quantity` y `Price Per Unit` estaban disponibles.

También verificamos la reconstrucción comparando el valor reconstruido con el cálculo esperado. Esa verificación fue clave para confirmar que la transformación se aplicó correctamente.

Finalmente, comparamos las estrategias y reforzamos la importancia de trabajar con copias durante la limpieza. Usar `df.copy()` nos permite probar transformaciones, comparar resultados y conservar el dataset original como punto de referencia.

La idea principal de este capítulo fue:

```
Tratar valores faltantes implica decidir entre eliminar, imputar o reconstruir, según el
contexto y el impacto de cada decisión.
```

No existe una única estrategia correcta para todos los casos. Una limpieza responsable debe diagnosticar, decidir, transformar, verificar y documentar.

## Próximo paso

---

El siguiente paso será trabajar con duplicados.

En los datasets reales puede ocurrir que una misma observación aparezca más de una vez. A veces se trata de un error de carga o una repetición accidental. Otras veces, en cambio, dos filas parecidas pueden representar eventos reales distintos.

Vamos a aprender a detectar duplicados, contar cuántos hay, observarlos, decidir si corresponde eliminarlos y verificar el resultado después de hacerlo.

## Capítulo 16 — Duplicados en un DataFrame

---

### Breve repaso

---

En el trabajo anterior sobre valores faltantes aprendimos que detectar un problema no alcanza para decidir automáticamente cómo resolverlo.

Vimos que los valores faltantes pueden tratarse de distintas maneras: eliminar filas o columnas, imputar valores con algún criterio o reconstruir información a partir de otras columnas. También vimos que cada estrategia tiene riesgos y que toda transformación debe verificarse después de aplicarse.

En este capítulo vamos a trabajar con otro problema frecuente en la limpieza de datos: los duplicados.

Un duplicado aparece cuando una observación parece estar repetida dentro del dataset. A veces se trata de una fila exactamente igual a otra. Otras veces, el duplicado no es completo, pero se repite una columna importante, como un identificador de transacción.

Para estudiar este tema vamos a usar un dataset sintético pequeño. Esto nos permitirá observar con claridad distintos tipos de duplicados y discutir qué decisiones podrían tomarse en cada caso.

El objetivo no será eliminar filas automáticamente, sino aprender a detectar duplicados, interpretarlos y verificar qué ocurre después de tratarlos.

Al finalizar este capítulo deberías poder:

- Comprender qué significa que una fila esté duplicada.
- Diferenciar duplicados completos de duplicados según columnas específicas.
- Usar `duplicated()` para detectar duplicados.
- Contar duplicados dentro de un `DataFrame`.
- Observar las filas duplicadas antes de eliminarlas.
- Usar `drop_duplicates()` para eliminar duplicados.
- Usar `subset` para revisar duplicados según columnas específicas.
- Decidir cuándo conviene eliminar duplicados y cuándo conviene revisarlos con más cuidado.
- Verificar el resultado después de eliminar duplicados.



## Dataset de trabajo

---

Para estudiar duplicados vamos a usar un dataset pequeño de ventas.

Cada fila representa una transacción. El dataset incluye un identificador de transacción, el producto vendido, la cantidad, el precio unitario, la sucursal y la fecha.

A diferencia de los datasets anteriores, este fue construido especialmente para mostrar distintos casos de duplicados. Algunos registros están repetidos exactamente, otros comparten el mismo identificador de transacción y otros son parecidos pero podrían representar ventas reales distintas.

```
import pandas as pd

datos = {
 "id_transaccion": [
 "T001", "T002", "T003", "T004", "T004",
 "T005", "T006", "T007", "T008", "T009",
 "T010", "T010", "T011", "T012", "T013"
],
 "producto": [
 "Cafe", "Medialuna", "Cafe", "Te", "Te",
 "Tostado", "Cafe", "Jugo", "Cafe", "Medialuna",
 "Tostado", "Tostado", "Cafe", "Cafe", "Te"
],
 "cantidad": [
 2, 3, 1, 2, 2,
 1, 2, 1, 2, 3,
 1, 1, 2, 2, 2
],
 "precio_unitario": [
 1200, 800, 1200, 900, 900,
 2500, 1200, 1500, 1200, 800,
 2500, 2500, 1200, 1200, 900
],
 "sucursal": [
 "Centro", "Centro", "Norte", "Sur", "Sur",
 "Centro", "Centro", "Norte", "Centro", "Centro",
 "Sur", "Sur", "Centro", "Centro", "Sur"
],
 "fecha": [
 "2024-04-01", "2024-04-01", "2024-04-01", "2024-04-02", "2024-04-02",
 "2024-04-02", "2024-04-03", "2024-04-03", "2024-04-03", "2024-04-04",
 "2024-04-04", "2024-04-04", "2024-04-05", "2024-04-05", "2024-04-05"
]
}

df = pd.DataFrame(datos)

df
```

índice	id_transaccion	producto	cantidad	precio_unitario	sucursal	fecha
0	T001	Cafe	2	1200	Centro	2024-04-01
1	T002	Medialuna	3	800	Centro	2024-04-01
2	T003	Cafe	1	1200	Norte	2024-04-01
3	T004	Te	2	900	Sur	2024-04-02
4	T004	Te	2	900	Sur	2024-04-02
5	T005	Tostado	1	2500	Centro	2024-04-02
6	T006	Cafe	2	1200	Centro	2024-04-03
7	T007	Jugo	1	1500	Norte	2024-04-03
8	T008	Cafe	2	1200	Centro	2024-04-03
9	T009	Medialuna	3	800	Centro	2024-04-04
10	T010	Tostado	1	2500	Sur	2024-04-04
11	T010	Tostado	1	2500	Sur	2024-04-04
12	T011	Cafe	2	1200	Centro	2024-04-05
13	T012	Cafe	2	1200	Centro	2024-04-05
14	T013	Te	2	900	Sur	2024-04-05

## Qué significa que una fila esté duplicada

Una fila duplicada es una fila que aparece repetida dentro de un `DataFrame`.

En el caso más simple, dos filas son duplicadas cuando todos sus valores son exactamente iguales. Es decir, tienen el mismo identificador, el mismo producto, la misma cantidad, el mismo precio unitario, la misma sucursal y la misma fecha.

Pero en datasets reales la situación puede ser más compleja. A veces dos filas no son idénticas en todas sus columnas, pero coinciden en una columna clave, como un identificador de transacción. En otros casos, dos filas pueden parecer muy similares, aunque representen operaciones reales distintas.

Por ejemplo, dos clientes podrían comprar el mismo producto, en la misma sucursal, el mismo día y al mismo precio. Si esas ventas tienen identificadores diferentes, no deberíamos eliminarlas automáticamente solo porque se parecen.

Por eso, antes de eliminar duplicados necesitamos distinguir entre:

```
filas exactamente repetidas
filas con identificadores repetidos
filas parecidas, pero posiblemente válidas
```

Esta diferencia es importante. Detectar duplicados es una tarea técnica, pero decidir qué hacer con ellos requiere interpretar qué representa cada fila dentro del dataset.

## Detectar duplicados completos

Para detectar filas duplicadas podemos usar el método `uplicated()`.

Cuando aplicamos `uplicated()` sobre un `DataFrame`, Pandas compara las filas completas. Si una fila ya apareció antes con los mismos valores en todas las columnas, la marca como duplicada.

Veamos qué ocurre con nuestro dataset.

```
df.duplicated()
```

índice	0
0	False
1	False
2	False
3	False
4	True
5	False
6	False
7	False
8	False
9	False
10	False
11	True
12	False
13	False
14	False

```
dtype: bool
```

La salida es una serie booleana.

Cada valor indica si la fila correspondiente se considera duplicada o no. `False` significa que esa fila no fue detectada como duplicada. `True` significa que esa fila repite una combinación de valores que ya había aparecido antes.

Como una serie booleana completa puede ser difícil de leer, normalmente contamos cuántos duplicados fueron detectados.

```
df.duplicated().sum()
```

```
np.int64(2)
```

El resultado indica cuántas filas están duplicadas de manera completa.

Recordemos que, por defecto, Pandas conserva la primera aparición de una fila y marca como duplicadas las repeticiones posteriores. Por eso, si una fila aparece dos veces, solo la segunda aparición se marca como duplicada.

## Ver las filas duplicadas

Contar duplicados nos da una primera medida del problema, pero no alcanza para decidir qué hacer.

Antes de eliminar filas, conviene observar cuáles son los registros involucrados. Para eso podemos usar la máscara booleana que produce `duplicated()`.

Si escribimos:

```
df[df.duplicated()]
```

Pandas muestra las filas que fueron marcadas como duplicadas.

Recordemos que, por defecto, `duplicated()` marca como duplicadas las repeticiones posteriores, pero no la primera aparición. Por eso, en esta vista veremos solamente la segunda aparición de cada fila repetida.

```
df[df.duplicated()]
```

índice	id_transaccion	producto	cantidad	precio_unitario	sucursal	fecha
4	T004	Te	2	900	Sur	2024-04-02
11	T010	Tostado	1	2500	Sur	2024-04-04

Esta salida muestra las filas que Pandas considera duplicadas completas.

Sin embargo, a veces queremos ver tanto la fila original como su repetición. Eso facilita la comparación y permite revisar el problema con más claridad.

Para eso podemos usar el parámetro `keep=False`.

```
df[df.duplicated(keep=False)]
```

índice	id_transaccion	producto	cantidad	precio_unitario	sucursal	fecha
3	T004	Te	2	900	Sur	2024-04-02
4	T004	Te	2	900	Sur	2024-04-02
10	T010	Tostado	1	2500	Sur	2024-04-04
11	T010	Tostado	1	2500	Sur	2024-04-04

Con `keep=False`, Pandas marca como duplicadas todas las apariciones de las filas repetidas, incluyendo la primera.

Esta vista es más útil para revisar duplicados, porque nos permite ver los grupos completos de filas repetidas.

En nuestro dataset aparecen dos grupos de duplicados completos. Cada grupo contiene una fila original y una repetición exacta.

Antes de eliminar duplicados, conviene observar estos casos y confirmar que realmente se trata de repeticiones accidentales.

## Eliminar duplicados completos

Después de detectar y revisar las filas duplicadas, podemos decidir eliminarlas.

Para eliminar duplicados completos en Pandas usamos `drop_duplicates()`.

Por defecto, este método conserva la primera aparición de cada fila repetida y elimina las repeticiones posteriores.

```
df_sin_duplicados = df.drop_duplicates()

df_sin_duplicados
```

índice	id_transaccion	producto	cantidad	precio_unitario	sucursal	fecha
0	T001	Cafe	2	1200	Centro	2024-04-01

índice	id_transaccion	producto	cantidad	precio_unitario	sucursal	fecha
1	T002	Medialuna	3	800	Centro	2024-04-01
2	T003	Cafe	1	1200	Norte	2024-04-01
3	T004	Te	2	900	Sur	2024-04-02
5	T005	Tostado	1	2500	Centro	2024-04-02
6	T006	Cafe	2	1200	Centro	2024-04-03
7	T007	Jugo	1	1500	Norte	2024-04-03
8	T008	Cafe	2	1200	Centro	2024-04-03
9	T009	Medialuna	3	800	Centro	2024-04-04
10	T010	Tostado	1	2500	Sur	2024-04-04
12	T011	Cafe	2	1200	Centro	2024-04-05
13	T012	Cafe	2	1200	Centro	2024-04-05
14	T013	Te	2	900	Sur	2024-04-05

Ahora el nuevo `DataFrame` conserva una sola aparición de cada fila duplicada.

Es importante notar que guardamos el resultado en una nueva variable llamada `df_sin_duplicados`. De esta manera conservamos `df` como dataset original y podemos comparar antes y después.

Podemos revisar las dimensiones de ambos `DataFrames`.

```
print("Tamaño original:")
print(df.shape)

print()

print("Tamaño sin duplicados completos:")
print(df_sin_duplicados.shape)
```

```
Tamaño original:
(15, 6)

Tamaño sin duplicados completos:
(13, 6)
```

También podemos contar nuevamente los duplicados completos en la versión limpia.

```
df_sin_duplicados.duplicated().sum()
```

```
np.int64(0)
```

Si el resultado es `0`, significa que ya no quedan filas duplicadas completas.

Esta verificación es fundamental. Eliminar duplicados no debería terminar en la aplicación del método. Después de transformar el dataset, necesitamos comprobar que el problema que queríamos resolver efectivamente fue tratado.

En este caso, eliminamos duplicados completos: filas que eran iguales en todas sus columnas. Más adelante veremos que también podemos analizar duplicados considerando solo algunas columnas.

## Duplicados según una columna específica

Hasta ahora buscamos duplicados completos, es decir, filas idénticas en todas sus columnas.

Pero en muchos datasets hay columnas que deberían identificar de manera única cada registro. En nuestro ejemplo, esa columna es `id_transaccion`.

Si cada fila representa una transacción, entonces cada `id_transaccion` debería aparecer una sola vez. Si un mismo identificador aparece más de una vez, puede ser una señal de problema, aunque las filas no sean idénticas en todas sus columnas.

Podemos revisar duplicados considerando solamente la columna `id_transaccion`.

```
df.duplicated(subset=["id_transaccion"]).sum()
```

```
np.int64(2)
```

El parámetro `subset` permite indicar qué columnas queremos usar para detectar duplicados.

En este caso, Pandas no compara todas las columnas del `DataFrame`. Solo revisa si el valor de `id_transaccion` ya apareció antes.

Podemos ver las filas involucradas usando nuevamente `keep=False`.

```
df[df.duplicated(subset=["id_transaccion"], keep=False)]
```

índice	id_transaccion	producto	cantidad	precio_unitario	sucursal	fecha
3	T004	Te	2	900	Sur	2024-04-02

índice	id_transaccion	producto	cantidad	precio_unitario	sucursal	fecha
4	T004	Te	2	900	Sur	2024-04-02
10	T010	Tostado	1	2500	Sur	2024-04-04
11	T010	Tostado	1	2500	Sur	2024-04-04

Esta salida muestra todas las filas cuyos identificadores de transacción aparecen más de una vez.

En nuestro dataset, estos casos coinciden con duplicados completos. Pero en datasets reales podría ocurrir que dos filas compartan el mismo identificador y tengan diferencias en alguna otra columna. Eso requeriría una revisión más cuidadosa, porque podría tratarse de una carga duplicada con algún campo modificado o de un error en el identificador.

Por eso, revisar duplicados por columnas específicas es una parte importante del diagnóstico.

## Filas parecidas no siempre son duplicadas

Una parte importante del análisis de duplicados consiste en no eliminar registros solo porque se parecen.

En nuestro dataset hay algunas filas que comparten varios valores: mismo producto, misma cantidad, mismo precio unitario, misma sucursal y misma fecha. Sin embargo, si tienen distinto `id_transaccion`, podrían representar ventas reales distintas.

Veamos un ejemplo.

```
df[
 (df["producto"] == "Cafe")
 & (df["cantidad"] == 2)
 & (df["precio_unitario"] == 1200)
 & (df["sucursal"] == "Centro")
]
```

índice	id_transaccion	producto	cantidad	precio_unitario	sucursal	fecha
0	T001	Cafe	2	1200	Centro	2024-04-01
6	T006	Cafe	2	1200	Centro	2024-04-03
8	T008	Cafe	2	1200	Centro	2024-04-03
12	T011	Cafe	2	1200	Centro	2024-04-05
13	T012	Cafe	2	1200	Centro	2024-04-05



En esta salida pueden aparecer filas muy parecidas.

La pregunta importante es si esas filas representan una repetición accidental o ventas reales distintas.

Si tienen identificadores de transacción diferentes, no deberíamos eliminarlas automáticamente. Podría tratarse de dos clientes distintos que compraron el mismo producto, en la misma sucursal, con la misma cantidad y al mismo precio.

Este ejemplo muestra por qué la detección de duplicados no debe basarse solamente en una impresión visual.

En datasets de transacciones, el identificador suele ser clave. Si dos filas comparten todos los datos pero tienen distinto identificador, conviene revisar el contexto antes de decidir. Si dos filas comparten el mismo identificador, la sospecha de duplicado es más fuerte.

## Duplicados según varias columnas

---

Además de buscar duplicados completos o duplicados por identificador, también podemos revisar duplicados usando un conjunto de columnas.

Esto puede ser útil cuando no tenemos una columna identificadora confiable o cuando queremos investigar registros que se parecen demasiado.

Por ejemplo, podríamos revisar si hay filas repetidas considerando estas columnas:

```
producto
cantidad
precio_unitario
sucursal
fecha
```

Esta revisión no usa `id_transaccion`. Por eso, no nos dice automáticamente qué filas debemos eliminar. Solo nos ayuda a encontrar registros que comparten las mismas características principales de venta.

```
columnas_para_revisar = [
 "producto",
 "cantidad",
 "precio_unitario",
 "sucursal",
 "fecha"
]

df[df.duplicated(subset=columnas_para_revisar, keep=False)]
```

índice	id_transaccion	producto	cantidad	precio_unitario	sucursal	fecha
3	T004	Te	2	900	Sur	2024-04-02
4	T004	Te	2	900	Sur	2024-04-02
6	T006	Cafe	2	1200	Centro	2024-04-03
8	T008	Cafe	2	1200	Centro	2024-04-03
10	T010	Tostado	1	2500	Sur	2024-04-04
11	T010	Tostado	1	2500	Sur	2024-04-04
12	T011	Cafe	2	1200	Centro	2024-04-05
13	T012	Cafe	2	1200	Centro	2024-04-05

La salida muestra filas que coinciden en las columnas seleccionadas.

Algunas de esas filas pueden ser duplicados reales. Otras pueden ser ventas diferentes que casualmente tienen los mismos valores en esas columnas.

Por eso, esta revisión debe interpretarse con cuidado. Cuando usamos `subset`, estamos cambiando el criterio de duplicación. Ya no estamos preguntando si toda la fila está repetida, sino si se repite una combinación específica de columnas.

Este tipo de análisis puede servir como señal de alerta, pero no reemplaza la interpretación del contexto.

Si el dataset tiene un identificador confiable, como `id_transaccion`, ese identificador suele ser una referencia muy importante. Si no lo tiene, debemos apoyarnos en varias columnas y revisar los casos con más atención.

## Qué significa el parámetro `keep`

Cuando usamos `duplicated()` o `drop_duplicates()`, Pandas necesita decidir qué aparición de una fila repetida debe conservarse y cuál debe marcarse como duplicada.

Para eso existe el parámetro `keep`.

Por defecto, `keep="first"`. Eso significa que Pandas conserva la primera aparición y considera duplicadas las apariciones posteriores.

Podemos verlo con `duplicated()`:

```
df.duplicated(keep="first")
```

índice	0
0	False
1	False
2	False
3	False
4	True
5	False
6	False
7	False
8	False
9	False
10	False
11	True
12	False
13	False
14	False

`dtype: bool`

Si queremos marcar como duplicada la primera aparición y conservar la última, podemos usar `keep="last"`.

```
df.duplicated(keep="last")
```

índice	0
0	False
1	False
2	False
3	True
4	False
5	False
6	False
7	False
8	False

índice	0
9	False
10	True
11	False
12	False
13	False
14	False

`dtype: bool`

Y si queremos marcar todas las apariciones de las filas repetidas, usamos `keep=False`.

```
df.duplicated(keep=False)
```

índice	0
0	False
1	False
2	False
3	True
4	True
5	False
6	False
7	False
8	False
9	False
10	True
11	True
12	False
13	False
14	False

`dtype: bool`

Esta última opción es muy útil para revisar duplicados, porque permite ver tanto la fila original como sus repeticiones.

También podemos usar `keep` con `drop_duplicates()`.

Por ejemplo, si eliminamos duplicados conservando la primera aparición:

```
df.drop_duplicates(keep="first")
```

índice	id_transaccion	producto	cantidad	precio_unitario	sucursal	fecha
0	T001	Cafe	2	1200	Centro	2024-04-01
1	T002	Medialuna	3	800	Centro	2024-04-01
2	T003	Cafe	1	1200	Norte	2024-04-01
3	T004	Te	2	900	Sur	2024-04-02
5	T005	Tostado	1	2500	Centro	2024-04-02
6	T006	Cafe	2	1200	Centro	2024-04-03
7	T007	Jugo	1	1500	Norte	2024-04-03
8	T008	Cafe	2	1200	Centro	2024-04-03
9	T009	Medialuna	3	800	Centro	2024-04-04
10	T010	Tostado	1	2500	Sur	2024-04-04
12	T011	Cafe	2	1200	Centro	2024-04-05
13	T012	Cafe	2	1200	Centro	2024-04-05
14	T013	Te	2	900	Sur	2024-04-05

Si quisiéramos conservar la última aparición:

```
df.drop_duplicates(keep="last")
```

índice	id_transaccion	producto	cantidad	precio_unitario	sucursal	fecha
0	T001	Cafe	2	1200	Centro	2024-04-01
1	T002	Medialuna	3	800	Centro	2024-04-01
2	T003	Cafe	1	1200	Norte	2024-04-01
4	T004	Te	2	900	Sur	2024-04-02
5	T005	Tostado	1	2500	Centro	2024-04-02
6	T006	Cafe	2	1200	Centro	2024-04-03
7	T007	Jugo	1	1500	Norte	2024-04-03
8	T008	Cafe	2	1200	Centro	2024-04-03
9	T009	Medialuna	3	800	Centro	2024-04-04

índice	id_transaccion	producto	cantidad	precio_unitario	sucursal	fecha
11	T010	Tostado	1	2500	Sur	2024-04-04
12	T011	Cafe	2	1200	Centro	2024-04-05
13	T012	Cafe	2	1200	Centro	2024-04-05
14	T013	Te	2	900	Sur	2024-04-05

En este dataset, como los duplicados completos son exactamente iguales, conservar la primera o la última aparición no cambia los valores finales. Pero en otros casos puede importar mucho, especialmente si las filas tienen el mismo identificador y alguna diferencia en otras columnas.

Por eso, antes de eliminar duplicados, conviene revisar qué criterio de conservación estamos usando.

## Verificar después de eliminar duplicados

Después de eliminar duplicados, siempre conviene verificar el resultado.

En limpieza de datos, no alcanza con aplicar una transformación. También necesitamos comprobar qué cambió y si el problema que queríamos resolver efectivamente quedó tratado.

Primero podemos crear una versión sin duplicados completos:

```
df_sin_duplicados = df.drop_duplicates()

df_sin_duplicados
```

índice	id_transaccion	producto	cantidad	precio_unitario	sucursal	fecha
0	T001	Cafe	2	1200	Centro	2024-04-01
1	T002	Medialuna	3	800	Centro	2024-04-01
2	T003	Cafe	1	1200	Norte	2024-04-01
3	T004	Te	2	900	Sur	2024-04-02
5	T005	Tostado	1	2500	Centro	2024-04-02
6	T006	Cafe	2	1200	Centro	2024-04-03
7	T007	Jugo	1	1500	Norte	2024-04-03
8	T008	Cafe	2	1200	Centro	2024-04-03
9	T009	Medialuna	3	800	Centro	2024-04-04

índice	id_transaccion	producto	cantidad	precio_unitario	sucursal	fecha
10	T010	Tostado	1	2500	Sur	2024-04-04
12	T011	Cafe	2	1200	Centro	2024-04-05
13	T012	Cafe	2	1200	Centro	2024-04-05
14	T013	Te	2	900	Sur	2024-04-05

Ahora podemos comparar el tamaño del dataset original con el tamaño del dataset sin duplicados:

```
print("Tamaño original:")
print(df.shape)

print()

print("Tamaño sin duplicados:")
print(df_sin_duplicados.shape)
```

```
Tamaño original:
(15, 6)

Tamaño sin duplicados:
(13, 6)
```

También podemos calcular cuántas filas se eliminaron:

```
filas_eliminadas = df.shape[0] - df_sin_duplicados.shape[0]

filas_eliminadas
```

```
2
```

Y podemos verificar que ya no queden duplicados completos:

```
df_sin_duplicados.duplicated().sum()
```

```
np.int64(0)
```

Si el resultado es `0`, significa que no quedan filas duplicadas completas.

También podemos revisar si siguen existiendo identificadores de transacción repetidos:

```
df_sin_duplicados.duplicated(subset=["id_transaccion"]).sum()
```

```
np.int64(0)
```

En este caso, si eliminamos duplicados completos y los identificadores repetidos correspondían a filas exactamente iguales, el conteo también debería quedar en `0`.

Estas verificaciones son importantes porque nos permiten confirmar el efecto de la limpieza. Además, nos ayudan a documentar el proceso:

```
cuántas filas había antes
cuántas filas quedaron después
cuántas filas fueron eliminadas
si todavía quedan duplicados según el criterio elegido
```

Eliminar duplicados no debe ser una acción invisible. Debemos poder explicar qué criterio usamos y qué impacto tuvo sobre el dataset.

## Errores frecuentes al trabajar con duplicados

---

Al trabajar con duplicados, uno de los errores más frecuentes es eliminar filas sin revisarlas antes.

Aunque `drop_duplicates()` es una herramienta muy útil, no deberíamos aplicarla automáticamente sin observar qué registros está eliminando. En algunos casos, las filas repetidas pueden ser errores claros. En otros, dos registros parecidos pueden representar eventos reales distintos.

Otro error frecuente es pensar que solo existen duplicados cuando toda la fila es exactamente igual. A veces el problema está en una columna específica, como un identificador de transacción repetido. Por eso, además de buscar duplicados completos, puede ser necesario revisar duplicados usando `subset`.

Por ejemplo:

```
df.duplicated(subset=["id_transaccion"]).sum()
```

```
np.int64(2)
```

Esta instrucción no compara toda la fila. Solo revisa si hay identificadores de transacción repetidos.



También debemos tener cuidado con el criterio usado para conservar registros. Por defecto, Pandas usa `keep="first"`, es decir, conserva la primera aparición y elimina o marca las siguientes. En algunos casos podría convenir conservar la última aparición, especialmente si el dataset fue actualizado y la fila más reciente contiene información corregida.

Otro error posible es confundir filas parecidas con duplicados. Dos ventas del mismo producto, con la misma cantidad, el mismo precio, la misma sucursal y la misma fecha pueden parecer repetidas, pero si tienen identificadores distintos podrían ser ventas reales diferentes.

Por eso, antes de eliminar duplicados conviene seguir esta rutina:

```
detectar duplicados
observar las filas involucradas
definir el criterio de duplicación
eliminar solo si corresponde
verificar el resultado
```

El objetivo no es borrar registros hasta que no quede ninguna repetición aparente. El objetivo es decidir, con criterio, qué registros representan duplicaciones reales y cuáles deben conservarse.

## Resumen del capítulo

---

En este capítulo trabajamos con duplicados dentro de un `DataFrame`.

Comenzamos usando un dataset sintético de ventas, construido especialmente para mostrar distintos casos posibles: filas exactamente repetidas, identificadores de transacción repetidos y filas parecidas que no necesariamente debían eliminarse.

La idea inicial fue que un duplicado puede entenderse de distintas maneras. A veces dos filas son duplicadas porque todos sus valores son exactamente iguales. Otras veces, el problema aparece porque se repite una columna importante, como `id_transaccion`. También vimos que dos filas pueden ser muy parecidas y, aun así, representar ventas reales distintas.

Para detectar duplicados completos usamos:

```
df.duplicated()
```

y para contarlos:

```
df.duplicated().sum()
```

Luego observamos las filas duplicadas. Primero vimos solamente las repeticiones posteriores:

```
df[df.duplicated()]
```

y después usamos `keep=False` para ver tanto la fila original como sus repeticiones:

```
df[df.duplicated(keep=False)]
```

Esta revisión fue importante porque antes de eliminar registros conviene observar qué filas están involucradas.

Después usamos `drop_duplicates()` para eliminar duplicados completos:

```
df_sin_duplicados = df.drop_duplicates()
```

También verificamos el resultado comparando el tamaño del dataset antes y después de la eliminación, contando cuántas filas se eliminaron y revisando si todavía quedaban duplicados.

Más adelante analizamos duplicados según una columna específica:

```
df.duplicated(subset=["id_transaccion"]).sum()
```

Esto nos permitió mostrar que, en muchos datasets, no alcanza con buscar filas completamente idénticas. Si una columna funciona como identificador único, su repetición puede ser una señal de problema aunque el resto de la fila no sea exactamente igual.

También revisamos duplicados según varias columnas usando `subset`. Esta técnica puede ser útil cuando no tenemos un identificador confiable o cuando queremos investigar registros demasiado parecidos. Sin embargo, vimos que esa revisión debe interpretarse con cuidado: filas parecidas no siempre son duplicadas reales.

Finalmente trabajamos con el parámetro `keep`, que permite controlar qué aparición se conserva o se marca como duplicada:

```
df.duplicated(keep="first")
df.duplicated(keep="last")
df.duplicated(keep=False)
```

La idea principal de este capítulo fue:

Detectar duplicados es una tarea técnica, pero decidir si eliminarlos requiere interpretar qué representa cada fila.

Eliminar duplicados puede mejorar la calidad de un dataset, pero solo si tenemos claro qué criterio estamos usando y qué registros estamos quitando.

## Próximo paso

---

Ya trabajamos con valores faltantes y duplicados, dos problemas muy frecuentes en la limpieza de datos.

El siguiente paso será volver sobre la limpieza de textos y categorías, pero ahora dentro de esta etapa de preparación de datos. Vamos a revisar espacios innecesarios, diferencias entre mayúsculas y minúsculas, valores escritos de forma inconsistente y reemplazos necesarios para unificar categorías.

Ese trabajo será importante porque muchas columnas categóricas pueden parecer correctas a simple vista, pero generar conteos fragmentados si los valores no están escritos de manera uniforme.

## Capítulo 17 — Limpieza de textos y categorías

---

### Breve repaso

---

En el trabajo anterior sobre duplicados vimos que una parte importante de la limpieza de datos consiste en interpretar los problemas antes de resolverlos. No alcanza con detectar filas repetidas: también necesitamos comprender qué representa cada fila, qué criterio usamos para definir un duplicado y qué información podríamos perder al eliminar registros.

En este capítulo vamos a trabajar con otro problema muy frecuente en la preparación de datos: la inconsistencia en columnas de texto y categorías.

Muchas columnas categóricas parecen simples al principio. Por ejemplo, una columna puede indicar el producto vendido, el método de pago o la ubicación de una venta. Sin embargo, cuando los valores fueron cargados manualmente, combinados desde distintas fuentes o exportados desde sistemas diferentes, pueden aparecer pequeñas diferencias de escritura que afectan el análisis.

Para una persona, valores como `"Coffee"`, `" coffee"` y `"COFFEE"` pueden representar el mismo producto. Para Pandas, en cambio, son valores distintos. Lo mismo puede ocurrir con `"Credit Card"`, `"credit card"`, `"Credito"` y `"Crédito"`, o con variantes como `"Takeaway"`, `"Take away"` y `"take-away"`.

Estas diferencias no son solo un problema estético. Si las categorías están escritas de manera inconsistente, los conteos se fragmentan, los filtros pueden dejar registros afuera, los agrupamientos pueden generar resultados incorrectos y las conclusiones pueden ser engañosas.

Por eso, limpiar textos y categorías es una parte importante de la calidad de los datos. En este capítulo vamos a revisar problemas como espacios innecesarios, diferencias entre mayúsculas y minúsculas, tildes, variantes de escritura, valores desconocidos y valores que indican error.

Al finalizar este capítulo deberías poder:

- Comprender por qué las inconsistencias de texto afectan el análisis.
- Revisar valores únicos y frecuencias en columnas categóricas.
- Detectar diferencias causadas por espacios, mayúsculas o minúsculas.
- Usar métodos de texto como `.str.strip()` y `.str.lower()`.
- Reemplazar valores inconsistentes usando `replace()`.

- Decidir cómo tratar valores como "UNKNOWN", "ERROR" o valores faltantes reales.
- Crear versiones limpias de columnas categóricas.
- Verificar que una limpieza de categorías haya funcionado.

## Dataset de trabajo

Para estudiar limpieza de textos y categorías vamos a usar un dataset sintético de ventas de cafetería.

A diferencia del dataset real de Kaggle, este dataset fue construido especialmente para mostrar problemas frecuentes en columnas categóricas. Incluye valores con espacios al comienzo o al final, diferencias entre mayúsculas y minúsculas, variantes de escritura, valores desconocidos, valores de error y datos faltantes reales.

Usar un dataset pequeño y controlado nos permite observar con claridad el problema. La idea no es simular todos los detalles de un archivo real, sino construir un ejemplo didáctico que nos permita entender por qué estas inconsistencias afectan el análisis.

```
import pandas as pd

datos = {
 "id_venta": [
 "V001", "V002", "V003", "V004", "V005",
 "V006", "V007", "V008", "V009", "V010",
 "V011", "V012", "V013", "V014", "V015"
],
 "producto": [
 "Coffee", " coffee", "COFFEE", "Tea", "tea ",
 "Sandwich", "sandwich", "Cake", "cake ", "Juice",
 "UNKNOWN", "ERROR", None, "Smoothie", "smoothie "
],
 "metodo_pago": [
 "Cash", "cash", " CASH ", "Credit Card", "credit card",
 "Credito", "Crédito", "Digital Wallet", "digital wallet", "wallet",
 "UNKNOWN", "ERROR", None, "Debit Card", "debit card "
],
 "ubicacion": [
 "In-store", "in-store", " In-store ", "Takeaway", "takeaway",
 "TAKEAWAY", "Delivery", "delivery ", "DELIVERY", "In store",
 "UNKNOWN", "ERROR", None, "Take away", "take-away"
],
 "cantidad": [
 2, 1, 3, 2, 1,
 1, 2, 1, 3, 2,
 1, 1, 2, 1, 2
],
 "precio_unitario": [
```

```

 1200, 1200, 1200, 900, 900,
 2500, 2500, 1800, 1800, 1500,
 0, 0, 1200, 2200, 2200
]
}

df = pd.DataFrame(datos)

df

```

índice	id_venta	producto	metodo_pago	ubicacion	cantidad	precio_unitario
0	V001	Coffee	Cash	In-store	2	1200
1	V002	coffee	cash	in-store	1	1200
2	V003	COFFEE	CASH	In-store	3	1200
3	V004	Tea	Credit Card	Takeaway	2	900
4	V005	tea	credit card	takeaway	1	900
5	V006	Sandwich	Credito	TAKEAWAY	1	2500
6	V007	sandwich	Crédito	Delivery	2	2500
7	V008	Cake	Digital Wallet	delivery	1	1800
8	V009	cake	digital wallet	DELIVERY	3	1800
9	V010	Juice	wallet	In store	2	1500
10	V011	UNKNOWN	UNKNOWN	UNKNOWN	1	0
11	V012	ERROR	ERROR	ERROR	1	0
12	V013	None	None	None	2	1200
13	V014	Smoothie	Debit Card	Take away	1	2200
14	V015	smoothie	debit card	take-away	2	2200

Al observar el `DataFrame`, algunos problemas se ven rápidamente. Por ejemplo, en la columna `producto` aparecen valores como `"Coffee"`, `" coffee"` y `"COFFEE"`. Aunque probablemente representan el mismo producto, Pandas los interpreta como valores diferentes.

También aparecen valores como `"UNKNOWN"`, `"ERROR"` y `None`. Estos casos requieren una decisión distinta. No son simplemente variantes de escritura de un producto, sino señales de información desconocida, errores de carga o datos faltantes.

En este capítulo vamos a trabajar paso a paso para detectar, limpiar y verificar este tipo de problemas.

## Revisar valores únicos y frecuencias

Antes de limpiar una columna de texto, necesitamos revisar qué valores contiene.

Una forma directa de hacerlo es usar `value_counts()`. Esta herramienta permite contar cuántas veces aparece cada valor dentro de una columna.

Vamos a empezar con la columna `producto`.

```
df["producto"].value_counts(dropna=False)
```

producto	count
Coffee	1
coffee	1
COFFEE	1
Tea	1
tea	1
Sandwich	1
sandwich	1
Cake	1
cake	1
Juice	1
UNKNOWN	1
ERROR	1
None	1
Smoothie	1
smoothie	1

```
dtype: int64
```

La salida muestra un problema importante: Pandas cuenta como productos diferentes valores que, para una persona, probablemente representan lo mismo.

Por ejemplo:

```
Coffee
coffee
COFFEE
```

aparecen como valores separados. Lo mismo ocurre con:

```
Tea
tea
```

o con:

```
Cake
cake
```

Esto ocurre porque Pandas compara los textos de manera exacta. Si un valor tiene un espacio al comienzo, un espacio al final o una diferencia de mayúsculas, Pandas lo considera distinto.

Este problema afecta directamente el análisis. Si queremos saber cuántas ventas hubo de café, no obtendremos un único conteo para "Coffee", sino varios conteos separados para cada variante de escritura.

También podemos revisar otras columnas categóricas.

```
df["metodo_pago"].value_counts(dropna=False)
```

metodo_pago	count
Cash	1
cash	1
CASH	1
Credit Card	1
credit card	1
Credito	1
Crédito	1
Digital Wallet	1
digital wallet	1
wallet	1
UNKNOWN	1
ERROR	1
None	1
Debit Card	1
debit card	1



dtype: int64

```
df["ubicacion"].value_counts(dropna=False)
```

ubicacion	count
In-store	1
in-store	1
In-store	1
Takeaway	1
takeaway	1
TAKEAWAY	1
Delivery	1
delivery	1
DELIVERY	1
In store	1
UNKNOWN	1
ERROR	1
None	1
Take away	1
take-away	1

dtype: int64

Estas salidas muestran que el problema no aparece en una sola columna. Hay inconsistencias en productos, métodos de pago y ubicaciones.

Por eso, antes de analizar datos categóricos, conviene revisar sus valores únicos y frecuencias. Si no hacemos esta revisión, podríamos construir filtros, conteos o agrupamientos sobre categorías fragmentadas.

## Espacios innecesarios al comienzo o al final

Uno de los problemas más comunes en columnas de texto es la presencia de espacios innecesarios.

Un valor como "Coffee" no es igual a " coffee" para Pandas, porque el segundo tiene un espacio al comienzo. Lo mismo ocurre con "tea " o "cake ", que tienen un espacio al final.

Estos espacios pueden ser difíciles de ver a simple vista, pero afectan conteos, filtros y agrupamientos. Por ejemplo, si filtramos las filas donde el producto es exactamente "Coffee", no vamos a incluir automáticamente las filas donde el producto aparece como " coffee" o "COFFEE".

Podemos observar algunos valores de la columna `producto` para notar estas diferencias.

```
df["producto"].unique()
```

```
array(['Coffee', ' coffee', 'COFFEE', 'Tea', 'tea ', 'Sandwich',
 'sandwich', 'Cake', 'cake ', 'Juice', 'UNKNOWN', 'ERROR', None,
 'Smoothie', 'smoothie '], dtype=object)
```

Para quitar espacios al comienzo y al final de cada texto, usamos `.str.strip()`.

En lugar de modificar directamente la columna original, vamos a crear una nueva columna llamada `producto_limpio`. Esto nos permitirá comparar los valores originales con los valores transformados.

```
df["producto_limpio"] = df["producto"].str.strip()

df[["producto", "producto_limpio"]]
```

índice	producto	producto_limpio
0	Coffee	Coffee
1	coffee	coffee
2	COFFEE	COFFEE
3	Tea	Tea
4	tea	tea
5	Sandwich	Sandwich
6	sandwich	sandwich
7	Cake	Cake
8	cake	cake
9	Juice	Juice
10	UNKNOWN	UNKNOWN
11	ERROR	ERROR
12	None	None

índice	producto	producto_limpio
13	Smoothie	Smoothie
14	smoothie	smoothie

Ahora los valores que tenían espacios al comienzo o al final quedaron corregidos en `producto_limpio`.

Observemos también que los valores faltantes reales no se transformaron en texto. Pandas conserva esos casos como valores ausentes, de modo que la limpieza de texto no reemplaza automáticamente los datos faltantes.

Esto es importante porque limpiar textos y tratar valores faltantes son tareas relacionadas, pero no idénticas. Quitar espacios o cambiar mayúsculas puede corregir problemas de escritura, pero no completa información que realmente falta.

Sin embargo, todavía no resolvimos todas las inconsistencias. Por ejemplo, "Coffee" y "COFFEE" siguen siendo valores diferentes, porque tienen distintas combinaciones de mayúsculas y minúsculas.

Esto muestra una idea importante: la limpieza de textos suele requerir varios pasos. Quitar espacios puede resolver una parte del problema, pero no necesariamente toda la inconsistencia.

## Unificar mayúsculas y minúsculas

Después de quitar espacios innecesarios, todavía puede quedar otro problema: la misma categoría escrita con distintas combinaciones de mayúsculas y minúsculas.

Por ejemplo, estos valores no son iguales para Pandas:

```
Coffee
COFFEE
coffee
```

Aunque para nosotros probablemente representan el mismo producto, Pandas los interpreta como textos distintos.

Para unificar esta diferencia podemos convertir todos los textos a minúsculas usando `.str.lower()`.

```
df["producto_limpio"] = df["producto_limpio"].str.lower()

df[["producto", "producto_limpio"]]
```

índice	producto	producto_limpio
0	Coffee	coffee
1	coffee	coffee
2	COFFEE	coffee
3	Tea	tea
4	tea	tea
5	Sandwich	sandwich
6	sandwich	sandwich
7	Cake	cake
8	cake	cake
9	Juice	juice
10	UNKNOWN	unknown
11	ERROR	error
12	None	None
13	Smoothie	smoothie
14	smoothie	smoothie

Ahora los valores de `producto_limpio` quedaron en minúsculas.

Esto permite que variantes como "Coffee", " coffee" y "COFFEE" se unifiquen como "coffee".

Podemos revisar nuevamente las frecuencias de la columna limpia.

```
df["producto_limpio"].value_counts(dropna=False)
```

producto_limpio	count
coffee	3
tea	2
sandwich	2
cake	2
smoothie	2
juice	1
unknown	1
error	1

producto_limpio	count
None	1

`dtype: int64`

El conteo ahora es más coherente que el original. Las variantes causadas por espacios y mayúsculas quedaron agrupadas.

Sin embargo, todavía aparecen valores que no representan productos reales, como "unknown" y "error", además de valores faltantes. Esos casos no se resuelven simplemente con `strip()` o `lower()`, porque no son variantes de escritura de un producto, sino valores problemáticos que requieren una decisión específica.

## Aplicar la misma limpieza a otras columnas

La combinación de `.str.strip()` y `.str.lower()` también puede aplicarse a otras columnas de texto.

En nuestro dataset, `metodo_pago` y `ubicacion` también tienen inconsistencias de escritura. Antes de limpiarlas, revisemos sus valores.

```
df["metodo_pago"].value_counts(dropna=False)
```

metodo_pago	count
Cash	1
cash	1
CASH	1
Credit Card	1
credit card	1
Credito	1
Crédito	1
Digital Wallet	1
digital wallet	1
wallet	1
UNKNOWN	1
ERROR	1
None	1
Debit Card	1

metodo_pago	count
debit card	1

dtype: int64

```
df["ubicacion"].value_counts(dropna=False)
```

ubicacion	count
In-store	1
in-store	1
In-store	1
Takeaway	1
takeaway	1
TAKEAWAY	1
Delivery	1
delivery	1
DELIVERY	1
In store	1
UNKNOWN	1
ERROR	1
None	1
Take away	1
take-away	1

dtype: int64

En ambas columnas aparecen valores que deberían unificarse.

Por ejemplo, en `metodo_pago`, valores como "Cash", "cash" y " CASH " representan la misma forma de pago, pero Pandas los cuenta por separado. En `ubicacion`, ocurre algo similar con "In-store", "in-store" y " In-store ".

Vamos a crear columnas limpias para conservar los valores originales y poder comparar antes y después.

```
df["metodo_pago_limpio"] = df["metodo_pago"].str.strip().str.lower()
df["ubicacion_limpia"] = df["ubicacion"].str.strip().str.lower()

df[
```

```
[
 "metodo_pago",
 "metodo_pago_limpio",
 "ubicacion",
 "ubicacion_limpia"
]
```

índice	metodo_pago	metodo_pago_limpio	ubicacion	ubicacion_limpia
0	Cash	cash	In-store	in-store
1	cash	cash	in-store	in-store
2	CASH	cash	In-store	in-store
3	Credit Card	credit card	Takeaway	takeaway
4	credit card	credit card	takeaway	takeaway
5	Credito	credito	TAKEAWAY	takeaway
6	Crédito	crédito	Delivery	delivery
7	Digital Wallet	digital wallet	delivery	delivery
8	digital wallet	digital wallet	DELIVERY	delivery
9	wallet	wallet	In store	in store
10	UNKNOWN	unknown	UNKNOWN	unknown
11	ERROR	error	ERROR	error
12	None	None	None	None
13	Debit Card	debit card	Take away	take away
14	debit card	debit card	take-away	take-away

Ahora tenemos una versión normalizada de cada columna.

Esta transformación corrige diferencias producidas por espacios al comienzo o al final y por mayúsculas o minúsculas. Sin embargo, todavía pueden quedar variantes más profundas.

Por ejemplo, "credit card" y "credito" no se unifican con `strip()` y `lower()`, porque son textos distintos. Lo mismo ocurre con "takeaway", "take away" y "take-away".

Para resolver esos casos necesitamos una estrategia adicional: reemplazar valores inconsistentes.

## Reemplazar valores inconsistentes

Después de quitar espacios y unificar mayúsculas/minúsculas, algunas categorías todavía pueden quedar separadas.

Esto ocurre cuando los valores no solo difieren en formato, sino también en la forma de escribir la categoría.

Por ejemplo, en la columna `metodo_pago_limpio` podemos encontrar valores como:

```
credit card
credito
crédito
```

Según el contexto, podríamos decidir que todos esos valores representan pagos con tarjeta de crédito. Del mismo modo, en la columna `ubicacion_limpia`, valores como:

```
takeaway
take away
take-away
```

podrían representar la misma modalidad de retiro.

Para unificar estos casos podemos usar `replace()`.

```
df["metodo_pago_limpio"].value_counts(dropna=False)
```

metodo_pago_limpio	count
cash	3
credit card	2
digital wallet	2
debit card	2
crédito	1
credito	1
wallet	1
unknown	1
error	1
None	1

```
dtype: int64
```



```
df["ubicacion_limpia"].value_counts(dropna=False)
```

ubicacion_limpia	count
in-store	3
takeaway	3
delivery	3
in store	1
unknown	1
error	1
None	1
take away	1
take-away	1

```
dtype: int64
```

Primero revisamos los valores actuales. Esto es importante porque no deberíamos reemplazar valores sin saber exactamente qué variantes existen.

En este ejemplo vamos a resolver la variante "credito" / "crédito" mediante reemplazos explícitos. Existen técnicas más generales para normalizar tildes y acentos, pero por ahora nos alcanza con una decisión puntual y visible, porque queremos que cada reemplazo quede claro.

Ahora podemos definir diccionarios de reemplazo. En cada diccionario, las claves son los valores actuales y los valores son las categorías finales que queremos conservar.

```
reemplazos_pago = {
 "cash": "cash",
 "credit card": "credit_card",
 "credito": "credit_card",
 "crédito": "credit_card",
 "digital wallet": "digital_wallet",
 "wallet": "digital_wallet",
 "debit card": "debit_card",
 "unknown": "unknown",
 "error": "error"
}

reemplazos_ubicacion = {
 "in-store": "in_store",
 "in store": "in_store",
 "takeaway": "takeaway",
 "take away": "takeaway",
 "take-away": "takeaway",
}
```

```

 "delivery": "delivery",
 "unknown": "unknown",
 "error": "error"
}

df["metodo_pago_limpio"] = df["metodo_pago_limpio"].replace(reemplazos_pago)
df["ubicacion_limpia"] = df["ubicacion_limpia"].replace(reemplazos_ubicacion)

```

Después de aplicar los reemplazos, podemos revisar nuevamente las frecuencias.

```
df["metodo_pago_limpio"].value_counts(dropna=False)
```

metodo_pago_limpio	count
credit_card	4
cash	3
digital_wallet	3
debit_card	2
unknown	1
error	1
None	1

dtype: int64

```
df["ubicacion_limpia"].value_counts(dropna=False)
```

ubicacion_limpia	count
takeaway	5
in_store	4
delivery	3
unknown	1
error	1
None	1

dtype: int64

Ahora las categorías quedaron más unificadas.

El método `replace()` es útil cuando conocemos las variantes que queremos corregir. A diferencia de `strip()` o `lower()`, que aplican una transformación general, `replace()` permite tomar decisiones específicas para cada valor.

Esto también implica una responsabilidad: cada reemplazo debe tener sentido dentro del contexto. No deberíamos unificar categorías solo porque se parecen. Necesitamos saber o decidir razonablemente que representan lo mismo.

Por ejemplo, unificar `"wallet"` con `"digital_wallet"` puede tener sentido si en nuestro criterio ambas expresiones representan la misma forma de pago. Pero esa decisión debería quedar documentada, porque afecta los resultados posteriores.

## Tratar valores desconocidos, errores y faltantes

Después de normalizar espacios, mayúsculas, minúsculas y algunas variantes de escritura, todavía quedan valores que requieren una decisión especial.

En nuestras columnas limpias aparecen valores como:

```
unknown
error
```

También pueden aparecer valores faltantes reales, representados como `NaN`.

Estos casos no son equivalentes a una categoría común. Por ejemplo, `cash`, `credit_card` o `digital_wallet` representan métodos de pago reales. En cambio, `unknown` indica que el valor no se conoce, y `error` sugiere que hubo algún problema de carga o registro.

Antes de decidir qué hacer, conviene revisar cuántos casos hay de cada tipo.

```
df["producto_limpio"].value_counts(dropna=False)
```

producto_limpio	count
coffee	3
tea	2
sandwich	2
cake	2
smoothie	2
juice	1
unknown	1
error	1
None	1

dtype: int64

```
df["metodo_pago_limpio"].value_counts(dropna=False)
```

metodo_pago_limpio	count
credit_card	4
cash	3
digital_wallet	3
debit_card	2
unknown	1
error	1
None	1

dtype: int64

```
df["ubicacion_limpia"].value_counts(dropna=False)
```

ubicacion_limpia	count
takeaway	5
in_store	4
delivery	3
unknown	1
error	1
None	1

dtype: int64

Una opción posible es conservar `unknown` como una categoría especial. Esto puede ser razonable cuando queremos indicar que el dato no está disponible, pero sin eliminar la fila.

El valor `error`, en cambio, puede requerir otro tratamiento. Podríamos decidir conservarlo como señal de problema, reemplazarlo por `unknown`, convertirlo en faltante o revisar la fila completa antes de tomar una decisión.

En este ejemplo vamos a tomar una decisión simple: vamos a unificar `error` dentro de `unknown`. La idea será tratar ambos casos como valores no confiables o no disponibles para el análisis categórico.

Esta decisión no es universal. En otro contexto, `error` podría analizarse por separado para investigar fallas del sistema de carga. Lo importante es que la decisión sea explícita.

```
df["producto_limpio"] = df["producto_limpio"].replace({
 "error": "unknown"
})

df["metodo_pago_limpio"] = df["metodo_pago_limpio"].replace({
 "error": "unknown"
})

df["ubicacion_limpia"] = df["ubicacion_limpia"].replace({
 "error": "unknown"
})
```

Ahora revisamos nuevamente los conteos.

```
df["producto_limpio"].value_counts(dropna=False)
```

producto_limpio	count
coffee	3
tea	2
sandwich	2
cake	2
unknown	2
smoothie	2
juice	1
None	1

`dtype: int64`

```
df["metodo_pago_limpio"].value_counts(dropna=False)
```

metodo_pago_limpio	count
credit_card	4
cash	3
digital_wallet	3
unknown	2
debit_card	2

metodo_pago_limpio	count
None	1

dtype: int64

```
df["ubicacion_limpia"].value_counts(dropna=False)
```

ubicacion_limpia	count
takeaway	5
in_store	4
delivery	3
unknown	2
None	1

dtype: int64

Después del reemplazo, `error` ya no aparece como categoría separada. Sus casos quedaron agrupados dentro de `unknown`.

Todavía pueden quedar valores faltantes reales, como `NaN`. En este capítulo no vamos a profundizar en imputación, porque ya trabajamos ese tema en el tratamiento de valores faltantes. Lo importante aquí es distinguir entre tres situaciones:

```

categorías válidas
categorías desconocidas o problemáticas
valores faltantes reales

```

Una limpieza de categorías no siempre busca eliminar todos los valores problemáticos. A veces busca hacerlos visibles, agruparlos de manera consistente y evitar que fragmenten el análisis.

## Comparar antes y después de la limpieza

Una buena limpieza de datos debe poder verificarse.

No alcanza con aplicar transformaciones como `.str.strip()`, `.str.lower()` o `replace()`. Después de transformar una columna, necesitamos comprobar si el resultado realmente mejoró el problema que queríamos resolver.

En este caso, podemos comparar los conteos originales con los conteos de las columnas limpias.

Empecemos con `producto`.

```
print("Conteo original de producto:")
print(df["producto"].value_counts(dropna=False))

print()

print("Conteo de producto_limpio:")
print(df["producto_limpio"].value_counts(dropna=False))
```

```
Conteo original de producto:
producto
Coffee 1
 coffee 1
COFFEE 1
Tea 1
 tea 1
Sandwich 1
 sandwich 1
Cake 1
 cake 1
Juice 1
UNKNOWN 1
ERROR 1
None 1
Smoothie 1
 smoothie 1
Name: count, dtype: int64
```

```
Conteo de producto_limpio:
producto_limpio
coffee 3
 tea 2
 sandwich 2
 cake 2
 unknown 2
 smoothie 2
 juice 1
 None 1
Name: count, dtype: int64
```

La comparación muestra que las variantes de escritura quedaron agrupadas.

Antes, Pandas contaba por separado valores como `"Coffee"`, `" coffee"` y `"COFFEE"`. Después de la limpieza, esas variantes quedaron reunidas bajo una misma categoría: `"coffee"`.

También podemos hacer la misma comparación con el método de pago.

```
print("Conteo original de metodo_pago:")
print(df["metodo_pago"].value_counts(dropna=False))
```

```
print()

print("Conteo de metodo_pago_limpio:")
print(df["metodo_pago_limpio"].value_counts(dropna=False))
```

Conteo original de metodo\_pago:

```
metodo_pago
Cash 1
cash 1
CASH 1
Credit Card 1
credit card 1
Credito 1
Crédito 1
Digital Wallet 1
digital wallet 1
wallet 1
UNKNOWN 1
ERROR 1
None 1
Debit Card 1
debit card 1
Name: count, dtype: int64
```

Conteo de metodo\_pago\_limpio:

```
metodo_pago_limpio
credit_card 4
cash 3
digital_wallet 3
unknown 2
debit_card 2
None 1
Name: count, dtype: int64
```

En este caso, la limpieza no solo corrigió espacios y mayúsculas. También unificó variantes como "Credit Card", "credit card", "Credito" y "Crédito" bajo una categoría común.

Finalmente, revisemos la ubicación.

```
print("Conteo original de ubicacion:")
print(df["ubicacion"].value_counts(dropna=False))

print()

print("Conteo de ubicacion_limpia:")
print(df["ubicacion_limpia"].value_counts(dropna=False))
```

Conteo original de ubicacion:  
ubicacion



```

In-store 1
in-store 1
In-store 1
Takeaway 1
takeaway 1
TAKEAWAY 1
Delivery 1
delivery 1
DELIVERY 1
In store 1
UNKNOWN 1
ERROR 1
None 1
Take away 1
take-away 1
Name: count, dtype: int64

Conteo de ubicacion_limpia:
ubicacion_limpia
takeaway 5
in_store 4
delivery 3
unknown 2
None 1
Name: count, dtype: int64

```

Esta comparación permite ver el efecto de la limpieza.

Antes, algunas categorías estaban fragmentadas por diferencias de escritura. Después, las columnas limpias tienen menos variantes y representan mejor las categorías que queremos analizar.

Esto es importante porque los conteos, filtros y agrupamientos posteriores van a depender de estas columnas. Si las categorías quedan fragmentadas, el análisis puede subestimar o sobreestimar ciertos grupos.

Por eso, después de limpiar columnas categóricas, conviene revisar siempre los valores únicos o las frecuencias resultantes.

## Crear una versión limpia del dataset

Hasta ahora fuimos creando columnas limpias junto a las columnas originales.

Esta estrategia tiene una ventaja importante: permite comparar antes y después de la limpieza. Si algo no queda bien, todavía podemos revisar los valores originales y corregir la transformación.

Sin embargo, para continuar un análisis, muchas veces conviene construir una versión más ordenada del dataset. Esa versión puede conservar las columnas limpias y dejar afuera las columnas originales que ya no vamos a usar.

Por ejemplo, podríamos crear un nuevo `DataFrame` llamado `df_limpio`.

```
df_limpio = df[
 [
 "id_venta",
 "producto_limpio",
 "metodo_pago_limpio",
 "ubicacion_limpia",
 "cantidad",
 "precio_unitario"
]
].copy()

df_limpio
```

índice	id_venta	producto_limpio	metodo_pago_limpio	ubicacion_limpia	cantidad	precio_uni
0	V001	coffee	cash	in_store	2	1200
1	V002	coffee	cash	in_store	1	1200
2	V003	coffee	cash	in_store	3	1200
3	V004	tea	credit_card	takeaway	2	900
4	V005	tea	credit_card	takeaway	1	900
5	V006	sandwich	credit_card	takeaway	1	2500
6	V007	sandwich	credit_card	delivery	2	2500
7	V008	cake	digital_wallet	delivery	1	1800
8	V009	cake	digital_wallet	delivery	3	1800
9	V010	juice	digital_wallet	in_store	2	1500
10	V011	unknown	unknown	unknown	1	0
11	V012	unknown	unknown	unknown	1	0
12	V013	None	None	None	2	1200
13	V014	smoothie	debit_card	takeaway	1	2200
14	V015	smoothie	debit_card	takeaway	2	2200

Ahora `df_limpio` contiene solamente las columnas que queremos conservar para el análisis.

Podemos renombrar las columnas limpias para que tengan nombres más simples:

```
df_limpio = df_limpio.rename(columns={
 "producto_limpio": "producto",
 "metodo_pago_limpio": "metodo_pago",
 "ubicacion_limpia": "ubicacion"
})

df_limpio
```

índice	id_venta	producto	metodo_pago	ubicacion	cantidad	precio_unitario
0	V001	coffee	cash	in_store	2	1200
1	V002	coffee	cash	in_store	1	1200
2	V003	coffee	cash	in_store	3	1200
3	V004	tea	credit_card	takeaway	2	900
4	V005	tea	credit_card	takeaway	1	900
5	V006	sandwich	credit_card	takeaway	1	2500
6	V007	sandwich	credit_card	delivery	2	2500
7	V008	cake	digital_wallet	delivery	1	1800
8	V009	cake	digital_wallet	delivery	3	1800
9	V010	juice	digital_wallet	in_store	2	1500
10	V011	unknown	unknown	unknown	1	0
11	V012	unknown	unknown	unknown	1	0
12	V013	None	None	None	2	1200
13	V014	smoothie	debit_card	takeaway	1	2200
14	V015	smoothie	debit_card	takeaway	2	2200

Esta versión del dataset es más clara para trabajar.

La tabla original `df` todavía conserva las columnas originales y las columnas limpias. En cambio, `df_limpio` representa una versión preparada para continuar el análisis.

Esta práctica es recomendable porque separa dos momentos del trabajo:

```
dataset original o de trabajo → permite revisar y comparar transformaciones
dataset limpio → permite continuar el análisis con columnas consistentes
```

Crear una versión limpia no significa borrar la historia del proceso. Significa construir una tabla más adecuada para seguir trabajando, después de haber verificado las transformaciones realizadas.

## Errores frecuentes al limpiar textos y categorías

Al limpiar columnas de texto, uno de los errores más frecuentes es aplicar transformaciones sin revisar primero los valores existentes.

Por ejemplo, convertir todo a minúsculas o quitar espacios puede ser útil, pero no resuelve todos los problemas. Si una columna contiene valores como "Credit Card", "Credito" y "Crédito", aplicar `.str.lower()` no alcanza para unificarlos. Esos valores seguirán siendo distintos hasta que tomemos una decisión explícita de reemplazo.

Otro error común es unificar categorías solo porque se parecen. En algunos casos, dos textos parecidos representan lo mismo, pero en otros pueden tener significados diferentes. Por eso, antes de reemplazar valores conviene revisar el contexto y documentar el criterio usado.

También debemos tener cuidado con valores como "UNKNOWN" y "ERROR". No son categorías comunes. Pueden representar ausencia de información, errores de carga o valores que necesitan una revisión especial. Agruparlos dentro de una categoría como "unknown" puede ser razonable para ciertos análisis, pero no significa que hayamos recuperado el dato real.

Otro punto importante es no sobrescribir columnas originales demasiado pronto. Durante la limpieza puede ser conveniente crear columnas auxiliares, como `producto_limpio`, `metodo_pago_limpio` o `ubicacion_limpia`, para comparar el antes y el después. Una vez verificada la transformación, podemos construir una versión limpia del dataset.

Finalmente, un error frecuente es no verificar el resultado. Después de limpiar una columna categórica, siempre conviene revisar los valores únicos o los conteos.

```
df_limpio["producto"].value_counts(dropna=False)
```

producto	count
coffee	3
tea	2
sandwich	2
cake	2
unknown	2
smoothie	2
juice	1

producto	count
None	1

dtype: int64

```
df_limpio["metodo_pago"].value_counts(dropna=False)
```

metodo_pago	count
credit_card	4
cash	3
digital_wallet	3
unknown	2
debit_card	2
None	1

dtype: int64

```
df_limpio["ubicacion"].value_counts(dropna=False)
```

ubicacion	count
takeaway	5
in_store	4
delivery	3
unknown	2
None	1

dtype: int64

Estas verificaciones permiten confirmar que las categorías quedaron más consistentes.

En general, una buena rutina para limpiar textos y categorías podría ser:

```
revisar valores originales
detectar inconsistencias
aplicar transformaciones generales
reemplazar variantes específicas
decidir qué hacer con valores problemáticos
verificar conteos finales
construir una versión limpia del dataset
```

La limpieza de textos no consiste solo en “prolijar” valores. Su objetivo es evitar que el análisis trate como distintas categorías que en realidad deberían estar agrupadas, o que mezcle categorías válidas con valores desconocidos o erróneos.

## Resumen del capítulo

---

En este capítulo trabajamos con la limpieza de textos y categorías.

Partimos de un dataset sintético de ventas de cafetería construido especialmente para mostrar problemas frecuentes en columnas categóricas. El objetivo fue observar cómo pequeñas diferencias de escritura pueden afectar el análisis.

Vimos que, para Pandas, valores como "Coffee", " coffee" y "COFFEE" son distintos. Lo mismo ocurre con textos que tienen espacios al comienzo o al final, diferencias entre mayúsculas y minúsculas, variantes con tildes o distintas formas de escribir una misma categoría.

Primero revisamos los valores originales usando `value_counts(dropna=False)`. Esa revisión nos permitió ver que las categorías estaban fragmentadas:

```
df["producto"].value_counts(dropna=False)
```

Después empezamos a limpiar la columna `producto`. Usamos `.str.strip()` para quitar espacios innecesarios al comienzo y al final:

```
df["producto_limpio"] = df["producto"].str.strip()
```

Luego usamos `.str.lower()` para unificar mayúsculas y minúsculas:

```
df["producto_limpio"] = df["producto_limpio"].str.lower()
```

Esta combinación permitió agrupar variantes como "Coffee", " coffee" y "COFFEE" bajo una misma categoría.

También aplicamos transformaciones similares en las columnas `metodo_pago` y `ubicacion`, creando versiones limpias de cada una:

```
df["metodo_pago_limpio"] = df["metodo_pago"].str.strip().str.lower()
df["ubicacion_limpia"] = df["ubicacion"].str.strip().str.lower()
```

Más adelante vimos que algunas inconsistencias no se resuelven solo con espacios y minúsculas. Por ejemplo, valores como "credit card", "credito" y "crédito"

pueden representar la misma forma de pago, pero siguen siendo textos diferentes. Para unificar esos casos usamos `replace()` con diccionarios de reemplazo.

También discutimos cómo tratar valores como `"UNKNOWN"`, `"ERROR"` y valores faltantes reales. Vimos que no son categorías comunes y que requieren una decisión explícita. En este ejemplo decidimos agrupar `"error"` dentro de `"unknown"`, interpretando ambos como valores no confiables o no disponibles para el análisis categórico.

Después comparamos los conteos antes y después de la limpieza. Esta verificación fue importante porque mostró el efecto real de las transformaciones: categorías que antes aparecían fragmentadas quedaron agrupadas de manera más consistente.

Finalmente construimos una versión limpia del dataset:

```
df_limpio = df[
 [
 "id_venta",
 "producto_limpio",
 "metodo_pago_limpio",
 "ubicacion_limpia",
 "cantidad",
 "precio_unitario"
]
].copy()
```

y luego renombramos las columnas limpias para continuar trabajando con nombres más simples.

La idea principal de este capítulo fue:

Limpiar textos y categorías permite evitar que el análisis trate como distintos valores que deberían representar la misma categoría.

Esta limpieza no es solo una cuestión de presentación. Afecta directamente conteos, filtros, agrupamientos, gráficos y conclusiones.

## Próximo paso

---

Ya trabajamos con valores faltantes, duplicados y limpieza de categorías.

El siguiente paso será estudiar la conversión de tipos de datos. En datasets reales, muchas columnas que deberían ser numéricas pueden aparecer cargadas como texto. También puede ocurrir que las fechas no estén interpretadas como fechas, sino como cadenas de caracteres.

Vamos a aprender a revisar tipos de datos, convertir columnas numéricas con control de errores y preparar columnas para que puedan usarse correctamente en cálculos y análisis posteriores.



## Capítulo 18 — Conversión de tipos de datos

---

### Breve repaso

---

En el trabajo anterior sobre limpieza de textos y categorías vimos que no alcanza con mirar el contenido de una columna de manera superficial. Dos valores pueden parecer equivalentes para una persona, pero ser distintos para Pandas si tienen espacios, mayúsculas, tildes o variantes de escritura.

En este capítulo vamos a trabajar con otro aspecto fundamental de la preparación de datos: los tipos de datos.

Cuando Pandas carga un archivo, intenta interpretar automáticamente qué tipo de información contiene cada columna. Algunas columnas quedan como números, otras como texto y otras pueden necesitar una conversión especial. Sin embargo, esa interpretación automática no siempre coincide con lo que necesitamos para analizar.

Una columna puede “parecer” numérica porque contiene valores como `2`, `3.5` o `1200`, pero si está cargada como texto, Pandas no la tratará como una variable numérica real. Algo similar ocurre con las fechas: una fecha puede verse como `2024-03-01`, pero si Pandas la interpreta como texto, no podremos trabajarla correctamente como dato temporal.

En este capítulo vamos a revisar tipos de datos, detectar columnas que necesitan conversión y transformar valores usando herramientas como `pd.to_numeric()` y `pd.to_datetime()`.

Al finalizar este capítulo deberías poder:

- Comprender por qué los tipos de datos son importantes para el análisis.
- Revisar los tipos de columnas con `dtypes` e `info()`.
- Detectar columnas numéricas cargadas como texto.
- Convertir columnas a formato numérico usando `pd.to_numeric()`.
- Comprender el uso de `errors="coerce"`.
- Verificar si la conversión generó nuevos valores faltantes.
- Convertir una columna de texto a fecha usando `pd.to_datetime()`.
- Validar la estructura del `DataFrame` después de convertir tipos.

## Dataset de trabajo

Para estudiar conversión de tipos de datos vamos a volver al dataset real **Cafe Sales — Dirty Data for Cleaning Training**.

Este dataset contiene transacciones de venta de una cafetería. Algunas columnas deberían poder analizarse como valores numéricos, por ejemplo `Quantity`, `Price Per Unit` y `Total Spent`. También hay una columna de fecha, `Transaction Date`, que debería poder interpretarse como dato temporal.

Sin embargo, como el dataset contiene valores problemáticos, Pandas puede cargar algunas de estas columnas con tipos de datos que no son los más adecuados para el análisis.

Como cada capítulo debe poder ejecutarse de manera independiente, vamos a descargar y cargar nuevamente el dataset.

```
import kagglehub
import pandas as pd
import os

ruta_dataset = kagglehub.dataset_download(
 "ahmedmohamed2003/cafe-sales-dirty-data-for-cleaning-training"
)

archivos = os.listdir(ruta_dataset)

archivos_csv = [
 archivo for archivo in archivos
 if archivo.endswith(".csv")
]

ruta_csv = os.path.join(ruta_dataset, archivos_csv[0])

df = pd.read_csv(ruta_csv)

df.head()
```

Using Colab cache for faster access to the 'cafe-sales-dirty-data-for-cleaning-training' dataset.

índice	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date
0	TXN_1961373	Coffee	2	2.0	4.0	Credit Card	Takeaway	2023-09-08
1	TXN_4977031	Cake	4	3.0	12.0	Cash	In-store	2023-05-16

índice	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date
2	TXN_4271903	Cookie	4	1.0	ERROR	Credit Card	In-store	2023-07-19
3	TXN_7034554	Salad	2	5.0	10.0	UNKNOWN	UNKNOWN	2023-04-27
4	TXN_3160411	Coffee	2	2.0	4.0	Digital Wallet	In-store	2023-06-11

La salida de `head()` nos permite confirmar que el archivo fue cargado correctamente.

En este capítulo no vamos a concentrarnos todavía en limpiar todos los problemas del dataset. El foco estará puesto en revisar cómo Pandas interpretó los tipos de datos y en convertir algunas columnas para que puedan usarse correctamente en cálculos y análisis posteriores.

## Revisar los tipos de datos

Después de cargar un dataset, una de las primeras tareas de diagnóstico consiste en revisar qué tipo de dato asignó Pandas a cada columna.

Esto es importante porque muchas operaciones dependen del tipo de dato. Si una columna fue interpretada como numérica, podremos calcular promedios, medianas, sumas o rankings. Si una columna fue interpretada como texto, esas operaciones pueden fallar o producir resultados que no tienen sentido.

Podemos revisar los tipos de datos con `dtypes`.

```
df.dtypes
```

índice	0
Transaction ID	object
Item	object
Quantity	object
Price Per Unit	object
Total Spent	object
Payment Method	object
Location	object
Transaction Date	object

```
dtype: object
```

La salida muestra el tipo de dato de cada columna.

En Pandas, el tipo `object` suele indicar texto o valores mezclados. No siempre significa que la columna esté mal, pero sí nos invita a revisar con cuidado.

En este dataset, columnas como `Item`, `Payment Method` o `Location` pueden ser textuales. Eso es esperable. Pero si columnas como `Quantity`, `Price Per Unit` o `Total Spent` aparecen como `object`, tenemos una señal de problema: son columnas que deberían poder analizarse como numéricas.

También podemos usar `info()` para obtener una visión general de tipos y valores no nulos.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 8 columns):
Column Non-Null Count Dtype
--- -
0 Transaction ID 10000 non-null object
1 Item 9667 non-null object
2 Quantity 9862 non-null object
3 Price Per Unit 9821 non-null object
4 Total Spent 9827 non-null object
5 Payment Method 7421 non-null object
6 Location 6735 non-null object
7 Transaction Date 9841 non-null object
dtypes: object(8)
memory usage: 625.1+ KB
```

La salida de `info()` permite ver, en una misma tabla, el nombre de cada columna, la cantidad de valores no nulos y el tipo de dato asignado por Pandas.

Esta revisión es parte del diagnóstico inicial. Antes de convertir columnas, necesitamos identificar cuáles parecen tener un tipo de dato inadecuado y por qué eso puede afectar el análisis.

## Interpretar la salida de `info()`

La salida de `info()` nos muestra algo importante: todas las columnas del dataset fueron cargadas con tipo `object`.

Esto significa que Pandas está tratando todas las columnas como texto o como columnas con valores mezclados.

En algunas columnas esto es esperable. Por ejemplo, `Transaction ID` es un identificador, por lo tanto no necesitamos tratarlo como número. Lo mismo ocurre con `Item`, `Payment Method` y `Location`, que representan categorías o textos.

Pero hay otras columnas que deberían poder analizarse de otra manera:

<code>Quantity</code>	→ debería ser numérica
<code>Price Per Unit</code>	→ debería ser numérica
<code>Total Spent</code>	→ debería ser numérica
<code>Transaction Date</code>	→ debería ser una fecha

Si `Quantity`, `Price Per Unit` y `Total Spent` permanecen como `object`, no podremos trabajar con ellas de manera confiable como columnas numéricas. Esto puede afectar cálculos como sumas, promedios, medianas, rankings o validaciones entre columnas.

Algo similar ocurre con `Transaction Date`. Aunque sus valores parezcan fechas, mientras Pandas la interprete como `object`, esa columna sigue siendo texto. Para analizarla temporalmente, ordenar fechas, extraer meses o agrupar ventas por día, vamos a necesitar convertirla a un tipo de dato temporal.

También vemos que varias columnas tienen menos de 10000 valores no nulos. Eso indica que hay valores faltantes. En este capítulo no vamos a centrarnos en el tratamiento de faltantes, porque ya lo trabajamos antes. Sin embargo, al convertir tipos debemos tenerlos en cuenta, porque algunos errores de conversión también pueden generar nuevos valores faltantes.

Antes de analizar este dataset, necesitamos convertir algunas columnas al tipo de dato adecuado.

## Observar columnas que deberían ser numéricas

---

Antes de convertir columnas, conviene observar algunos valores.

En este dataset hay tres columnas que deberían representar cantidades o importes:

<code>Quantity</code>
<code>Price Per Unit</code>
<code>Total Spent</code>

Aunque visualmente puedan parecer numéricas, Pandas las cargó como `object`. Eso suele ocurrir cuando una columna contiene números mezclados con textos, valores problemáticos o datos faltantes.

Vamos a observar algunos valores de esas columnas.

```
df[["Quantity", "Price Per Unit", "Total Spent"]].head(15)
```

índice	Quantity	Price Per Unit	Total Spent
0	2	2.0	4.0
1	4	3.0	12.0
2	4	1.0	ERROR
3	2	5.0	10.0
4	2	2.0	4.0
5	5	4.0	20.0
6	3	3.0	9.0
7	4	4.0	16.0
8	5	3.0	15.0
9	5	4.0	20.0
10	5	5.0	25.0
11	2	4.0	8.0
12	2	4.0	8.0
13	5	1.0	5.0
14	2	1.5	3.0

Esta vista nos permite revisar ejemplos concretos.

Si una columna contiene valores como `"ERROR"` o `"UNKNOWN"`, Pandas no puede convertirla automáticamente a número durante la carga del archivo. Por eso la conserva como `object`.

También podemos revisar las frecuencias de valores en cada una de estas columnas. Esto puede ayudarnos a detectar valores problemáticos.

```
df["Quantity"].value_counts(dropna=False).head(15)
```

Quantity	count
5	2013
2	1974
4	1863
3	1849
1	1822
UNKNOWN	171
ERROR	170
NaN	138

dtype: int64

```
df["Price Per Unit"].value_counts(dropna=False).head(15)
```

Price Per Unit	count
3.0	2429
4.0	2331
2.0	1227
5.0	1204
1.0	1143
1.5	1133
ERROR	190
NaN	179
UNKNOWN	164

dtype: int64

```
df["Total Spent"].value_counts(dropna=False).head(15)
```

Total Spent	count
6.0	979
12.0	939
3.0	930
4.0	923
20.0	746

Total Spent	count
15.0	734
8.0	677
10.0	524
2.0	497
9.0	479
5.0	468
16.0	444
25.0	259
7.5	237
1.0	232

`dtype: int64`

En estas salidas debemos prestar atención a valores que no sean números reales.

Si aparecen valores como `"UNKNOWN"` o `"ERROR"`, ya tenemos una explicación para el tipo `object`: la columna mezcla valores numéricos con textos problemáticos.

Antes de convertir, es importante detectar estos casos. Convertir tipos no debería ser una acción ciega. Necesitamos saber qué clase de valores contiene la columna y qué ocurrirá con aquellos que no puedan convertirse.

## Por qué no alcanza con que los valores parezcan números

Una columna puede verse numérica cuando observamos algunas filas, pero eso no garantiza que Pandas pueda tratarla como una columna numérica.

Por ejemplo, `Quantity`, `Price Per Unit` y `Total Spent` representan valores que deberían permitir cálculos. Sin embargo, como fueron cargadas como `object`, necesitamos revisar si pueden usarse directamente en operaciones numéricas.

Intentemos calcular algunas estadísticas sobre `Price Per Unit`.

```
df["Price Per Unit"].describe()
```

índice	Price Per Unit
count	9821
unique	8



índice	Price Per Unit
top	3.0
freq	2429

`dtype: object`

Como la columna fue cargada como `object`, `describe()` produce un resumen propio de datos categóricos o textuales, no un resumen numérico.

En lugar de mostrar medidas como media, mediana, mínimo o máximo, Pandas muestra información como cantidad de valores, cantidad de valores únicos, valor más frecuente y frecuencia del valor más frecuente.

Esto nos indica que, para Pandas, `Price Per Unit` todavía no es una variable numérica.

Algo similar ocurre si intentamos ordenar o comparar valores. Cuando una columna está como texto, el orden puede ser alfabético y no necesariamente numérico. Por eso, antes de hacer análisis cuantitativo, necesitamos convertir estas columnas al tipo adecuado.

```
df["Price Per Unit"].sort_values().head(10)
```

índice	Price Per Unit
1207	1.0
5841	1.0
2606	1.0
3933	1.0
1331	1.0
5834	1.0
3935	1.0
5832	1.0
3942	1.0
8544	1.0

`dtype: object`

Esta salida puede resultar engañosa, porque el ordenamiento se realiza sobre textos.

En análisis de datos, no alcanza con que una columna parezca contener números. Necesitamos que Pandas la reconozca como numérica para poder calcular, comparar y ordenar correctamente.

El siguiente paso será convertir estas columnas usando `pd.to_numeric()`.

## Convertir columnas con `pd.to_numeric()`

---

Para convertir una columna a formato numérico podemos usar `pd.to_numeric()`.

Esta función intenta transformar los valores de una columna en números. Si todos los valores son compatibles con un formato numérico, la conversión suele ser directa.

Probemos primero con la columna `Price Per Unit`.

```
pd.to_numeric(df["Price Per Unit"])
```

Es posible que esta instrucción produzca un error.

Ese error aparece porque la columna no contiene solamente números. También contiene valores textuales problemáticos, como `"ERROR"` o `"UNKNOWN"`. Pandas no puede convertir esos textos en números, porque no representan cantidades válidas.

Este resultado es importante: confirma que el problema no era solamente el tipo `object`, sino la presencia de valores no numéricos dentro de una columna que debería ser numérica.

Para manejar este tipo de situación podemos usar el parámetro `errors`.

## Controlar errores de conversión

---

Cuando usamos `pd.to_numeric()`, podemos indicar qué debe hacer Pandas si encuentra un valor que no puede convertir.

Para eso usamos el parámetro `errors`.

Una opción muy útil es `errors="coerce"`.

Con esta opción, Pandas intenta convertir cada valor a número. Si encuentra un valor que no puede convertir, como `"ERROR"` o `"UNKNOWN"`, lo transforma en `NaN`.

Esto no significa que el problema desaparezca. Significa que los valores no convertibles quedan marcados como faltantes, lo cual nos permite detectarlos y tratarlos con las herramientas que ya conocemos.

```
precio_convertido = pd.to_numeric(
 df["Price Per Unit"],
 errors="coerce"
)

precio_convertido.head(15)
```

índice	Price Per Unit
0	2.0
1	3.0
2	1.0
3	5.0
4	2.0
5	4.0
6	3.0
7	4.0
8	3.0
9	4.0
10	5.0
11	4.0
12	4.0
13	1.0
14	1.5

`dtype: float64`

Ahora la conversión no se detiene con un error.

Los valores numéricos se transforman correctamente, y los valores que no pudieron convertirse pasan a ser `NaN`.

Podemos revisar el tipo de dato resultante:

```
precio_convertido.dtype
```

```
dtype('float64')
```

El resultado ya no es `object`. Ahora tenemos una serie numérica.

También podemos revisar cuántos valores faltantes aparecen después de la conversión:

```
precio_convertido.isna().sum()
```

```
np.int64(533)
```

Este conteo incluye los valores que ya eran faltantes y también los valores que se convirtieron en `NaN` porque no pudieron interpretarse como números.

Por eso, después de usar `errors="coerce"`, siempre debemos verificar cuántos valores quedaron como faltantes. La conversión no debe ocultar el problema: debe hacerlo más fácil de diagnosticar.

## Convertir varias columnas numéricas

En nuestro dataset no necesitamos convertir una sola columna. Hay tres columnas que deberían poder tratarse como numéricas:

```
Quantity
Price Per Unit
Total Spent
```

Como no queremos modificar el dataset original directamente, vamos a crear una copia llamada `df_convertido`.

Luego aplicaremos `pd.to_numeric()` a cada una de esas columnas.

```
df_convertido = df.copy()

columnas_numericas = [
 "Quantity",
 "Price Per Unit",
 "Total Spent"
]

for columna in columnas_numericas:
 df_convertido[columna] = pd.to_numeric(
 df_convertido[columna],
 errors="coerce"
)

df_convertido[columnas_numericas].head()
```

índice	Quantity	Price Per Unit	Total Spent
0	2.0	2.0	4.0
1	4.0	3.0	12.0
2	4.0	1.0	NaN
3	2.0	5.0	10.0
4	2.0	2.0	4.0

Ahora las columnas seleccionadas fueron convertidas a formato numérico dentro de `df_convertido`.

Usamos un bucle `for` para aplicar la misma transformación a varias columnas. Esto evita repetir tres veces una estructura muy parecida.

La lógica fue la misma en cada caso: intentar convertir los valores a número y transformar en `NaN` aquellos valores que no podían interpretarse como numéricos.

Después de convertir, debemos verificar los tipos de datos.

```
df_convertido[columnas_numericas].dtypes
```

índice	0
Quantity	float64
Price Per Unit	float64
Total Spent	float64

`dtype: object`

Ahora estas columnas ya no aparecen como `object`. Pandas puede tratarlas como columnas numéricas.

Esto nos permite calcular estadísticas, ordenar correctamente, comparar valores y construir nuevas columnas a partir de operaciones matemáticas.

Sin embargo, todavía debemos revisar algo importante: al usar `errors="coerce"`, algunos valores problemáticos pueden haberse convertido en `NaN`. Por eso, después de convertir, siempre necesitamos verificar los faltantes.

## Verificar faltantes después de convertir

Cuando usamos `pd.to_numeric()` con `errors="coerce"`, los valores que no pueden convertirse pasan a ser `NaN`.

Eso significa que la conversión puede aumentar la cantidad de valores faltantes detectados por Pandas. No porque hayamos perdido datos, sino porque valores como "ERROR" o "UNKNOWN" fueron transformados en faltantes reconocidos.

Por eso, conviene comparar los faltantes antes y después de la conversión.

```
faltantes_antes = df[columnas_numericas].isna().sum()
faltantes_despues = df_convertido[columnas_numericas].isna().sum()

comparacion_faltantes = pd.DataFrame({
 "faltantes_antes": faltantes_antes,
 "faltantes_despues": faltantes_despues,
 "nuevos_faltantes": faltantes_despues - faltantes_antes
})

comparacion_faltantes
```

índice	faltantes_antes	faltantes_despues	nuevos_faltantes
Quantity	138	479	341
Price Per Unit	179	533	354
Total Spent	173	502	329

La columna `faltantes_antes` muestra los valores que Pandas ya reconocía como faltantes antes de la conversión.

La columna `faltantes_despues` muestra los faltantes luego de convertir las columnas a formato numérico.

La columna `nuevos_faltantes` indica cuántos valores adicionales pasaron a ser NaN porque no pudieron convertirse.

Esta comparación es muy importante. Si solo miramos el resultado final, podríamos pensar que la columna simplemente tenía muchos faltantes. Pero al comparar antes y después vemos que una parte de esos faltantes proviene de valores problemáticos que estaban escritos como texto.

La conversión de tipos no solo cambia el formato de una columna. También puede revelar problemas de calidad que antes estaban ocultos.

## Calcular estadísticas después de convertir

Una vez convertidas las columnas numéricas, Pandas puede analizarlas como cantidades reales.

Antes de la conversión, `describe()` sobre `Price Per Unit` producía un resumen de tipo textual. Ahora, sobre `df_convertido`, podemos obtener estadísticas numéricas.

```
df_convertido["Price Per Unit"].describe()
```

índice	Price Per Unit
count	9467.000000
mean	2.949984
std	1.278450
min	1.000000
25%	2.000000
50%	3.000000
75%	4.000000
max	5.000000

`dtype: float64`

Ahora la salida incluye medidas como media, desviación estándar, mínimo, cuartiles y máximo.

Esto confirma que la columna ya puede usarse como una variable numérica.

También podemos obtener un resumen de las tres columnas convertidas:

```
df_convertido[columnas_numericas].describe()
```

índice	Quantity	Price Per Unit	Total Spent
count	9521.000000	9467.000000	9498.000000
mean	3.028463	2.949984	8.924352
std	1.419007	1.278450	6.009919
min	1.000000	1.000000	1.000000
25%	2.000000	2.000000	4.000000
50%	3.000000	3.000000	8.000000
75%	4.000000	4.000000	12.000000
max	5.000000	5.000000	25.000000

Este resumen permite revisar rangos y valores generales de las columnas numéricas.

A partir de esta conversión, podemos hacer operaciones que antes no eran confiables. Por ejemplo, podemos ordenar por precio unitario de mayor a menor:

```
df_convertido.sort_values("Price Per Unit", ascending=False).head()
```

índice	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date
9958	TXN_4125474	ERROR	2.0	5.0	10.0	Credit Card	In-store	2023-08-02
9942	TXN_5344848	Salad	1.0	5.0	5.0	Digital Wallet	Takeaway	2023-09-27
9940	TXN_8273780	Salad	2.0	5.0	10.0	Digital Wallet	Takeaway	2023-10-15
9928	TXN_5171519	Salad	3.0	5.0	15.0	NaN	In-store	2023-04-06
9920	TXN_4365321	Salad	4.0	5.0	20.0	Cash	In-store	2023-03-09

También podemos crear o verificar relaciones entre columnas numéricas.

Por ejemplo, más adelante podremos revisar si `Total Spent` coincide con `Quantity * Price Per Unit`.

La conversión de tipos no es solo un cambio técnico. Es lo que permite que ciertas columnas puedan participar correctamente en cálculos, comparaciones, ordenamientos y validaciones.

## Convertir una columna de fecha

Además de columnas numéricas, muchos datasets incluyen columnas de fecha.

En nuestro caso, la columna `Transaction Date` representa la fecha de cada transacción. Sin embargo, al cargar el archivo, Pandas la interpretó como `object`. Eso significa que, por ahora, la está tratando como texto.

Podemos verificarlo:

```
df_convertido["Transaction Date"].dtype
```

```
dtype('O')
```



Aunque los valores se vean como fechas, todavía no son datos temporales para Pandas.

Para convertir una columna a formato de fecha usamos `pd.to_datetime()`.

```
fecha_convertida = pd.to_datetime(
 df_convertido["Transaction Date"],
 errors="coerce"
)

fecha_convertida.head()
```

índice	Transaction Date
0	2023-09-08
1	2023-05-16
2	2023-07-19
3	2023-04-27
4	2023-06-11

```
dtype: datetime64[ns]
```

La función `pd.to_datetime()` intenta interpretar los valores como fechas.

Usamos nuevamente `errors="coerce"` para evitar que la conversión se detenga si aparece un valor que no puede interpretarse como fecha. En esos casos, Pandas convierte el valor problemático en `NaT`.

`NaT` significa *Not a Time* y es el equivalente temporal de un valor faltante. Así como `NaN` representa un faltante en muchos tipos de columnas, `NaT` representa una fecha faltante o no interpretable.

Ahora podemos revisar el tipo de dato resultante:

```
fecha_convertida.dtype
```

```
dtype('<M8[ns]')
```

El tipo `datetime64[ns]` indica que Pandas ya interpreta la columna como una fecha.

Podemos guardar esta conversión en el `DataFrame` convertido:

```
df_convertido["Transaction Date"] = pd.to_datetime(
 df_convertido["Transaction Date"],
```

```
 errors="coerce"
)

df_convertido["Transaction Date"].dtype
```

```
dtype('<M8[ns]')
```

A partir de esta conversión, la columna `Transaction Date` puede usarse como dato temporal.

Todavía no vamos a profundizar en operaciones con fechas. Más adelante podremos extraer el año, el mes, el día, ordenar correctamente las transacciones en el tiempo o agrupar ventas por período.

En este capítulo nos alcanza con comprender la idea principal: una fecha escrita como texto no es lo mismo que una fecha interpretada como dato temporal por Pandas.

## Verificar la conversión de fechas

Así como hicimos con las columnas numéricas, después de convertir una columna de fecha conviene verificar el resultado.

Primero podemos revisar cuántos valores faltantes tenía originalmente la columna `Transaction Date` y cuántos valores faltantes o no interpretables aparecen después de la conversión.

```
faltantes_fecha_antes = df["Transaction Date"].isna().sum()
faltantes_fecha_despues = df_convertido["Transaction Date"].isna().sum()

print("Faltantes antes de convertir:")
print(faltantes_fecha_antes)

print()

print("Faltantes después de convertir:")
print(faltantes_fecha_despues)
```

```
Faltantes antes de convertir:
159
```

```
Faltantes después de convertir:
460
```

Si la cantidad de faltantes aumenta después de la conversión, eso significa que algunos valores que antes estaban escritos como texto no pudieron interpretarse como fechas y fueron convertidos en `NaT`.

También podemos revisar algunas estadísticas básicas de la columna convertida.

```
df_convertido["Transaction Date"].describe()
```

índice	Transaction Date
count	9540
mean	2023-07-01 23:00:31.698113280
min	2023-01-01 00:00:00
25%	2023-04-01 00:00:00
50%	2023-07-02 00:00:00
75%	2023-10-02 00:00:00
max	2023-12-31 00:00:00

`dtype: object`

El resumen de una columna temporal nos permite observar, entre otras cosas, la fecha mínima y la fecha máxima registradas.

Esto puede ser útil para detectar fechas fuera de rango o valores extraños. Por ejemplo, si un dataset de ventas de 2024 tuviera una fecha del año 2099, esa fecha debería llamar nuestra atención.

También podemos ordenar el dataset por fecha:

```
df_convertido.sort_values("Transaction Date").head()
```

índice	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date
8015	TXN_4801947	Juice	1.0	3.0	3.0	Digital Wallet	Takeaway	2023-01-01
9063	TXN_9161256	Smoothie	2.0	4.0	8.0	Digital Wallet	In-store	2023-01-01
7309	TXN_6093955	Tea	5.0	1.5	NaN	UNKNOWN	Takeaway	2023-01-01
1425	TXN_8842223	Sandwich	5.0	NaN	20.0	Digital Wallet	In-store	2023-01-01

índice	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date
1777	TXN_7367474	Juice	5.0	3.0	15.0	Digital Wallet	Takeaway	2023-01-01

Ahora el ordenamiento se realiza con una columna temporal, no con texto.

En muchos casos esto no cambia visualmente el resultado si las fechas estaban escritas en un formato ordenable, como `YYYY-MM-DD`. Pero conceptualmente sí es importante: Pandas ahora reconoce la columna como fecha y puede aplicar operaciones temporales sobre ella.

La conversión de fechas será la base para trabajar más adelante con análisis por día, mes, año o períodos.

## Validar los tipos finales

Después de convertir columnas numéricas y temporales, conviene revisar nuevamente la estructura general del `DataFrame`.

El objetivo es comprobar que las columnas que necesitaban conversión ya tengan tipos de datos más adecuados para el análisis.

```
df_convertido.dtypes
```

índice	0
Transaction ID	object
Item	object
Quantity	float64
Price Per Unit	float64
Total Spent	float64
Payment Method	object
Location	object
Transaction Date	datetime64[ns]

```
dtype: object
```

Ahora deberíamos observar que:

```
Quantity → tipo numérico
Price Per Unit → tipo numérico
Total Spent → tipo numérico
Transaction Date → tipo fecha
```

Mientras tanto, columnas como `Transaction ID`, `Item`, `Payment Method` y `Location` pueden seguir como `object`, porque representan identificadores o categorías textuales.

También podemos usar `info()` para ver la estructura completa:

```
df_convertido.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 8 columns):
Column Non-Null Count Dtype
--- -
0 Transaction ID 10000 non-null object
1 Item 9667 non-null object
2 Quantity 9521 non-null float64
3 Price Per Unit 9467 non-null float64
4 Total Spent 9498 non-null float64
5 Payment Method 7421 non-null object
6 Location 6735 non-null object
7 Transaction Date 9540 non-null datetime64[ns]
dtypes: datetime64[ns](1), float64(3), object(4)
memory usage: 625.1+ KB
```

Esta revisión final permite confirmar que el dataset está en una mejor condición para el análisis.

Convertir tipos no significa que el dataset ya esté completamente limpio. Todavía pueden quedar valores faltantes, categorías problemáticas, duplicados u otras inconsistencias. Pero sí significa que algunas columnas ya están en un formato más apropiado para operar con ellas.

A partir de este momento, las columnas numéricas pueden participar en cálculos y comparaciones, y la columna de fecha puede usarse para análisis temporal.

La validación final es una parte esencial del proceso. No basta con aplicar conversiones: necesitamos comprobar que el resultado tenga sentido.

## Errores frecuentes al convertir tipos de datos

---

Al convertir tipos de datos, uno de los errores más comunes es asumir que una columna está bien solo porque sus valores “parecen” numéricos o “parecen” fechas.

Para Pandas, no alcanza con la apariencia visual. Si una columna fue cargada como `object`, puede contener textos, valores problemáticos o datos mezclados. Antes de usarla en cálculos, conviene revisar su tipo y convertirla si corresponde.

Otro error frecuente es usar `pd.to_numeric()` o `pd.to_datetime()` sin verificar qué ocurrió después. Cuando usamos `errors="coerce"`, los valores que no pueden convertirse pasan a ser `NaN` o `NaT`. Eso es útil porque evita que la conversión se detenga, pero también puede aumentar la cantidad de valores faltantes.

Por eso, después de convertir, siempre conviene comparar faltantes antes y después.

También debemos evitar sobrescribir el dataset original demasiado pronto. En este capítulo trabajamos con una copia llamada `df_convertido`, lo que nos permitió transformar columnas sin perder el punto de partida.

Otro error posible es convertir columnas que no necesitan conversión. Por ejemplo, `Transaction ID` contiene identificadores. Aunque tenga números dentro del texto, no conviene tratarlo como una variable numérica si no representa una cantidad. Un identificador no se suma, no se promedia y no se interpreta como magnitud.

Algo similar ocurre con ciertas categorías codificadas. Que una columna tenga números no significa necesariamente que sea numérica en sentido analítico. Siempre debemos preguntarnos qué representa la columna.

Una rutina razonable para convertir tipos podría ser:

```
revisar tipos actuales
identificar columnas que necesitan conversión
observar valores problemáticos
convertir en una copia del DataFrame
verificar nuevos tipos
comparar faltantes antes y después
confirmar que las columnas convertidas se comportan como esperamos
```

La conversión de tipos es una parte central de la preparación de datos. Si las columnas no tienen el tipo adecuado, muchas operaciones posteriores pueden fallar o producir resultados engañosos.

## Resumen del capítulo

---

En este capítulo trabajamos con la conversión de tipos de datos.

Partimos de una idea importante: no alcanza con que una columna parezca numérica o parezca una fecha. Para que Pandas pueda operar correctamente con esos valores, necesita interpretarlos con el tipo de dato adecuado.

Primero revisamos los tipos de datos con:

```
df.dtypes
```

y con:

```
df.info()
```

La salida de `info()` mostró que todas las columnas del dataset habían sido cargadas como `object`. Esto era esperable para columnas como `Transaction ID`, `Item`, `Payment Method` o `Location`, pero no para columnas como `Quantity`, `Price Per Unit`, `Total Spent` o `Transaction Date`.

Después observamos las columnas que deberían ser numéricas:

```
df[["Quantity", "Price Per Unit", "Total Spent"]].head(15)
```

y vimos que podían contener valores problemáticos como `"ERROR"` o `"UNKNOWN"`. Esa mezcla de números y textos explica por qué Pandas las cargó como `object`.

También comprobamos que una columna cargada como texto no produce un resumen numérico adecuado. Por ejemplo, `describe()` sobre `Price Per Unit` antes de la conversión genera un resumen propio de datos textuales, no estadísticas numéricas.

Luego usamos `pd.to_numeric()` para convertir valores a formato numérico. Como algunas celdas contenían valores no convertibles, usamos:

```
pd.to_numeric(df["Price Per Unit"], errors="coerce")
```

El parámetro `errors="coerce"` permite transformar en `NaN` los valores que no pueden convertirse a número.

Más adelante aplicamos esa conversión a varias columnas:

```
columnas_numericas = [
 "Quantity",
```

```

 "Price Per Unit",
 "Total Spent"
]

for columna in columnas_numericas:
 df_convertido[columna] = pd.to_numeric(
 df_convertido[columna],
 errors="coerce"
)

```

Después verificamos los tipos resultantes y comparamos los faltantes antes y después de la conversión. Esta comparación fue importante porque algunos valores problemáticos que antes estaban escritos como texto pasaron a ser `NaN`.

También convertimos la columna `Transaction Date` usando:

```
pd.to_datetime(df_convertido["Transaction Date"], errors="coerce")
```

En este caso, los valores que no pudieron interpretarse como fechas pasaron a ser `NaT`, que representa valores temporales faltantes o no interpretables.

Finalmente validamos la estructura final con `dtypes` e `info()`, confirmando que las columnas numéricas y la columna de fecha ya estaban en tipos más adecuados para el análisis.

La idea principal de este capítulo fue:

```

Convertir tipos de datos permite que Pandas interprete correctamente qué clase de
información contiene cada columna.

```

Esta conversión es necesaria para calcular, ordenar, comparar, validar y analizar datos de manera confiable.

## Próximo paso

Ya trabajamos con valores faltantes, duplicados, limpieza de textos y conversión de tipos de datos.

El siguiente paso será profundizar en fechas y datos temporales. Ahora que sabemos convertir una columna a formato de fecha, podemos aprender a extraer información útil de ella: año, mes, día, día de la semana o períodos.

También vamos a ver por qué las fechas son especialmente importantes en muchos análisis, ya que permiten estudiar evolución, estacionalidad, cambios en el tiempo y comportamiento por períodos.



## Capítulo 19 — Fechas y datos temporales

---

### Breve repaso

---

En el trabajo anterior sobre conversión de tipos de datos vimos que no alcanza con que una columna parezca numérica o parezca una fecha. Para que Pandas pueda trabajar correctamente con esos valores, necesita interpretarlos con el tipo adecuado.

Convertimos columnas como `Quantity`, `Price Per Unit` y `Total Spent` a formato numérico usando `pd.to_numeric()`. También hicimos una primera conversión de la columna `Transaction Date` usando `pd.to_datetime()`.

En este capítulo vamos a profundizar en el trabajo con fechas.

Las fechas son especialmente importantes porque permiten analizar los datos a lo largo del tiempo. En un dataset de ventas, por ejemplo, una columna de fecha puede ayudarnos a responder preguntas como: ¿en qué días hubo más transacciones?, ¿qué período cubre el dataset?, ¿hay ventas concentradas en ciertos días?, ¿cómo se distribuyen las operaciones a lo largo del tiempo?

Para responder este tipo de preguntas, no alcanza con que la fecha esté escrita como texto. Necesitamos que Pandas la interprete como dato temporal.

Una vez que una columna fue convertida a fecha, podemos ordenar registros cronológicamente, detectar valores fuera de rango, extraer el año, el mes o el día, y construir nuevas variables temporales útiles para el análisis.

También vamos a hacer un primer uso acotado de `groupby()`. Esta es una herramienta muy potente de Pandas para agrupar datos y calcular resúmenes por grupo. Más adelante la vamos a estudiar con mayor profundidad. En este capítulo la usaremos solamente para mostrar cómo una fecha bien convertida permite resumir información por períodos.

Al finalizar este capítulo deberías poder:

- Comprender por qué las fechas necesitan un tratamiento especial.
- Convertir una columna de texto a fecha usando `pd.to_datetime()`.
- Identificar valores temporales no interpretables.
- Revisar el rango temporal de un dataset.
- Ordenar datos usando una columna de fecha.

- Extraer componentes como año, mes, día y día de la semana.
- Crear columnas nuevas a partir de una fecha.
- Realizar primeras exploraciones temporales simples.

## Dataset de trabajo

---

Para trabajar con fechas vamos a volver al dataset real **Cafe Sales — Dirty Data for Cleaning Training**.

Este dataset contiene transacciones de venta de una cafetería. Una de sus columnas, `Transaction Date`, registra la fecha de cada operación.

Esa columna será el centro de este capítulo. Primero vamos a cargar el dataset, revisar cómo Pandas interpreta la fecha y luego convertirla a un tipo temporal adecuado.

Como cada capítulo debe poder ejecutarse de manera independiente, vamos a descargar y cargar nuevamente el dataset.

```
import kagglehub
import pandas as pd
import os

ruta_dataset = kagglehub.dataset_download(
 "ahmedmohamed2003/cafe-sales-dirty-data-for-cleaning-training"
)

archivos = os.listdir(ruta_dataset)

archivos_csv = [
 archivo for archivo in archivos
 if archivo.endswith(".csv")
]

ruta_csv = os.path.join(ruta_dataset, archivos_csv[0])

df = pd.read_csv(ruta_csv)

df.head()
```

Using Colab cache for faster access to the 'cafe-sales-dirty-data-for-cleaning-training' dataset.

índice	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date
0	TXN_1961373	Coffee	2	2.0	4.0	Credit Card	Takeaway	2023-09-08
1	TXN_4977031	Cake	4	3.0	12.0	Cash	In-store	2023-05-16
2	TXN_4271903	Cookie	4	1.0	ERROR	Credit Card	In-store	2023-07-19
3	TXN_7034554	Salad	2	5.0	10.0	UNKNOWN	UNKNOWN	2023-04-27
4	TXN_3160411	Coffee	2	2.0	4.0	Digital Wallet	In-store	2023-06-11

La salida de `head()` nos permite confirmar que el archivo fue cargado correctamente.

En este capítulo vamos a concentrarnos en la columna `Transaction Date`. Antes de transformarla, necesitamos revisar cómo fue cargada por Pandas.

## Revisar la columna de fecha

Antes de convertir una columna, conviene observar cómo fue cargada.

En este dataset, la columna que registra la fecha de cada transacción se llama `Transaction Date`.

Podemos revisar su tipo de dato:

```
df["Transaction Date"].dtype
```

```
dtype('O')
```

La salida indica que la columna fue cargada como `object`.

Eso significa que, por ahora, Pandas la está tratando como texto. Aunque los valores parezcan fechas, todavía no son fechas en sentido técnico para Pandas.

Podemos observar algunos valores:

```
df["Transaction Date"].head(10)
```

índice	Transaction Date
0	2023-09-08
1	2023-05-16

índice	Transaction Date
2	2023-07-19
3	2023-04-27
4	2023-06-11
5	2023-03-31
6	2023-10-06
7	2023-10-28
8	2023-07-28
9	2023-12-31

`dtype: object`

Visualmente, los valores pueden parecer fechas válidas. Pero mientras la columna esté como `object`, no podremos aprovechar correctamente las herramientas temporales de Pandas.

También podemos revisar cuántos valores faltantes tiene esta columna antes de convertirla:

```
df["Transaction Date"].isna().sum()
```

```
np.int64(159)
```

Este conteo nos da un primer diagnóstico.

Más adelante vamos a convertir la columna con `pd.to_datetime()` y compararemos si la cantidad de valores faltantes cambia. Si aumenta, eso indicará que había valores escritos como texto que no pudieron interpretarse como fechas.

## Convertir la columna a fecha

---

Ya vimos que `Transaction Date` fue cargada como `object`. Aunque sus valores parecen fechas, Pandas todavía la trata como texto.

Para convertirla a un tipo temporal usamos `pd.to_datetime()`.

Vamos a crear una copia del dataset para trabajar sin modificar directamente el `DataFrame` original.

```
df_fechas = df.copy()

df_fechas["Transaction Date"] = pd.to_datetime(
 df_fechas["Transaction Date"],
 errors="coerce"
)

df_fechas["Transaction Date"].head(10)
```

índice	Transaction Date
0	2023-09-08
1	2023-05-16
2	2023-07-19
3	2023-04-27
4	2023-06-11
5	2023-03-31
6	2023-10-06
7	2023-10-28
8	2023-07-28
9	2023-12-31

```
dtype: datetime64[ns]
```

Ahora la columna fue convertida a formato de fecha.

Usamos `errors="coerce"` para que cualquier valor que no pueda interpretarse como fecha se convierta en `NaT`.

`NaT` significa *Not a Time*. Es el equivalente temporal de un valor faltante. Así como `NaN` representa un dato faltante en muchas columnas, `NaT` representa una fecha faltante o no interpretable.

Después de convertir, debemos verificar el tipo de dato resultante.

```
df_fechas["Transaction Date"].dtype
```

```
dtype('<M8[ns]')
```

El tipo `datetime64[ns]` indica que Pandas ya reconoce la columna como temporal.

Esto nos permite trabajar con herramientas específicas para fechas: ordenar cronológicamente, extraer año, mes o día, calcular rangos temporales y construir nuevas columnas derivadas de la fecha.

## Verificar valores no interpretables

---

Antes de convertir la columna `Transaction Date`, ya habíamos detectado 159 valores faltantes.

Después de usar `pd.to_datetime()` con `errors="coerce"`, conviene revisar si esa cantidad cambió.

Si la cantidad de faltantes aumenta, significa que algunos valores que no estaban vacíos no pudieron interpretarse como fechas y fueron convertidos en `NaT`.

```
faltantes_fecha_antes = df["Transaction Date"].isna().sum()
faltantes_fecha_despues = df_fechas["Transaction Date"].isna().sum()

print("Faltantes antes de convertir:")
print(faltantes_fecha_antes)

print()

print("Faltantes después de convertir:")
print(faltantes_fecha_despues)
```

```
Faltantes antes de convertir:
159

Faltantes después de convertir:
460
```

Si ambos valores coinciden, entonces la conversión no generó nuevos faltantes. Eso significa que los valores no faltantes de la columna pudieron interpretarse correctamente como fechas.

Si el segundo valor fuera mayor que el primero, deberíamos investigar qué valores no pudieron convertirse.

Esta comparación es importante porque `errors="coerce"` evita que la conversión se detenga con un error, pero también puede transformar valores problemáticos en `NaT`. Por eso, después de convertir fechas, siempre debemos verificar cuántos valores temporales faltantes quedaron.

El resultado muestra algo importante: después de convertir, la cantidad de valores faltantes aumentó.

Antes de la conversión había 159 valores faltantes en `Transaction Date`. Después de aplicar `pd.to_datetime()` con `errors="coerce"`, aparecen 460 valores temporales faltantes.

Eso significa que había valores no vacíos que Pandas no pudo interpretar como fechas. Al usar `errors="coerce"`, esos valores fueron convertidos en `NaT`.

Calculemos cuántos valores nuevos se volvieron faltantes durante la conversión.

```
nuevos_nat = faltantes_fecha_despues - faltantes_fecha_antes

nuevos_nat
```

```
np.int64(301)
```

El resultado indica cuántos valores no faltantes se convirtieron en `NaT`.

Estos casos merecen una revisión adicional. No eran valores vacíos, pero tampoco pudieron convertirse correctamente a fecha. Podrían ser textos como `"ERROR"`, `"UNKNOWN"` u otros valores problemáticos.

El siguiente paso es identificar cuáles eran esos valores originales.

## Revisar los valores que no pudieron convertirse

---

Ahora sabemos que la conversión generó nuevos valores `NaT`.

Para investigar esos casos, necesitamos comparar dos cosas:

```
la columna original antes de convertir
la columna convertida a fecha
```

Nos interesan las filas donde la columna original no estaba vacía, pero la columna convertida quedó como `NaT`.

Podemos construir una condición para identificar esos casos.

```
fechas_no_convertidas = (
 df["Transaction Date"].notna()
 & df_fechas["Transaction Date"].isna()
)

fechas_no_convertidas.sum()
```

```
np.int64(301)
```

El resultado debería coincidir con la cantidad de nuevos valores `NaT`.

Ahora podemos observar qué valores originales tenían esas filas.

```
df.loc[fechas_no_convertidas, "Transaction Date"].value_counts(dropna=False)
```

Transaction Date	count
UNKNOWN	159
ERROR	142

```
dtype: int64
```

Esta salida muestra cuáles eran los valores originales que no pudieron interpretarse como fechas.

Si aparecen valores como `"UNKNOWN"` o `"ERROR"`, eso confirma que la columna no tenía solamente fechas válidas y valores faltantes, sino también textos problemáticos.

Esta revisión es importante porque nos ayuda a diferenciar situaciones:

```
NaN original → la fecha ya estaba faltante antes de convertir
NaT por conversión → había un valor escrito, pero no era una fecha interpretable
```

Ambos casos aparecen como faltantes después de la conversión, pero no significan exactamente lo mismo. Esa diferencia puede ser importante si queremos documentar la calidad del dataset.

## Revisar el rango temporal del dataset

Una vez que la columna fue convertida a fecha, podemos empezar a obtener información temporal útil.

Una primera pregunta importante es:

```
¿Qué período cubre el dataset?
```

Para responderla podemos buscar la fecha mínima y la fecha máxima registradas.

```
fecha_minima = df_fechas["Transaction Date"].min()
fecha_maxima = df_fechas["Transaction Date"].max()
```



```
print("Fecha mínima:")
print(fecha_minima)

print()

print("Fecha máxima:")
print(fecha_maxima)
```

```
Fecha mínima:
2023-01-01 00:00:00

Fecha máxima:
2023-12-31 00:00:00
```

La fecha mínima indica el primer momento registrado en el dataset. La fecha máxima indica el último.

Esta revisión nos permite entender el rango temporal de los datos. Por ejemplo, podríamos descubrir que el dataset cubre varios meses, un año completo o solo un período breve.

También puede servir para detectar valores sospechosos. Si esperamos trabajar con ventas de un año determinado y aparece una fecha demasiado antigua o demasiado futura, esa fecha debería revisarse.

Cuando una columna está cargada como texto, este tipo de revisión puede ser menos confiable. Al convertirla a fecha, Pandas puede comparar los valores temporalmente y encontrar correctamente el mínimo y el máximo.

También podemos calcular la amplitud temporal del dataset, es decir, cuánto tiempo transcurre entre la primera y la última fecha registrada.

```
fecha_maxima - fecha_minima
```

```
Timedelta('364 days 00:00:00')
```

El resultado es una diferencia de tiempo.

Este valor nos da una idea general de la extensión temporal del dataset. A partir de esta información, más adelante podríamos decidir si tiene sentido analizar los datos por día, por mes, por trimestre o por otro período.

## Ordenar datos por fecha

Una vez que una columna fue convertida a tipo temporal, podemos usarla para ordenar el dataset cronológicamente.

Ordenar por fecha permite leer las transacciones en el orden en que ocurrieron. Esto puede ser útil para revisar la evolución de los datos, detectar períodos con mayor actividad o preparar análisis posteriores.

Vamos a ordenar el `DataFrame` según `Transaction Date`.

```
df_fechas.sort_values("Transaction Date").head(10)
```

índice	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date
8015	TXN_4801947	Juice	1	3.0	3.0	Digital Wallet	Takeaway	2023-01-01
9063	TXN_9161256	Smoothie	2	4.0	8.0	Digital Wallet	In-store	2023-01-01
7309	TXN_6093955	Tea	5	1.5	NaN	UNKNOWN	Takeaway	2023-01-01
1425	TXN_8842223	Sandwich	5	NaN	20.0	Digital Wallet	In-store	2023-01-01
1777	TXN_7367474	Juice	5	3.0	15.0	Digital Wallet	Takeaway	2023-01-01
2557	TXN_2690222	ERROR	4	3.0	12.0	NaN	NaN	2023-01-01
768	TXN_5728991	Salad	ERROR	5.0	10.0	NaN	NaN	2023-01-01
1806	TXN_2192787	Sandwich	5	4.0	20.0	Cash	In-store	2023-01-01
1912	TXN_5563675	Tea	3	1.5	4.5	Digital Wallet	In-store	2023-01-01
9356	TXN_2104473	Cake	3	3.0	9.0	Digital Wallet	Takeaway	2023-01-01

La salida muestra las primeras transacciones según el orden temporal.

Como la columna ya fue convertida a fecha, Pandas no ordena los valores como simples textos, sino como fechas reales. Esto es importante porque el orden cronológico depende del significado temporal de los valores, no solamente de cómo están escritos.

También podemos mirar las últimas fechas del dataset:

```
df_fechas.sort_values("Transaction Date").tail(10)
```

índice	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date
9838	TXN_6744733	Sandwich	1	4.0	NaN	Cash	Takeaway	NaT
9877	TXN_1964893	Smoothie	1	4.0	4.0	NaN	ERROR	NaT
9885	TXN_4659954	NaN	3	4.0	12.0	Credit Card	In-store	NaT
9907	TXN_8104914	Cookie	1	1.0	1.0	NaN	Takeaway	NaT
9931	TXN_8344810	Smoothie	2	4.0	8.0	NaN	UNKNOWN	NaT
9933	TXN_9460419	Cake	1	3.0	3.0	NaN	Takeaway	NaT
9937	TXN_8253472	Cake	1	3.0	3.0	NaN	NaN	NaT
9949	TXN_3130865	Juice	3	3.0	9.0	NaN	In-store	NaT
9983	TXN_9226047	Smoothie	3	4.0	12.0	Cash	NaN	NaT
9988	TXN_9594133	Cake	5	3.0	NaN	ERROR	NaN	NaT

Esta vista muestra las transacciones ubicadas al final del período registrado.

Al ordenar por fecha, es posible que los valores `NaT` aparezcan al final del resultado. Esto ocurre porque esos registros no tienen una fecha válida para ser ubicados cronológicamente.

Por eso, cuando trabajamos con fechas, debemos recordar que no todos los registros necesariamente podrán participar de un análisis temporal. Las filas con fechas faltantes o no interpretables requieren una decisión específica: conservarlas para otros análisis, excluirlas de ciertos cálculos temporales o revisarlas con más detalle.

## Extraer componentes de una fecha

Una vez que una columna está convertida a tipo fecha, podemos extraer partes específicas de cada valor temporal.

Por ejemplo, de una fecha podemos obtener:

```
año
mes
día
día de la semana
```

Esto es útil porque muchas preguntas de análisis no se hacen sobre la fecha completa, sino sobre alguna de sus partes. Por ejemplo, podríamos querer saber si hay más transacciones en ciertos meses, si algunos días de la semana tienen más actividad o si el dataset cubre un solo año o varios.

En Pandas, para acceder a componentes de una fecha usamos `.dt`.

Primero vamos a crear una copia de trabajo con la columna de fecha ya convertida.

```
df_temporal = df_fechas.copy()

df_temporal["anio"] = df_temporal["Transaction Date"].dt.year
df_temporal["mes"] = df_temporal["Transaction Date"].dt.month
df_temporal["dia"] = df_temporal["Transaction Date"].dt.day

df_temporal[
 [
 "Transaction Date",
 "anio",
 "mes",
 "dia"
]
].head(10)
```

índice	Transaction Date	anio	mes	dia
0	2023-09-08	2023.0	9.0	8.0
1	2023-05-16	2023.0	5.0	16.0
2	2023-07-19	2023.0	7.0	19.0
3	2023-04-27	2023.0	4.0	27.0
4	2023-06-11	2023.0	6.0	11.0
5	2023-03-31	2023.0	3.0	31.0
6	2023-10-06	2023.0	10.0	6.0
7	2023-10-28	2023.0	10.0	28.0
8	2023-07-28	2023.0	7.0	28.0
9	2023-12-31	2023.0	12.0	31.0

Ahora el `DataFrame` tiene nuevas columnas derivadas de la fecha.

La columna `anio` contiene el año de la transacción. La columna `mes` contiene el número de mes. La columna `dia` contiene el día del mes.

Observemos que los valores pueden aparecer con `.0`, por ejemplo `2023.0` o `9.0`. Esto ocurre porque algunas filas tienen `NaT` en la fecha original. Al extraer año, mes

o día de una fecha faltante, Pandas genera valores faltantes en las columnas derivadas, y por eso usa un formato numérico que puede representar esos faltantes.

Lo importante en este punto no es el formato visual, sino la idea: las columnas `año`, `mes` y `día` fueron construidas a partir de la fecha convertida.

Estas columnas no estaban en el dataset original. Las construimos a partir de `Transaction Date`.

Este tipo de transformación es muy útil porque permite preparar el dataset para preguntas temporales más específicas. En lugar de trabajar siempre con la fecha completa, podemos analizar los datos por año, mes o día.

## Extraer el día de la semana

Además del año, el mes y el día del mes, también podemos extraer el día de la semana.

Esto puede ser útil para analizar patrones de comportamiento. Por ejemplo, en un comercio podríamos preguntarnos si hay más transacciones los fines de semana, si ciertos productos se venden más en determinados días o si la actividad cambia entre días laborales y no laborales.

Pandas permite obtener el nombre del día usando `.dt.day_name()`.

```
df_temporal["dia_semana"] = df_temporal["Transaction Date"].dt.day_name()

df_temporal[
 [
 "Transaction Date",
 "dia_semana"
]
].head(10)
```

índice	Transaction Date	dia_semana
0	2023-09-08	Friday
1	2023-05-16	Tuesday
2	2023-07-19	Wednesday
3	2023-04-27	Thursday
4	2023-06-11	Sunday
5	2023-03-31	Friday
6	2023-10-06	Friday

índice	Transaction Date	dia_semana
7	2023-10-28	Saturday
8	2023-07-28	Friday
9	2023-12-31	Sunday

La columna `dia_semana` contiene el nombre del día correspondiente a cada fecha.

Por defecto, en este entorno los nombres de los días aparecen en inglés, por ejemplo `Friday`, `Tuesday` o `Wednesday`. Esto no afecta el análisis: siguen representando correctamente el día de la semana. Más adelante, si necesitáramos presentar los resultados en español, podríamos crear un reemplazo o mapeo de nombres.

Esta información no estaba escrita directamente en el dataset original. La obtuvimos a partir de la columna temporal.

```
df_temporal["numero_dia_semana"] = df_temporal["Transaction Date"].dt.dayofweek

df_temporal[
 [
 "Transaction Date",
 "dia_semana",
 "numero_dia_semana"
]
].head(10)
```

índice	Transaction Date	dia_semana	numero_dia_semana
0	2023-09-08	Friday	4.0
1	2023-05-16	Tuesday	1.0
2	2023-07-19	Wednesday	2.0
3	2023-04-27	Thursday	3.0
4	2023-06-11	Sunday	6.0
5	2023-03-31	Friday	4.0
6	2023-10-06	Friday	4.0
7	2023-10-28	Saturday	5.0
8	2023-07-28	Friday	4.0
9	2023-12-31	Sunday	6.0

En Pandas, `dayofweek` usa la siguiente convención:

```
0 → lunes
1 → martes
```

```
2 → miércoles
3 → jueves
4 → viernes
5 → sábado
6 → domingo
```

El nombre del día resulta más fácil de leer, mientras que el número puede ser útil para ordenar los días correctamente o para construir algunos tipos de análisis.

En este capítulo vamos a usar estas columnas solo para primeras exploraciones temporales. Más adelante podremos profundizar en agrupamientos, resúmenes y visualizaciones por períodos.

## Primeras exploraciones temporales

Después de crear columnas como `anio`, `mes`, `dia` y `dia_semana`, podemos empezar a hacer preguntas simples sobre la distribución temporal de las transacciones.

Por ejemplo, podemos revisar cuántas transacciones hay por año.

```
df_temporal["anio"].value_counts(dropna=False)
```

anio	count
2023.0	9540
NaN	460

```
dtype: int64
```

Este conteo nos permite ver si el dataset contiene registros de un solo año o de varios.

También podemos contar transacciones por mes:

```
df_temporal["mes"].value_counts(dropna=False).sort_index()
```

mes	count
1.0	818
2.0	727
3.0	827
4.0	774
5.0	777

mes	count
6.0	818
7.0	791
8.0	803
9.0	788
10.0	838
11.0	784
12.0	795
NaN	460

`dtype: int64`

En este caso usamos `sort_index()` para que los meses aparezcan ordenados numéricamente, del 1 al 12.

Este tipo de conteo todavía es simple, pero ya muestra el valor de haber convertido la columna de fecha. Ahora podemos analizar partes de la fecha que antes no estaban disponibles como columnas.

También podemos revisar las transacciones por día de la semana:

```
df_temporal["dia_semana"].value_counts(dropna=False)
```

dia_semana	count
Friday	1388
Monday	1382
Sunday	1380
Thursday	1380
Saturday	1358
Wednesday	1341
Tuesday	1311
NaN	460

`dtype: int64`

Este conteo muestra cuántas transacciones corresponden a cada día de la semana.



Es posible que los días no aparezcan en orden calendario, porque `value_counts()` ordena por frecuencia de manera descendente. Más adelante podremos trabajar formas más ordenadas de presentar este tipo de resultados.

Por ahora, lo importante es notar que la columna de fecha nos permitió construir nuevas variables y empezar a explorar patrones temporales.

## Una primera agrupación temporal con `groupby()`

Hasta ahora usamos `value_counts()` para contar transacciones según año, mes o día de la semana.

Otra forma de resumir información por grupos es usar `groupby()`.

`groupby()` es una herramienta muy potente de Pandas. Permite agrupar filas según los valores de una o más columnas y luego calcular resúmenes para cada grupo. Más adelante vamos a estudiarla con mayor profundidad.

En este capítulo la vamos a usar solo de manera introductoria, para mostrar cómo una columna temporal puede servir para resumir información por períodos.

Por ejemplo, podemos contar cuántas transacciones hubo en cada fecha.

```
transacciones_por_fecha = (
 df_temporal
 .groupby("Transaction Date")["Transaction ID"]
 .count()
)

transacciones_por_fecha.head()
```

Transaction Date	Transaction ID
2023-01-01	20
2023-01-02	21
2023-01-03	21
2023-01-04	27
2023-01-05	38

`dtype: int64`

En esta instrucción agrupamos el dataset por `Transaction Date` y contamos cuántos identificadores de transacción aparecen en cada fecha.

El resultado muestra la cantidad de transacciones registradas por día.

También podemos ordenar ese resultado para ver los días con mayor cantidad de transacciones:

```
transacciones_por_fecha.sort_values(ascending=False).head(10)
```

Transaction Date	Transaction ID
2023-02-06	40
2023-06-16	40
2023-09-21	39
2023-07-21	39
2023-07-24	39
2023-03-13	39
2023-01-05	38
2023-10-22	37
2023-01-25	37
2023-06-18	37

`dtype: int64`

Esta salida permite identificar las fechas con más transacciones registradas.

El uso de `groupby()` nos permite pasar de una tabla de transacciones individuales a un resumen por período. Esa es una de las razones por las que convertir correctamente las fechas es tan importante: una vez que Pandas entiende la columna como temporal, podemos construir análisis organizados por día, mes, año u otros períodos.

## Fechas faltantes o no interpretables

Cuando convertimos `Transaction Date`, vimos que no todas las filas quedaron con una fecha válida.

Antes de la conversión había valores faltantes reales. Después de convertir con `errors="coerce"`, algunos valores adicionales pasaron a ser `NaT` porque no pudieron interpretarse como fechas.

Esto es importante porque las filas con `NaT` no pueden ubicarse correctamente en una línea de tiempo. No pueden asignarse a un año, a un mes, a un día ni a un día de la semana.

Podemos revisar cuántas fechas no interpretables o faltantes quedaron en `df_temporal`.

```
df_temporal["Transaction Date"].isna().sum()
```

```
np.int64(460)
```

Ese conteo nos indica cuántas filas no tienen una fecha válida después de la conversión.

También podemos observar algunas de esas filas.

```
df_temporal[df_temporal["Transaction Date"].isna()].head(10)
```

índice	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date	an
11	TXN_3051279	Sandwich	2	4.0	8.0	Credit Card	Takeaway	NaT	Na
29	TXN_7640952	Cake	4	3.0	12.0	Digital Wallet	Takeaway	NaT	Na
33	TXN_7710508	UNKNOWN	5	1.0	5.0	Cash	NaN	NaT	Na
77	TXN_2091733	Salad	1	5.0	5.0	NaN	In-store	NaT	Na
103	TXN_7028009	Cake	4	3.0	12.0	NaN	Takeaway	NaT	Na
104	TXN_7447872	Juice	2	NaN	6.0	NaN	NaN	NaT	Na
115	TXN_1001832	Salad	2	5.0	10.0	Cash	Takeaway	NaT	Na
142	TXN_7943008	Coffee	1	2.0	2.0	Credit Card	NaN	NaT	Na
160	TXN_1093800	Sandwich	3	4.0	12.0	Cash	Takeaway	NaT	Na
175	TXN_6463132	Cookie	5	1.0	5.0	Credit Card	Takeaway	NaT	Na

Estas filas no deberían desaparecer automáticamente del dataset. Pueden seguir siendo útiles para otros análisis, como productos vendidos, métodos de pago o importes, siempre que esas columnas tengan datos válidos.

Sin embargo, para análisis temporales específicos, estas filas requieren una decisión. Podemos excluirlas de ciertos conteos por fecha, conservarlas como registros sin fecha, revisar si el valor original era "UNKNOWN" o "ERROR", o intentar recuperar la fecha desde otra fuente si existiera.

La decisión depende del contexto y del objetivo del análisis.

En este capítulo no vamos a corregir esas fechas. Lo importante es reconocer que una conversión temporal puede dejar filas fuera del análisis por período, y que eso debe ser registrado.

## Errores frecuentes al trabajar con fechas

---

Al trabajar con fechas, uno de los errores más frecuentes es asumir que una columna es temporal solo porque sus valores se ven como fechas.

Una fecha escrita como "2023-09-08" puede parecer correcta para una persona, pero si Pandas la cargó como `object`, todavía es texto. En ese estado, la columna no permite aprovechar correctamente las herramientas temporales de Pandas.

Por eso, antes de analizar fechas, conviene revisar el tipo de dato:

```
df["Transaction Date"].dtype
```

Otro error frecuente es convertir fechas sin verificar cuántos valores quedaron como `NaT`. Cuando usamos `pd.to_datetime()` con `errors="coerce"`, los valores que no pueden interpretarse como fechas se convierten en `NaT`. Esto evita que el código se detenga, pero también puede ocultar problemas si no revisamos el resultado.

Por eso, después de convertir, conviene comparar faltantes antes y después:

```
df["Transaction Date"].isna().sum()
df_fechas["Transaction Date"].isna().sum()
```

También puede ser un error eliminar automáticamente las filas con fechas faltantes o no interpretables. Esas filas no sirven para ciertos análisis temporales, pero pueden seguir siendo útiles para otros análisis. Por ejemplo, una venta sin fecha válida todavía podría aportar información sobre producto, cantidad, importe o método de pago.

Otro punto importante es no abrir demasiadas conclusiones solo a partir de conteos temporales simples. Saber que un día tuvo más transacciones que otro puede ser una señal interesante, pero antes de sacar conclusiones deberíamos considerar el

contexto: cantidad de días registrados, posibles datos faltantes, períodos incompletos o sesgos de carga.

Finalmente, cuando extraemos columnas como `año`, `mes`, `día` o `día_semana`, debemos recordar que esas columnas dependen de que la fecha original esté correctamente convertida. Si una fila tiene `NaT`, las columnas derivadas también quedarán sin información válida.

Una buena rutina para trabajar con fechas podría ser:

```
revisar el tipo de dato original
convertir la columna con pd.to_datetime()
comparar faltantes antes y después
revisar valores no interpretables
analizar el rango temporal
crear columnas temporales derivadas
verificar los resultados antes de sacar conclusiones
```

Trabajar con fechas no consiste solamente en cambiar el tipo de una columna. Implica asegurarse de que los valores temporales sean interpretables, revisar qué registros quedan fuera del análisis temporal y usar las nuevas variables con cuidado.

## Resumen del capítulo

---

En este capítulo trabajamos con fechas y datos temporales.

Partimos de una idea central: una fecha escrita como texto no es lo mismo que una fecha interpretada como dato temporal por Pandas. Aunque valores como `"2023-09-08"` parezcan fechas válidas a simple vista, si la columna está cargada como `object`, Pandas todavía la trata como texto.

Primero revisamos la columna `Transaction Date` y comprobamos que había sido cargada como `object`:

```
df["Transaction Date"].dtype
```

También observamos algunos valores y contamos los faltantes antes de la conversión. En ese momento detectamos 159 valores faltantes reales en la columna original.

Luego convertimos la columna usando `pd.to_datetime()`:

```
df_fechas["Transaction Date"] = pd.to_datetime(
 df_fechas["Transaction Date"],
 errors="coerce"
)
```

El parámetro `errors="coerce"` permitió convertir en `NaT` los valores que no podían interpretarse como fechas. Después de la conversión, la cantidad de valores faltantes temporales aumentó a 460. Esto indicó que, además de los faltantes originales, había valores escritos que no podían interpretarse como fechas.

Para investigar esos casos, construimos una condición que identificó las filas donde la columna original no estaba vacía, pero la columna convertida quedó como `NaT`:

```
fechas_no_convertidas = (
 df["Transaction Date"].notna()
 & df_fechas["Transaction Date"].isna()
)
```

Ese paso nos permitió diferenciar entre valores que ya estaban faltantes antes de convertir y valores que se volvieron `NaT` durante la conversión.

Después revisamos el rango temporal del dataset usando la fecha mínima y la fecha máxima. Esto nos permitió saber qué período cubren los datos y también pensar en posibles valores fuera de rango.

Más adelante ordenamos el dataset por fecha y vimos que, una vez convertida la columna, Pandas puede ordenar cronológicamente los registros.

Luego creamos nuevas columnas temporales a partir de `Transaction Date`:

```
df_temporal["anio"] = df_temporal["Transaction Date"].dt.year
df_temporal["mes"] = df_temporal["Transaction Date"].dt.month
df_temporal["dia"] = df_temporal["Transaction Date"].dt.day
```

También extraímos el día de la semana:

```
df_temporal["dia_semana"] = df_temporal["Transaction Date"].dt.day_name()
df_temporal["numero_dia_semana"] = df_temporal["Transaction Date"].dt.dayofweek
```

Estas nuevas columnas permiten hacer preguntas temporales más específicas, como cuántas transacciones hubo por año, por mes o por día de la semana.

Finalmente hicimos una primera agrupación temporal con `groupby()`. Usamos esta herramienta de manera introductoria para contar transacciones por fecha:

```
transacciones_por_fecha = (
 df_temporal
 .groupby("Transaction Date")["Transaction ID"]
 .count()
)
```

Más adelante vamos a estudiar `groupby()` con mayor profundidad. En este capítulo lo usamos solo para mostrar que una columna temporal bien convertida permite resumir información por períodos.

La idea principal de este capítulo fue:

Trabajar con fechas requiere convertirlas, verificarlas y luego usarlas para construir nuevas variables temporales.

Las fechas permiten analizar evolución, períodos, patrones y distribución temporal. Pero antes de hacerlo, necesitamos asegurarnos de que Pandas las interprete correctamente y de que sepamos qué registros quedaron sin fecha válida.

## Próximo paso

---

Ya trabajamos con valores faltantes, duplicados, limpieza de textos, conversión de tipos y fechas.

El siguiente paso será comenzar a integrar varias de estas tareas en un proceso de limpieza más completo. En lugar de tratar cada problema por separado, vamos a trabajar sobre un flujo de preparación de datos paso a paso.

La idea será partir de un dataset con varios problemas, aplicar transformaciones ordenadas y verificar el resultado final. Esto nos acercará a una situación más realista de trabajo: preparar una tabla para que quede lista para analizar.

## Capítulo 20 — Estandarizar nombres y crear nuevas columnas

---

### Breve repaso

---

En los capítulos anteriores avanzamos sobre varias tareas centrales de preparación de datos. Trabajamos con valores faltantes, analizamos distintas estrategias para tratarlos, revisamos duplicados, limpiamos textos y categorías, convertimos columnas al tipo de dato adecuado y profundizamos en el trabajo con fechas.

Ese recorrido nos permitió ver una idea importante: preparar un dataset no consiste en aplicar una sola transformación, sino en tomar varias decisiones conectadas entre sí. Cada paso mejora algún aspecto de la tabla y deja los datos en mejores condiciones para el análisis.

En este capítulo vamos a trabajar con dos tareas que suelen aparecer en una etapa avanzada de preparación: estandarizar nombres de columnas y crear nuevas columnas útiles para el análisis.

Ya trabajamos antes con nombres de columnas, así que no vamos a tratar este tema como algo completamente nuevo. Esta vez lo retomaremos dentro de un flujo de limpieza más realista. La idea será justificar por qué conviene usar nombres consistentes y aplicar una estrategia simple para transformar columnas como `Transaction ID`, `Price Per Unit` o `Transaction Date` en nombres más cómodos para trabajar.

Luego vamos a crear nuevas columnas a partir de datos ya existentes. Por ejemplo, si tenemos una columna de cantidad y otra de precio unitario, podemos calcular un importe total. También podemos crear columnas de validación para comparar un valor registrado con un valor calculado, o columnas derivadas de una fecha.

Estas nuevas columnas no son adornos. Muchas veces permiten responder preguntas que el dataset original no podía responder directamente, o ayudan a verificar la consistencia interna de los datos.

Al finalizar este capítulo deberías poder:

- Comprender por qué conviene estandarizar nombres de columnas.
- Renombrar columnas usando un criterio uniforme.
- Verificar que los nuevos nombres quedaron correctamente aplicados.
- Crear columnas nuevas a partir de operaciones entre columnas existentes.



- Crear columnas de validación para comparar valores registrados y calculados.
- Crear columnas derivadas de una fecha.
- Comprender que crear columnas nuevas puede enriquecer el análisis y también ayudar a controlar la calidad de los datos.

## Dataset de trabajo

---

Para este capítulo vamos a volver al dataset real **Cafe Sales — Dirty Data for Cleaning Training**.

Este dataset nos permite trabajar con varias situaciones ya vistas: columnas con nombres que contienen espacios y mayúsculas, columnas numéricas que necesitan conversión, una columna de fecha y una relación interna entre cantidad, precio unitario e importe total.

Como venimos haciendo en esta parte del recorrido, vamos a cargar el dataset desde Kaggle. En Google Colab, `kagglehub` ya suele estar disponible, por lo que comenzamos directamente con las importaciones necesarias.

```
import kagglehub
import pandas as pd
import os

ruta_dataset = kagglehub.dataset_download(
 "ahmedmohamed2003/cafe-sales-dirty-data-for-cleaning-training"
)

archivos = os.listdir(ruta_dataset)

archivos_csv = [
 archivo for archivo in archivos
 if archivo.endswith(".csv")
]

ruta_csv = os.path.join(ruta_dataset, archivos_csv[0])

df = pd.read_csv(ruta_csv)

df.head()
```

Using Colab cache for faster access to the 'cafe-sales-dirty-data-for-cleaning-training' dataset.

índice	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date
0	TXN_1961373	Coffee	2	2.0	4.0	Credit Card	Takeaway	2023-09-08
1	TXN_4977031	Cake	4	3.0	12.0	Cash	In-store	2023-05-16
2	TXN_4271903	Cookie	4	1.0	ERROR	Credit Card	In-store	2023-07-19
3	TXN_7034554	Salad	2	5.0	10.0	UNKNOWN	UNKNOWN	2023-04-27
4	TXN_3160411	Coffee	2	2.0	4.0	Digital Wallet	In-store	2023-06-11

La salida de `head()` nos permite confirmar que el archivo fue cargado correctamente.

En este capítulo vamos a trabajar sobre una copia del dataset. De esa manera podemos transformar nombres, convertir columnas y crear nuevas variables sin perder el punto de partida original.

## Por qué estandarizar nombres de columnas

Los nombres de columnas son una parte importante de la estructura del dataset.

En el archivo original encontramos nombres como:

```
Transaction ID
Price Per Unit
Total Spent
Payment Method
Transaction Date
```

Estos nombres son comprensibles, pero no siempre son los más cómodos para trabajar en código. Tienen espacios, usan mayúsculas y están escritos con un estilo más cercano a un encabezado de planilla que a un nombre de variable.

En Pandas podemos trabajar con nombres así sin problema, usando corchetes:

```
df["Price Per Unit"]
```

Sin embargo, en un flujo de análisis largo puede ser más cómodo usar nombres consistentes, en minúsculas y con guiones bajos:

```
df["price_per_unit"]
```

Estandarizar nombres no cambia los valores del dataset. Lo que cambia es la forma en que nos referimos a las columnas. Esto puede reducir errores de escritura, hacer que el código sea más legible y facilitar la aplicación de transformaciones encadenadas.

En esta etapa vamos a adoptar un criterio simple:

```
usar minúsculas
reemplazar espacios por guiones bajos
mantener nombres descriptivos
```

Ese criterio suele conocerse como estilo `snake_case`, porque las palabras se escriben en minúsculas y separadas por guiones bajos.

```
df.columns
```

```
Index(['Transaction ID', 'Item', 'Quantity', 'Price Per Unit', 'Total Spent',
 'Payment Method', 'Location', 'Transaction Date'],
 dtype='object')
```

La salida muestra los nombres originales de las columnas.

Antes de crear nuevas columnas o aplicar transformaciones más largas, vamos a construir una copia del dataset y renombrar sus columnas con un criterio más uniforme.

## Renombrar columnas con un criterio uniforme

---

Para estandarizar los nombres de columnas podemos usar `rename()`.

En este caso vamos a crear una copia llamada `df_trabajo` y vamos a renombrar las columnas principales usando nombres en minúsculas y con guiones bajos.

No estamos cambiando los datos. Solo estamos cambiando los nombres con los que vamos a referirnos a cada columna.

```
df_trabajo = df.copy()

df_trabajo = df_trabajo.rename(columns={
 "Transaction ID": "transaction_id",
 "Item": "item",
 "Quantity": "quantity",
 "Price Per Unit": "price_per_unit",
 "Total Spent": "total_spent",
 "Payment Method": "payment_method",
```

```

 "Location": "location",
 "Transaction Date": "transaction_date"
})

df_trabajo.head()

```

índice	transaction_id	item	quantity	price_per_unit	total_spent	payment_method	location
0	TXN_1961373	Coffee	2	2.0	4.0	Credit Card	Takeawa
1	TXN_4977031	Cake	4	3.0	12.0	Cash	In-store
2	TXN_4271903	Cookie	4	1.0	ERROR	Credit Card	In-store
3	TXN_7034554	Salad	2	5.0	10.0	UNKNOWN	UNKNO
4	TXN_3160411	Coffee	2	2.0	4.0	Digital Wallet	In-store

Ahora el `DataFrame` tiene nombres de columnas más cómodos para trabajar.

Podemos verificar la estructura revisando nuevamente los nombres:

```
df_trabajo.columns
```

```

Index(['transaction_id', 'item', 'quantity', 'price_per_unit', 'total_spent',
 'payment_method', 'location', 'transaction_date'],
 dtype='object')

```

Los nombres quedaron en un formato más uniforme.

Este paso parece pequeño, pero puede facilitar mucho el trabajo posterior. Por ejemplo, al escribir transformaciones más largas, nombres como `price_per_unit` o `transaction_date` suelen ser más cómodos y menos propensos a errores que nombres con espacios y mayúsculas.

También prepara el terreno para crear nuevas columnas con nombres consistentes.

## Preparar columnas antes de crear nuevas variables

Antes de crear columnas nuevas a partir de operaciones matemáticas, necesitamos asegurarnos de que las columnas involucradas sean realmente numéricas.

En nuestro dataset, las columnas `quantity`, `price_per_unit` y `total_spent` deberían representar cantidades o importes. Sin embargo, como vimos antes, el archivo original puede contener valores problemáticos como `"UNKNOWN"` o `"ERROR"`, y por eso Pandas puede haberlas cargado como texto.

Si intentamos multiplicar columnas que todavía están como texto, el resultado puede fallar o no representar un cálculo numérico real. Por eso, antes de crear una columna calculada, vamos a convertir esas variables a formato numérico.

```
columnas_numericas = [
 "quantity",
 "price_per_unit",
 "total_spent"
]

for columna in columnas_numericas:
 df_trabajo[columna] = pd.to_numeric(
 df_trabajo[columna],
 errors="coerce"
)

df_trabajo[columnas_numericas].dtypes
```

índice	0
quantity	float64
price_per_unit	float64
total_spent	float64

dtype: object

Ahora las columnas seleccionadas fueron convertidas a tipo numérico.

Usamos `errors="coerce"` para que los valores que no puedan convertirse, como `"UNKNOWN"` o `"ERROR"`, pasen a ser `NaN`. Esto permite continuar con los cálculos, pero también nos obliga a revisar los faltantes resultantes.

Antes de crear nuevas columnas, conviene verificar cuántos valores faltantes tenemos en estas variables.

```
df_trabajo[columnas_numericas].isna().sum()
```

índice	0
quantity	479
price_per_unit	533
total_spent	502

dtype: int64

Esta revisión nos recuerda que convertir tipos no elimina los problemas del dataset. Algunos valores quedan como faltantes porque no pudieron interpretarse como números.

Aun así, tener estas columnas en formato numérico nos permite crear nuevas variables calculadas y verificar relaciones internas.

## Crear una columna calculada

Una vez que `quantity` y `price_per_unit` están en formato numérico, podemos crear una nueva columna a partir de ellas.

En un dataset de ventas, una relación esperada es:

```
importe calculado = cantidad × precio unitario
```

En nuestro caso, vamos a crear una columna llamada `calculated_total`.

Esta columna representa el total que se obtiene al multiplicar la cantidad vendida por el precio unitario.

```
df_trabajo["calculated_total"] = (
 df_trabajo["quantity"] * df_trabajo["price_per_unit"]
)

df_trabajo[
 [
 "quantity",
 "price_per_unit",
 "total_spent",
 "calculated_total"
]
].head(10)
```

índice	quantity	price_per_unit	total_spent	calculated_total
0	2.0	2.0	4.0	4.0
1	4.0	3.0	12.0	12.0
2	4.0	1.0	NaN	4.0
3	2.0	5.0	10.0	10.0
4	2.0	2.0	4.0	4.0
5	5.0	4.0	20.0	20.0
6	3.0	3.0	9.0	9.0

índice	quantity	price_per_unit	total_spent	calculated_total
7	4.0	4.0	16.0	16.0
8	5.0	3.0	15.0	15.0
9	5.0	4.0	20.0	20.0

La nueva columna `calculated_total` no estaba en el dataset original. La construimos a partir de columnas existentes.

Este tipo de columna puede servir para enriquecer el análisis, pero también para validar datos. En este caso, el dataset ya tenía una columna llamada `total_spent`, que representa el total registrado de la transacción.

Entonces podemos comparar dos valores:

```
total_spent → total registrado en el archivo
calculated_total → total calculado a partir de cantidad y precio unitario
```

Si ambos valores coinciden, la transacción parece consistente desde el punto de vista del importe. Si no coinciden, puede haber un error en alguna de las columnas o un problema de carga.

Esta es una idea muy importante: crear columnas nuevas no solo permite agregar información, también puede ayudar a controlar la calidad del dataset.

## Crear una columna de validación

La columna `calculated_total` nos permite comparar el importe registrado con el importe calculado.

Para eso podemos crear una nueva columna booleana llamada `total_matches`.

Esta columna indicará si `total_spent` coincide con `calculated_total`.

```
df_trabajo["total_matches"] = (
 df_trabajo["total_spent"] == df_trabajo["calculated_total"]
)

df_trabajo[
 [
 "quantity",
 "price_per_unit",
 "total_spent",
 "calculated_total",
 "total_matches"
]
]
```

```
]
].head(10)
```

índice	quantity	price_per_unit	total_spent	calculated_total	total_matches
0	2.0	2.0	4.0	4.0	True
1	4.0	3.0	12.0	12.0	True
2	4.0	1.0	NaN	4.0	False
3	2.0	5.0	10.0	10.0	True
4	2.0	2.0	4.0	4.0	True
5	5.0	4.0	20.0	20.0	True
6	3.0	3.0	9.0	9.0	True
7	4.0	4.0	16.0	16.0	True
8	5.0	3.0	15.0	15.0	True
9	5.0	4.0	20.0	20.0	True

La columna `total_matches` contiene valores `True` o `False`.

`True` significa que el total registrado coincide con el total calculado. `False` significa que no coinciden.

Sin embargo, debemos interpretar esta columna con cuidado. Si alguna de las columnas necesarias para el cálculo tiene un valor faltante, el resultado de la comparación puede ser `False`, aunque no necesariamente exista una inconsistencia real. Puede ocurrir simplemente que no tengamos suficiente información para validar esa fila.

Por eso, muchas veces conviene distinguir entre tres situaciones:

```
el total coincide
el total no coincide
no hay datos suficientes para comparar
```

Vamos a crear una versión más expresiva de esta validación en la siguiente sección.

## **Distinguir coincidencias, diferencias y datos insuficientes**

La columna `total_matches` nos da una primera validación, pero es demasiado simple.

Cuando devuelve `False`, pueden estar ocurriendo dos cosas distintas. Tal vez `total_spent` y `calculated_total` realmente no coinciden. Pero también puede pasar que falte alguno de los datos necesarios para hacer la comparación.



Por ejemplo, si falta `quantity`, `price_per_unit` o `total_spent`, no tenemos información suficiente para validar esa fila.

Para evitar confusiones, podemos crear una columna más descriptiva llamada `estado_total`.

```
df_trabajo["estado_total"] = "sin_datos_suficientes"

condicion_datos_completos = (
 df_trabajo["quantity"].notna()
 & df_trabajo["price_per_unit"].notna()
 & df_trabajo["total_spent"].notna()
)

condicion_coincide = (
 condicion_datos_completos
 & (df_trabajo["total_spent"] == df_trabajo["calculated_total"])
)

condicion_no_coincide = (
 condicion_datos_completos
 & (df_trabajo["total_spent"] != df_trabajo["calculated_total"])
)

df_trabajo.loc[condicion_coincide, "estado_total"] = "coincide"
df_trabajo.loc[condicion_no_coincide, "estado_total"] = "no_coincide"

df_trabajo[
 [
 "quantity",
 "price_per_unit",
 "total_spent",
 "calculated_total",
 "estado_total"
]
].head(15)
```

índice	quantity	price_per_unit	total_spent	calculated_total	estado_total
0	2.0	2.0	4.0	4.0	coincide
1	4.0	3.0	12.0	12.0	coincide
2	4.0	1.0	NaN	4.0	sin_datos_suficientes
3	2.0	5.0	10.0	10.0	coincide
4	2.0	2.0	4.0	4.0	coincide
5	5.0	4.0	20.0	20.0	coincide
6	3.0	3.0	9.0	9.0	coincide
7	4.0	4.0	16.0	16.0	coincide

índice	quantity	price_per_unit	total_spent	calculated_total	estado_total
8	5.0	3.0	15.0	15.0	coincide
9	5.0	4.0	20.0	20.0	coincide
10	5.0	5.0	25.0	25.0	coincide
11	2.0	4.0	8.0	8.0	coincide
12	2.0	4.0	8.0	8.0	coincide
13	5.0	1.0	5.0	5.0	coincide
14	2.0	1.5	3.0	3.0	coincide

Ahora la validación es más clara.

La columna `estado_total` puede tomar tres valores:

```
coincide
no_coincide
sin_datos_suficientes
```

Esto evita interpretar como error una fila que simplemente no tiene datos suficientes para ser evaluada.

Podemos revisar cuántos casos hay en cada grupo.

```
df_trabajo["estado_total"].value_counts(dropna=False)
```

estado_total	count
coincide	8544
sin_datos_suficientes	1456

```
dtype: int64
```

Este conteo nos permite evaluar la consistencia interna del dataset. En la salida aparecen casos `coincide` y `sin_datos_suficientes`, pero no aparecen casos `no_coincide`. Eso indica que, dentro de las filas que tienen los datos necesarios para hacer la comparación, el total registrado coincide con el total calculado.

Las filas clasificadas como `sin_datos_suficientes` no indican una diferencia entre importes, sino falta de información para poder validar la relación.

Si hubiese muchas filas en `no_coincide`, deberíamos revisar si existe un problema en `quantity`, `price_per_unit`, `total_spent` o en la forma en que fue registrada la transacción.

Si hay muchas filas en `sin_datos_suficientes`, el problema principal no es una diferencia entre importes, sino la falta de información necesaria para validar.

Este tipo de columna es muy útil porque convierte una comparación técnica en una señal más interpretable para el análisis.

## Crear columnas a partir de una fecha

Además de crear columnas numéricas y columnas de validación, también podemos crear nuevas columnas a partir de una fecha.

En nuestro dataset, la columna `transaction_date` todavía necesita convertirse a formato temporal. Una vez convertida, podremos extraer información como el año, el mes o el día de la semana.

Primero convertimos la columna:

```
df_trabajo["transaction_date"] = pd.to_datetime(
 df_trabajo["transaction_date"],
 errors="coerce"
)

df_trabajo["transaction_date"].dtype
```

```
dtype('<M8[ns]')
```

Ahora `transaction_date` está en formato temporal.

Podemos crear algunas columnas derivadas:

```
df_trabajo["year"] = df_trabajo["transaction_date"].dt.year
df_trabajo["month"] = df_trabajo["transaction_date"].dt.month
df_trabajo["day_of_week"] = df_trabajo["transaction_date"].dt.day_name()

df_trabajo[
 [
 "transaction_date",
 "year",
 "month",
 "day_of_week"
]
].head(10)
```

índice	transaction_date	year	month	day_of_week
0	2023-09-08	2023.0	9.0	Friday

índice	transaction_date	year	month	day_of_week
1	2023-05-16	2023.0	5.0	Tuesday
2	2023-07-19	2023.0	7.0	Wednesday
3	2023-04-27	2023.0	4.0	Thursday
4	2023-06-11	2023.0	6.0	Sunday
5	2023-03-31	2023.0	3.0	Friday
6	2023-10-06	2023.0	10.0	Friday
7	2023-10-28	2023.0	10.0	Saturday
8	2023-07-28	2023.0	7.0	Friday
9	2023-12-31	2023.0	12.0	Sunday

Estas columnas derivadas permiten analizar las transacciones desde una perspectiva temporal.

La columna `year` indica el año de la transacción. La columna `month` indica el número de mes. La columna `day_of_week` indica el día de la semana.

Observemos que `year` y `month` pueden aparecer con decimales, como `2023.0` o `9.0`. Esto ocurre porque algunas fechas no pudieron convertirse y quedaron como `NaT`. Al derivar columnas desde fechas faltantes, Pandas necesita usar un formato que también pueda representar esos valores ausentes.

También vemos que los nombres de los días aparecen en inglés, como `Friday`, `Tuesday` o `Wednesday`. Esto depende del entorno y no afecta el análisis: los valores siguen representando correctamente el día de la semana. Si más adelante necesitaríamos presentar esos nombres en español, podríamos aplicar un reemplazo o mapeo de valores.

Al igual que con las columnas numéricas, estas nuevas variables no estaban explícitas en el dataset original. Las construimos a partir de una columna existente.

Crear columnas temporales puede ser útil para analizar períodos, detectar patrones por mes o estudiar diferencias entre días de la semana.

Si algunas filas tienen fechas faltantes o no interpretables, las columnas derivadas también quedarán con valores faltantes. Por eso, cada nueva columna debe interpretarse teniendo en cuenta la calidad de la columna original.

## Verificar las nuevas columnas

Después de renombrar columnas, convertir tipos y crear nuevas variables, conviene revisar cómo quedó el `DataFrame`.

La verificación nos ayuda a confirmar que las transformaciones se aplicaron correctamente y que las nuevas columnas tienen sentido.

Podemos comenzar revisando los nombres de columnas actuales.

```
df_trabajo.columns
```

```
Index(['transaction_id', 'item', 'quantity', 'price_per_unit', 'total_spent',
 'payment_method', 'location', 'transaction_date', 'calculated_total',
 'total_matches', 'estado_total', 'year', 'month', 'day_of_week'],
 dtype='object')
```

Ahora el `DataFrame` contiene tanto columnas originales renombradas como columnas nuevas creadas durante el proceso.

Entre las nuevas columnas tenemos:

```
calculated_total
total_matches
estado_total
year
month
day_of_week
```

También podemos revisar una vista general de algunas columnas importantes.

```
df_trabajo[
 [
 "transaction_id",
 "item",
 "quantity",
 "price_per_unit",
 "total_spent",
 "calculated_total",
 "estado_total",
 "transaction_date",
 "year",
 "month",
 "day_of_week"
]
].head(10)
```

índice	transaction_id	item	quantity	price_per_unit	total_spent	calculated_total	estado
0	TXN_1961373	Coffee	2.0	2.0	4.0	4.0	coinced
1	TXN_4977031	Cake	4.0	3.0	12.0	12.0	coinced
2	TXN_4271903	Cookie	4.0	1.0	NaN	4.0	sin_d
3	TXN_7034554	Salad	2.0	5.0	10.0	10.0	coinced
4	TXN_3160411	Coffee	2.0	2.0	4.0	4.0	coinced
5	TXN_2602893	Smoothie	5.0	4.0	20.0	20.0	coinced
6	TXN_4433211	UNKNOWN	3.0	3.0	9.0	9.0	coinced
7	TXN_6699534	Sandwich	4.0	4.0	16.0	16.0	coinced
8	TXN_4717867	NaN	5.0	3.0	15.0	15.0	coinced
9	TXN_2064365	Sandwich	5.0	4.0	20.0	20.0	coinced

Esta vista permite observar, en una misma tabla, las variables originales ya preparadas y las nuevas columnas derivadas.

También podemos revisar los tipos de datos:

```
df_trabajo.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
Column Non-Null Count Dtype
--- -
0 transaction_id 10000 non-null object
1 item 9667 non-null object
2 quantity 9521 non-null float64
3 price_per_unit 9467 non-null float64
4 total_spent 9498 non-null float64
5 payment_method 7421 non-null object
6 location 6735 non-null object
7 transaction_date 9540 non-null datetime64[ns]
8 calculated_total 9006 non-null float64
9 total_matches 10000 non-null bool
10 estado_total 10000 non-null object
11 year 9540 non-null float64
12 month 9540 non-null float64
13 day_of_week 9540 non-null object
dtypes: bool(1), datetime64[ns](1), float64(6), object(6)
memory usage: 1.0+ MB
```

Esta revisión final es importante porque el `DataFrame` cambió bastante desde la carga inicial.

Primero estandarizamos nombres de columnas. Luego convertimos columnas numéricas. Después creamos columnas calculadas y columnas de validación. Finalmente convertimos la fecha y generamos columnas temporales.

Cada una de esas transformaciones modifica la estructura del dataset. Por eso, después de crear nuevas columnas, conviene revisar tanto los nombres como los tipos de datos y algunos ejemplos concretos.

Preparar datos no es solo transformar. También es verificar que cada transformación haya dejado el dataset en mejores condiciones para el análisis.

## Errores frecuentes al estandarizar nombres y crear columnas

---

Al estandarizar nombres de columnas, uno de los errores más frecuentes es cambiar nombres sin verificar después la estructura del `DataFrame`.

Si renombramos una columna y luego intentamos usar el nombre anterior, Pandas no va a encontrarla. Por eso, después de aplicar `rename()`, conviene revisar `df.columns` y asegurarnos de usar los nombres nuevos en las celdas siguientes.

Otro error posible es crear nuevas columnas antes de preparar los tipos de datos. Por ejemplo, si `quantity` o `price_per_unit` todavía estuvieran cargadas como texto, la columna `calculated_total` no sería confiable. Antes de crear variables numéricas derivadas, debemos convertir las columnas necesarias y revisar si aparecieron nuevos faltantes.

También debemos tener cuidado con las columnas de validación. Una comparación como:

```
df_trabajo["total_spent"] == df_trabajo["calculated_total"]
```

puede devolver `False` cuando falta algún dato necesario para comparar. Eso no siempre significa que haya una inconsistencia real. Por eso creamos `estado_total`, que distingue entre coincidencia, diferencia y falta de datos suficientes.

Otro error frecuente es crear muchas columnas nuevas sin un propósito claro. Cada nueva variable debería tener una función: enriquecer el análisis, simplificar una consulta, validar otra columna o preparar el dataset para una etapa posterior.

Finalmente, al crear columnas derivadas de fechas, debemos recordar que dependen de la calidad de la fecha original. Si `transaction_date` no pudo convertirse

correctamente y quedó como `NaT`, las columnas derivadas como `year`, `month` o `day_of_week` también quedarán sin información válida.

Una buena rutina para esta etapa podría ser:

```
renombrar columnas con un criterio claro
verificar los nombres nuevos
convertir columnas necesarias antes de calcular
crear columnas derivadas con un propósito definido
crear columnas de validación cuando haya relaciones internas
verificar tipos, faltantes y ejemplos concretos
```

Estandarizar nombres y crear columnas nuevas no son pasos aislados. Forman parte de un proceso de preparación que busca dejar el dataset más claro, más consistente y más útil para el análisis.

## Resumen del capítulo

---

En este capítulo trabajamos con dos tareas importantes dentro de la preparación de datos: estandarizar nombres de columnas y crear nuevas columnas útiles para el análisis.

Primero retomamos la importancia de los nombres de columnas. El dataset original tenía nombres como `Transaction ID`, `Price Per Unit`, `Total Spent`, `Payment Method` y `Transaction Date`. Esos nombres son comprensibles, pero tienen espacios y mayúsculas, lo que puede volver el código menos cómodo de escribir.

Por eso creamos una copia del dataset y renombramos las columnas con un criterio uniforme:

```
df_trabajo = df_trabajo.rename(columns={
 "Transaction ID": "transaction_id",
 "Item": "item",
 "Quantity": "quantity",
 "Price Per Unit": "price_per_unit",
 "Total Spent": "total_spent",
 "Payment Method": "payment_method",
 "Location": "location",
 "Transaction Date": "transaction_date"
})
```

El criterio elegido fue usar nombres en minúsculas, sin espacios y con guiones bajos. Este estilo facilita la escritura del código y ayuda a mantener una estructura más consistente.



Después preparamos algunas columnas antes de crear nuevas variables. Convertimos `quantity`, `price_per_unit` y `total_spent` a formato numérico usando `pd.to_numeric()` con `errors="coerce"`:

```
for columna in columnas_numericas:
 df_trabajo[columna] = pd.to_numeric(
 df_trabajo[columna],
 errors="coerce"
)
```

Este paso fue necesario porque no conviene crear columnas calculadas a partir de valores que todavía están cargados como texto.

Luego creamos una nueva columna llamada `calculated_total`:

```
df_trabajo["calculated_total"] = (
 df_trabajo["quantity"] * df_trabajo["price_per_unit"]
)
```

Esta columna representa el total calculado a partir de la cantidad y el precio unitario.

Como el dataset ya tenía una columna `total_spent`, pudimos comparar el total registrado con el total calculado. Primero creamos una validación simple llamada `total_matches`, pero luego vimos que esa comparación podía ser insuficiente cuando faltaban datos.

Por eso creamos una columna más expresiva, `estado_total`, con tres estados posibles:

```
coincide
no_coincide
sin_datos_suficientes
```

Esta columna permitió distinguir entre una inconsistencia real y una fila que no tenía datos suficientes para ser evaluada.

También convertimos `transaction_date` a formato temporal y creamos columnas derivadas de la fecha:

```
df_trabajo["year"] = df_trabajo["transaction_date"].dt.year
df_trabajo["month"] = df_trabajo["transaction_date"].dt.month
df_trabajo["day_of_week"] = df_trabajo["transaction_date"].dt.day_name()
```

Estas columnas permiten analizar las transacciones desde una perspectiva temporal.

Finalmente verificamos las columnas creadas, los tipos de datos y algunos ejemplos concretos del `DataFrame` transformado.

La idea principal de este capítulo fue:

Estandarizar nombres y crear columnas nuevas ayuda a transformar un dataset crudo en una tabla más clara, verificable y útil para el análisis.

Crear columnas no es solo agregar información. También puede servir para validar relaciones internas, detectar inconsistencias y preparar el dataset para análisis posteriores.

## Próximo paso

---

Ya trabajamos por separado varias tareas de preparación: valores faltantes, duplicados, limpieza de textos, conversión de tipos, fechas, estandarización de nombres y creación de nuevas columnas.

El siguiente paso será integrar varias de estas tareas en un flujo de limpieza encadenado.

En lugar de trabajar cada problema por separado, vamos a partir de un dataset crudo y construir una versión preparada paso a paso. La idea será aplicar transformaciones en un orden razonable, verificar resultados intermedios y obtener un `DataFrame` final más limpio y listo para analizar.

## Capítulo 21 — Limpieza encadenada paso a paso

---

### Breve repaso

---

En los capítulos anteriores trabajamos distintas tareas de preparación de datos por separado. Analizamos valores faltantes, estudiamos estrategias para tratarlos, revisamos duplicados, limpiamos textos y categorías, convertimos tipos de datos, trabajamos con fechas, estandarizamos nombres de columnas y creamos nuevas variables útiles para el análisis.

Cada una de esas tareas es importante, pero en un trabajo real rara vez aparecen completamente separadas. Cuando recibimos un dataset, los problemas suelen estar mezclados: una columna puede tener un nombre poco cómodo, estar cargada como texto, contener valores faltantes y además incluir códigos como `"UNKNOWN"` o `"ERROR"`.

Por eso, en este capítulo vamos a construir un flujo de limpieza paso a paso.

La idea no será presentar herramientas nuevas, sino integrar varias herramientas ya trabajadas dentro de una secuencia razonable. Vamos a partir de un dataset crudo, aplicar transformaciones en orden, verificar resultados intermedios y construir una versión más preparada para analizar.

Este tipo de flujo suele llamarse pipeline de limpieza o proceso de preparación de datos. No significa que exista una única receta válida para todos los datasets. Significa que organizamos las transformaciones de manera clara, repetible y verificable.

Al finalizar este capítulo deberías poder:

- Comprender por qué conviene ordenar las tareas de limpieza.
- Construir una copia de trabajo a partir de un dataset crudo.
- Estandarizar nombres de columnas como primer paso estructural.
- Convertir columnas numéricas y temporales.
- Tratar valores problemáticos en columnas categóricas.
- Crear columnas calculadas y de validación.
- Revisar el estado del dataset después de cada etapa.
- Construir una versión final preparada para análisis.

## Dataset de trabajo

Para construir este flujo de limpieza vamos a volver al dataset real **Cafe Sales — Dirty Data for Cleaning Training**.

Este dataset es adecuado para integrar varias tareas porque contiene distintos tipos de problemas: columnas con nombres poco cómodos para programar, valores faltantes, columnas numéricas cargadas como texto, fechas que necesitan conversión y valores categóricos problemáticos como "UNKNOWN" o "ERROR".

En este capítulo vamos a trabajar sobre una copia del dataset original. La idea será conservar el punto de partida y construir, paso a paso, una versión preparada para el análisis.

```
import kagglehub
import pandas as pd
import os

ruta_dataset = kagglehub.dataset_download(
 "ahmedmohamed2003/cafe-sales-dirty-data-for-cleaning-training"
)

archivos = os.listdir(ruta_dataset)

archivos_csv = [
 archivo for archivo in archivos
 if archivo.endswith(".csv")
]

ruta_csv = os.path.join(ruta_dataset, archivos_csv[0])

df_original = pd.read_csv(ruta_csv)

df_original.head()
```

Using Colab cache for faster access to the 'cafe-sales-dirty-data-for-cleaning-training' dataset.

índice	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date
0	TXN_1961373	Coffee	2	2.0	4.0	Credit Card	Takeaway	2023-09-08
1	TXN_4977031	Cake	4	3.0	12.0	Cash	In-store	2023-05-16
2	TXN_4271903	Cookie	4	1.0	ERROR	Credit Card	In-store	2023-07-19
3	TXN_7034554	Salad	2	5.0	10.0	UNKNOWN	UNKNOWN	2023-04-27

índice	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date
4	TXN_3160411	Coffee	2	2.0	4.0	Digital Wallet	In-store	2023-06-11

El `DataFrame` `df_original` representa el dataset tal como fue cargado desde el archivo.

Antes de transformarlo, conviene hacer una inspección general. No vamos a explicar nuevamente cada herramienta, porque ya las usamos en capítulos anteriores. Aquí las utilizamos como parte de una rutina inicial de diagnóstico.

```
df_original.shape
```

```
(10000, 8)
```

```
df_original.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 8 columns):
 # Column Non-Null Count Dtype
--- -
 0 Transaction ID 10000 non-null object
 1 Item 9667 non-null object
 2 Quantity 9862 non-null object
 3 Price Per Unit 9821 non-null object
 4 Total Spent 9827 non-null object
 5 Payment Method 7421 non-null object
 6 Location 6735 non-null object
 7 Transaction Date 9841 non-null object
dtypes: object(8)
memory usage: 625.1+ KB
```

```
df_original.isna().sum()
```

índice	0
Transaction ID	0
Item	333
Quantity	138
Price Per Unit	179
Total Spent	173

índice	0
Payment Method	2579
Location	3265
Transaction Date	159

`dtype: int64`

Estas primeras salidas nos permiten recuperar el estado inicial del dataset: cuántas filas y columnas tiene, qué tipos de datos asignó Pandas y cuántos valores faltantes reconoce en cada columna.

A partir de este diagnóstico inicial vamos a crear una copia de trabajo y comenzar el proceso de limpieza.

## Crear una copia de trabajo

Después del diagnóstico inicial, vamos a crear una copia del dataset.

La copia nos permite aplicar transformaciones sin perder el archivo original cargado desde Kaggle. Esto es especialmente útil cuando construimos un flujo de limpieza, porque necesitamos poder comparar el estado inicial con el estado final.

Vamos a llamar `df_limpieza` a la copia de trabajo.

```
df_limpieza = df_original.copy()

df_limpieza.head()
```

índice	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date
0	TXN_1961373	Coffee	2	2.0	4.0	Credit Card	Takeaway	2023-09-08
1	TXN_4977031	Cake	4	3.0	12.0	Cash	In-store	2023-05-16
2	TXN_4271903	Cookie	4	1.0	ERROR	Credit Card	In-store	2023-07-19
3	TXN_7034554	Salad	2	5.0	10.0	UNKNOWN	UNKNOWN	2023-04-27
4	TXN_3160411	Coffee	2	2.0	4.0	Digital Wallet	In-store	2023-06-11

A partir de ahora, las transformaciones se aplicarán sobre `df_limpieza`.

El dataset original queda disponible en `df_original`, por si necesitamos revisar valores iniciales o comparar resultados.

## Paso 1: estandarizar nombres de columnas

---

El primer paso del flujo será estandarizar los nombres de columnas.

El dataset original tiene nombres como `Transaction ID`, `Price Per Unit`, `Payment Method` y `Transaction Date`. Estos nombres son claros, pero tienen espacios y mayúsculas. Para trabajar con mayor comodidad, vamos a convertirlos a un formato uniforme en minúsculas y con guiones bajos.

Este paso no cambia los datos. Solo mejora la estructura del `DataFrame` y facilita las transformaciones posteriores.

```
df_limpieza = df_limpieza.rename(columns={
 "Transaction ID": "transaction_id",
 "Item": "item",
 "Quantity": "quantity",
 "Price Per Unit": "price_per_unit",
 "Total Spent": "total_spent",
 "Payment Method": "payment_method",
 "Location": "location",
 "Transaction Date": "transaction_date"
})

df_limpieza.columns
```

```
Index(['transaction_id', 'item', 'quantity', 'price_per_unit', 'total_spent',
 'payment_method', 'location', 'transaction_date'],
 dtype='object')
```

Ahora los nombres de columnas siguen un criterio más uniforme.

Este cambio nos permitirá escribir las próximas transformaciones con nombres más cómodos y consistentes.

## Paso 2: convertir columnas numéricas

---

Después de estandarizar los nombres, vamos a convertir las columnas que deberían ser numéricas.

En el diagnóstico inicial vimos que todas las columnas fueron cargadas como `object`. Esto incluye columnas que representan cantidades o importes:

```
quantity
price_per_unit
total_spent
```

Antes de hacer cálculos con estas columnas, necesitamos convertirlas a formato numérico. Como el dataset contiene valores problemáticos, usaremos `pd.to_numeric()` con `errors="coerce"`.

Esto transformará en `NaN` los valores que no puedan convertirse, como `"UNKNOWN"` o `"ERROR"`.

```
columnas_numericas = [
 "quantity",
 "price_per_unit",
 "total_spent"
]

for columna in columnas_numericas:
 df_limpieza[columna] = pd.to_numeric(
 df_limpieza[columna],
 errors="coerce"
)

df_limpieza[columnas_numericas].dtypes
```

índice	0
quantity	float64
price_per_unit	float64
total_spent	float64

`dtype: object`

Ahora las columnas seleccionadas quedaron en formato numérico.

Como usamos `errors="coerce"`, es importante revisar cuántos valores faltantes quedaron después de la conversión. Esta revisión nos permite ver el efecto de transformar valores problemáticos en `NaN`.

```
df_limpieza[columnas_numericas].isna().sum()
```

índice	0
quantity	479
price_per_unit	533
total_spent	502

`dtype: int64`



Este conteo puede ser mayor que el conteo inicial de faltantes, porque ahora también se consideran faltantes los valores que estaban escritos como texto problemático y no pudieron convertirse a número.

Este paso deja preparadas las columnas numéricas para cálculos posteriores, como validar si el total registrado coincide con la cantidad multiplicada por el precio unitario.

### Paso 3: convertir la columna de fecha

---

Además de las columnas numéricas, necesitamos convertir la columna `transaction_date`.

En el diagnóstico inicial vimos que esta columna también fue cargada como `object`. Eso significa que Pandas la interpretó como texto, aunque visualmente sus valores parezcan fechas.

Para poder trabajar con información temporal, vamos a convertirla usando `pd.to_datetime()` con `errors="coerce"`.

```
df_limpieza["transaction_date"] = pd.to_datetime(
 df_limpieza["transaction_date"],
 errors="coerce"
)

df_limpieza["transaction_date"].dtype
```

```
dtype('<M8[ns]')
```

Ahora la columna `transaction_date` quedó en formato temporal.

Como usamos `errors="coerce"`, los valores que no pudieron interpretarse como fechas quedaron como `NaT`. Por eso, igual que hicimos con las columnas numéricas, conviene revisar cuántos valores faltantes temporales quedaron después de la conversión.

```
df_limpieza["transaction_date"].isna().sum()
```

```
np.int64(460)
```

Este conteo incluye los valores que ya estaban faltantes antes de convertir y también los valores textuales que no pudieron interpretarse como fechas.

A partir de esta conversión, el dataset queda preparado para crear columnas temporales o realizar análisis por fecha.

## Paso 4: revisar valores categóricos problemáticos

Después de convertir columnas numéricas y fechas, vamos a revisar algunas columnas categóricas.

En este dataset hay tres columnas categóricas importantes:

```
item
payment_method
location
```

Estas columnas pueden contener valores válidos, valores faltantes y valores problemáticos como "UNKNOWN" o "ERROR".

Antes de reemplazar nada, conviene revisar sus frecuencias.

```
columnas_categoricas = [
 "item",
 "payment_method",
 "location"
]

for columna in columnas_categoricas:
 print(f"\nColumna: {columna}")
 print("-" * 40)
 print(df_limpieza[columna].value_counts(dropna=False))
```

```
Columna: item

item
Juice 1171
Coffee 1165
Salad 1148
Cake 1139
Sandwich 1131
Smoothie 1096
Cookie 1092
Tea 1089
UNKNOWN 344
NaN 333
ERROR 292
Name: count, dtype: int64

Columna: payment_method

payment_method
```

```

NaN 2579
Digital Wallet 2291
Credit Card 2273
Cash 2258
ERROR 306
UNKNOWN 293
Name: count, dtype: int64

```

```

Columna: location

```

```

location
NaN 3265
Takeaway 3022
In-store 3017
ERROR 358
UNKNOWN 338
Name: count, dtype: int64

```

Esta revisión permite distinguir entre categorías válidas y valores problemáticos.

Por ejemplo, en `payment_method`, valores como "Cash", "Credit Card" o "Digital Wallet" representan formas de pago reales. En cambio, "UNKNOWN" y "ERROR" no deberían interpretarse como métodos de pago normales.

Lo mismo ocurre con `item` y `location`. "UNKNOWN" puede indicar que el dato no se conoce, mientras que "ERROR" sugiere que hubo algún problema de carga o registro.

En este flujo vamos a tomar una decisión simple: convertir "ERROR" en "UNKNOWN" dentro de estas columnas categóricas. De esta manera agrupamos los valores no confiables en una misma categoría de dato desconocido.

Esta decisión no es universal. En otro contexto, podríamos conservar "ERROR" por separado para investigar fallas del sistema. Aquí lo hacemos para construir una versión más simple y consistente del dataset.

```

for columna in columnas_categoricas:
 df_limpieza[columna] = df_limpieza[columna].replace({
 "ERROR": "UNKNOWN"
 })

for columna in columnas_categoricas:
 print(f"\nColumna: {columna}")
 print("-" * 40)
 print(df_limpieza[columna].value_counts(dropna=False))

```

```

Columna: item

```

```

item
Juice 1171

```

```
Coffee 1165
Salad 1148
Cake 1139
Sandwich 1131
Smoothie 1096
Cookie 1092
Tea 1089
UNKNOWN 636
NaN 333
Name: count, dtype: int64
```

Columna: payment\_method

```

payment_method
NaN 2579
Digital Wallet 2291
Credit Card 2273
Cash 2258
UNKNOWN 599
Name: count, dtype: int64
```

Columna: location

```

location
NaN 3265
Takeaway 3022
In-store 3017
UNKNOWN 696
Name: count, dtype: int64
```

Después del reemplazo, "ERROR" ya no aparece como categoría separada en estas columnas. Sus casos quedaron agrupados dentro de "UNKNOWN".

Todavía pueden quedar valores faltantes reales, representados como NaN. En este flujo no vamos a completar todos esos valores de manera automática. Preferimos conservarlos para no introducir información que no estaba en el dataset original.

La decisión tomada en este paso fue acotada: unificar valores categóricos problemáticos que cumplen una función similar dentro del análisis.

## Paso 5: crear columnas calculadas y de validación

Una vez que las columnas numéricas fueron convertidas, podemos crear nuevas variables útiles para el análisis.

En este dataset existe una relación esperada:

```
total_spent = quantity × price_per_unit
```

La columna `total_spent` representa el total registrado en el archivo. Podemos crear una nueva columna, `calculated_total`, para calcular ese total a partir de `quantity` y `price_per_unit`.

Esta nueva columna nos permitirá validar si el total registrado coincide con el total que se obtiene mediante el cálculo.

```
df_limpieza["calculated_total"] = (
 df_limpieza["quantity"] * df_limpieza["price_per_unit"]
)

df_limpieza[
 [
 "quantity",
 "price_per_unit",
 "total_spent",
 "calculated_total"
]
].head(10)
```

índice	quantity	price_per_unit	total_spent	calculated_total
0	2.0	2.0	4.0	4.0
1	4.0	3.0	12.0	12.0
2	4.0	1.0	NaN	4.0
3	2.0	5.0	10.0	10.0
4	2.0	2.0	4.0	4.0
5	5.0	4.0	20.0	20.0
6	3.0	3.0	9.0	9.0
7	4.0	4.0	16.0	16.0
8	5.0	3.0	15.0	15.0
9	5.0	4.0	20.0	20.0

Ahora tenemos dos valores relacionados:

```
total_spent → total registrado en el dataset
calculated_total → total calculado a partir de quantity y price_per_unit
```

Para comparar estos valores, vamos a crear una columna de validación. Esta vez no vamos a usar una comparación booleana simple, porque si falta alguno de los datos necesarios, no podemos afirmar que haya una inconsistencia. En esos casos, simplemente no hay información suficiente para validar.

```

df_limpieza["estado_total"] = "sin_datos_suficientes"

condicion_datos_completos = (
 df_limpieza["quantity"].notna()
 & df_limpieza["price_per_unit"].notna()
 & df_limpieza["total_spent"].notna()
)

condicion_coincide = (
 condicion_datos_completos
 & (df_limpieza["total_spent"] == df_limpieza["calculated_total"])
)

condicion_no_coincide = (
 condicion_datos_completos
 & (df_limpieza["total_spent"] != df_limpieza["calculated_total"])
)

df_limpieza.loc[condicion_coincide, "estado_total"] = "coincide"
df_limpieza.loc[condicion_no_coincide, "estado_total"] = "no_coincide"

df_limpieza["estado_total"].value_counts(dropna=False)

```

estado_total	count
coincide	8544
sin_datos_suficientes	1456

dtype: int64

La columna `estado_total` resume la validación de la relación entre cantidad, precio unitario y total registrado.

En este caso, la salida muestra dos estados:

```

coincide
sin_datos_suficientes

```

No aparecen casos `no_coincide`. Esto indica que, entre las filas que tienen datos suficientes para comparar `quantity`, `price_per_unit` y `total_spent`, el total registrado coincide con el total calculado.

Las filas clasificadas como `sin_datos_suficientes` no indican una inconsistencia entre importes. Indican que falta alguno de los datos necesarios para hacer la validación.

En otros datasets, o incluso en otra versión de este mismo archivo, podría aparecer también el estado `no_coincide`. Si aparecieran muchos casos de ese tipo, deberíamos revisar posibles errores en `quantity`, `price_per_unit` o `total_spent`.

Crear columnas de validación ayuda a convertir una relación interna del dataset en una señal concreta de calidad de datos.

## Paso 6: crear columnas temporales

Como ya convertimos `transaction_date` a formato temporal, podemos crear columnas derivadas de esa fecha.

Estas columnas ayudan a preparar el dataset para análisis por períodos. Por ejemplo, podemos extraer el año, el mes y el día de la semana de cada transacción.

No estamos agregando información externa. Estamos reorganizando información que ya estaba dentro de la fecha original.

```
df_limpieza["year"] = df_limpieza["transaction_date"].dt.year
df_limpieza["month"] = df_limpieza["transaction_date"].dt.month
df_limpieza["day_of_week"] = df_limpieza["transaction_date"].dt.day_name()

df_limpieza[
 [
 "transaction_date",
 "year",
 "month",
 "day_of_week"
]
].head(10)
```

índice	transaction_date	year	month	day_of_week
0	2023-09-08	2023.0	9.0	Friday
1	2023-05-16	2023.0	5.0	Tuesday
2	2023-07-19	2023.0	7.0	Wednesday
3	2023-04-27	2023.0	4.0	Thursday
4	2023-06-11	2023.0	6.0	Sunday
5	2023-03-31	2023.0	3.0	Friday
6	2023-10-06	2023.0	10.0	Friday
7	2023-10-28	2023.0	10.0	Saturday
8	2023-07-28	2023.0	7.0	Friday
9	2023-12-31	2023.0	12.0	Sunday

Las columnas `year`, `month` y `day_of_week` permiten analizar las transacciones desde una perspectiva temporal.

Como vimos antes, es posible que `year` y `month` aparezcan con decimales, como `2023.0` o `9.0`. Esto ocurre porque algunas filas tienen fechas faltantes o no interpretables. Al derivar columnas desde una fecha faltante, Pandas necesita usar un formato que pueda representar esos valores ausentes.

También es posible que los nombres de los días aparezcan en inglés, como `Friday`, `Tuesday` o `Wednesday`. Esto depende del entorno y no afecta el análisis. Si más adelante necesitáramos presentar esos valores en español, podríamos aplicar un reemplazo o mapeo de nombres.

Estas columnas derivadas serán útiles para análisis posteriores. Por ejemplo, podríamos revisar ventas por mes, transacciones por día de la semana o períodos con mayor actividad.

## Paso 7: verificar el estado del DataFrame preparado

Después de aplicar varias transformaciones, conviene hacer una revisión general del `DataFrame`.

Hasta este punto realizamos varias acciones:

```
creamos una copia de trabajo
estandarizamos nombres de columnas
convertimos columnas numéricas
convertimos la columna de fecha
unificamos valores categóricos problemáticos
creamos una columna calculada
creamos una columna de validación
creamos columnas temporales
```

Cada una de estas transformaciones modificó de alguna manera la estructura del dataset. Por eso, antes de continuar, necesitamos verificar cómo quedó.

```
df_limpieza.head()
```

índice	transaction_id	item	quantity	price_per_unit	total_spent	payment_method	location
0	TXN_1961373	Coffee	2.0	2.0	4.0	Credit Card	Takeawa
1	TXN_4977031	Cake	4.0	3.0	12.0	Cash	In-store
2	TXN_4271903	Cookie	4.0	1.0	NaN	Credit Card	In-store
3	TXN_7034554	Salad	2.0	5.0	10.0	UNKNOWN	UNKNOV



índice	transaction_id	item	quantity	price_per_unit	total_spent	payment_method	location
4	TXN_3160411	Coffee	2.0	2.0	4.0	Digital Wallet	In-store

```
df_limpieza.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 13 columns):
 # Column Non-Null Count Dtype
--- -
 0 transaction_id 10000 non-null object
 1 item 9667 non-null object
 2 quantity 9521 non-null float64
 3 price_per_unit 9467 non-null float64
 4 total_spent 9498 non-null float64
 5 payment_method 7421 non-null object
 6 location 6735 non-null object
 7 transaction_date 9540 non-null datetime64[ns]
 8 calculated_total 9006 non-null float64
 9 estado_total 10000 non-null object
10 year 9540 non-null float64
11 month 9540 non-null float64
12 day_of_week 9540 non-null object
dtypes: datetime64[ns](1), float64(6), object(6)
memory usage: 1015.8+ KB
```

La salida de `info()` nos permite revisar los tipos de datos actuales y la cantidad de valores no nulos por columna.

Ahora deberíamos observar que las columnas numéricas ya no están cargadas como texto, y que `transaction_date` aparece como columna temporal.

También podemos revisar los faltantes actuales.

```
df_limpieza.isna().sum()
```

índice	0
transaction_id	0
item	333
quantity	479
price_per_unit	533
total_spent	502
payment_method	2579
location	3265

índice	0
transaction_date	460
calculated_total	994
estado_total	0
year	460
month	460
day_of_week	460

`dtype: int64`

Este conteo muestra los valores faltantes después de las transformaciones realizadas.

Es esperable que algunas columnas tengan más faltantes que en el diagnóstico inicial, especialmente las columnas numéricas y la columna de fecha. Esto ocurre porque valores como "UNKNOWN" o "ERROR" fueron convertidos a NaN o NaT durante la conversión de tipos.

Por ejemplo, `quantity` pasó de 138 faltantes iniciales a 479 después de la conversión; `price_per_unit`, de 179 a 533; `total_spent`, de 173 a 502; y `transaction_date`, de 159 a 460. Esa diferencia se explica por los valores textuales problemáticos que ahora quedaron representados como NaN o NaT.

Esto no significa necesariamente que el dataset haya empeorado. En realidad, ahora esos problemas quedaron representados de una manera más clara para Pandas, lo que facilita su análisis y tratamiento posterior.

## Paso 8: construir una versión final de trabajo

Después de aplicar y verificar las transformaciones principales, podemos construir una versión final del dataset preparado.

Esta versión no tiene por qué eliminar toda la información problemática. Su objetivo es dejar una tabla más clara, con nombres consistentes, tipos de datos adecuados y columnas útiles para el análisis.

Vamos a llamar `df_preparado` a esta versión.

```
columnas_finales = [
 "transaction_id",
 "item",
 "quantity",
 "price_per_unit",
 "total_spent",
```

```

 "calculated_total",
 "estado_total",
 "payment_method",
 "location",
 "transaction_date",
 "year",
 "month",
 "day_of_week"
]

df_preparado = df_limpieza[columnas_finales].copy()

df_preparado.head()

```

índice	transaction_id	item	quantity	price_per_unit	total_spent	calculated_total	estado_to
0	TXN_1961373	Coffee	2.0	2.0	4.0	4.0	coincide
1	TXN_4977031	Cake	4.0	3.0	12.0	12.0	coincide
2	TXN_4271903	Cookie	4.0	1.0	NaN	4.0	sin_datos_
3	TXN_7034554	Salad	2.0	5.0	10.0	10.0	coincide
4	TXN_3160411	Coffee	2.0	2.0	4.0	4.0	coincide

El nuevo `DataFrame` conserva las columnas originales más importantes y agrega algunas columnas creadas durante el proceso de limpieza.

La columna `calculated_total` permite comparar el total calculado con el total registrado. La columna `estado_total` resume esa validación. Las columnas `year`, `month` y `day_of_week` permiten trabajar con la dimensión temporal del dataset.

También podemos revisar la estructura final.

```
df_preparado.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 13 columns):
 # Column Non-Null Count Dtype
--- -
 0 transaction_id 10000 non-null object
 1 item 9667 non-null object
 2 quantity 9521 non-null float64
 3 price_per_unit 9467 non-null float64
 4 total_spent 9498 non-null float64
 5 calculated_total 9006 non-null float64
 6 estado_total 10000 non-null object
 7 payment_method 7421 non-null object
 8 location 6735 non-null object
 9 transaction_date 9540 non-null datetime64[ns]

```

```
10 year 9540 non-null float64
11 month 9540 non-null float64
12 day_of_week 9540 non-null object
dtypes: datetime64[ns](1), float64(6), object(6)
memory usage: 1015.8+ KB
```

Esta versión preparada no representa una limpieza definitiva para cualquier objetivo posible. Todavía pueden quedar decisiones pendientes, especialmente sobre valores faltantes y categorías desconocidas.

Sin embargo, sí representa una mejora importante respecto del dataset crudo. Ahora tenemos nombres de columnas consistentes, tipos de datos más adecuados, valores categóricos problemáticos parcialmente unificados y columnas nuevas que ayudan a validar y analizar la información.

En un proyecto real, esta versión podría ser el punto de partida para análisis exploratorio, visualizaciones o nuevas transformaciones más específicas.

## Errores frecuentes en una limpieza encadenada

Cuando aplicamos varias transformaciones en secuencia, uno de los errores más frecuentes es perder de vista el orden del proceso.

Por ejemplo, no conviene crear una columna calculada antes de convertir las columnas necesarias a formato numérico. Si `quantity` o `price_per_unit` todavía están como texto, el cálculo de `calculated_total` puede fallar o producir resultados no confiables.

Otro error frecuente es modificar el dataset original sin conservar una copia. En este capítulo trabajamos con `df_original`, `df_limpieza` y `df_preparado`. Esta separación permite distinguir entre el punto de partida, la copia de trabajo y la versión preparada para continuar el análisis.

También debemos tener cuidado con los valores faltantes que aparecen después de convertir tipos. Al usar `errors="coerce"`, valores como `"UNKNOWN"` o `"ERROR"` pueden transformarse en `NaN` o `NaT`. Esto no significa que el problema haya aparecido recién en ese momento, sino que ahora Pandas lo representa de una manera más adecuada para el análisis.

Otro error posible es aplicar reemplazos categóricos sin revisar primero los valores existentes. En este capítulo reemplazamos `"ERROR"` por `"UNKNOWN"` solo después de revisar las frecuencias de `item`, `payment_method` y `location`.

También puede ser un problema crear muchas columnas nuevas sin un propósito claro. Las columnas agregadas deberían tener una función dentro del análisis o

dentro del control de calidad. En este caso, `calculated_total` y `estado_total` sirven para validar la relación entre cantidad, precio unitario y total registrado. Las columnas `year`, `month` y `day_of_week` sirven para preparar análisis temporales.

Una buena rutina para una limpieza encadenada podría ser:

```
diagnosticar el estado inicial
crear una copia de trabajo
estandarizar nombres
convertir tipos necesarios
revisar valores problemáticos
crear columnas útiles
verificar resultados intermedios
construir una versión final de trabajo
```

El objetivo de un flujo de limpieza no es aplicar muchas transformaciones, sino aplicar las transformaciones necesarias en un orden razonable y con verificaciones intermedias.

## Resumen del capítulo

---

En este capítulo integramos varias tareas de preparación de datos dentro de un flujo de limpieza paso a paso.

Hasta ahora habíamos trabajado muchos temas por separado: valores faltantes, duplicados, limpieza de textos, conversión de tipos, fechas, nombres de columnas y creación de nuevas variables. En este capítulo reunimos varias de esas herramientas en una secuencia más cercana a una situación real de trabajo.

Partimos del dataset original cargado desde Kaggle y realizamos un diagnóstico inicial. Revisamos sus dimensiones, su estructura, los tipos de datos asignados por Pandas y la cantidad de valores faltantes por columna.

Luego creamos una copia de trabajo:

```
df_limpieza = df_original.copy()
```

Esto nos permitió conservar el dataset original y aplicar transformaciones sobre una versión editable.

El primer paso de limpieza fue estandarizar los nombres de columnas. Cambiamos nombres como `Transaction ID`, `Price Per Unit` y `Transaction Date` por nombres en minúsculas y con guiones bajos, como `transaction_id`, `price_per_unit` y `transaction_date`.

Después convertimos columnas numéricas usando `pd.to_numeric()` con `errors="coerce"`:

```
for columna in columnas_numericas:
 df_limpieza[columna] = pd.to_numeric(
 df_limpieza[columna],
 errors="coerce"
)
```

Esta conversión permitió transformar `quantity`, `price_per_unit` y `total_spent` en columnas numéricas. Los valores que no pudieron convertirse quedaron representados como `NaN`.

También convertimos la columna `transaction_date` usando `pd.to_datetime()`:

```
df_limpieza["transaction_date"] = pd.to_datetime(
 df_limpieza["transaction_date"],
 errors="coerce"
)
```

Los valores que no pudieron interpretarse como fechas quedaron como `NaT`.

Luego revisamos columnas categóricas como `item`, `payment_method` y `location`. En esas columnas unificamos el valor `"ERROR"` dentro de `"UNKNOWN"`, tomando una decisión simple para agrupar valores problemáticos o no confiables.

Más adelante creamos una columna calculada:

```
df_limpieza["calculated_total"] = (
 df_limpieza["quantity"] * df_limpieza["price_per_unit"]
)
```

Esta columna permitió comparar el total calculado con el total registrado en `total_spent`.

También creamos una columna de validación llamada `estado_total`, que distingue tres situaciones:

```
coincide
no_coincide
sin_datos_suficientes
```

Esta columna resultó más informativa que una comparación booleana simple, porque permite diferenciar una inconsistencia real de una fila que no tiene datos suficientes para ser evaluada.

Después creamos columnas temporales derivadas de `transaction_date`:

```
df_limpieza["year"] = df_limpieza["transaction_date"].dt.year
df_limpieza["month"] = df_limpieza["transaction_date"].dt.month
df_limpieza["day_of_week"] = df_limpieza["transaction_date"].dt.day_name()
```

Estas columnas preparan el dataset para futuros análisis por períodos.

Finalmente construimos una versión final de trabajo llamada `df_preparado`, seleccionando las columnas más relevantes para continuar el análisis.

La idea principal de este capítulo fue:

Un flujo de limpieza organiza varias transformaciones en una secuencia razonable, verificable y orientada al análisis.

La limpieza encadenada no consiste en aplicar muchos comandos seguidos. Consiste en ordenar decisiones: diagnosticar, transformar, verificar y construir una versión del dataset más clara, consistente y útil.

## Próximo paso

---

El siguiente paso será trabajar con la validación posterior a la limpieza.

Vamos a revisar cómo evaluar si el dataset preparado quedó en buenas condiciones: tipos de datos, valores faltantes, categorías problemáticas, duplicados, rangos numéricos, fechas válidas, consistencia interna y dimensiones antes/después.

Ese capítulo funcionará como cierre técnico de esta etapa de preparación de datos.

## Capítulo 22 — Validación posterior a la limpieza

---

### Breve repaso

---

En el trabajo anterior construimos un flujo de limpieza encadenado. Partimos de un dataset crudo, creamos una copia de trabajo, estandarizamos nombres de columnas, convertimos tipos de datos, revisamos valores categóricos problemáticos, creamos columnas calculadas, agregamos una columna de validación y construimos una versión preparada del `DataFrame`.

Ese recorrido mostró que la limpieza de datos no suele resolverse con una sola instrucción. Preparar un dataset implica aplicar varias transformaciones en un orden razonable y verificar el resultado de cada etapa.

En este capítulo vamos a concentrarnos en el paso posterior: validar la limpieza realizada.

Validar no significa demostrar que el dataset quedó perfecto. Significa revisar si la versión preparada quedó en mejores condiciones para el análisis, qué problemas fueron tratados, qué problemas siguen presentes y qué decisiones deberían quedar documentadas.

La validación posterior es una especie de control de calidad. Nos ayuda a no asumir que el dataset está listo solo porque aplicamos varias transformaciones.

A lo largo del capítulo vamos a revisar dimensiones, tipos de datos, valores faltantes, duplicados, categorías problemáticas, rangos numéricos, fechas válidas y consistencia interna.

Al finalizar este capítulo deberías poder:

- Comprender por qué es necesario validar después de limpiar.
- Comparar el dataset original con el dataset preparado.
- Revisar tipos de datos finales.
- Revisar valores faltantes después de la limpieza.
- Detectar si quedan duplicados.
- Revisar categorías problemáticas pendientes.
- Revisar rangos numéricos y fechas válidas.
- Validar relaciones internas entre columnas.
- Construir una síntesis del estado final del dataset.



- Reconocer qué problemas quedan pendientes y documentarlos.

## Dataset de trabajo

Para validar una limpieza necesitamos contar con dos versiones del dataset:

```
df_original → el dataset tal como fue cargado desde el archivo
df_preparado → una versión transformada y más lista para analizar
```

En este capítulo vamos a reconstruir una versión preparada aplicando el mismo flujo general que trabajamos antes. No vamos a explicar cada transformación desde cero, porque esas herramientas ya fueron desarrolladas. El foco estará puesto en validar el resultado final.

```
import kagglehub
import pandas as pd
import os

ruta_dataset = kagglehub.dataset_download(
 "ahmedmohamed2003/caffe-sales-dirty-data-for-cleaning-training"
)

archivos = os.listdir(ruta_dataset)

archivos_csv = [
 archivo for archivo in archivos
 if archivo.endswith(".csv")
]

ruta_csv = os.path.join(ruta_dataset, archivos_csv[0])

df_original = pd.read_csv(ruta_csv)

df_original.head()
```

Using Colab cache for faster access to the 'caffe-sales-dirty-data-for-cleaning-training' dataset.

índice	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date
0	TXN_1961373	Coffee	2	2.0	4.0	Credit Card	Takeaway	2023-09-08
1	TXN_4977031	Cake	4	3.0	12.0	Cash	In-store	2023-05-16
2	TXN_4271903	Cookie	4	1.0	ERROR	Credit Card	In-store	2023-07-19
3	TXN_7034554	Salad	2	5.0	10.0	UNKNOWN	UNKNOWN	2023-04-27

índice	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date
4	TXN_3160411	Coffee	2	2.0	4.0	Digital Wallet	In-store	2023-06-11

Ahora tenemos cargado el dataset original.

A continuación vamos a construir una versión preparada usando una secuencia compacta de transformaciones: renombrar columnas, convertir tipos, tratar algunos valores categóricos problemáticos y crear columnas útiles para validar y analizar.

```
df_limpieza = df_original.copy()

df_limpieza = df_limpieza.rename(columns={
 "Transaction ID": "transaction_id",
 "Item": "item",
 "Quantity": "quantity",
 "Price Per Unit": "price_per_unit",
 "Total Spent": "total_spent",
 "Payment Method": "payment_method",
 "Location": "location",
 "Transaction Date": "transaction_date"
})

columnas_numericas = [
 "quantity",
 "price_per_unit",
 "total_spent"
]

for columna in columnas_numericas:
 df_limpieza[columna] = pd.to_numeric(
 df_limpieza[columna],
 errors="coerce"
)

df_limpieza["transaction_date"] = pd.to_datetime(
 df_limpieza["transaction_date"],
 errors="coerce"
)

columnas_categoricas = [
 "item",
 "payment_method",
 "location"
]

for columna in columnas_categoricas:
 df_limpieza[columna] = df_limpieza[columna].replace({
 "ERROR": "UNKNOWN"
 })
```

```

df_limpieza["calculated_total"] = (
 df_limpieza["quantity"] * df_limpieza["price_per_unit"]
)

df_limpieza["estado_total"] = "sin_datos_suficientes"

condicion_datos_completos = (
 df_limpieza["quantity"].notna()
 & df_limpieza["price_per_unit"].notna()
 & df_limpieza["total_spent"].notna()
)

condicion_coincide = (
 condicion_datos_completos
 & (df_limpieza["total_spent"] == df_limpieza["calculated_total"])
)

condicion_no_coincide = (
 condicion_datos_completos
 & (df_limpieza["total_spent"] != df_limpieza["calculated_total"])
)

df_limpieza.loc[condicion_coincide, "estado_total"] = "coincide"
df_limpieza.loc[condicion_no_coincide, "estado_total"] = "no_coincide"

df_limpieza["year"] = df_limpieza["transaction_date"].dt.year
df_limpieza["month"] = df_limpieza["transaction_date"].dt.month
df_limpieza["day_of_week"] = df_limpieza["transaction_date"].dt.day_name()

columnas_finales = [
 "transaction_id",
 "item",
 "quantity",
 "price_per_unit",
 "total_spent",
 "calculated_total",
 "estado_total",
 "payment_method",
 "location",
 "transaction_date",
 "year",
 "month",
 "day_of_week"
]

df_preparado = df_limpieza[columnas_finales].copy()

df_preparado.head()

```

índice	transaction_id	item	quantity	price_per_unit	total_spent	calculated_total	estado_to
0	TXN_1961373	Coffee	2.0	2.0	4.0	4.0	coincide
1	TXN_4977031	Cake	4.0	3.0	12.0	12.0	coincide

índice	transaction_id	item	quantity	price_per_unit	total_spent	calculated_total	estado_to
2	TXN_4271903	Cookie	4.0	1.0	NaN	4.0	sin_datos_
3	TXN_7034554	Salad	2.0	5.0	10.0	10.0	coincide
4	TXN_3160411	Coffee	2.0	2.0	4.0	4.0	coincide

Ya tenemos una versión preparada del dataset.

A partir de ahora, el objetivo no será seguir transformando datos, sino validar el estado de `df_preparado` y compararlo cuando sea necesario con `df_original`.

## Comparar dimensiones antes y después

Una primera validación consiste en comparar las dimensiones del dataset original y del dataset preparado.

Esto permite saber si durante el proceso cambiaron la cantidad de filas o columnas.

En nuestro flujo de limpieza no eliminamos filas. Sí agregamos columnas nuevas y seleccionamos una estructura final diferente. Por eso esperamos que la cantidad de filas se mantenga, pero que la cantidad de columnas cambie.

```
print("Dimensiones del dataset original:")
print(df_original.shape)

print()

print("Dimensiones del dataset preparado:")
print(df_preparado.shape)
```

```
Dimensiones del dataset original:
(10000, 8)
```

```
Dimensiones del dataset preparado:
(10000, 13)
```

La comparación entre dimensiones nos permite responder dos preguntas:

```
¿Se conservaron todas las filas?
¿Cambió la cantidad de columnas?
```

Si la cantidad de filas se mantiene, significa que el flujo aplicado no eliminó registros. Esto es importante porque algunas decisiones de limpieza, como `dropna()` o `drop_duplicates()`, pueden reducir el dataset.

En este caso, el cambio principal está en la estructura de columnas. El dataset preparado conserva las variables principales, pero también incluye columnas nuevas como `calculated_total`, `estado_total`, `year`, `month` y `day_of_week`.

Comparar dimensiones es una validación simple, pero muy útil. Nos ayuda a detectar cambios estructurales inesperados.

## Revisar tipos de datos finales

Después de preparar el dataset, necesitamos revisar si las columnas quedaron con tipos de datos adecuados.

En el dataset original, todas las columnas habían sido cargadas como `object`. Eso incluía columnas que deberían ser numéricas y una columna que debería ser temporal.

En el dataset preparado esperamos observar una estructura más adecuada:

```
quantity, price_per_unit, total_spent, calculated_total → columnas numéricas
transaction_date → columna temporal
item, payment_method, location, estado_total → columnas categóricas o
textuales
```

Podemos revisar los tipos de datos finales con `info()`.

```
df_preparado.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 13 columns):
Column Non-Null Count Dtype
--- -
0 transaction_id 10000 non-null object
1 item 9667 non-null object
2 quantity 9521 non-null float64
3 price_per_unit 9467 non-null float64
4 total_spent 9498 non-null float64
5 calculated_total 9006 non-null float64
6 estado_total 10000 non-null object
7 payment_method 7421 non-null object
8 location 6735 non-null object
9 transaction_date 9540 non-null datetime64[ns]
10 year 9540 non-null float64
11 month 9540 non-null float64
12 day_of_week 9540 non-null object
```

```
dtypes: datetime64[ns](1), float64(6), object(6)
memory usage: 1015.8+ KB
```

Esta salida permite comprobar si la conversión de tipos se aplicó correctamente.

Las columnas numéricas deberían aparecer como `float64`, porque contienen valores numéricos y también pueden contener `NaN`. La columna `transaction_date` debería aparecer como `datetime64[ns]`.

Las columnas de texto o categorías pueden seguir apareciendo como `object`. Eso no es necesariamente un problema: un producto, una ubicación o un método de pago son datos categóricos, no magnitudes numéricas.

La validación de tipos es importante porque muchas operaciones posteriores dependen de esta estructura. Si una columna de importes siguiera como texto, los cálculos y resúmenes numéricos podrían fallar o producir resultados incorrectos.

## Revisar valores faltantes finales

Después de preparar el dataset, conviene revisar cuántos valores faltantes quedan.

Es importante recordar que el objetivo de esta limpieza no fue eliminar todos los faltantes. En algunos casos decidimos conservarlos porque no teníamos una justificación suficiente para completarlos. En otros casos, algunos valores problemáticos pasaron a ser `NaN` o `NaT` durante la conversión de tipos.

Por eso, la presencia de faltantes en `df_preparado` no significa automáticamente que la limpieza haya fallado. Lo importante es saber dónde están, cuántos son y qué representan.

```
df_preparado.isna().sum()
```

índice	0
transaction_id	0
item	333
quantity	479
price_per_unit	533
total_spent	502
calculated_total	994
estado_total	0
payment_method	2579

índice	0
location	3265
transaction_date	460
year	460
month	460
day_of_week	460

dtype: int64

Esta salida muestra la cantidad de valores faltantes por columna en el dataset preparado.

Algunas columnas pueden tener más faltantes que en el dataset original. Por ejemplo, las columnas numéricas pueden haber recibido nuevos `NaN` cuando valores como `"UNKNOWN"` o `"ERROR"` no pudieron convertirse a número. La columna `transaction_date` también puede tener más faltantes porque algunos textos no pudieron interpretarse como fechas y quedaron como `NaT`.

Podemos construir una tabla con cantidad y porcentaje de faltantes para interpretar mejor el estado final.

```
diagnostico_faltantes_final = pd.DataFrame({
 "faltantes": df_preparado.isna().sum(),
 "porcentaje": df_preparado.isna().mean() * 100
})

diagnostico_faltantes_final.sort_values("porcentaje", ascending=False)
```

índice	faltantes	porcentaje
location	3265	32.65
payment_method	2579	25.79
calculated_total	994	9.94
price_per_unit	533	5.33
total_spent	502	5.02
quantity	479	4.79
year	460	4.60
month	460	4.60
transaction_date	460	4.60
day_of_week	460	4.60

índice	faltantes	porcentaje
item	333	3.33
transaction_id	0	0.00
estado_total	0	0.00

Esta tabla permite priorizar la revisión.

Una columna con muchos faltantes puede limitar ciertos análisis. Por ejemplo, si `location` tiene muchos valores ausentes, los análisis por ubicación deberán interpretarse con cuidado. Si `transaction_date` tiene valores faltantes o no interpretables, algunas transacciones no podrán incluirse en análisis temporales.

La validación posterior no exige resolver todos los problemas en este momento. Su función es dejar claro qué tan preparado está el dataset y qué limitaciones todavía deben considerarse.

## Revisar duplicados

Otra parte importante de la validación posterior consiste en revisar si el dataset preparado contiene duplicados.

En este flujo de limpieza no eliminamos duplicados. Sin embargo, conviene comprobar si existen filas repetidas o identificadores de transacción duplicados.

Primero revisamos duplicados completos.

```
df_preparado.duplicated().sum()
```

```
np.int64(0)
```

Este conteo indica cuántas filas completas están duplicadas en `df_preparado`.

Si el resultado es `0`, significa que no hay filas exactamente iguales en todas sus columnas.

También podemos revisar duplicados según el identificador de transacción. En este dataset, `transaction_id` debería identificar cada operación.

```
df_preparado.duplicated(subset=["transaction_id"]).sum()
```

```
np.int64(0)
```



Este conteo indica si hay identificadores de transacción repetidos.

La revisión por identificador es importante porque, aunque dos filas no sean exactamente iguales, un identificador repetido podría indicar una carga duplicada o un problema en el registro.

Si aparecieran identificadores duplicados, deberíamos observar esas filas antes de decidir qué hacer.

```
df_preparado[
 df_preparado.duplicated(subset=["transaction_id"], keep=False)
].head()
```

índice	transaction_id	item	quantity	price_per_unit	total_spent	calculated_total	estado_tot
--------	----------------	------	----------	----------------	-------------	------------------	------------

Esta última vista muestra las filas involucradas si existen identificadores repetidos.

Si la salida está vacía, significa que no se encontraron casos para revisar bajo ese criterio.

Validar duplicados ayuda a confirmar que el dataset preparado no contiene repeticiones evidentes que puedan afectar conteos, importes o análisis posteriores.

## Revisar categorías problemáticas

Después de la limpieza, también conviene revisar las columnas categóricas.

En nuestro flujo reemplazamos "ERROR" por "UNKNOWN" en las columnas `item`, `payment_method` y `location`. Esa decisión agrupó valores problemáticos dentro de una categoría común de dato desconocido.

Ahora necesitamos verificar si ese reemplazo se aplicó correctamente y qué categorías siguen presentes.

```
columnas_categoricas = [
 "item",
 "payment_method",
 "location"
]

for columna in columnas_categoricas:
 print(f"\nColumna: {columna}")
 print("-" * 40)
 print(df_preparado[columna].value_counts(dropna=False))
```

```

Columna: item

item
Juice 1171
Coffee 1165
Salad 1148
Cake 1139
Sandwich 1131
Smoothie 1096
Cookie 1092
Tea 1089
UNKNOWN 636
NaN 333
Name: count, dtype: int64

Columna: payment_method

payment_method
NaN 2579
Digital Wallet 2291
Credit Card 2273
Cash 2258
UNKNOWN 599
Name: count, dtype: int64

Columna: location

location
NaN 3265
Takeaway 3022
In-store 3017
UNKNOWN 696
Name: count, dtype: int64

```

Esta revisión permite observar las categorías finales de cada columna.

Si el reemplazo se aplicó correctamente, "ERROR" ya no debería aparecer como categoría separada. En cambio, sus casos deberían estar agrupados dentro de "UNKNOWN".

Podemos verificarlo de manera más directa:

```

for columna in columnas_categoricas:
 cantidad_error = (df_preparado[columna] == "ERROR").sum()
 print(f"{columna}: {cantidad_error} valores ERROR")

```

```

item: 0 valores ERROR
payment_method: 0 valores ERROR
location: 0 valores ERROR

```

Si todos los conteos dan 0, significa que no quedan valores "ERROR" en esas columnas.

Esto no significa que las columnas categóricas estén perfectas. Todavía pueden quedar valores "UNKNOWN" y valores faltantes reales. Lo importante es que ahora esos casos están representados de manera más consistente.

Validar categorías problemáticas ayuda a confirmar que las decisiones de limpieza se aplicaron como esperábamos.

## Revisar rangos numéricos

Después de convertir columnas numéricas, conviene revisar sus rangos.

Esta validación permite detectar valores sospechosos. Por ejemplo, en un dataset de ventas podríamos prestar atención a cantidades negativas, precios iguales a cero, importes negativos o valores extremadamente altos.

En nuestro dataset preparado, las columnas numéricas principales son:

```
quantity
price_per_unit
total_spent
calculated_total
```

Podemos comenzar con un resumen estadístico.

```
columnas_numericas_validar = [
 "quantity",
 "price_per_unit",
 "total_spent",
 "calculated_total"
]

df_preparado[columnas_numericas_validar].describe()
```

índice	quantity	price_per_unit	total_spent	calculated_total
count	9521.000000	9467.000000	9498.000000	9006.000000
mean	3.028463	2.949984	8.924352	8.936098
std	1.419007	1.278450	6.009919	5.994375
min	1.000000	1.000000	1.000000	1.000000
25%	2.000000	2.000000	4.000000	4.000000
50%	3.000000	3.000000	8.000000	8.000000

índice	quantity	price_per_unit	total_spent	calculated_total
75%	4.000000	4.000000	12.000000	12.000000
max	5.000000	5.000000	25.000000	25.000000

La salida de `describe()` nos permite revisar valores mínimos, máximos, promedios y cuartiles.

En esta etapa no buscamos interpretar cada número en profundidad. El objetivo es detectar señales generales: valores negativos, ceros inesperados, importes demasiado grandes o rangos que no tengan sentido para el contexto.

También podemos revisar explícitamente si hay cantidades o importes negativos.

```
for columna in columnas_numericas_validar:
 cantidad_negativos = (df_preparado[columna] < 0).sum()
 print(f"{columna}: {cantidad_negativos} valores negativos")
```

```
quantity: 0 valores negativos
price_per_unit: 0 valores negativos
total_spent: 0 valores negativos
calculated_total: 0 valores negativos
```

Si los conteos son `0`, significa que no se detectaron valores negativos en esas columnas.

También podemos revisar valores iguales a cero. En algunos contextos, un precio o un total igual a cero podría ser válido, por ejemplo una promoción o una devolución. En otros contextos, podría indicar un error de carga. Por eso no debemos corregirlos automáticamente, pero sí registrarlos si aparecen.

```
for columna in columnas_numericas_validar:
 cantidad_ceros = (df_preparado[columna] == 0).sum()
 print(f"{columna}: {cantidad_ceros} valores iguales a cero")
```

```
quantity: 0 valores iguales a cero
price_per_unit: 0 valores iguales a cero
total_spent: 0 valores iguales a cero
calculated_total: 0 valores iguales a cero
```

Esta revisión ayuda a detectar posibles valores sospechosos.

La validación de rangos numéricos no implica eliminar o modificar datos de inmediato. Primero necesitamos identificar los casos, luego decidir si realmente son errores y recién después aplicar alguna transformación si corresponde.

En un proceso de limpieza responsable, los valores extremos o inesperados se revisan antes de ser corregidos.

## Revisar fechas válidas

---

Después de convertir una columna a formato temporal, también conviene validar sus fechas.

En nuestro dataset preparado, la columna temporal principal es `transaction_date`.

Durante la conversión, algunos valores quedaron como `NaT`, porque estaban faltantes o no pudieron interpretarse como fechas. Ahora vamos a revisar cuántos casos hay y qué rango temporal cubren las fechas válidas.

```
df_preparado["transaction_date"].isna().sum()
```

```
np.int64(460)
```

Este conteo muestra cuántas filas no tienen una fecha válida.

Esas filas pueden seguir siendo útiles para otros análisis, pero no podrán ubicarse correctamente en una línea de tiempo.

Ahora revisemos la fecha mínima y la fecha máxima del dataset preparado.

```
fecha_minima = df_preparado["transaction_date"].min()
fecha_maxima = df_preparado["transaction_date"].max()

print("Fecha mínima:")
print(fecha_minima)

print()

print("Fecha máxima:")
print(fecha_maxima)
```

```
Fecha mínima:
2023-01-01 00:00:00

Fecha máxima:
2023-12-31 00:00:00
```

Esta revisión permite conocer el período cubierto por el dataset.

También ayuda a detectar fechas fuera de rango. Por ejemplo, si esperaríamos trabajar con datos de 2023 y apareciera una fecha de 2099, deberíamos revisarla con atención.

Podemos observar cuántas transacciones válidas hay por año.

```
df_preparado["year"].value_counts(dropna=False).sort_index()
```

year	count
2023.0	9540
NaN	460

dtype: int64

Esta salida permite confirmar si las fechas válidas se concentran en un año o si el dataset contiene registros de varios años.

Los valores faltantes en `year` corresponden a filas cuya `transaction_date` quedó como `NaT`.

La validación temporal no busca corregir fechas automáticamente. Su objetivo es comprobar si el rango de fechas tiene sentido y registrar cuántas filas no pueden participar de análisis temporales.

## Revisar consistencia interna

Una parte importante de la validación posterior consiste en revisar si ciertas columnas son coherentes entre sí.

En este dataset existe una relación esperada:

```
total_spent = quantity × price_per_unit
```

Durante la limpieza creamos la columna `calculated_total` y luego la columna `estado_total`, que resume si el total registrado coincide con el total calculado.

Ahora podemos revisar el resultado de esa validación.

```
df_preparado["estado_total"].value_counts(dropna=False)
```

estado_total	count
coincide	8544

estado_total	count
sin_datos_suficientes	1456

dtype: int64

Esta salida muestra el resultado de la validación interna entre `quantity`, `price_per_unit`, `total_spent` y `calculated_total`.

En este caso aparecen dos estados:

```
coincide
sin_datos_suficientes
```

No aparecen casos `no_coincide`. Esto significa que, entre las filas que tienen datos suficientes para hacer la comparación, el total registrado coincide con el total calculado.

Las filas clasificadas como `sin_datos_suficientes` no indican una inconsistencia entre importes. Indican que falta alguno de los datos necesarios para validar la relación.

En otro dataset, o en otra versión de este archivo, podrían aparecer casos `no_coincide`. Si aparecieran, convendría observarlos con detalle.

```
df_preparado[df_preparado["estado_total"] == "no_coincide"][
 [
 "quantity",
 "price_per_unit",
 "total_spent",
 "calculated_total",
 "estado_total"
]
].head(10)
```

índice	quantity	price_per_unit	total_spent	calculated_total	estado_total
--------	----------	----------------	-------------	------------------	--------------

Si esta salida está vacía, significa que no hay filas clasificadas como `no_coincide`.

Eso no significa que todos los registros estén completos. Las filas con `sin_datos_suficientes` indican que falta alguno de los datos necesarios para validar la relación.

Podemos observar algunos de esos casos:

```
df_preparado[df_preparado["estado_total"] == "sin_datos_suficientes"][
 [
 "quantity",
 "price_per_unit",
 "total_spent",
 "calculated_total",
 "estado_total"
]
].head(10)
```

índice	quantity	price_per_unit	total_spent	calculated_total	estado_total
2	4.0	1.0	NaN	4.0	sin_datos_suficientes
20	NaN	4.0	20.0	NaN	sin_datos_suficientes
25	3.0	4.0	NaN	12.0	sin_datos_suficientes
31	2.0	1.0	NaN	2.0	sin_datos_suficientes
42	2.0	1.5	NaN	3.0	sin_datos_suficientes
55	NaN	1.0	2.0	NaN	sin_datos_suficientes
56	5.0	NaN	15.0	NaN	sin_datos_suficientes
57	NaN	3.0	3.0	NaN	sin_datos_suficientes
65	3.0	NaN	NaN	NaN	sin_datos_suficientes
66	NaN	3.0	6.0	NaN	sin_datos_suficientes

Esta revisión nos permite diferenciar dos situaciones:

```
no_coincide → hay datos suficientes, pero el total registrado no coincide
con el calculado
sin_datos_suficientes → no hay información suficiente para validar la relación
```

Esa diferencia es importante. No debemos interpretar todas las filas no coincidentes o incompletas como el mismo tipo de problema.

La validación de consistencia interna ayuda a detectar errores más profundos que no siempre aparecen al revisar tipos de datos o valores faltantes por separado.

## Construir una tabla resumen de validación

Después de revisar distintos aspectos del dataset preparado, podemos construir una tabla resumen.

Esta tabla no reemplaza las revisiones detalladas, pero ayuda a reunir en un solo lugar algunas señales importantes del estado final del dataset.



Podemos incluir información como:

```
cantidad de filas
cantidad de columnas
duplicados completos
identificadores duplicados
faltantes en columnas clave
fechas no válidas
casos sin datos suficientes para validar el total
casos donde el total no coincide
```

Construir este tipo de resumen es útil porque permite documentar rápidamente el resultado de la limpieza.

```
resumen_validacion = pd.DataFrame({
 "aspecto": [
 "filas",
 "columnas",
 "duplicados_completos",
 "transaction_id_duplicados",
 "faltantes_item",
 "faltantes_quantity",
 "faltantes_price_per_unit",
 "faltantes_total_spent",
 "faltantes_payment_method",
 "faltantes_location",
 "fechas_no_validas",
 "totales_sin_datos_suficientes",
 "totales_no_coinciden"
],
 "valor": [
 df_preparado.shape[0],
 df_preparado.shape[1],
 df_preparado.duplicated().sum(),
 df_preparado.duplicated(subset=["transaction_id"]).sum(),
 df_preparado["item"].isna().sum(),
 df_preparado["quantity"].isna().sum(),
 df_preparado["price_per_unit"].isna().sum(),
 df_preparado["total_spent"].isna().sum(),
 df_preparado["payment_method"].isna().sum(),
 df_preparado["location"].isna().sum(),
 df_preparado["transaction_date"].isna().sum(),
 (df_preparado["estado_total"] == "sin_datos_suficientes").sum(),
 (df_preparado["estado_total"] == "no_coincide").sum()
]
})

resumen_validacion
```

índice	aspecto	valor
0	filas	10000

índice	aspecto	valor
1	columnas	13
2	duplicados_completos	0
3	transaction_id_duplicados	0
4	faltantes_item	333
5	faltantes_quantity	479
6	faltantes_price_per_unit	533
7	faltantes_total_spent	502
8	faltantes_payment_method	2579
9	faltantes_location	3265
10	fechas_no_validas	460
11	totales_sin_datos_suficientes	1456
12	totales_no_coinciden	0

Esta tabla resume varios controles realizados durante la validación.

Por ejemplo, permite ver rápidamente si se conservaron las filas, si hay duplicados, cuántos faltantes quedan en columnas importantes, cuántas fechas no son válidas y qué resultado tuvo la validación del total.

En un proyecto real, una tabla como esta puede servir como registro del estado final del dataset preparado.

También ayuda a comunicar que limpiar datos no significa necesariamente dejar todo en cero. Puede quedar información faltante o desconocida, pero lo importante es que esos problemas estén detectados, representados correctamente y documentados.

## Qué problemas quedan pendientes

---

Después de validar el dataset preparado, podemos identificar qué aspectos todavía requieren atención.

La validación nos permite ver que el dataset está mejor estructurado que al comienzo: tiene nombres de columnas consistentes, tipos de datos más adecuados, una columna temporal convertida, valores categóricos problemáticos parcialmente unificados y columnas nuevas para validar relaciones internas.

Sin embargo, eso no significa que el dataset esté perfecto.

Todavía pueden quedar valores faltantes en columnas importantes, como `payment_method`, `location`, `quantity`, `price_per_unit`, `total_spent` o `transaction_date`. También pueden quedar valores `"UNKNOWN"` en columnas categóricas, que representan información desconocida o no confiable.

Estos problemas no necesariamente deben resolverse todos de la misma manera. Por ejemplo, una fila sin `location` puede seguir sirviendo para analizar productos vendidos o totales de venta. En cambio, una fila sin `quantity` o sin `price_per_unit` no permite calcular correctamente el importe esperado.

Por eso, después de validar, conviene separar dos ideas:

```
problemas tratados
problemas pendientes
```

Los problemas tratados son aquellos sobre los que ya tomamos una decisión concreta. Por ejemplo, convertimos tipos, unificamos `"ERROR"` dentro de `"UNKNOWN"` y creamos una validación de totales.

Los problemas pendientes son aquellos que decidimos conservar para no aplicar una corrección apresurada. Por ejemplo, no imputamos automáticamente todas las ubicaciones faltantes ni eliminamos todas las filas incompletas.

En un proyecto real, esta distinción es muy importante. Un dataset preparado no es necesariamente un dataset sin problemas. Es un dataset cuyos problemas fueron identificados, tratados cuando correspondía y documentados cuando quedaron pendientes.

## Errores frecuentes al validar una limpieza

---

Un error frecuente después de limpiar un dataset es asumir que, porque el código se ejecutó sin errores, el dataset ya quedó listo para cualquier análisis.

Que una celda no produzca errores no significa necesariamente que la limpieza haya sido correcta. Podemos haber convertido tipos, reemplazado valores o creado columnas nuevas, pero todavía necesitamos verificar si esas transformaciones produjeron el resultado esperado.

Otro error común es revisar solo una parte del dataset. Por ejemplo, podríamos mirar los tipos de datos finales y confirmar que las columnas numéricas quedaron como `float64`, pero olvidarnos de revisar cuántos valores se convirtieron en `NaN`. También podríamos verificar que no hay duplicados completos, pero no revisar si hay identificadores repetidos.

También es un error esperar que un dataset preparado no tenga ningún valor faltante. En muchos casos, una limpieza responsable no completa todos los datos ausentes. A veces es mejor conservar un faltante que inventar un valor sin fundamento.

Otro punto importante es no confundir valores desconocidos con valores válidos. Una categoría como "UNKNOWN" puede ser útil para señalar que el dato no está disponible, pero no debería interpretarse como una categoría común del negocio. No es una ubicación real ni un método de pago real; es una marca de información desconocida.

También debemos tener cuidado con las columnas nuevas. Crear `calculated_total`, `estado_total`, `year`, `month` o `day_of_week` puede enriquecer el dataset, pero esas columnas dependen de la calidad de las columnas originales. Si faltan datos en `quantity`, `price_per_unit` o `transaction_date`, las columnas derivadas también pueden quedar incompletas.

Una rutina razonable de validación posterior podría incluir:

```
comparar dimensiones antes y después
revisar tipos de datos finales
contar valores faltantes
revisar duplicados completos y por identificador
verificar categorías problemáticas
revisar rangos numéricos
validar fechas
comprobar relaciones internas
documentar problemas pendientes
```

La validación posterior no busca declarar que el dataset es perfecto. Busca dejar claro qué tan confiable es, qué transformaciones se aplicaron y qué limitaciones siguen presentes.

## Resumen del capítulo

---

En este capítulo trabajamos con la validación posterior a la limpieza.

Partimos de una idea central: limpiar un dataset no alcanza. Después de aplicar transformaciones, necesitamos comprobar si el resultado quedó en mejores condiciones para el análisis y qué problemas siguen presentes.

Primero reconstruimos una versión preparada del dataset. Conservamos `df_original` como punto de partida y creamos `df_preparado` como versión transformada. Esa versión incluyó nombres de columnas estandarizados, columnas numéricas convertidas, fecha convertida, valores categóricos problemáticos unificados parcialmente, columnas calculadas y columnas de validación.

Luego comparamos las dimensiones del dataset original y del dataset preparado:

```
df_original.shape
df_preparado.shape
```

Esta comparación permitió verificar si se conservaron las filas y si cambió la cantidad de columnas.

Después revisamos los tipos de datos finales usando `info()`. Esta validación permitió comprobar que columnas como `quantity`, `price_per_unit`, `total_spent` y `calculated_total` quedaron como numéricas, mientras que `transaction_date` quedó como columna temporal.

También revisamos los valores faltantes finales:

```
df_preparado.isna().sum()
```

y construimos una tabla con cantidad y porcentaje de faltantes por columna. Esta revisión mostró que el dataset preparado todavía conserva valores faltantes, lo cual no significa necesariamente que la limpieza haya fallado. En varios casos, esos faltantes representan información ausente o valores problemáticos que fueron convertidos a `NaN` o `NaT`.

Más adelante revisamos duplicados completos y duplicados por `transaction_id`:

```
df_preparado.duplicated().sum()
df_preparado.duplicated(subset=["transaction_id"]).sum()
```

Esta validación permitió comprobar si el dataset preparado contenía registros repetidos o identificadores duplicados.

También revisamos categorías problemáticas. Verificamos que `"ERROR"` ya no apareciera como categoría separada en `item`, `payment_method` y `location`, porque durante el flujo de limpieza habíamos decidido agrupar esos casos dentro de `"UNKNOWN"`.

Luego revisamos rangos numéricos usando `describe()` y conteos de valores negativos o iguales a cero. Esta validación ayuda a detectar valores sospechosos que podrían requerir una revisión posterior.

También validamos fechas. Revisamos cuántas fechas quedaron como faltantes o no interpretables, cuál fue el rango temporal del dataset y cómo se distribuyeron los años registrados.

Después revisamos la consistencia interna entre `quantity`, `price_per_unit`, `total_spent` y `calculated_total` mediante la columna `estado_total`. Esta columna permitió distinguir entre casos que coinciden, casos que no coinciden y casos sin datos suficientes para validar la relación.

Finalmente construimos una tabla resumen de validación, reuniendo en un solo lugar varios controles importantes: cantidad de filas, columnas, duplicados, faltantes, fechas no válidas y resultados de la validación de totales.

La idea principal de este capítulo fue:

Validar después de limpiar permite comprobar qué mejoró, qué problemas siguen presentes y qué limitaciones deben documentarse.

Un dataset preparado no es necesariamente un dataset perfecto. Es un dataset cuyo estado conocemos mejor: sabemos qué transformaciones se aplicaron, qué problemas fueron tratados y qué aspectos todavía requieren cuidado.

## Próximo paso

---

Con este capítulo cerramos la parte técnica de la etapa de preparación y limpieza de datos.

Antes de pasar al siguiente eje temático, conviene detenernos y hacer un balance de lo trabajado hasta aquí. Podemos revisar qué aprendimos, cómo se conectan las herramientas entre sí y qué significa, en la práctica, dejar un dataset listo para analizar.

Ese cierre nos va a permitir ordenar el recorrido completo: desde el diagnóstico inicial hasta la construcción y validación de una versión preparada del dataset.

## Capítulo 23 — Del primer DataFrame al dataset preparado

---

### Por qué hacer un balance

---

Llegamos a un punto importante del recorrido.

Hasta ahora trabajamos muchos temas relacionados con Pandas y con el uso de `DataFrames`. Al comienzo, el objetivo era familiarizarnos con la estructura básica de una tabla de datos: observar sus filas, sus columnas, sus índices y algunas de sus características principales.

Luego empezamos a seleccionar información, filtrar registros, combinar condiciones, ordenar datos, construir rankings, crear nuevas columnas y modificar columnas existentes.

Más adelante dimos un paso más grande: dejamos de pensar en operaciones aisladas y empezamos a trabajar con preparación y limpieza de datos. Allí aparecieron problemas más realistas, como valores faltantes, duplicados, categorías inconsistentes, tipos de datos incorrectos, fechas cargadas como texto y la necesidad de validar el resultado después de transformar un dataset.

Este capítulo funciona como un cierre de etapa.

No vamos a incorporar una herramienta nueva de Pandas ni vamos a construir otro flujo de limpieza desde cero. La idea es mirar hacia atrás y ordenar lo aprendido: cómo se conectan los temas, qué tipo de pensamiento fuimos construyendo y qué significa, en la práctica, dejar un dataset preparado para analizar.

Hacer este balance es importante porque el trabajo con datos no consiste en memorizar instrucciones sueltas. Las instrucciones son herramientas. Lo central es aprender a usarlas dentro de un proceso: observar, seleccionar, transformar, limpiar, validar y recién después avanzar hacia el análisis.

### De mirar una tabla a comprender un DataFrame

---

El recorrido comenzó con una idea básica: un `DataFrame` es una estructura de datos organizada en filas y columnas.

A simple vista puede parecer una tabla común, parecida a una hoja de cálculo. Sin embargo, en Pandas un `DataFrame` es mucho más que una tabla visible. Es una

estructura que permite seleccionar datos, filtrarlos, ordenarlos, transformarlos, combinarlos, resumirlos y analizarlos mediante código.

Por eso, los primeros pasos fueron importantes. Antes de modificar un dataset, necesitamos aprender a observarlo.

Herramientas como `head()`, `tail()`, `shape`, `columns` e `info()` nos ayudaron a responder preguntas iniciales:

```
¿Cuántas filas y columnas tiene el dataset?
¿Cómo se llaman sus columnas?
Qué tipos de datos interpretó Pandas?
¿Hay valores faltantes?
¿Cómo se ven los primeros registros?
```

Estas preguntas pueden parecer simples, pero forman parte de una práctica fundamental: no trabajar a ciegas.

Cuando recibimos un dataset, la primera tarea no es aplicar transformaciones ni calcular resultados. La primera tarea es entender qué tenemos delante.

Esta idea se mantuvo durante toda la etapa. Cada vez que avanzamos hacia una transformación más compleja, volvimos a la misma lógica: observar primero, decidir después.

## Seleccionar, filtrar y ordenar información

---

Después de aprender a inspeccionar un `DataFrame`, el siguiente paso fue aprender a trabajar con partes específicas de la tabla.

No siempre necesitamos usar todos los datos al mismo tiempo. Muchas preguntas se responden seleccionando ciertas columnas, ciertas filas o ciertos registros que cumplen una condición.

Por eso trabajamos con selección de columnas, selección de filas, uso de etiquetas con `loc`, filtros con condiciones y combinación de condiciones.

Ese bloque del recorrido nos permitió empezar a formular preguntas más concretas:

```
¿Qué columnas necesito para responder esta pregunta?
¿Qué filas cumplen cierta condición?
¿Qué registros pertenecen a una categoría específica?
¿Qué ventas superan determinado valor?
¿Qué casos cumplen varias condiciones al mismo tiempo?
```



Aprender a filtrar fue un paso muy importante, porque los filtros transforman una tabla general en una respuesta más precisa. En lugar de mirar todo el dataset, empezamos a recortar la información según un criterio.

Luego incorporamos el ordenamiento. Ordenar datos nos permitió observar los registros desde otra perspectiva: productos más caros, ventas con mayor cantidad, valores más bajos, primeros y últimos registros según algún criterio.

A partir de ahí aparecieron los primeros rankings y la búsqueda de valores extremos. Eso nos ayudó a entender que el orden de una tabla no es neutro: según la columna elegida para ordenar, podemos responder preguntas distintas.

Una idea importante de esta parte fue que seleccionar, filtrar y ordenar no son operaciones aisladas. Muchas veces se combinan. Podemos filtrar un subconjunto y luego ordenarlo, seleccionar algunas columnas para leer mejor el resultado, o construir un ranking solo dentro de una categoría.

Ese fue un paso importante: empezamos a usar Pandas no solo para mostrar datos, sino para interrogarlos.

## Crear, modificar y organizar columnas

---

Una vez que aprendimos a seleccionar, filtrar y ordenar datos, dimos otro paso importante: empezamos a transformar la estructura del `DataFrame`.

Crear nuevas columnas nos permitió construir información que no estaba explícita en el dataset original. Por ejemplo, si tenemos una cantidad y un precio unitario, podemos calcular un importe total. Si tenemos una fecha, podemos extraer el año, el mes o el día de la semana. Si tenemos una condición, podemos crear una nueva columna que clasifique los registros según algún criterio.

Este tipo de transformación es muy importante porque muchos datasets no vienen con todas las variables que necesitamos. A veces los datos disponibles son solo el punto de partida. El análisis requiere construir nuevas variables que representen mejor la pregunta que queremos responder.

También aprendimos a modificar columnas existentes. Eso nos permitió corregir textos, reemplazar valores, ajustar formatos y decidir si convenía sobrescribir una columna o crear una versión nueva. Esa decisión no es menor: conservar una columna original puede ser útil para comparar antes y después de una transformación.

Luego trabajamos con la organización general del `DataFrame`: renombrar columnas, eliminar columnas que no necesitamos y reordenar variables. Estas acciones no

siempre cambian los datos en sí mismos, pero hacen que la tabla sea más clara, más cómoda de usar y más fácil de interpretar.

La idea central de esta parte fue que un `DataFrame` no es una estructura fija. Podemos transformarlo para que represente mejor nuestro problema de análisis.

Pero esa transformación debe ser cuidadosa. Cada columna nueva o modificada debería tener un propósito: facilitar una pregunta, corregir un problema, validar una relación o preparar el dataset para una etapa posterior.

## De operaciones aisladas a un proceso de limpieza

---

En la primera parte del recorrido aprendimos muchas operaciones importantes de Pandas. Cada una tenía un objetivo relativamente claro: seleccionar columnas, filtrar filas, ordenar datos, crear una columna, renombrar variables o revisar la estructura del `DataFrame`.

Pero, cuando empezamos a trabajar con limpieza de datos, el enfoque cambió.

Ya no se trataba solamente de aprender una instrucción nueva, sino de pensar un proceso completo. Un dataset real puede tener varios problemas al mismo tiempo: valores faltantes, duplicados, textos inconsistentes, columnas con tipos incorrectos, fechas mal interpretadas o valores que no tienen sentido dentro del contexto.

Por eso, la limpieza de datos nos obligó a trabajar con una lógica más amplia:

```
diagnosticar
decidir
transformar
verificar
```

Diagnosticar significa observar el problema antes de actuar. Decidir implica elegir una estrategia razonable según el contexto. Transformar consiste en aplicar una modificación concreta. Verificar significa comprobar que el cambio produjo el resultado esperado y no generó nuevos problemas.

Esta forma de trabajo fue apareciendo en distintos temas. Con los valores faltantes, primero los contamos y analizamos antes de eliminarlos o imputarlos. Con los duplicados, revisamos si las filas eran realmente repeticiones o si podían representar casos válidos. Con las categorías, verificamos si las diferencias eran solo de escritura o si implicaban significados distintos. Con los tipos de datos, convertimos columnas pero luego revisamos cuántos valores se transformaron en `NaN` o `NaT`.

Ese fue uno de los aprendizajes más importantes de la etapa: limpiar datos no es aplicar comandos automáticamente. Es tomar decisiones justificadas y dejar evidencia de esas decisiones.

## Qué problemas aprendimos a detectar y tratar

---

Durante la etapa de limpieza aparecieron varios problemas frecuentes en el trabajo con datos.

El primero fue el de los valores faltantes. Aprendimos que un valor ausente no siempre significa lo mismo. Puede tratarse de un dato que no fue registrado, de un campo que no correspondía, de un error de carga o de un valor problemático que todavía no fue interpretado como faltante. Por eso, antes de eliminar o completar datos, analizamos cuántos faltantes había, qué porcentaje representaban y qué columnas estaban afectadas.

Luego trabajamos con estrategias para tratar esos faltantes. Vimos que eliminar filas puede ser razonable en algunos casos, pero también puede reducir el dataset o sesgar el análisis. Imputar permite conservar registros, pero introduce valores que no estaban originalmente en los datos. Reconstruir valores a partir de otras columnas puede ser una alternativa más sólida, siempre que exista una relación confiable.

También revisamos duplicados. Allí aprendimos que no todas las filas parecidas son necesariamente registros repetidos. En algunos contextos, dos filas iguales pueden indicar una carga duplicada. En otros, dos filas similares pueden representar eventos reales distintos. La clave fue definir el criterio de duplicación antes de eliminar registros.

Más adelante trabajamos con textos y categorías. Vimos que diferencias pequeñas, como espacios, mayúsculas, minúsculas, tildes o variantes de escritura, pueden fragmentar los conteos. Para una persona, dos valores pueden parecer equivalentes; para Pandas, si no están escritos exactamente igual, son valores distintos. Por eso limpiamos textos, unificamos categorías y revisamos valores problemáticos como `UNKNOWN` o `ERROR`.

Después avanzamos sobre tipos de datos. Aprendimos que una columna puede parecer numérica pero estar cargada como texto. También vimos que una fecha puede verse como fecha, pero seguir siendo un `object` para Pandas. Convertir tipos con cuidado fue fundamental para poder calcular, ordenar, comparar y analizar correctamente.

Finalmente trabajamos con fechas y datos temporales. Allí aprendimos que las fechas permiten crear nuevas variables, revisar rangos temporales, ordenar cronológicamente y comenzar a analizar datos por períodos.

Todos estos problemas tienen algo en común: no se resuelven solo con una instrucción. Requieren interpretar qué representan dentro del dataset y qué impacto pueden tener sobre el análisis.

## Qué significa tener un dataset preparado

---

Después de todo este recorrido, podemos preguntarnos qué significa realmente tener un dataset preparado.

Un dataset preparado no es necesariamente un dataset perfecto. En muchos casos, la perfección no existe: puede haber información faltante que no podemos recuperar, valores desconocidos que deben conservarse, registros que requieren revisión externa o decisiones que dependen del objetivo del análisis.

Preparar un dataset significa otra cosa.

Significa que entendemos mejor qué contiene la tabla, qué problemas tenía, qué transformaciones aplicamos y qué limitaciones siguen presentes.

Un dataset preparado debería tener, al menos, algunas características:

```
nombres de columnas claros
tipos de datos adecuados
valores problemáticos identificados
duplicados revisados
categorías más consistentes
fechas interpretadas correctamente
columnas útiles para el análisis
validaciones básicas realizadas
problemas pendientes documentados
```

Esto no garantiza que todas las conclusiones futuras sean correctas. Pero reduce el riesgo de trabajar sobre datos mal interpretados.

Por ejemplo, si una columna de importes estaba cargada como texto y no lo advertimos, podríamos calcular mal o no poder calcular. Si una categoría aparece escrita de cinco formas distintas, podríamos subestimar su frecuencia real. Si una fecha no fue convertida correctamente, podríamos excluir registros del análisis temporal sin darnos cuenta.

La preparación de datos busca evitar ese tipo de errores.

Por eso, preparar datos no es una etapa secundaria. Es una parte central del análisis. Antes de buscar patrones, construir gráficos o sacar conclusiones, necesitamos asegurarnos de que el dataset esté en condiciones razonables para responder las preguntas que queremos formular.

## Buenas prácticas que conviene conservar

---

A partir de lo trabajado hasta aquí, podemos reconocer algunas buenas prácticas que conviene mantener en los próximos capítulos.

La primera es inspeccionar antes de transformar. Cada vez que recibimos un dataset, necesitamos observar su estructura, sus columnas, sus tipos de datos y sus primeras filas. Esa inspección inicial evita trabajar a ciegas y permite detectar señales tempranas de problemas.

La segunda es no aplicar soluciones automáticas sin contexto. Métodos como `dropna()`, `fillna()`, `replace()` o `drop_duplicates()` son útiles, pero no deberían usarse sin pensar qué representan los datos afectados. Una instrucción puede ser correcta desde el punto de vista técnico y, al mismo tiempo, ser una mala decisión para el análisis.

La tercera es trabajar con copias cuando vamos a realizar transformaciones importantes. Conservar una versión original o una copia de referencia permite comparar antes y después, revisar decisiones y volver atrás si una transformación no resulta adecuada.

La cuarta es verificar después de cada cambio relevante. Si convertimos tipos, revisamos `dtypes` o `info()`. Si limpiamos categorías, revisamos `value_counts()`. Si eliminamos duplicados, contamos nuevamente los duplicados. Si creamos columnas nuevas, observamos ejemplos y comprobamos que tengan sentido.

La quinta es documentar las decisiones. En limpieza de datos, muchas acciones dependen del criterio adoptado. Por ejemplo, decidir que `"ERROR"` se agrupe dentro de `"UNKNOWN"` puede ser razonable en un contexto, pero no en otro. Por eso, conviene dejar claro qué se hizo y por qué.

La sexta es distinguir entre datos corregidos y problemas pendientes. Un dataset puede quedar preparado aunque todavía conserve faltantes o valores desconocidos. Lo importante es saber que esos casos existen, cómo están representados y qué limitaciones generan para el análisis.

Estas prácticas no pertenecen solo a la etapa de limpieza. Van a seguir siendo útiles cuando avancemos hacia resúmenes, agrupamientos, visualizaciones e interpretación de resultados.

## Dejar una base para lo que viene

---

Este cierre no marca el final del trabajo con datos. Marca el final de una primera etapa.

Hasta aquí construimos una base necesaria: aprendimos a cargar, observar, seleccionar, filtrar, ordenar, transformar, limpiar y validar datos dentro de un `DataFrame`.

A partir de ahora, esa base nos permite avanzar hacia preguntas más analíticas.

Ya no se trata solamente de preparar la tabla, sino de empezar a obtener información a partir de ella. Para eso vamos a necesitar nuevas herramientas y nuevas formas de mirar los datos.

En los próximos capítulos vamos a comenzar a trabajar con algunos ejes generales:

```
resumir información
agrupar datos
comparar variables
construir tablas de resumen
visualizar resultados
interpretar hallazgos
```

No necesitamos definir todavía todos los detalles de ese recorrido. Lo importante es entender el cambio de foco.

Durante esta etapa nos preguntamos principalmente:

```
¿Está el dataset en condiciones razonables para trabajar?
```

En la etapa siguiente vamos a empezar a preguntarnos:

```
¿Qué nos dicen los datos?
```

Ese cambio es importante. Para responder buenas preguntas de análisis, primero necesitamos datos preparados. Por eso todo lo trabajado hasta aquí no fue un desvío, sino una condición necesaria para avanzar.

## Resumen de la etapa

---

En esta etapa recorrimos un camino amplio: partimos del primer contacto con un `DataFrame` y llegamos a construir una versión preparada y validada de un dataset.

Al comienzo, el foco estuvo en comprender la estructura básica de los datos: filas, columnas, índices, nombres de columnas, dimensiones y tipos de datos. Aprendimos a inspeccionar una tabla antes de operar sobre ella.

Después avanzamos hacia la selección de información. Trabajamos con columnas, filas, etiquetas, posiciones, filtros, condiciones combinadas, ordenamientos y rankings. Eso nos permitió pasar de mirar una tabla completa a formular preguntas más precisas sobre partes del dataset.

Luego empezamos a transformar la estructura del `DataFrame`. Creamos nuevas columnas, modificamos columnas existentes, renombramos variables, eliminamos columnas innecesarias y reordenamos la tabla para hacerla más clara.

Más adelante ingresamos en la preparación y limpieza de datos. Allí trabajamos con problemas frecuentes en datasets reales: valores faltantes, duplicados, textos inconsistentes, categorías problemáticas, tipos de datos incorrectos y fechas cargadas como texto.

También construimos un flujo de limpieza paso a paso. Ese flujo integró varias decisiones: estandarizar nombres, convertir tipos, revisar categorías, crear columnas calculadas, validar relaciones internas y construir una versión preparada del dataset.

Finalmente, validamos el resultado. Revisamos dimensiones, tipos de datos, faltantes, duplicados, categorías, rangos numéricos, fechas y consistencia interna. Ese paso fue clave para entender que limpiar datos no termina cuando el código se ejecuta, sino cuando comprobamos qué cambió y qué problemas siguen presentes.

La idea central de toda esta etapa puede resumirse así:

Trabajar con datos implica observar, seleccionar, transformar, limpiar, validar e interpretar.

A partir de ahora, vamos a apoyarnos en esta base para avanzar hacia nuevas preguntas. Ya no miraremos el `DataFrame` solo como una estructura que debemos preparar, sino como una fuente de información que podemos resumir, comparar, agrupar, visualizar e interpretar.

## Cierre

---

Llegamos al final de esta primera gran etapa del recorrido.

A lo largo de los capítulos anteriores aprendimos a trabajar con `DataFrames` desde distintos niveles: primero como estructuras de datos que necesitamos comprender, luego como tablas que podemos consultar y transformar, y finalmente como datasets que requieren preparación antes de ser analizados.

Este proceso nos permitió construir una idea fundamental: los datos no hablan por sí solos. Para que puedan ayudarnos a responder preguntas, primero necesitamos entender cómo están organizados, qué problemas presentan, qué decisiones tomamos sobre ellos y qué limitaciones siguen presentes.

La limpieza y preparación de datos no es una tarea menor ni un paso mecánico. Es una parte central del análisis, porque condiciona la calidad de las respuestas que podremos obtener después.

A partir de ahora vamos a avanzar hacia una nueva etapa. Con la base construida, podremos empezar a resumir información, agrupar datos, comparar variables, construir tablas de análisis y visualizar resultados.

En otras palabras, después de preparar los datos, empezaremos a usarlos para construir interpretaciones.





MATERIAL GRATUITO

## Última versión y apoyo al proyecto

Este documento forma parte de VintaBytes y se comparte de manera gratuita.

Escaneá el código QR para acceder a la última versión disponible, encontrar otros recursos del proyecto o colaborar de forma voluntaria.

Si este material te resultó útil, cualquier aporte ayuda a seguir preparando libros, cuadernos y ejemplos en español. También ayuda compartirlo, recomendarlo o avisarme si encontrás algo para mejorar.

Y si querés colaborar con el café que acompaña muchas de estas páginas, prometo usarlo con responsabilidad pedagógica.

Muchas gracias por formar parte de VintaBytes.



Escaneá para ir a la página oficial

También podés entrar desde:

<https://vintabytes.github.io/>